

SKALA

Full-Stack Engineering

Frontend-framework: Vue.js

2026.7

본 문서는 SK㈜ AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 AX



1. Vue.js 시작하기
2. Vue 문법
3. Composition API
4. Vue Component
5. Vue Router
6. Pinia
7. Axios
8. UI 라이브러리
9. Modern JavaScript
10. Vite 빌드 및 실무 배포

교수 소개



[소스 코드]

- https://github.com/bottletiger/skala-vue
  - Git Clone
  - Download ZIP
  - GitHub Fork

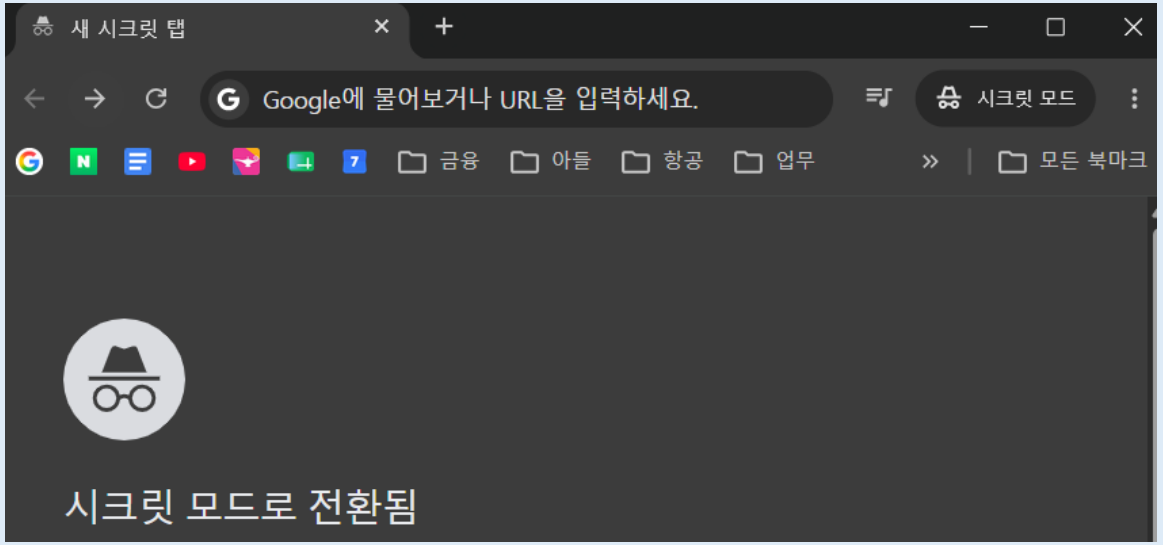
[과제 제출]

- 실습 내용을 제출한다.
  - (필수) 본인의 GitHub 계정에 **Public** 저장소를 생성하여 제출한다. (Private 설정 시 채점 불가).
    - 저장소 주소 예: https://github.com/교육생GitHub계정/skala-vue
  - (선택/권장) 호스팅(배포) URL
    - Vercel, Netlify, GitHub Pages 등을 통해 정적 웹 호스팅을 완료한 경우 제출한다.

- 제출 전 확인
  - Browser 시크릿 창을 켜고 저장소 주소로 접속했을 때, 로그인 요구 없이 소스코드가 정상적으로 잘 보이는지 더블 체크 후 제출한다.
  - Chrome Secret Mode 켜는 법

    Windows / Linux: Ctrl + Shift + N

    macOS (Mac): Command(⌘) + Shift + N



[과제 제출]

- 종합과제 평가 기준표 (Rubric)

점수 구간	평가 등급	기술적 및 실무적 평가 기준
91 ~ 100점	Excellent	• 기본 요구사항을 완벽히 충족 • 지시되지 않은 내용을 추가로 실습한 경우
81 ~ 90점	Good	• 기본 요구사항을 완벽히 충족 • 단순 복사-붙여넣기가 아닌 충분한 개인 실습 흔적이 보이는 경우
71 ~ 80점	Fair	• 안내된 기본 요구사항(최소 스펙)을 부분 충족 • 지정된 형식에 맞춰 GitHub Public Repository URL을 올바르게 제출한 경우
61 ~ 70점	Poor	• 과제를 제출했으나 요구사항의 핵심 기능 중 상당 부분이 누락된 경우 • 제출한 GitHub URL이 비공개(Private) 설정되어 있어 소스코드 확인 및 채점이 불가능한 경우



Full-Stack Engineering - Frontend-framework: Vue.js

1. Vue.js 시작하기

SK 주식회사 AX

Vue.js Overview

- Vue.js는 Front-end JavaScript Framework이다.

정식 명칭	Vue.js (뷰 제이에스)
주요 목적	사용자 인터페이스(UI) 구축
언어	JavaScript, TypeScript 지원
특징	간결한 문법, 반응형 데이터 바인딩, 컴포넌트 기반, 가상 DOM, 트랜지션 효과 지원 등

  - Similar frameworks to Vue are React and Angular, but Vue is more lightweight and easier to start with.
  - [참고] JavaScript vs. TypeScript

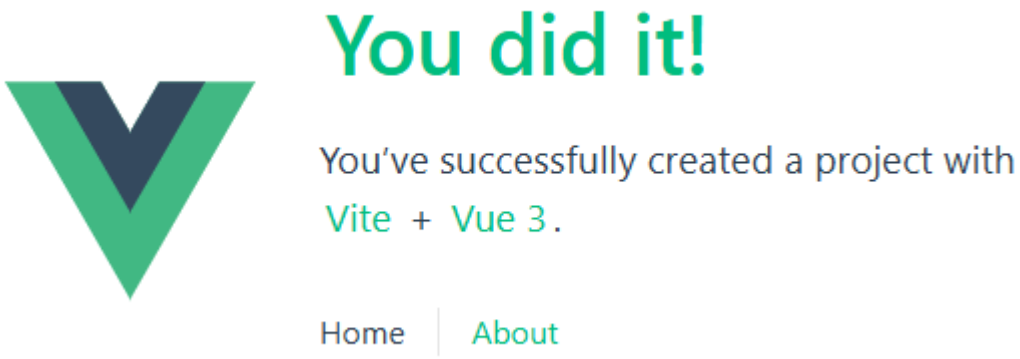
구분	JavaScript	TypeScript
정의	웹의 기본 스크립트 언어	JavaScript에 정적 타입을 추가한 상위 집합(Superset)
타입 시스템	동적 타이핑(Dynamic Typing): 런타임에 변수 타입이 결정됨	정적 타이핑(Static Typing): 컴파일 타임에 변수 타입을 명시 및 검증
에러 발견 시점	런타임(Execution Time): 코드를 실행하는 도중에 타입 에러 발생	컴파일 타임(Compile Time): 코드 작성/빌드 단계에서 타입 에러 감지
실행 방식	브라우저 및 Node.js에서 직접 해석 및 실행	브라우저가 직접 실행 불가능하며, JS로 트랜스파일(Transpile) 후 실행
생산성 & DX	소규모 프로젝트나 빠른 프로토타이핑에 유리	대규모 프로젝트, 자동 완성과 리팩토링 및 코드 가독성에 유리

  - Vue 2의 핵심 코드는 JavaScript로 작성되었으나 Vue 3는 코어 엔진으로부터 완전히 100% TypeScript로 재작성 되었다.
  - Vue 2와 Vue 3 모두 개발자가 프로젝트를 만들 때 JavaScript와 TypeScript 중 원하는 언어를 선택해서 개발할 수 있다.

Vue.js Overview - History

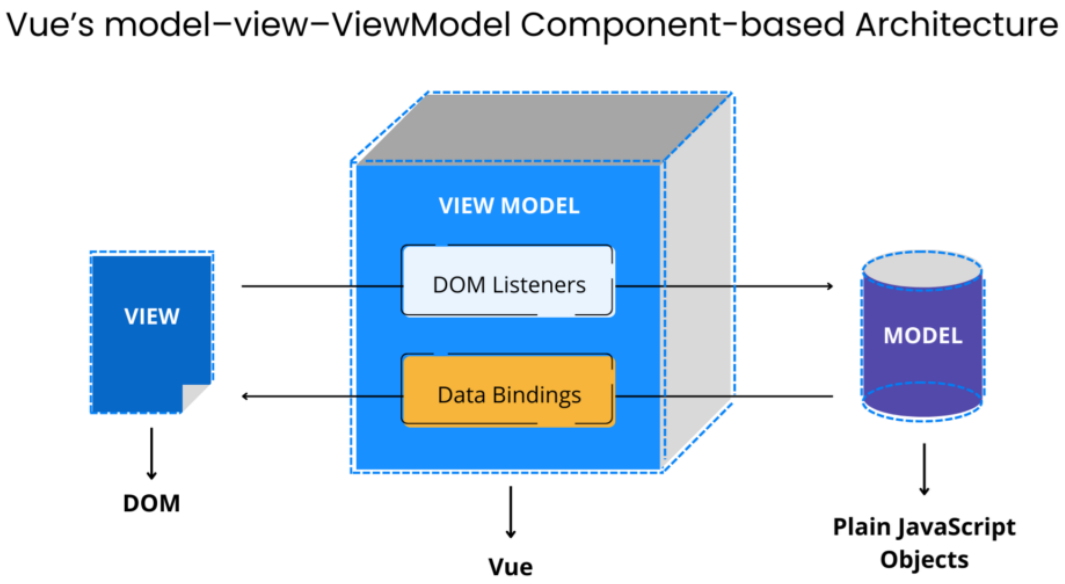
- Vue.js의 역사는 "기존 프레임워크들의 장점만 흡수하여 더 가볍고 쓰기 쉽게 만들자"는 아이디어에서 시작되었다.

연도	주요 이정표
2014년	Evan You에 의해 개발 – 참을 수 없는 AngularJS의 무거움
2015년	Vue.js 1.0 릴리스 – 안정화 버전. 데이터 바인딩과 기본 디렉티브 (v-for, v-if 등)
2016년	Vue.js 2.0 릴리스 – Virtual DOM 도입으로 렌더링 속도 향상
2020년	Vue.js 3.0 릴리스 – Composition API 도입, TypeScript 지원, 퍼포먼스 향상
2023년	Vue 3 표준 - 빌드 도구 Vite연계로 현대화



Vue.js Overview - MVVM

- Vue.js는 MVVM(Model-View-ViewModel) Architecture Pattern을 따른다.
  - MVVM Architecture Pattern은 데이터를 보여주는 view와 데이터를 처리하는 model 사이에 데이터를 중개하는 view model을 두어 UI를 그리는 부분과 Data 처리 로직를 완전히 분리하는 Architecture Pattern이다.



  - **Model**: 애플리케이션에서 사용하는 순수한 비즈니스 데이터로 <script> 내부에 정의된 JavaScript 객체(ref, reactive의 원본 데이터)나 Backend REST API에서 넘어오는 데이터 자체를 의미한다.
  - **View**: 사용자에게 실제로 보여지는 화면 인터페이스로 DOM 구조와 시각적 Styling을 담당한다. (<template>, <style> 영역)
  - **ViewModel**: View와 Model 사이를 연결 주는 중재자로 Vue.js 엔진과 <script>가 ViewModel 역할을 수행하며, DOM Listeners (DOM 이벤트 감지)와Data Bindings라는 메커니즘을 가동하여 데이터를 중개한다.

https://www.scalablepath.com/front-end/vue-js-popularity-framework

Vue.js Overview - UI Frameworks

- Frontend Framework 3대장





항목	Vue.js	React	Angular
출시연도	2014년	2013년	2010년
개발	Evan You (커뮤니티 주도)	Meta (구 Facebook)	Google
기술분류	점진적 프레임워크	UI 라이브러리	풀스택 프레임워크
개발 언어	JavaScript, TypeScript	JavaScript, TypeScript 지원	TypeScript 필수
학습 곡선	낮음	중간	높음
크기	약 33KB	약 42KB	약 500KB
주요 특징 (Key Features)	- Virtual DOM (가상 DOM)	- Virtual DOM	- 실제 DOM 사용
	- 양방향 데이터 바인딩 (v-model)	- 단방향 데이터 흐름	- 양방향 데이터 바인딩
	- 컴포넌트 기반	- Hooks 지원	- 종속성 주입
렌더링 방식	CSR, SSR 지원	CSR, SSR 지원	CSR, SSR 지원

Vue.js Features - Virtual DOM

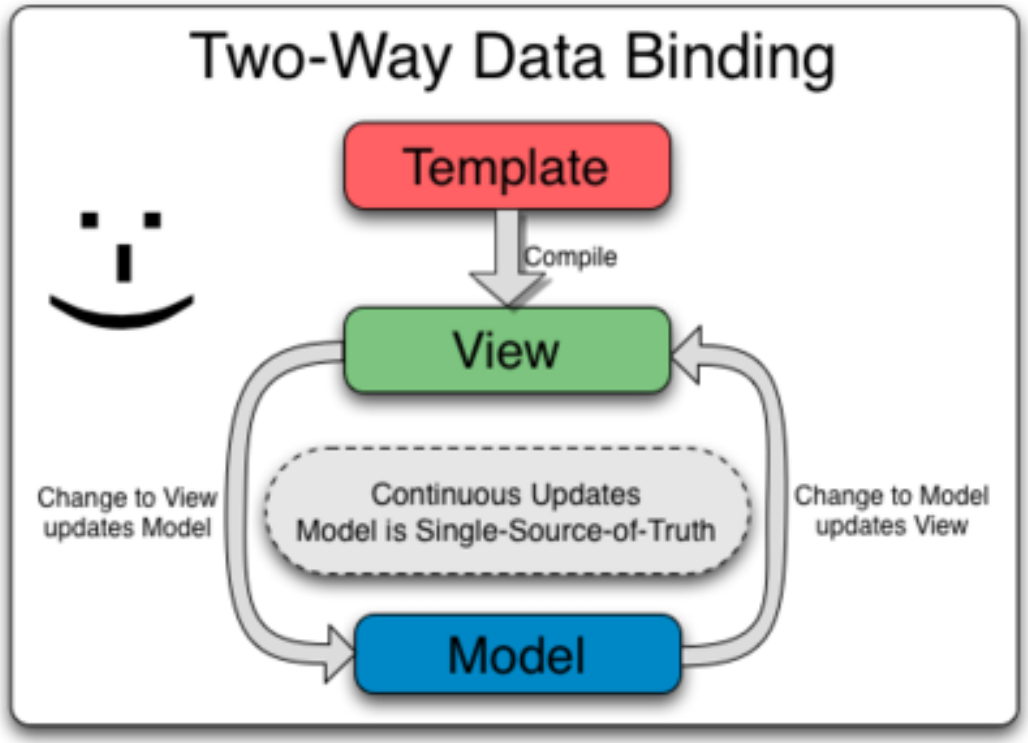
- 브라우저의 Real DOM 조작은 느리다.
  - DOM: Browser가 HTML 코드를 인식하여 Tree 구조로 만든 Object Model
  - 실제 DOM이 수정되면 Browser는 Layout을 다시 계산(Reflow)하고 화면을 다시 색칠하는 Repaint 하는 연산을 수행한다.
  - 만약 3,000개의 노드가 있는 상태에서 연속적인 상태 변경으로 인해 노드를 재구성하는 무분별한 DOM 조작이 일어난다면, 브라우저는 반복적인 Layout/Paint 연산 부하를 겪으며 화면이 느려지게 된다
  - 상태가 변경될 때마다 직접 Real DOM을 연속해서 조작하므로, 10번의 데이터 변화가 일어나면 10번의 브라우저 렌더링 연산이 즉시 발생한다.
- Vue.js는 Virtual DOM을 통해 렌더링 성능을 효율적으로 올린다.
  - **Virtual DOM**: 가상 DOM은 실제 브라우저의 DOM 조작 비용을 줄이기 위해 도입된 메모리상의 가짜 DOM이다.
  - 개발자가 직접 DOM을 일일이 제어하는 번거로움 없이, 상태 변화만 선언해도 Virtual DOM이 최적의 최소 DOM 조작 지점을 자동으로 추적해 준다.
  - Batch 처리를 통한 렌더링 횟수 최적화: 가상 DOM은 한 이벤트 안에 여러 번 데이터가 바뀌어도 모두 끝난 시점에 단 한번만 실제 DOM에 반영한다.





Vue.js Features - Two-Way Data Binding

- 양방향 데이터 바인딩 (Two-Way Data Binding)
  - 양방향 데이터 바인딩이란 "Model(JavaScript Data)이 바뀌면 View(화면)가 바뀌고, 반대로 View(화면 입력)가 바뀌면 Model도 동시에 바뀐다"는 것을 의미한다.
  - Vue에서는 주로 v-model directive를 사용하여 이를 구현한다.

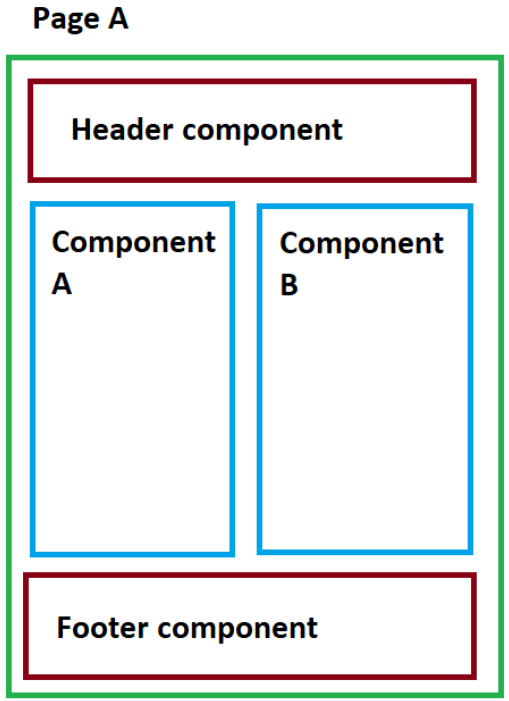


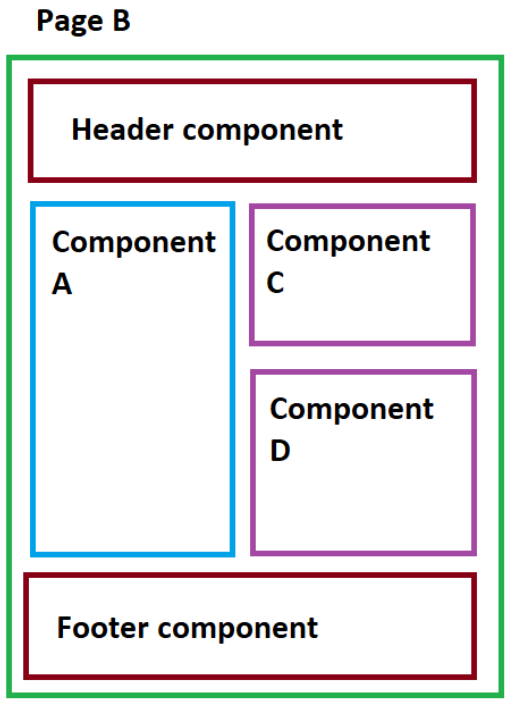
https://medium.com/front-end-weekly/what-is-2-way-data-binding-44dd8082e48e

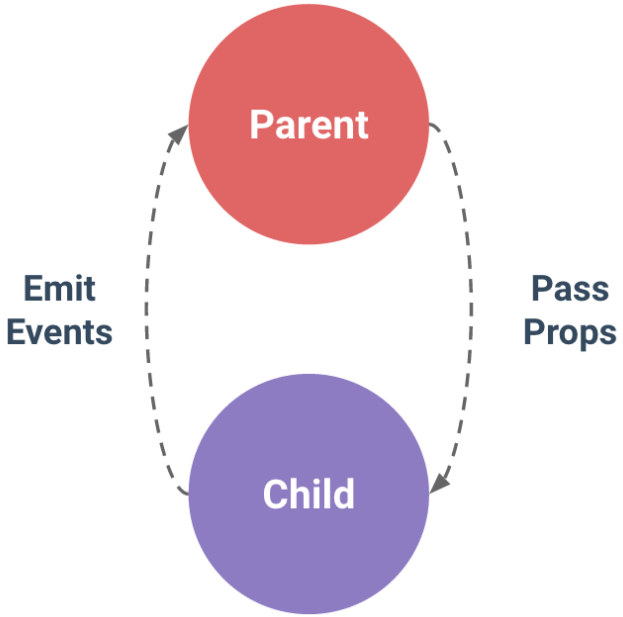
Vue.js Features - Component Based Architecture

- 컴포넌트 기반 개발
  - 웹페이지를 통째로 만들지 않고, 독립적인 UI 부품(Component)들을 각각 만든 뒤 조립하여 화면을 완성하는 방식
  - Encapsulation: 컴포넌트는 내부의 HTML(구조), JavaScript(로직), CSS(스타일)를 하나의 파일(.vue) 안에 응집시켜 놓는다.
  - Reusability: 잘 만든 Component는 여러 곳에서 다시 가져다 쓸 수 있어 중복 코드가 제거된다.
  - Tree Structure: Component들은 부모-자식 관계를 가지며 부모난 자식에게 데이터를 내려주고(Props Down), 자식은 부모에게 상태 변경을 알려준다(Event Up = Emit).









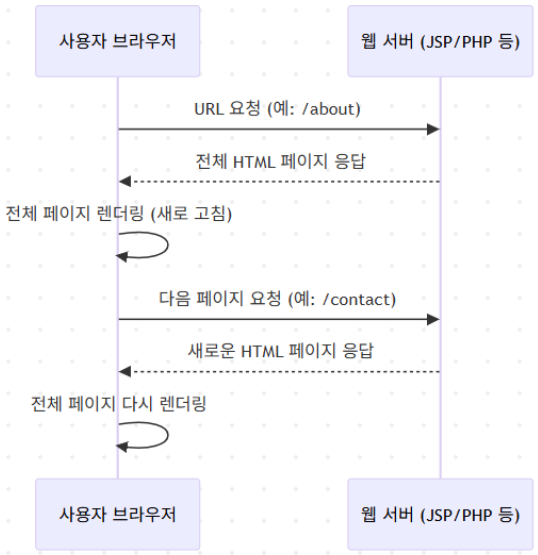
Vue.js Rendering

- Rendering (CSR vs. SSR)
  - 화면을 구성하는 HTML 파일을 어디서 최종적으로 완성하는가?
  - CSR (Client-Side Rendering): 브라우저(Client)가 자바스크립트를 실행하여 화면을 직접 그리는(Rendering) 방식으로 Vue.js의 기본 동작 렌더링 방식이다
  - SSR (Server-Side Rendering): 서버(Server)에서 데이터까지 모두 주입하여 완성된 HTML 파일을 만들어 브라우저에 내려주는 방식으로 Vue 생태계에서는 주로 Nuxt.js 프레임워크를 사용해 구현한다.

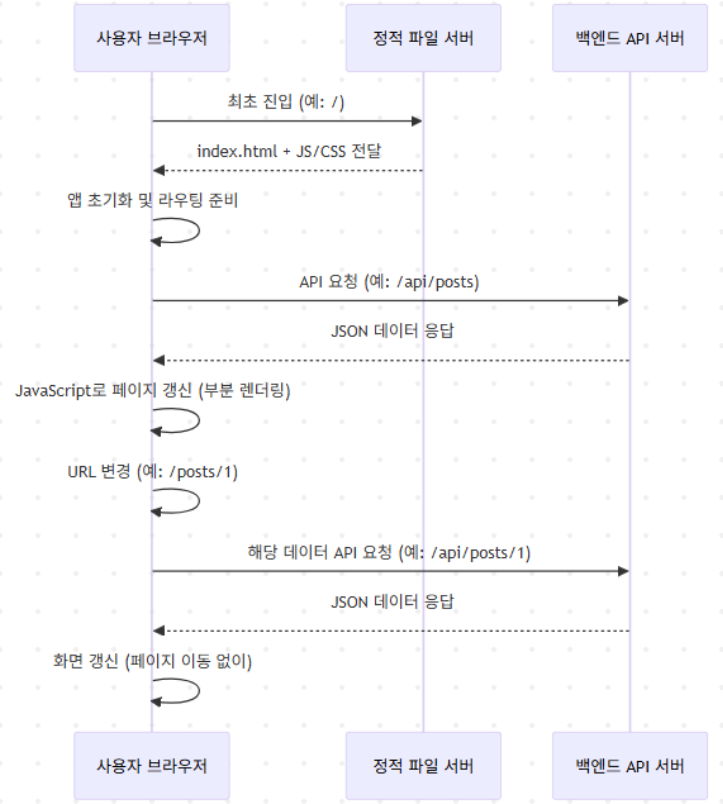
비교 항목	CSR (기본 Vue.js / SPA)	SSR (Vue + Nuxt.js)
HTML 완성 주체	**브라우저 (클라이언트)**	**웹 서버**
초기 서버 전송 데이터	빈 HTML + 대용량 JS 파일	데이터가 결합되어 완성된 HTML
초기 화면 표시 속도	느림 (JS 다운로드 및 실행 대기)	**매우 빠름 (HTML 즉시 렌더링)**
페이지 이동 속도	**매우 빠름 (서버 거치지 않음)**	다소 느림 (서버가 다음 페이지를 또 그려야 함)
검색엔진 최적화 (SEO)	불리함 (봇이 빈 화면으로 인식 가능)	**강력함 (완성된 텍스트 수집 가능)**

Vue.js Rendering - SPA (Single Page Application)

- SPA (Single Page Application)
  - 브라우저가 서버로부터 오직 딱 한 하나의 HTML 페이지(index.html)만 받아와서 구동되는 웹 애플리케이션 구조.
  - 전통적인 방식 (MPA - Multiple Page Application): 사용자가 메뉴를 클릭할 때마다 서버에 새 HTML 파일을 새로 요청
  - SPA 방식 (Vue.js): 서버에는 오직 단 하나의 index.html 파일만 존재하며, JavaScript가 이후 작업을 처리한다.



[Multi Page Application]



[Single Page Application]

Vue.js Rendering - SPA vs MPA

- MPA vs. SPA

구분	Multi Page Application	Single Page Application
페이지 구성 방식	요청할 때마다 새로운 HTML 페이지를 서버에서 전송	초기 로딩 시 하나의 HTML 페이지과 대용량 JS 로딩
페이지 전환	서버로 요청 → 전체 페이지 새로 고침	클라이언트에서 JS로 필요한 부분만 변경 (부분 렌더링)
요청 처리 방식	요청 시마다 서버에서 HTML 렌더링 (SSR)	데이터는 API로 받아오고 렌더링은 브라우저 수행 (CSR)
속도	초기 로딩 매우 빠름, 페이지 전환 속도 다소 느림	초기 로딩 느림, 페이지 전환 속도 매우 빠름
새로고침(F5)	자연스럽게 동작	전체 앱이 다시 로딩되어 **상태가 초기화**될 수 있음
SEO	HTML이 서버에서 렌더링되므로 **SEO에 유리**	JS 기반 렌더링으로 **SEO가 불리** (보완가능)
기술 스택 예시	JSP, PHP, ASP.NET, Spring MVC 등	Vue.js, React, Angular + REST API (ex: Spring Boot)

Vue.js Rendering - SPA 추가 기술 요소 및 장단점

- Vue.js외 SPA의 추가 기술 요소
  - **Vite**: Frontend Build Tool로 개발자가 작성한 수많은 .vue, .js, .css 파일들을 브라우저가 읽을 수 있는 최적화된 형태의 정적 파일로 묶어주고, 개발 서버를 띄워주는 빌드 도구(Bundler)이다.
  - **Vue Router**: 브라우저의 URL 주소와 Vue 컴포넌트를 연결해 주는 공식 라우팅 라이브러리
  - **Pinia**: 전역 상태 관리 라이브러리로 모든 컴포넌트가 접근할 수 있는 중앙 집중식 데이터 저장소(Store)를 메모리 상에 개설한다.
  - **Axios**: 백엔드 서버(API 서버)와 순수한 데이터(JSON)만 주고받는 통신 창구.
- SPA의 장점
  - 압도적인 사용자 경험(UX): 페이지 이동이 데스크톱 소프트웨어처럼 즉각적이다.
  - 네트워크 효율성: 한 번 로딩된 리소스(CSS, JS 등)를 재사용하고 데이터만 주고받으므로 전체적인 네트워크 트래픽이 크게 절감.
  - 프론트/백엔드 분리: 프론트엔드는 Vue만 담당하고, 백엔드는 데이터만 담당하므로 개발 팀 간의 협업 및 서버 분산이 명확
- SPA의 단점
  - 초기 로딩 속도: 웹사이트에 필요한 모든 자바스크립트 로직이 하나의 거대한 덩어리(app.js)로 묶여 처음에 다운로드되므로, 첫 접속 시 대기 시간이 발생할 수 있다.
  - SEO 취약: 검색 로봇이 들어왔을 때 알맹이가 없는 빈 index.html만 보이기 때문에 검색 노출 점수를 따기 어렵다.

개발환경 - WSL2 설치 (Windows 사용자)

- WSL(Windows Subsystem for Linux)2 설치
  - 윈도우 명령창에서 아래 명령어 실행하면 WSL2와 최신 Ubuntu Linux가 동시에 설치된다.

```
wsl --install
```

```
C:\Users\03295>wsl --install
다운로드 중: Linux용 Windows 하위 시스템 2.7.8
설치 중: Linux용 Windows 하위 시스템 2.7.8
Linux용 Windows 하위 시스템 2.7.8이(가) 설치되었습니다.
Windows 선택적 구성 요소 설치 중: VirtualMachinePlatform

배포 이미지 서비스 및 관리 도구
버전: 10.0.26100.5074

이미지 버전: 10.0.26100.8246

기능을 사용하도록 설정하는 중
[==========================100.0%==========================]
작업을 완료했습니다.
요청한 작업이 잘 실행되었습니다. 시스템을 다시 시작하면 변경 사항이 적용됩니다.
요청한 작업이 잘 실행되었습니다. 시스템을 다시 시작하면 변경 사항이 적용됩니다.
```

  - 설치가 완료되면 재부팅후 Linux 초기계정을 설정한다.
  - 참고로, Ubuntu Linux가 동시설치 안되는 경우도 있으며, 이 경우 Ubuntu를 추가로 설치한다.

개발환경 - Ubuntu 설치 (Windows 사용자)

- Ubuntu 설치 및 초기계정 생성
  - WSL 설치 후 Ubuntu가 설치되지 않았다면 Ubuntu 설치

```
wsl --install -d Ubuntu
```

  - 설치 되면서 초기 계정(default Unix user account)을 설정

```
PS C:\Users\03295> wsl --install -d Ubuntu
다운로드 중: Ubuntu
설치 중: Ubuntu
배포가 설치되었습니다. 'wsl.exe -d Ubuntu'을(를) 통해 시작할 수 있습니다.
Ubuntu 시작하는 중...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: bottletiger
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!

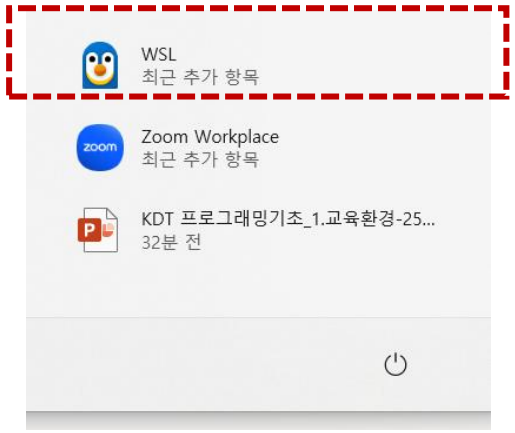
Help us improve Ubuntu features and compatibility by sharing system reports with Canonical.
Reports are sent anonymously and do not contain any personal data.
For legal details, please visit: https://ubuntu.com/legal/systems-information-notice

We will save your answer to Windows and will only ask you once.

Would you like to opt-in to platform metrics collection (Y/n)? To see an example of the data collected, enter 'e'.
[Y/n/e]: y
bottletiger@SKCC22N01104:/mnt/c/Users/03295$
```

개발환경 - Ubuntu 실행 (Windows 사용자)

- 메뉴에서 “WSL” 선택



- 명령 프롬프트에서 Ubuntu 터미널 선택



개발환경 - Node.js

- Vue.js 개발에서의 Node.js의 역할
  - **빌드도구(Vite)의 실행 엔진**:
    개발자가 작성한 .vue 파일은 브라우저가 직접 읽을 수 없어 브라우저가 이해할 수 있는 표준 자바스크립트, 스타일시트, HTML로 변환(컴파일/번들링)해야 한다. 이 변환을 담당하는 최신 빌드 도구인 Vite(비트)가 바로 Node.js 위에서 실행되는 프로그램이다.
  - **패키지 관리 (npm / nvm):**
    Vue 생태계의 수많은 오픈소스 라이브러리(Vue Router, Pinia, Axios, UI 프레임워크 등)를 전 세계 저장소에서 내 컴퓨터로 다운로드하고 버전을 관리해 주는 도구가 바로 npm(Node Package Manager)이다. 이 npm은 Node.js를 설치할 때 함께 설치된다.
  - **로컬 개발 서버 구동**:
    코드를 수정하면 브라우저 화면에 실시간으로 반영되는 핫 리로드(HMR = Hot Module Replacemenet) 기능을 갖춘 로컬 웹 서버(localhost:5173)를 컴퓨터에 띄워 가동하는 주체가 바로 Node.js이다.

개발환경 - Node.js 설치 (WSL)

- WSL에 Node.js 설치하기
  - Ubuntu Linux에 unzip 설치 - 압축해제 프로그램

```
bottletiger@SKCC22N01104:~$ sudo apt update
bottletiger@SKCC22N01104:~$ sudo apt install unzip -y
```

  - fnm(Fast Node Manager) 설치 - 다양한 Node.js 버전을 쉽게 설치하고 빠르게 전환할 수 있도록 도와주는 버전관리도구

```
bottletiger@SKCC22N01104:~$ curl -fsSL https://fnm.vercel.app/install | bash
```

  - 터미널 환경 설정 적용

```
bottletiger@SKCC22N01104:~$ source ~/.bashrc
```

  - Node.js LTS(안정화 버전) 설치 및 활성화

```
bottletiger@SKCC22N01104:~$ fnm install --lts
```

  - 설치 버전 확인

```
bottletiger@SKCC22N01104:~$ node -v
v24.16.0
bottletiger@SKCC22N01104:~$ npm -v
11.13.0
```

- fnm(Fast Node Manager) vs. nvm(Node Version Manager)
  - Node.js 버전을 관리하는 도구로는 nvm과 fnm이 있다.
  - 과거에 나온 nvm은 터미널을 켤 때마다 컴퓨터가 느려지는 단점이 있는데 반해, 최신 기술인 fnm은 속도가 압도적으로 빠르다.

개발환경 - Node.js 설치 (macOS)

- 맥북에 Node.js 설치하기
  - Homebrew 설치 - 맥북 패키지 관리자

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

설치 중간에 맥북 잠금 해제 비밀번호를 요구하면 입력해야 한다. 설치가 끝나면 화면 맨 아래 안내에 따라 Next steps에 나오는 두 줄의 환경설정 명령어를 복사해서 실행해 주어야 한다.

  - Node.js 설치

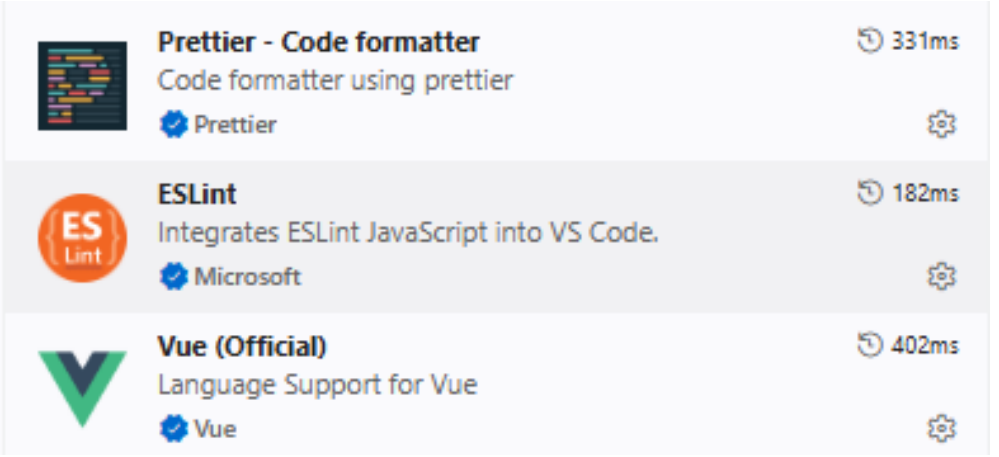
```
brew install node
```

  - 설치 버전 확인

```
bottletiger@SKCC22N01104:~$ node -v
v24.16.0
bottletiger@SKCC22N01104:~$ npm -v
11.13.0
```

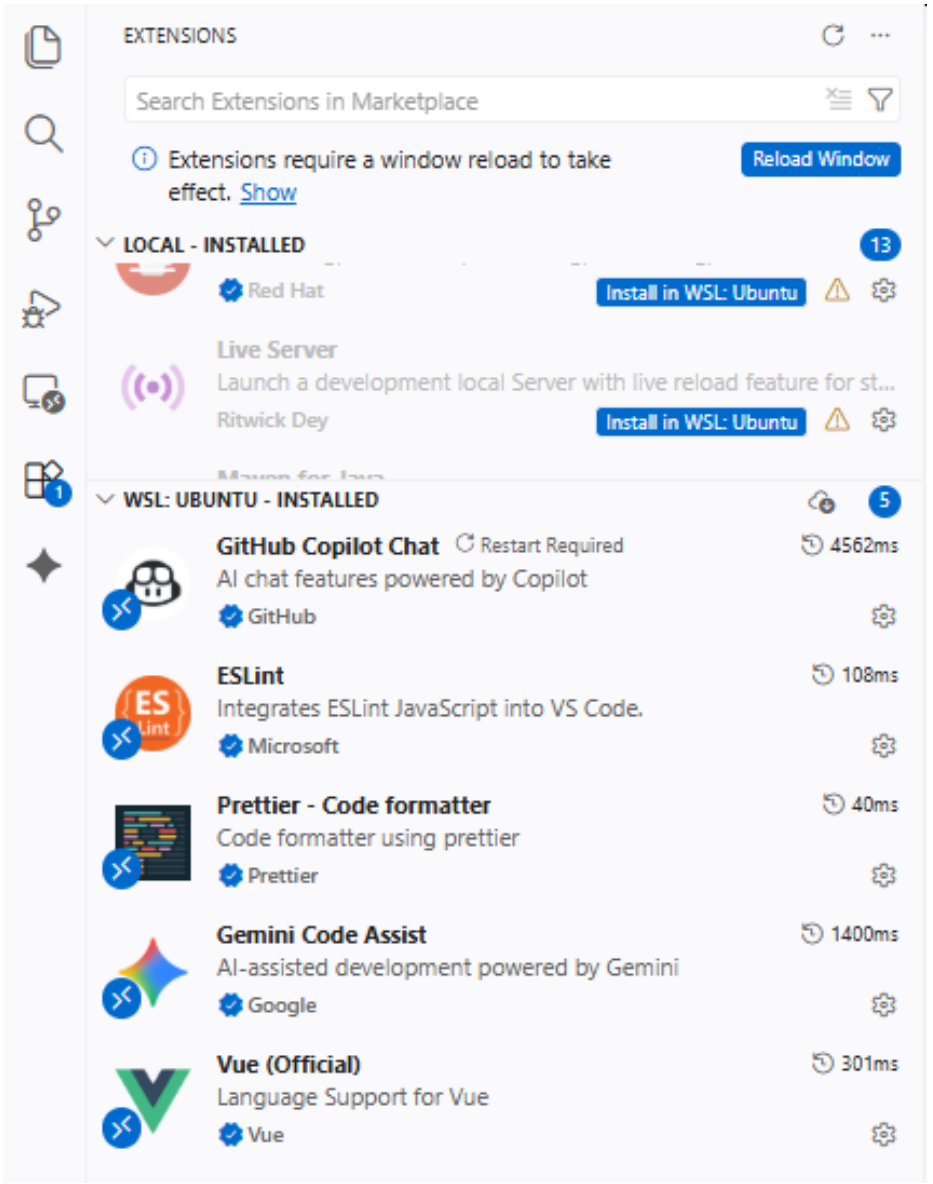
개발환경 - VS Code Extension

- Vue (Official) 설치
  - VS Code가 뷰에서 제공하는 문법과 파일을 인식할 수 있게 하는 확장 프로그램
  - 공식 게시자: Vue
- ESLint 설치
  - ESLint: JavaScript Code 구분에서 에러가 날 만한 부분을 실시간 감지
  - 공식 게시자: Microsoft
- Prettier - Code formatter 설치
  - Prettier: 코드를 작성하 sgn 저장하면 미리 정한 전역 규칙에 맞춰 코드를 자동으로 정렬
  - 공식 게시자: Prettier



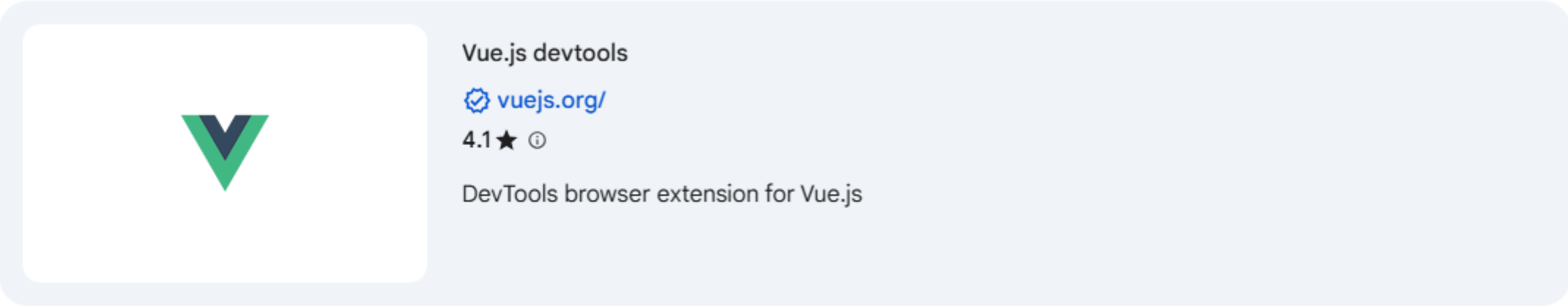
개발환경 - VS Code Extension (WSL)

- WSL 설치
  - Windows에서 VS Code를 통해 Windows Subsystem for Linux(WSL)에서 실행되는 Linux 애플리케이션을 빌드할 수 있게 한다.
  - 공식 게시자: Microsoft
- Connect to WSL
  - VS Code 왼쪽 최하단 구성에 있는 원격 연결 아이콘(><)을 클릭
  - 화면 중앙 상단 메뉴에서 Connect to WSL 선택
- 확장 프로그램 복사
  - 확장 탭을 누르면 Install in WSL: Ubuntu라는 파란색 버튼을 클릭하여 리눅스 구역으로 확장 프로그램 복사 처리



개발환경 - Chrome Extension

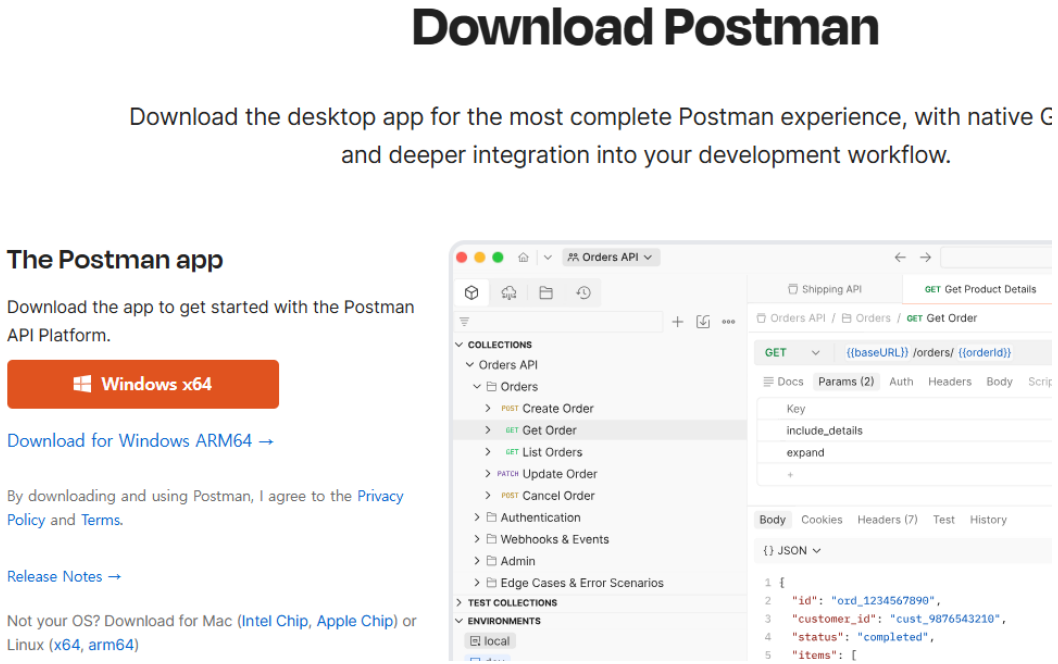
- Vue Devtools 설치
  - 브라우저 우측 상단 메뉴 클릭 > 확장 프로그램 > Chrome 웹 스토어 방문하기
  - 검색란에 “vue.js” 입력 후 Vue.js devtools 선택
  - Chrome에 추가 버튼 클릭

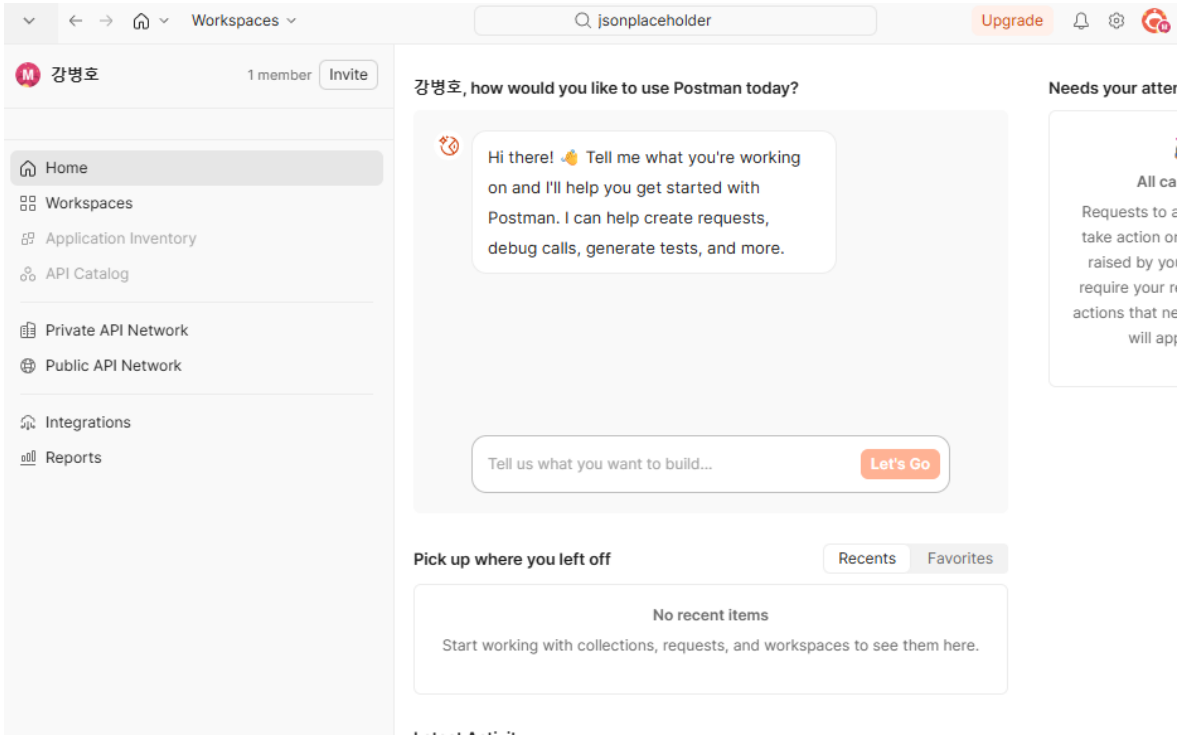


- Vue Devtools 기능
  - Component Tree 실시간 시각화
  - Event 및 Emit 추적
  - 전역 상태 관리(Pinia) 모니터링

개발환경 - Postman

- Postman 설치
  - https://www.postman.com/
  - The Postman app 설치 후 계정 생성
  - Postman 실행





뷰 프로젝트 - skala-vue 생성

- skaka-vue 프로젝트 생성
  - 작업 디렉토리(예. ~/projects)로 이동
  - 프로젝트 생성명령어 (npm create vue@latest) 실행
  - 프로젝트 설정
    - ✔ Project name: skala-vue
    - ✔ Add TypeScript? No
    - ✔ Add JSX Support? No
    - ✔ Add Vue Router for Single Page Application development? Yes
    - ✔ Add Pinia for state management? Yes
    - ✔ Add Vitest for Unit Testing? No
    - ✔ Add an End-to-End (E2E) Testing Solution? No
    - ✔ Add ESLint for code quality? Yes
    - ✔ Add Prettier for code formatting? Yes
- Project Scaffolding
  - 개발에 필요한 기본 디렉터리 구조, 빌드/스타일 설정, 공통 모듈 등을 자동으로 생성하여 초기 개발 환경(뼈대)을 구성하는 작업

```
bottletiger@SKCC22N01104:~$ cd projects
bottletiger@SKCC22N01104:~/projects$ npm create vue@latest
Need to install the following packages:
create-vue@3.22.3
Ok to proceed? (y) y
> npx
> "create-vue"
┌  Vue.js - The Progressive JavaScript Framework
◇  Project name (target directory):
│  skala-vue
◇  Use TypeScript?
│  No
...
Scaffolding project in /home/bottletiger/projects/skala-vue...
└  Done. Now run:
   cd skala-vue
   npm install
   npm run format
   npm run dev
...
```

- JSX(JavaScript XML): JavaScript 코드 내부에서 HTML과 유사한 구문을 직접 작성할 수 있게 해주는 자바스크립트 언어 확장(Syntax Extension)
- Vitest: Vite 환경에 맞춰 개발된 아주 빠른 자바스크립트/타입스크립트 Unit Testing Framework

뷰 프로젝트 - skala-vue 실행

- 생성된 폴더 진입 및 패키지 설치 (의존성 주입)
  - 프로젝트 폴더 진입(cd skala-vue)
  - 패키지 설치 (npm install) → node_modules 생성됨.

```
bottletiger@SKCC22N01104:~/projects$ cd skala-vue
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm install
```

- 로컬 개발서버 구동 및 웹 브라우저 확인
  - 개발서버 구동 (npm run dev)

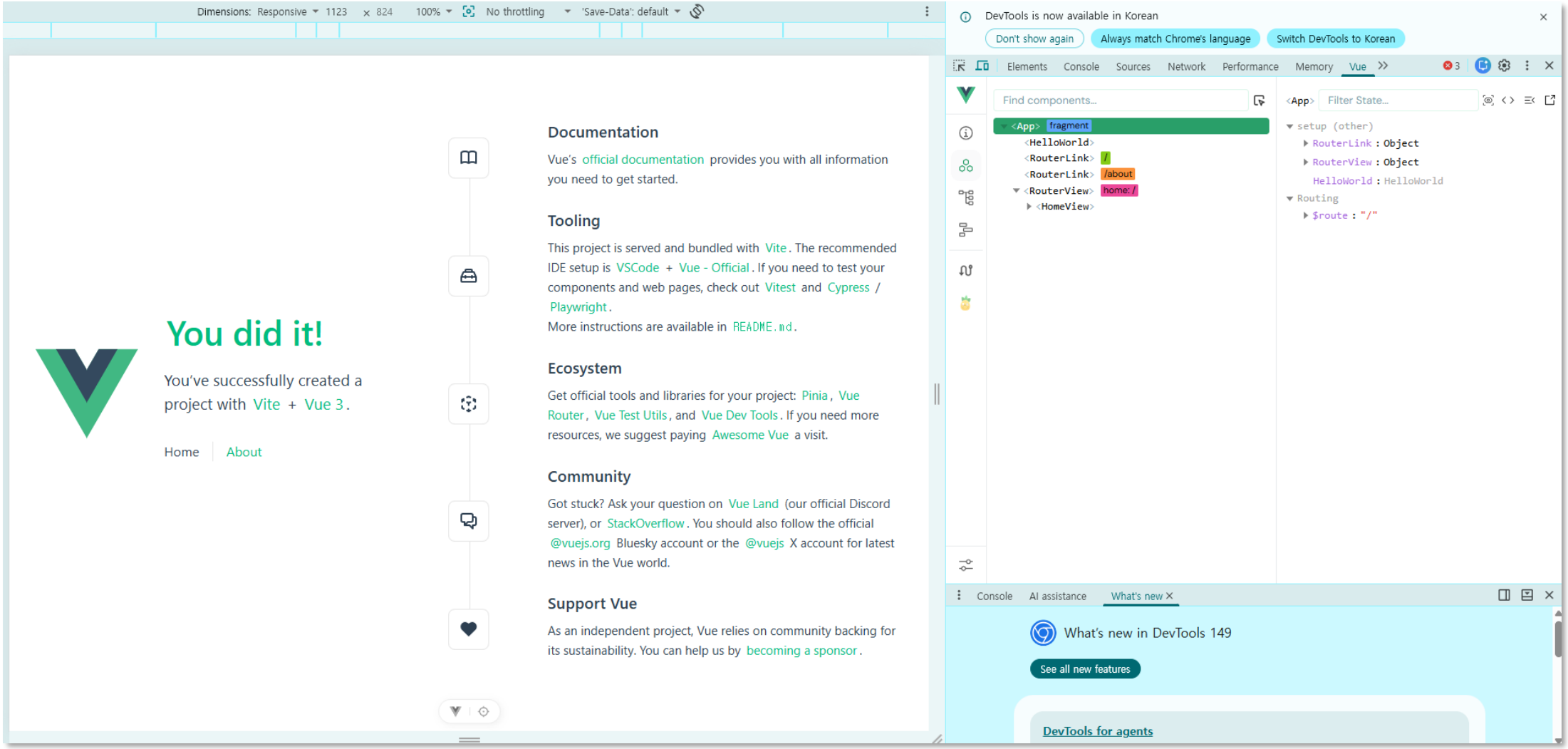
```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm run dev
> skala-vue@0.0.0 dev
> vite


  VITE v8.0.16  ready in 642 ms


  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  Vue DevTools: Open http://localhost:5173/__devtools__/ as a separate window
  ➜  Vue DevTools: Press Alt(⌥)+Shift(⇧)+D in App to toggle the Vue DevTools
  ➜  press h + enter to show help tall
```

뷰 프로젝트 - skala-vue 확인

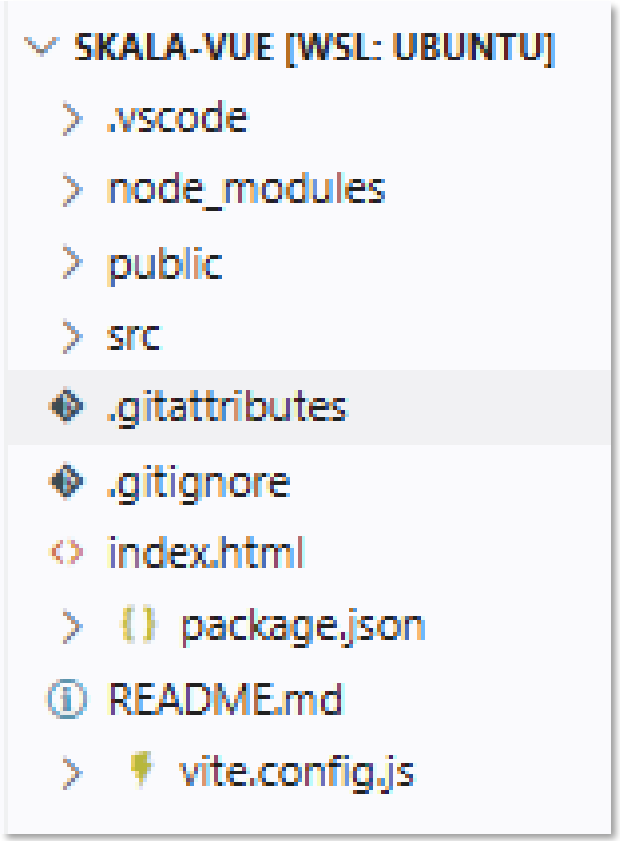
- http://localhost:5173/ 접속
- Chrome 개발자 도구에서 Vue Tab 확인



뷰 프로젝트 - skala-vue 소스

- VS Code에서 SKALA-VUE 프로젝트 폴더 오픈 후 확인

폴더/파일	역할
.vscode	에디터 개인 설정 폴더
node_modules	외부 라이브러리 물리 저장소 (npm install)
public	순수 정적 자원(Static Assets) 저장소. 컴파일하지 않는다.
src	실제 소스코드 작업 공간
.gitattributes	서로 다른 OS를 쓰는 개발자들이 협업할 때 사용
.gitignore	원격 서버에 업로드되지 않는 파일들을 명세
index.html	브라우저가 최초로 보여주는 단 하나의 진짜 HTML 파일
package.json	npm의 핵심 명세서
README.md	프로젝트 가이드 문서
vite.config.js	빌드 도구인 Vite의 전역 환경 설정 파일



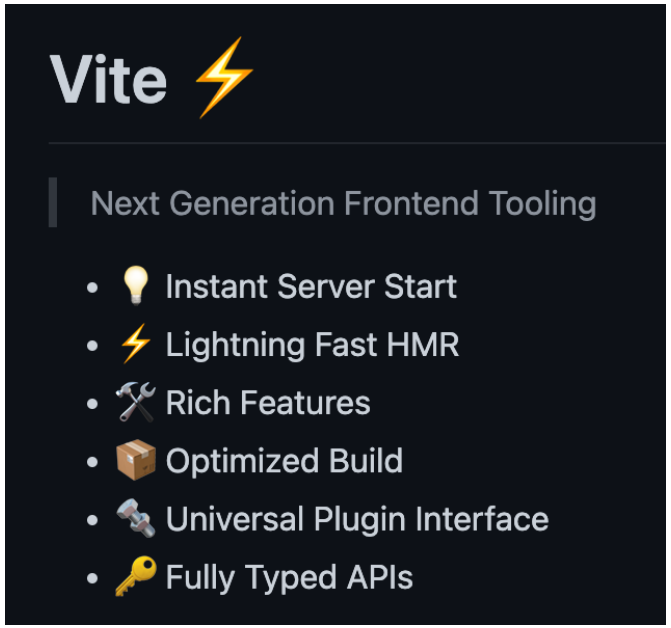
Frontend Project Tools - 도구의 필요성

- 웹 UI 개발 방식의 변화
  - 파일 분할과 모듈화: 유지보수와 재사용성을 위해 자바스크립트를 여러 파일로 나누어 작성
  - 사이즈 최적화: 애플리케이션 배포 시 전체 파일을 묶고 크기를 줄여야 함
  - 브라우저 호환: 구 버전 브라우저에서 ES6 이후의 문법, TypeScript 등을 사용할 수 있는 변환 과정 필요
- 개발 편의를 위한 자동화
  - 변경 시 자동 새로 고침 (Live Reload, HMR)
  - 코드 압축, 이미지 최적화, CSS 전처리
  - .vue, .scss, .ts 등 다양한 확장자 파일 처리
- 도구 유형

용어	의미/역할	예시
Packager	패키지를 다운로드하고 설치 (종속성 관리)	npm, yarn, pnpm
Compiler	코드를 다른 형식으로 변환 (ex. TypeScript → JavaScript)	Babel, TypeScript
Transpiler	같은 수준의 언어에서 문법을 변환	Babel (ES6 → ES5)
Task Runner	반복 작업 자동화 (빌드, 테스트, 린트 등)	Gulp, Grunt
Bundler	여러 JS/CSS 모듈을 묶어 하나의 파일 또는 청크로 만듦	Vite, Webpack, Rollup, Parcel
Build Tool	컴파일, 번들링, 최적화 등을 포함한 전체 빌드 과정 도구	Vite, Webpack, Parcel

Frontend Project Tools - Vite

- Vite는 Vue 소스 코드를 Build하고, 로컬 개발 서버를 띄워주는 Frontend Build 도구이다.
- Vite 주요 기능
  - Compile: .vue나 TypeScript를 순수한 HTML, JavaScript, CSS로 변환
  - Local 개발 서버 소스 제공: 코드를 실시간으로 반영(HMR, Hot Module Replacement)되도록 로컬 웹 서버에 컴파일된 소스 제공
  - Bundling: 수백 개의 작은 소스코드 파일들을 배포에 적합하도록 몇 개의 압축된 덩어리 파일로 묶어줌 (Staging/Production 서버용)



- 참고로, Vue-CLI + Webpack은 Vite 이전 기술로 더 이상 사용하지 않는다.

https://tech.wonderwall.kr/articles/vite/

Frontend Project Tools - npm

- NPM(Node Package Manager)는 오픈소스 라이브러리 저장소이자 Node.js 패키지 관리도구이다.
- NPM 구성 요소
  - npm Registry: JavaScript 라이브러리를 업로드 해 두는 공식 Cloud Server
  - npm CLI(Command Line Interface): npm install… 같은 명령어를 수행
  - npmjs.com: 등록된 패키지들을 웹 브라우저로 확인할 수 있는 공식 포털 사이트
- 주요 NPM 명령어

분류	명령어	상세 내용
프로젝트 생성	npm create vue@latest	Vue 공식 프로젝트 생성 스캐폴딩 도구(create-vue)를 실행하여 Vite 기반의 최신 Vue 3 뼈대 폴더와 설정 파일을 자동으로 생성한다.
의존성 설치	npm install	프로젝트 루트의 package.json 명세서를 읽어, 누락된 외부 라이브러리들을 인터넷에서 다운로드하여 node_modules 폴더를 생성한다.
패키지 추가	npm install 패키지명	특정 외부 라이브러리를 다운로드하여 node_modules에 넣고, package.json의 dependencies(배포용 라이브러리 목록)에 자동 등록한다.
패키지 제거	npm uninstall 패키지명	node_modules 내부에서 해당 라이브러리 물리 파일을 완전히 삭제하고, package.json 명세서에서도 이름을 깨끗이 지운다.
로컬서버 구동	npm run dev	package.json의 scripts에 정의된 vite 엔진을 가동하여 컴파일을 대기하고, 로컬 주소(localhost:5173)로 HTTP 개발 서버를 개설한다.
최종 배포	npm run build	내부의 **Rollup** 엔진을 깨워 프로젝트의 모든 소스를 샅샅이 뒤진 뒤, 쓸데없는 코드를 털어내고(Tree-shaking) 압축된 정적 파일(dist 폴더)을 생성한다

[실습] 개발환경 구성

- 개발에 필요한 환경을 구성한다.
  - WSL2 설치 (Windows 사용자 Only)
  - Ubuntu 설치 (Windows 사용자 Only)
  - Node.js 설치
  - VS Code Extension 설치 (Windows 사용자는 WSL도 추가 설치)
  - Chrome Extension 설치

https://whatismyipaddress.com/

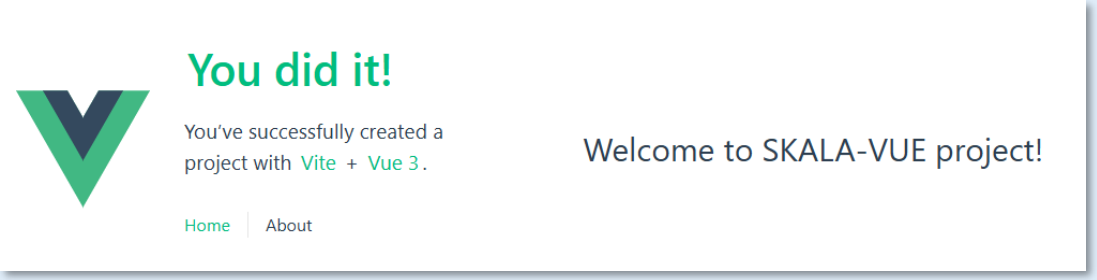
[실습] 프로젝트 생성 및 실행

- SKALA-VUE 프로젝트를 생성하고 실행한다.
  - skala-vue 프로젝트 생성
  - skala-vue 프로젝트 실행
  - skala-vue 프로젝트 동작 확인 및 Vue Tools 확인
  - skala-vue 프로젝트 소스 확인

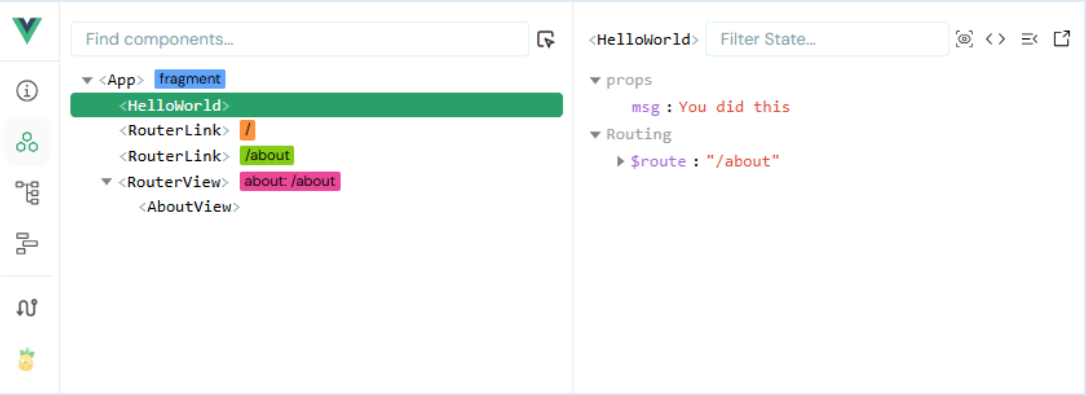
https://whatismyipaddress.com/

[실습] HMR 체감 및 Vue Devtools

- 소스 변경을 통해 HMR을 즉시 확인한다.
  - VS Code와 SKALA-VUE(http://localhost:5173/)에 접속하여 AboutView 확인
  - src/views/AboutView.vue 파일의 Template 부분 변경하고 변경한 내역 즉시 반영 확인



- Vue Devtools 상태 확인
  - 개발자도구에서 Vue Tab 확인
  - Components Tab에서 Vue 컴포넌트들의 부모-상속관계 확인
  - 단일 컴포넌트를 클릭하여 props, routing 등을 확인
  - 브라우저 메모리 상의 변수 값을 즉시 변경하면서 내용 확인



Full-Stack Engineering - Frontend-framework: Vue.js

2. Vue 문법



SK 주식회사 AX

Project 구조

- 생성된 핵심 폴더와 주요 파일 구조

디렉토리/파일	설명
index.html	어플리케이션 진입점(entry point). 브라우저가 최초로 읽는 단 하나의 진짜 HTML.
package.json	프로젝트 메타정보(이름, 버전), 실행 스크립트 명령어, 의존성 라이브러리 목록 기록.
vite.config.js	Vite 빌드 엔진 설정 파일
.gitignore	git ignore
public/	브라우저에 바로 제공되는 정적(static) 파일 폴더. 빌드 엔진이 컴파일하지 않는다.
src/	소스 코드 디렉토리
main.js	index.html에서 지정하는 어플리케이션 진입점 ← index.html에서 호출
App.vue	어플리케이션 루트 Vue 컴포넌트 ← main.js에서 호출
style.css	스타일(CSS) 파일
assets/	CSS, 로고 이미지, 폰트 파일 등 **Vite 빌드 엔진에 의해 컴파일 및 최적화가 필요한 소스 자원**
components/	화면의 일부분을 조각내어 만든 **재사용 가능한 작은 부품(Component)들을 보관**하는 곳
HelloWorld.vue	헬로 월드 컴포넌트 ← App.vue에서 호출
router/	SPA(싱글 페이지 앱)의 핵심인 페이지 이동 경로를 정의
stores/	전역 상태 관리 도구인 **Pinia(피니아)의 데이터 저장소**
views/	components 조각들을 조립해서 완성한 **하나의 거대한 독립된 '페이지 단위 화면'**

Project 구조 - package.json

- Meta 정보
  - name: 프로젝트 이름
  - Version: 프로젝트 버전 (0.0.0은 [major].[minor].[patch])
  - private: 이 프로젝트가 실수로 npm 레지스트리(공개 패키지 저장소)에 배포(publish)되는 것을 방지 (true 면 비공개)
  - type: module - 프로젝트 내 .js 파일들의 기본 모듈 시스템을 ESM (ECMAScript Modules, import/export)으로 지정
- "scripts": Script 명령어 정보 (npm run 명령어로 실행)
  - dev: dev 런타임 실행 (npm run dev)
  - build: 패키지 빌드(npm run build) → "dist" 폴더에 배포용 정적 자원 번들링
- 의존성 모듈 정보
  - dependencies : 애플리케이션 실행에 필요한 패키지
  - devDependencies : 개발 중에만 필요한 패키지
- "engines": 실행 환경 제약
  - node : 프로젝트가 정상 동작하기 위해 요구되는 Node.js 최소 실행 버전

{} package.json > ...

```
{
  "name": "skala-vue",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "run-s lint:*",
    "lint:oxlint": "oxlint . --fix",
    "lint:eslint": "eslint . --fix --cache",
    "format": "prettier --write --experimental-cli src/"
  },
  "dependencies": {
    "pinia": "^3.0.4",
    "vue": "^3.5.32",
    "vue-router": "^5.0.4"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@vitejs/plugin-vue": "^6.0.6",
    "eslint": "^10.2.1",
    "eslint-config-prettier": "^10.1.8",
    "eslint-plugin-oxlint": "~1.60.0",
    "eslint-plugin-vue": "~10.8.0",
    "globals": "^17.5.0",
    "npm-run-all2": "^8.0.4",
    "oxlint": "~1.60.0",
    "prettier": "3.8.3",
    "vite": "^8.0.8",
    "vite-plugin-vue-devtools": "^8.1.1"
  },
  "engines": {
    "node": "^20.19.0 || >=22.12.0"
  }
}
```

Project 구조 - index.html

- Application의 진입점(Entry Point)로 브라우저가 최초로 읽는 단 하나의 HTML.
  - Line 10: 안에 Vue엔진이 Component들을 실시간으로 바꿔 끼워 화면을 그린다.
  - Line 11: main.js를 모듈 형식으로 연결하여 이 때부터 뷰 어플리케이션에서 사용할 패키지와 코드가 실행된다.

index.html > ...

```
1   <!DOCTYPE html>
2   <html lang="">
3     <head>
4       <meta charset="UTF-8">
5       <link rel="icon" href="/favicon.ico">
6       <meta name="viewport" content="width=device-width, initial-scale=1.0">
7       <title>Vite App</title>
8     </head>
9     <body>
10      <div id="app"></div>
11      <script type="module" src="/src/main.js"></script>
12    </body>
13  </html>
```

Project 구조 - main.js

- main.js 파일은 뷰 어플리케이션을 초기화하고 구성하는 역할을 한다.
  - Line 1: 프로젝트 전체에 적용할 main.css를 불러온다. 모든 컴포넌트에 적용됨.
  - Line 3: 'vue' 패키지에서 createApp 함수를 불러온다.
  - Line 6: App.vue 파일을 불러온다.
  - Line 9: createApp()함수로 뷰 어플리케이션 인스턴스를 생성한다. 전달된 App.vue 파일이 루트 컴포넌트가 된다. 아직 화면에 그려지지 않은 상태이다.
  - Line 11~12: main.js에서는 Pinia와 Router 같은 모듈도 등록한다
  - Line 14: index.html의 에 물리적으로 넣으며(mount) 화면 렌더링을 시작한다.
- createApp()
  - 첫 번째 인자로 애플리케이션의 시작점이 되는 최상위 컴포넌트(App.vue)를 전달받는다.
  - 이 최상위 컴포넌트를 기점으로 하위 컴포넌트들이 Component Tree를 형성하며 렌더링 파이프라인을 구축한다.
  - createApp(App)이 반환하는 app 객체는 플러그인을 설치하거나 전역 자원을 등록할 수 있는 컨텍스트 API(use, component, directive 등)를 내장하고 있다.

src > JS main.js > ...

```
1   import './assets/main.css'
2
3   import { createApp } from 'vue'
4   import { createPinia } from 'pinia'
5
6   import App from './App.vue'
7   import router from './router'
8
9   const app = createApp(App)
10
11  app.use(createPinia())
12  app.use(router)
13
14  app.mount('#app')
```

Project 구조 - App.vue

- App.vue는 뷰 어플리케이션의 Root Component 역할을 한다.
  - Line 3: components 폴더의 HelloWorld.vue를 불러온다.
  - Line 11: HelloWorld를 화면에 배치하고, 컴포넌트 속성으로 msg를 전달한다.
- Vue-router
  - Line 2: 주소창에 따라 화면을 전환하기 위해 Vue Router가 제공하는 RouterLink와 RouterView를 가져온다.
  - Line 13~14: <RouterLink>는 HTML의 <a> 태그로 변환되지만 Browser의 새로고침을 막고 주소창만 바꾸고 <RouterView />구역에 해당 컴포넌트를 바꾼다.
  - Line 20: 주소창 변경에 따라 실제 화면이 갈이 끼워지는 가변형 주입 구역이다.

src > App.vue > { } script setup

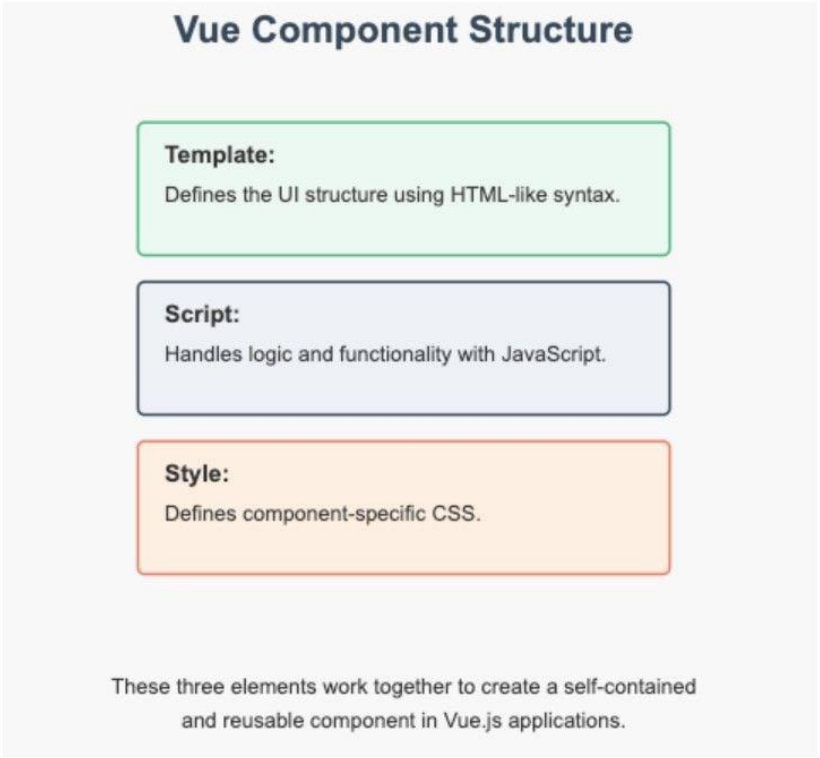
```
1  <script setup>
2  import { RouterLink, RouterView } from 'vue-router'
3  import HelloWorld from './components/HelloWorld.vue'
4  </script>
5
6  <template>
7    <header>
8      <img alt="Vue logo" class="logo" src="@/assets/logo.svg"
9
10     <div class="wrapper">
11       <HelloWorld msg="You did it!" />
12
13       <nav>
14         <RouterLink to="/">Home</RouterLink>
15         <RouterLink to="/about">About</RouterLink>
16       </nav>
17     </div>
18   </header>
19
20   <RouterView />
21 </template>
```

SFC (Single File Component)

- Vue 컴포넌트는 .vue 확장자를 가진 하나의 독립된 파일(SFC)로 구성된다.
- SFC (.vue 파일) 3단 구조

<script setup>	데이터, 함수 등 기능 로직을 JavaScript로 작성하는 곳
<template>	사용자에게 보여질 HTML 구조를 작성하는 곳
<style>	CSS 스타일을 작성하는 곳 (보통 scoped로 적용 범위를 제한함)

  - Vue3 생태계에서는 <script setup>을 맨 위에 먼저 쓰고, 그 아래에 <template>을 작성하는 방식이 트렌드가 되었다.
- 컴포넌트 파일명을 지을 때 Naming Convention은 두 단어 이상으로 조합된 PascalCase를 권장한다.



https://www.monterail.com/blog/guide-to-vue-components

SFC - HelloWorld.vue

src > components > HelloWorld.vue > ...

```vue
<script setup>
defineProps({
  msg: {
    type: String,
    required: true,
  },
})
</script>

<template>
  <div class="greetings">
    <h1 class="green">{{ msg }}</h1>
    <h3>
      You've successfully created a project with
      <a href="https://vite.dev/" target="_blank" rel="noopener">Vite</a> +
      <a href="https://vuejs.org/" target="_blank" rel="noopener">Vue 3</a>.
    </h3>
  </div>
</template>

<style scoped>
h1 {
  font-weight: 500;
  font-size: 2.6rem;
  position: relative;
  top: -10px;
}

h3 {
  font-size: 1.2rem;
}

.greetings h1,
.greetings h3 {
  text-align: center;
}

@media (min-width: 1024px) {
  .greetings h1,
  .greetings h3 {
    text-align: left;
  }
}
</style>
```

SFC - Options API vs. Composition API

- Vue 컴포넌트의 Script 영역을 작성하는 방법

비교 항목	Options API (Vue 2 방식)	Composition API (Vue 3 표준)
코드 선언 구조	<script>	<script **setup**>
작성 철학	역할별 격리 (Options 기반) 정해진 상자(data, methods, computed) 안에 코드를 나누어 배치	논리적 기능별 그룹화 (Function 기반) 순수 자바스크립트처럼 관련 있는 데이터와 함수를 한곳에 연달아 묶어 작성하는 방식.
코드 가독성 (규모가 커질 때)	하나의 기능을 수정하기 위해 파일 상단 data와 하단 methods를 계속 오르내려야 함.	하나의 기능에 필요한 데이터와 로직이 한 구역에 모여 있어 한눈에 파악 가능.
반응성 변수 선언	data() 함수가 반환하는 객체 내부에 선언	ref() 또는 reactive() 내장 함수를 사용해 선언
코드 재사용성	Mixin을 사용하나, 데이터 출처가 불분명해지고 이름 충돌 가능성이 높음	Composable 함수를 사용해 순수 자바스크립트 함수 형태로 완벽하게 격리 및 재사용 가능.
TypeScript 호환	구조적 한계로 인해 타입 추론 및 결합이 매우 복잡하고 부자연스러움.	순수 함수 및 변수 기반이므로 TypeScript와 완벽하게 100% 호환 및 자동 추론 가능.
공식 권장 여부	레거시 유지보수 외에는 신규 권장 안 함	현재 Vue 3 공식 문서 및 생태계 기본 표준

* Composable: Vue의 반응형 상태(Ref, Reactive)와 로직을 묶어 재사용할 수 있도록 많든 함수

SFC - Options API vs. Composition API Example

```vue
<template>
    <div>
        <h1>Options API Counter</h1>
        <p>Count: {{ count }}</p>
        <button @click="increment">Increment</button>
    </div>
</template>

<script>
export default {
    data() {
        return {
            count: 0, // 상태 정의
        };
    },
    methods: {
        increment() {
            this.count++; // 메서드 정의
        },
    },
};
</script>

<style>
button {
    padding: 8px 16px;
    font-size: 16px;
}
</style>
```

```vue
<template>
    <div>
        <h1>Composition API Counter</h1>
        <p>Count: {{ count }}</p>
        <button @click="increment">Increment</button>
    </div>
</template>

<script setup>
import { ref } from 'vue';

const count = ref(0); // 상태 정의
const increment = () => {
    count.value++; // 메서드 정의
};
</script>

<style>
button {
    padding: 8px 16px;
    font-size: 16px;
}
</style>
```

SFC - Interpolation & Directive

- Vue 컴포넌트의 Template 영역을 작성하는 방법
- Text Interpolation (텍스트 보간법)
  - Syntax: {{ 변수명 }}
  - 용도: JavaScript 변수 값을 그대로 문자열로 투사하고 싶을 때 사용
- Directive
  - Syntax: v-로 시작하는 Vue 전용 특수 속성 (v-bind, v-if, v-for, v-on 등)
  - 용도: 일반 HTML 태그 안에서 태그의 속성, 스타일, 조건문, 반복문, 이벤트 리스너 등을 자바스크립트 데이터와 연결하여 제어하기 위해 사용

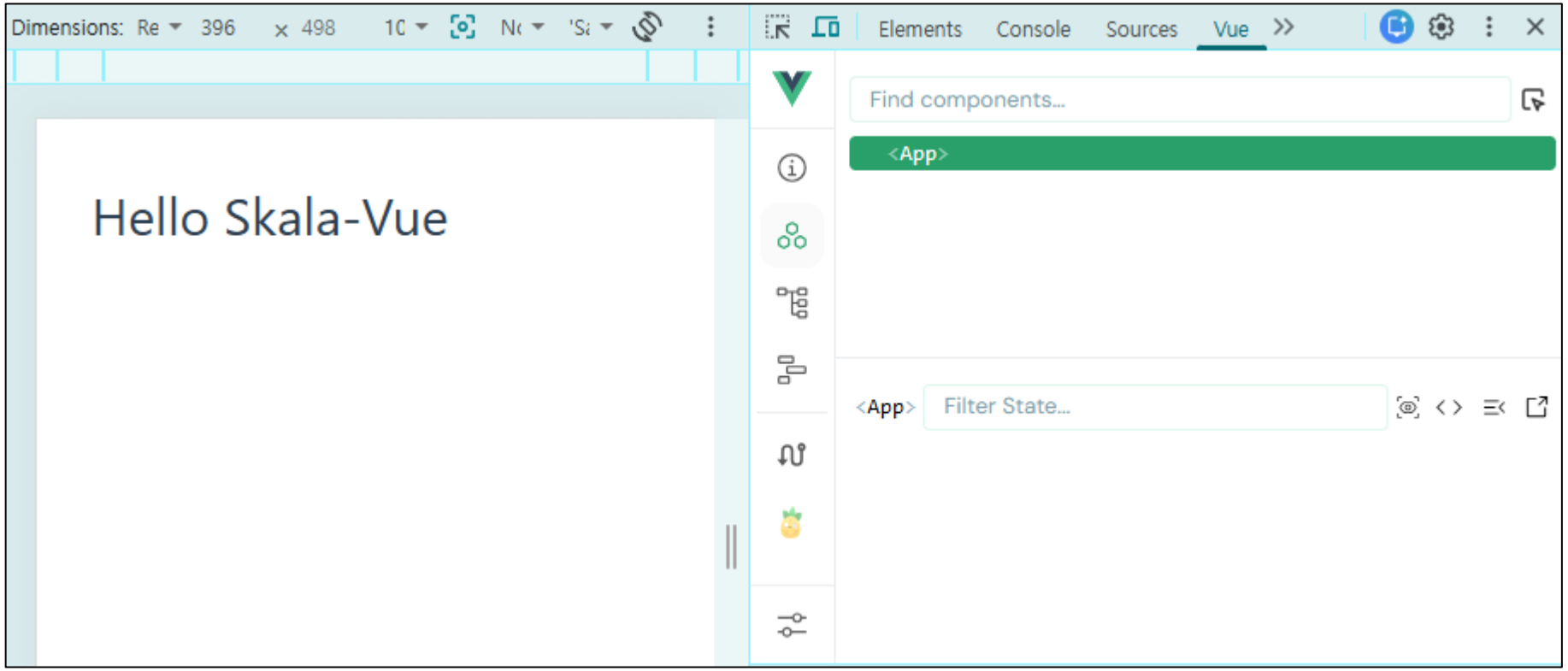
```
<template>
    <div>
        <h1>Composition API Counter</h1>
        <p>Count: {{ count }}</p>
        <button @click="increment">Increment</button>
    </div>
</template>
```

학습환경구성 - App.vue 비우기

- App.vue

```
<script setup>
// 자바스크립트 영역 (우선 비워둡니다)
</script>

<template>
  <h1>Hello Skala-Vue</h1>
</template>
```



학습환경구성 - App.vue 샘플 컴포넌트 채우기

- App.vue

```
<script setup>
import SampleOne from './components/practices/basic/SampleOne.vue'
</script>

<template>
  <div style="padding: 20px">
    <SampleOne />
  </div>
</template>
```

  - 작성된 vue 파일을 App.vue에서 자식 컴포넌트로 가져올 수 있도록 갈아 끼우면서 테스트 한다.
  - 교육과정에서 작성되는 component들은 특정 폴더에 잘 분류하여 넣는다.

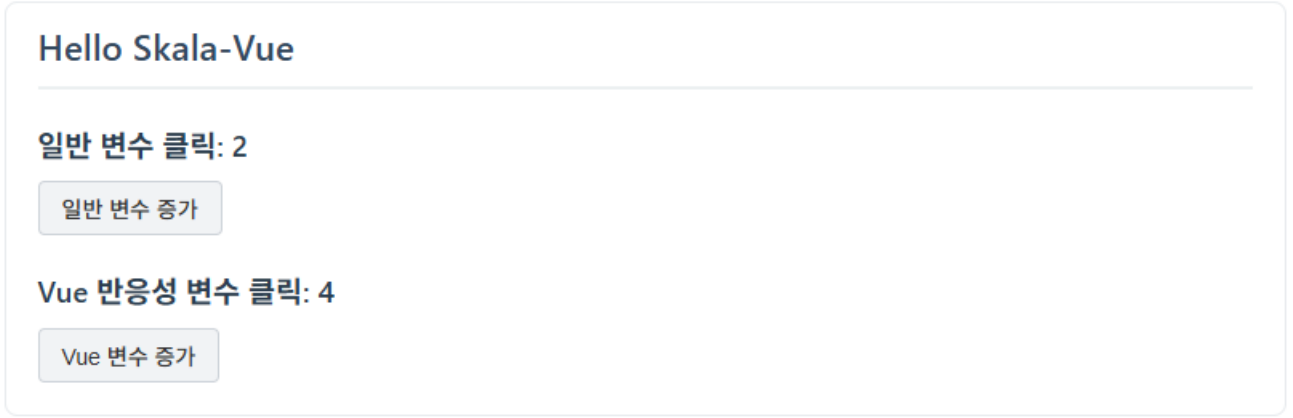
학습환경구성 - 반응성 데이터 (Reactivity) Example

- SampleOne.vue

```
<script setup>
import { ref } from 'vue'

// 1. 일반 변수 (화면이 실시간으로 바뀌지 않음)
let normalCount = 0
// 2. 반응성 변수 (화면이 실시간으로 바뀜)
const vueCount = ref(0)
</script>

<template>
  <div class="practice-section">
    <h2>Hello Skala-Vue</h2>
    <h3>일반 변수 클릭: {{ normalCount }}</h3>
    <button @click="normalCount++">일반 변수 증가</button>
    <br />
    <h3>Vue 반응성 변수 클릭: {{ vueCount }}</h3>
    <button @click="vueCount++">Vue 변수 증가</button>
  </div>
</template>
```



- 일반 변수는 버튼을 눌러도 화면의 숫자가 변경하지 않음. (하지만 변수의 값은 내부적으로 변함)
- ref로 감싼 반응형 변수는 버튼을 누르는 순간 숫자가 변경됨.
- 반응형 변수를 누르는 순간 화면을 새로 고침으로 일반 변수의 변경된 값도 같이 반영됨.
- 참고) JavaScript 소스의 끝 부분에 “;”를 빼먹어도 자바스크립트 내부 엔진에 ASI (Automatic Semicolon Insertion) 기능이 있어 잘 동작한다.

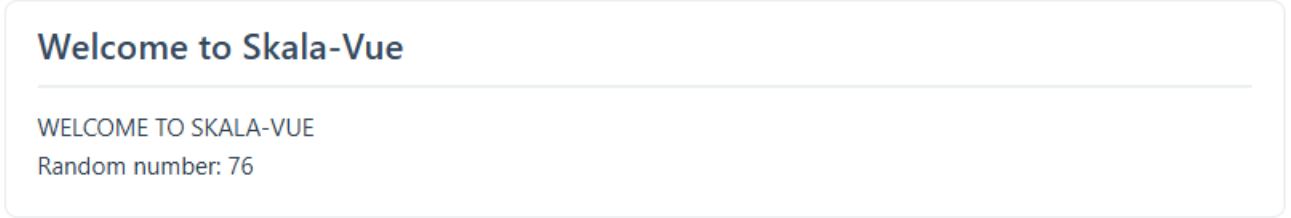
학습환경구성 - JavaScript in Text Interpolation Example

- SampleTwo.vue

```
<script setup>
import { ref } from 'vue'

const welcomeMessage = 'Welcome to Skala-Vue'
</script>

<template>
  <div class="practice-section">
    <h2>{{ welcomeMessage }}</h2>
    <p>{{ welcomeMessage.toUpperCase() }}</p>
    <p>{{ 'Random number: ' + Math.ceil(Math.random() * 100) }}</p>
  </div>
</template>
```



- Text Interpolation 에서 JavaScript Expressions (JavaScript 표현식)을 사용할 수도 있다.

Code Challenge

- 학습환경 구성
  - 반응성 데이터 (Reactivity) Example
  - JavaScript in Text Interpolation Example

Vue Directive

- Vue Directive
  - Vue Directive는 v- 접두사가 붙은 특수한 HTML 속성으로 Vue 인스턴스와 연동된다.
  - Directive 뒤에 오는 값인 v-명령어="값" 구역의 따옴표 내부(" ")는 단순 문자열이 아니라 **자바스크립트 변수나 연산식**이 작동하는 공간이다.
- Vue Directive

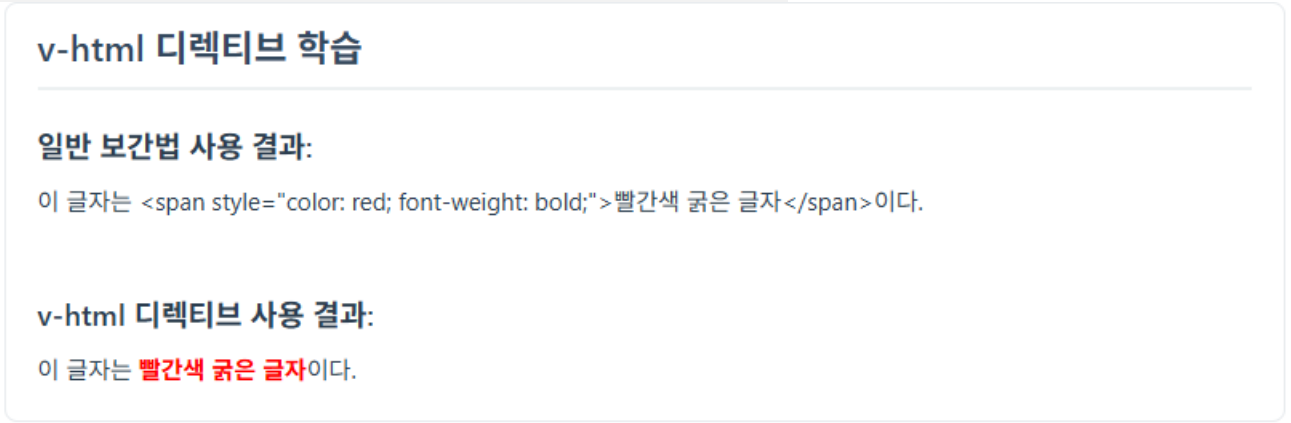
Directive	설명	비고
v-html	요소의 HTML 콘텐츠를 표현식의 값으로 업데이트	보안사고(XSS) 유의
v-text	텍스트 콘텐츠를 표현식의 값으로 업데이트.	Text Interpolation과 유사
v-bind	Elements의 Attributes에 표현식(변수, 함수, 객체 등)을 동적으로 바인딩	축약형 ➔ :
v-model	폼 입력<input>과 Vue 인스턴스 데이터 간에 양방향 데이터 바인딩	
v-if	표현식의 참/거짓에 따라 요소 또는 템플릿<template> 블록을 조건부로 렌더링	v-else-if, v-else
v-show	표현식의 참/거짓에 따라 HTML element를 보이거나 숨김.	v-if 와 구분
v-for	반복적으로 렌더링하는 HTML 요소를 생성	
v-on	클릭 또는 키 입력과 같은 사용자 이벤트에 응답하고자 지정 메서드를 실행	축약형 ➔ @
v-cloak	템플릿이 렌더링 되기 전까지 요소를 숨김.	자주 사용되지 않음
v-once	요소와 하위 콘텐츠를 한 번만 렌더링하고, 이후 변경하지 않음.	자주 사용되지 않음
v-pre	템플릿 구문을 무시하고 원본 HTML을 그대로 렌더링	자주 사용되지 않음

Vue Directive - v-html

- v-html
  - 자바스크립트 변수에 담긴 문자열을 단순 텍스트가 아니라, 실제 HTML Element로 해석하여 화면에 주입하는 Directive.
  - 내부적으로는 자바스크립트의 element.innerHTML 속성과 동일하게 동작

```
<script setup>
const rawHtmlData = '이 글자는 <span style="color: red; font-weight: bold;">빨간색 굵은 글자</span>이다.'
</script>

<template>
  <div class="practice-section">
    <h2>v-html 디렉티브 학습</h2>
    <h3>일반 보간법 {{}} 사용 결과:</h3>
    <p>{{ rawHtmlData }}</p>
    <br />
    <h3>v-html 디렉티브 사용 결과:</h3>
    <p v-html="rawHtmlData"></p>
  </div>
</template>
```



Vue Directive - v-html (XSS 위협)

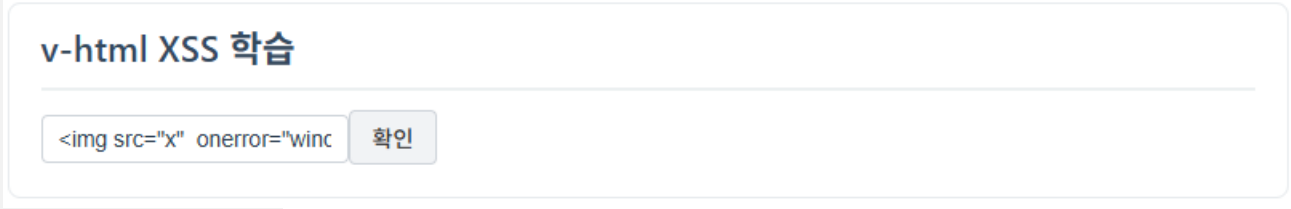
- XSS (Cross-Site Scripting)
  - 해커가 게시판 댓글이나 입력창에 악성 자바스크립트 코드를 심어두고, 다른 사용자가 그 글을 읽을 때 그 사용자의 브라우저에서 해커의 코드가 강제로 실행되게 만들어 쿠키, 세션 토큰, 로그인 정보를 탈취하는 해킹 기법
  - v-html은 XSS 공격에 노출된다.

```vue
<script setup>
import { ref } from 'vue'

const inputValue = ref('')
const message = ref('')

function showMessage() {
  message.value = inputValue.value
}
</script>

<template>
  <div class="practice-section">
    <h2>v-html XSS 학습</h2>
    <input v-model="inputValue" placeholder="내용을 입력하세요" />
    <button @click="showMessage">확인</button>
    <div v-html="message"></div>
  </div>
</template>
```



- 을 입력 후 클릭하면 다른 사이트로 이동한다.

Vue Directive - v-text

- v-text
  - 지정한 변수의 값을 태그의 텍스트 내용으로 채워 넣는다.
  - 내부적으로는 자바스크립트의 element.innerText 속성과 똑같이 동작한다.
  - Text Interpolation {{ }} 과 동일함으로 실무에서는 v-text 대신 Text Interpolation을 사용한다.

```
<script setup>
const content = '안녕하세요! <strong>Skala-Vue</strong> 강의입니다.'
</script>

<template>
  <div class="practice-section">
    <h2>v-text 디렉티브 학습</h2>
    <h3>1) 일반 보간법 {{}} 결과:</h3>
    <p>출력: {{ content }}</p>
    <br />

    <h3>2) v-text 디렉티브 결과:</h3>
    <p v-text="'출력: ' + content"></p>
    <br />

    <h3>3) v-html 결과 비교:</h3>
    <p v-html="content"></p>
  </div>
</template>
```



Vue Directive - v-bind

- v-bind : <div v-bind:[attribute]="[Vue data]">
  - HTML 태그 내부의 Attribute에 자바스크립트 값을 동적으로 연결(Binding)하는 지시자
  - 실무에서는 v-bind라는 긴 글자를 다 쓰지 않고, 앞에 콜론(:) 기호 하나만 남기는 축약형을 100% 사용

```
<script setup>
import { ref } from 'vue'

const dynamicUrl = 'https://www.naver.com'
const logoImgSrc = 'https://vuejs.org/images/logo.png'
const isButtonDisabled = ref(true)
</script>

<template>
  <div class="practice-section">
    <h2>v-bind 디렉티브 기본 (축약형: 콜론)</h2>
    <h3>1) 동적 링크 연결</h3>
    <a :href="dynamicUrl">여기를 클릭하면 네이버로 이동합니다</a>
    <br />

    <h3>2) 동적 이미지 연결</h3>
    <img :src="logoImgSrc" alt="Vue 로고" style="width: 100px" />
    <br />

    <h3>3) 버튼 비활성화 제어</h3>
    <p>현재 버튼 사용 불가능 상태: {{ isButtonDisabled }}</p>
    <button :disabled="isButtonDisabled">동의해야 클릭할 수 있는 버튼</button> 
    <button @click="isButtonDisabled = !isButtonDisabled">위 버튼 잠금 해제/토글하기</button>
  </div>
</template>
```



Vue Directive - v-bind (Class Binding)

- v-bind : class binding
  - 스타일시트(class)를 동적으로 붙였다 뗐다 하는 클래스 바인딩은 단순 문자열 주입이 아니라, "객체(Object) 형식"과 "배열(Array) 형식"을 지원하여 강력한 조건부 디자인을 가능하게 한다.
  - 객체(Object) 구문 패턴 기본 구조: :class="{ '클래스명': 조건불리언변수 }"
    조건 불리언 변수가 true 일 때만 해당 클래스를 태그에 활성화 한다.
  - 배열(Array) 구문 패턴 기본 구조: :class="[기본클래스변수, 조건 ? '클래스A' : '클래스B']"
    조건과 상관없어 여러 개의 클래스를 동시에 엮어서 주입하거나, 삼항 연산자를 이용해 상황별로 클래스를 주입할 수 있다.

형식	예시	설명
문자열	:class="active"	클래스 이름 문자열 하나
객체	:class="{'active-style': isActive}, :class="{active: isActive, primary: isPrimary}"	조건에 따라 클래스 포함 여부 지정
배열	:class="[active, primary]", :class="[baseClass, isError ? 'text-red' : 'text-blue']"	여러 클래스를 조건에 따라 조합
조합	class="기본스타일" :class="추가스타일"	정적 클래스와 동적 클래스 동시 적용

Vue Directive - v-bind (Class Binding Example)

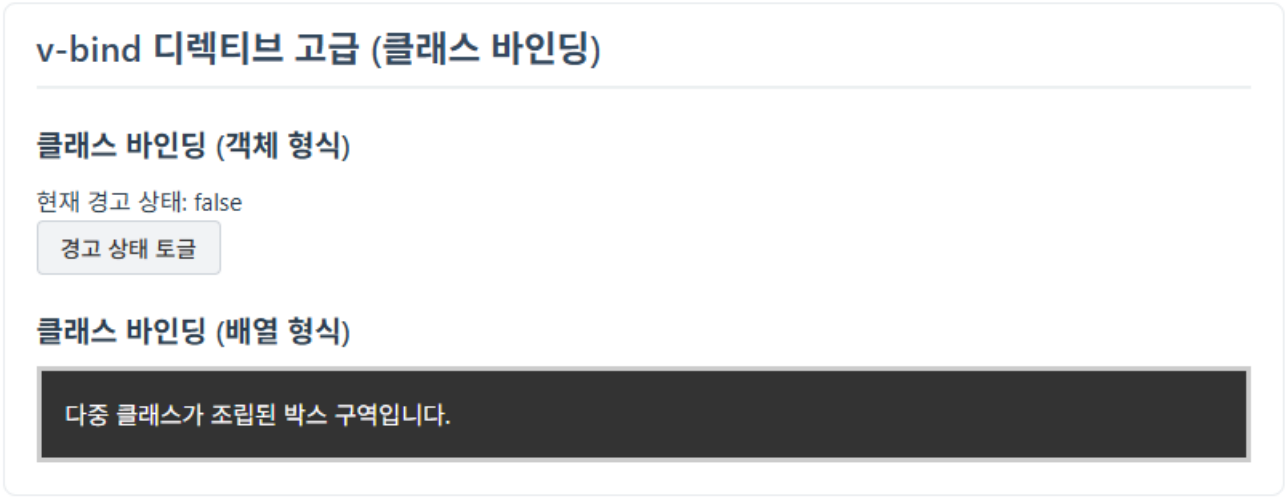
- Class Binding Example

```
<script setup>
import { ref } from 'vue'

const isWarning = ref(false) // 객체 바인딩용 스위치
const themeClass = ref('bg-dark') // 배열 바인딩용 고정 클래스
</script>

<template>
  <div class="practice-section">
    <h2>v-bind 디렉티브 고급 (클래스 바인딩)</h2>
    <h3>클래스 바인딩 (객체 형식)</h3>
    <p :class="{ 'text-danger': isWarning }">현재 경고 상태: {{ isWarning }}</p>
    <button @click="isWarning = !isWarning">경고 상태 토글</button>
    <br />
    <h3>클래스 바인딩 (배열 형식)</h3>
    <div :class="[themeClass, isWarning ? 'border-red' : 'border-gray']">다중 클래스가 조립된 박스 구역입니다.</div>
  </div>
</template>

<style scoped>
.text-danger {color: red; font-weight: bold;}
.bg-dark {background-color: #333; color: white; padding: 15px;}
.border-red {border: 3px solid red;}
.border-gray {border: 3px solid #ccc;}
</style>
```



Vue Directive - v-bind (Style Binding)

- v-bind : style binding
  - 클래스 바인딩처럼, Inline Style (style="...") 역시 JavaScript 객체(Object)와 배열(Array)을 지원하여 CSS 속성을 제어할 수 있다.
  - 객체 안에서 CSS 속성명을 적을 때, 속성 key는 camelCase로 사용하며, 'kebab-case' 문자열도 사용 가능
  - 객체(Object) 구문 패턴 기본 구조: :style="{ color: 변수명, fontSize: 변수명 + 'px' }"

```
<p :style="{ color: activeColor, fontSize: fontSize + 'px' }">텍스트</p>
```

  - 배열(Array) 구문 패턴: 여러 개의 스타일 객체 변수들을 하나로 합쳐서 태그에 주입

```
const baseStyle = ref({ color: 'blue', fontSize: '16px' })
const borderStyle = ref({ border: '1px solid black' })
```

```
<div :style="[baseStyle, borderStyle]">상자 구역</div>
```

Vue Directive - v-bind (Style Binding Example)

- Style Binding Example

```
<script setup>
import { ref } from 'vue'

// 1. 객체 바인딩용 변수
const textColor = ref('purple')
const boxWidth = ref(150) // 숫자만 제어
// 2. 배열 바인딩용 스타일 객체 무더기
const baseBoxStyle = ref({
  backgroundColor: '#42b883',
  height: '100px',
  transition: 'all 0.3s ease', // 부드러운 애니메이션 효과
})
</script>

<template>
  <div class="practice-section">
    <h2>v-bind 디렉티브 고급 (스타일 바인딩)</h2>
    <h3>1) 인라인 스타일 변수 조작 (객체 형식)</h3>
    <p :style="{ color: textColor, fontWeight: 'bold' }">이 글자의 색상은 실시간으로 바뀝니다.</p>
    <button @click="textColor = textColor === 'purple' ? 'blue' : 'purple'">글자 색상 토글</button>
    <br />

    <h3>2) 다중 스타일 객체 조립 (배열 형식)</h3>
    <label>박스 가로 크기(px): </label>
    <input type="number" v-model="boxWidth" step="50" />
    <br />
    <div :style="[baseBoxStyle, { width: boxWidth + 'px' }]">
      <p style="color: white; padding: 10px; text-align: center">가로 크기: {{ boxWidth }}px 박스</p>
    </div>
  </div>
</template>
```



Vue Directive - v-bind (Class Binding vs. Style Binding)

- v-bind : Class Binding vs. Style Binding

비교 항목	클래스 바인딩 (:class)	스타일 바인딩 (:style)
기술적 실체	HTML의 class 속성에 동적으로 문자열 주입	HTML의 인라인 style 속성에 동적으로 스타일 주입
주요 활용 목적	이미 정의된 디자인 옷을 갈아 입힐 때 (예: 활성화/비활성화, 다크모드 켜기, 경고 상태 등)	수치나 색상을 실시간으로 미세 가공할 때 (예: 슬라이더 수치 반영, 프로그레스바 게이지 등)
속성명 작성 규칙	CSS 클래스명을 문자열 그대로 작성 { 'text-danger': isWarning }	카멜 케이스(CamelCase) 표기법 권장 { backgroundColor: '#fff', fontSize: '14px' }
객체(Object) 구문 해석 방식	{ '클래스명': 조건불리언 } → 조건이 true일 때만 해당 클래스 추가.	{ CSS속성명: 자바스크립트변수 } → 속성값에 변수의 데이터가 실시간 매핑.
배열(Array) 구문 해석 방식	[고정클래스, 조건부삼항연산클래스] → 여러 개의 클래스 이름을 나열하여 동시 적용.	[스타일객체A, 스타일객체B] → 분리되어 있던 여러 스타일 변수를 하나로 병합.
실무 권장도	★★★ 압도적 적극 권장 (90%) (구조와 스타일을 완벽히 분리하여 유지보수에 유리)	★★ 특수 상황에서만 제한적 사용 (10%) (인라인 스타일 남용은 코드 복잡도를 높임)

Vue Directive - v-bind (Same-name Shorthand)

- v-bind : same-name shorthand
  - Vue 3.4 버전부터 공식 도입된 문법으로, "연결할 자바스크립트 변수명"과 "HTML 속성명"이 완전히 일치할 때 코드를 극단적으로 줄여주는 방법.

```
<img v-bind:src="src" />
<img :src="src" />
<img :src />
```

  - 동작 원리: 속성 앞에 콜론(:)만 붙이고 뒤의 ="src"를 생략하면, Vue 엔진이 "이 태그의 src 속성에 이름이 똑같은 src라는 자바스크립트 변수를 자동으로 매핑하라는 뜻이구나!" 하고 알아서 해석한다
  - 실무 활용: 변수명을 id, src, href, disabled 등 HTML 표준 속성명과 똑같치 맞춰 선언해 두면 코딩 속도가 빨라진다.
- Example

```
<script setup>
const id = 'user-profile-card'
const src = 'https://vuejs.org/images/logo.png'
</script>

<template>
  <div class="practice-section">
    <h2>v-bind 디렉티브 고급 (단축 문법)</h2>
    <div :id>
      <img :src alt="Vue 로고" style="width: 50px" />
    </div>
  </div>
</template>
```



Vue Directive - v-if/v-else-if/v-else

- v-if/v-else-if/v-else
  - JavaScript 조건식의 결과(true/false)에 따라 HTML 태그를 화면에 그릴지, 아니면 지울지 결정하는 제어문 역할

```
<script setup>
import { ref } from 'vue'

// 1. 조건부 온/오프 스위치 변수
const isLogged = ref(false)
// 2. 다중 조건 분기용 숫자 변수
const score = ref(85)
</script>

<template>
  <div class="practice-section">
    <h2>v-if, v-else-if, v-else 디렉티브 학습</h2>
    <h3>1) 기본 로그인 상태 스위치</h3>
    <p v-if="isLogged">환영합니다! 회원 전용 화면입니다.</p>
    <p v-else>로그인이 필요합니다. 먼저 로그인해 주세요.</p>
    <button @click="isLogged = !isLogged">
      {{ isLogged ? '로그아웃 하기' : '로그인 하기' }}
    </button>
    <br />
    <h3>2) 성적별 학점 등급 측정 (다중 조건문)</h3>
    <label>현재 점수 입력: </label>
    <input type="number" v-model="score" min="0" max="100" step="5" />
    <br />
    <div v-if="score >= 90" style="color: green; font-weight: bold">합격 등급: A 학점 (훌륭합니다!)</div>
    <div v-else-if="score >= 80" style="color: blue">합격 등급: B 학점 (양호합니다.)</div>
    <div v-else-if="score >= 70" style="color: orange">합격 등급: C 학점 (조금 더 분발하세요.)</div>
    <div v-else style="color: red; font-weight: bold">합격 등급: F 학점 (재시험 대상입니다.)</div>
  </div>
</template>
```



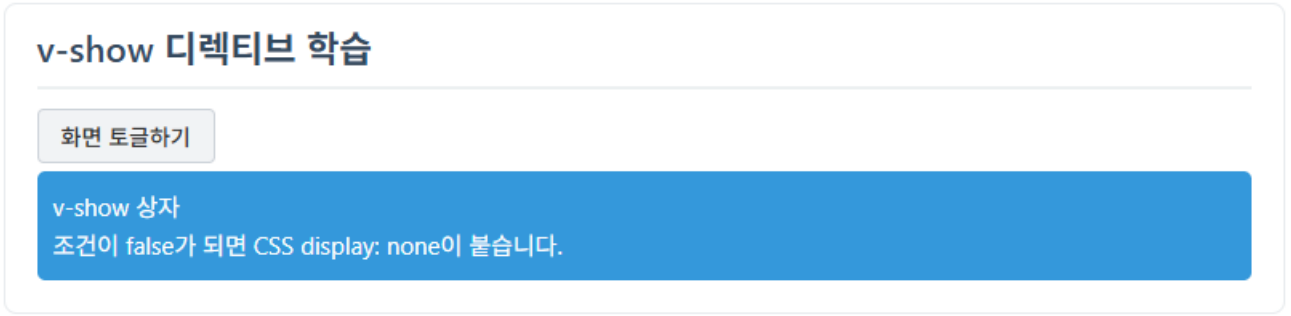
Vue Directive - v-show

- v-show
  - 조건식의 결과(true/false)에 따라 태그를 화면에 '보여줄지(Show)' 아니면 '숨길지(Hide)' 결정하는 디렉티브
  - 조건이 false가 되더라도 HTML DOM에서 태그를 삭제하지 않고, CSS 속성인 display: none;을 실시간으로 붙여서 숨긴다.

```vue
<script setup>
import { ref } from 'vue'
const isVisible = ref(true)
</script>

<template>
  <div class="practice-section">
    <h2>v-show 디렉티브 학습</h2>
    <button @click="isVisible = !isVisible">화면 토글하기</button>
    <br />
    <div v-show="isVisible" class="box">
      <p>v-show 상자</p>
      <p>조건이 false가 되면 CSS display: none이 붙습니다.</p>
    </div>
  </div>
</template>

<style scoped>
.box {
  padding: 10px;
  margin-top: 5px;
  color: white;
  border-radius: 5px;
  background-color: #3498db; /* 파란색 */
}
</style>
```



Vue Directive - v-if vs. v-show

- v-if vs. v-show

비교 항목	v-if (조건부 렌더링)	v-show (조건부 가시성)
렌더링 방식	**실제 DOM 파괴 및 생성** 조건이 맞지 않으면 태그 자체가 존재하지 않음.	**CSS display 속성 조작** 태그는 항상 존재하며 눈에만 숨김.
초기 렌더링 비용	**낮음** 처음에 false이면 아예 그리지 않으므로 빠름.	**높음** true/false 상관없이 일단 다 그려 놓으므로 느림.
화면 전환(Toggle) 비용	**높음** 바뀔 때마다 태그를 새로 부수고 지어야 해서 부담.	**낮음** 단순 CSS 속성 한 줄만 끄고 켜는 것이라 매우 가볍고 빠름.
v-else 조합 가능 여부	⭕ **가능** (v-else-if, v-else 연동 가능)	❌ **불가능** (오직 단독 조건으로만 사용)
<template> 태그 사용	⭕ **가능** (여러 태그를 한 번에 묶어서 제어 가능)	❌ **불가능** (실제 눈에 보이는 HTML 태그에만 작동)
실무 권장 선택 기준	화면 전환이 **드물게 일어나는** 경우 (예: 로그인 후 회원 화면, 권한별 메뉴 전환 등)	화면 전환이 **매우 빈번하게 일어나는** 경우 (예: 모달 팝업, 탭 메뉴, 아코디언 접고 펴기 등)

Vue Directive - v-for

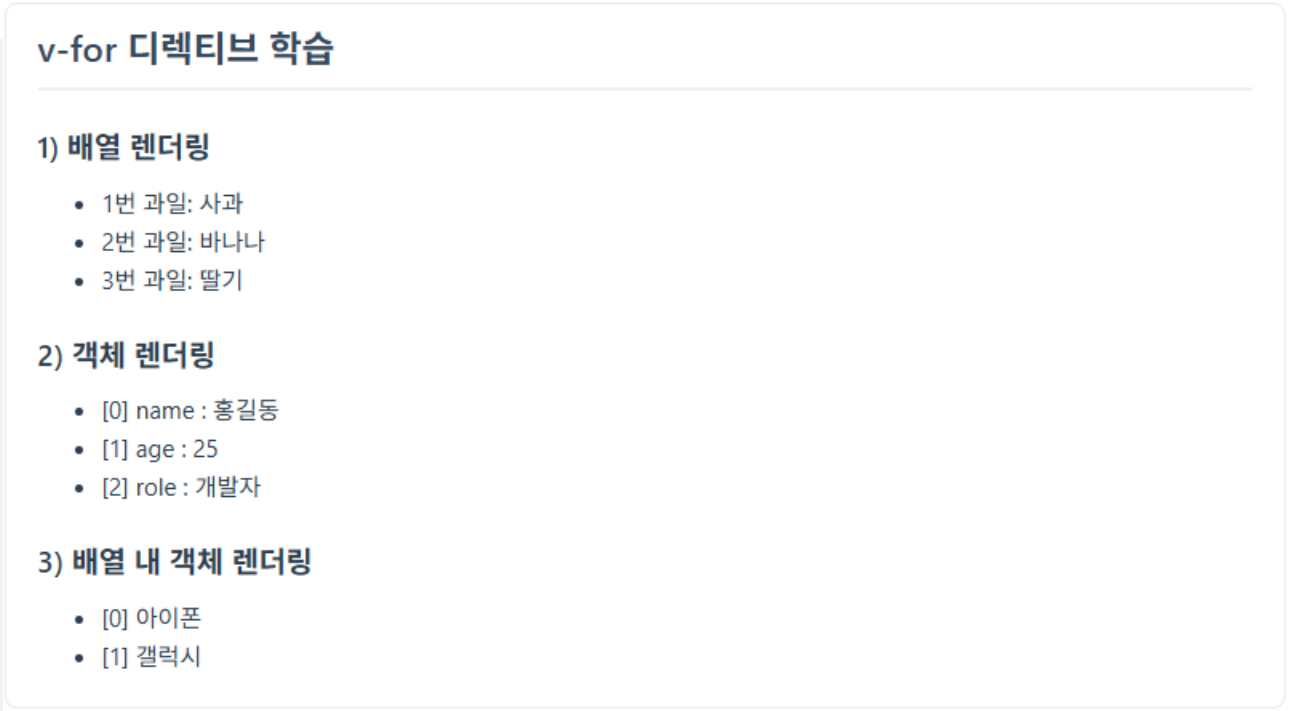
- v-for
  - 배열이나 객체를 사용해서 뷰에서 반복적으로 렌더링 하는 HTML Element를 생성하는 데 사용.
  - v-for를 쓸 때는 Vue 엔진이 각 태그를 고유하게 식별할 수 있도록 반드시 고유한 값을 :key 속성을 바인딩해야 한다. 그렇지 않으면 에러 또는 성능 저하가 발생한다.
- 배열 렌더링:
  - <div v-for="(item, index) in items" :key="고유값">
  - <div v-for="item in items" :key="고유값">
- 객체 렌더링:
  - <div v-for="(value, key, index) in object" :key="고유값">
  - <div v-for="(value, key) in object" :key="고유값">
  - <div v-for="value in object" :key="고유값">

Vue Directive - v-for (Example)

```vue
<script setup>
import { ref } from 'vue'

const fruits = ref(['사과', '바나나', '딸기'])
const user = ref({
  name: '홍길동',
  age: 25,
  role: '개발자',
})
const items = ref([
  { id: 'prod_101', name: '아이폰' },
  { id: 'prod_102', name: '갤럭시' },
])
</script>

<template>
  <div class="practice-section">
    <h2>v-for 디렉티브 학습</h2>
    <h3>1) 배열 렌더링</h3>
    <ul>
      <li v-for="(fruit, index) in fruits" :key="index">{{ index + 1 }}번 과일: {{ fruit }}</li>
    </ul>
    <h3>2) 객체 렌더링</h3>
    <ul>
      <li v-for="(value, key, index) in user" :key="key">[{{ index }}] {{ key }} : {{ value }}</li>
    </ul>
    <h3>3) 배열 내 객체 렌더링</h3>
    <ul>
      <li v-for="(item, index) in items" :key="item.id">[{{ index }}] {{ item.name }}</li>
    </ul>
  </div>
</template>
```



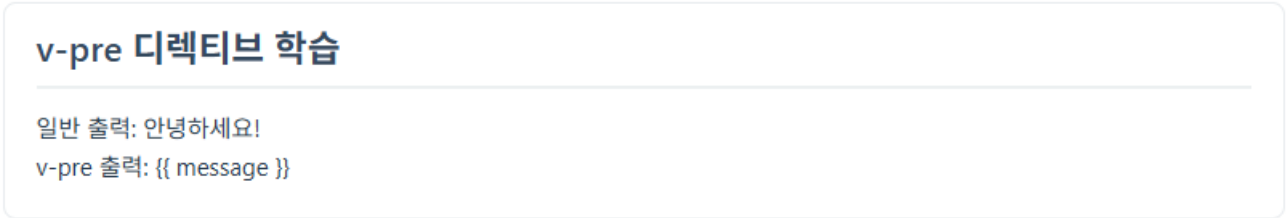
Vue Directive - v-pre

- v-pre
  - Vue의 템플릿 컴파일러가 Vue 문법으로 해석(Compile)하지 말고, 써진 그대로 HTML 텍스트로 화면에 표시하라고 지시
  - Vue 엔진은 원래 HTML을 읽다가 {{ }}나 v-디렉티브를 만나면 자바스크립트 데이터로 갈아 끼우는 연산을 한다. 하지만 v-pre가 붙은 태그와 그 자식 태그들은 아무런 연산 없이 그대로 출력한다.

```
<script setup>
import { ref } from 'vue'

const message = ref('안녕하세요!')
</script>

<template>
  <div class="practice-section">
    <h2>v-pre 디렉티브 학습</h2>
    <p>일반 출력: {{ message }}</p>
    <p v-pre>v-pre 출력: {{ message }}</p>
  </div>
</template>
```



Vue Directive - v-cloak

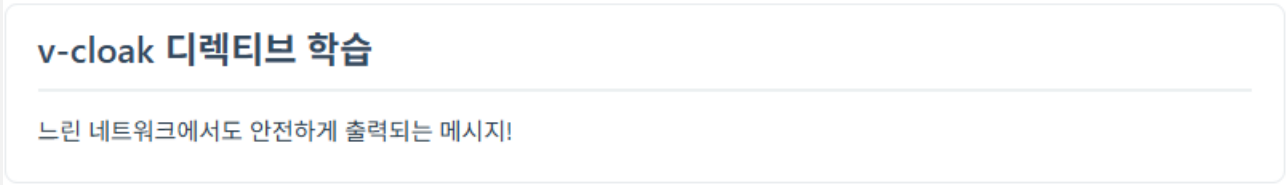
- v-cloak
  - Vue 어플리케이션의 렌더링 과정에서 데이터 바인딩이 완료되기 전에 Template을 노출하면 {{ message }} 같은 해석 안 된 뼈대 문자열이 그대로 노출되는 현상이 발생한다.
  - 네트워크가 아주 느린 환경에서 발생하는 현상으로, v-cloak은 이런 현상을 예방한다.
  - 이 디렉티브는 혼자서는 작동하지 않고, CSS의 속성 선택자([v-cloak])가 필요하다.

```
<script setup>
import { ref } from 'vue'

const message = ref('느린 네트워크에서도 안전하게 출력되는 메시지!')
</script>

<template>
  <div v-cloak class="practice-section">
    <h2>v-cloak 디렉티브 학습</h2>
    <p>{{ message }}</p>
  </div>
</template>

<style scoped>
/* ⚠ 필수: Vue가 로딩되기 전까지 해당 구역을 물리적으로 숨기는 CSS 규칙 */
[v-cloak] {
  display: none !important;
}
</style>
```



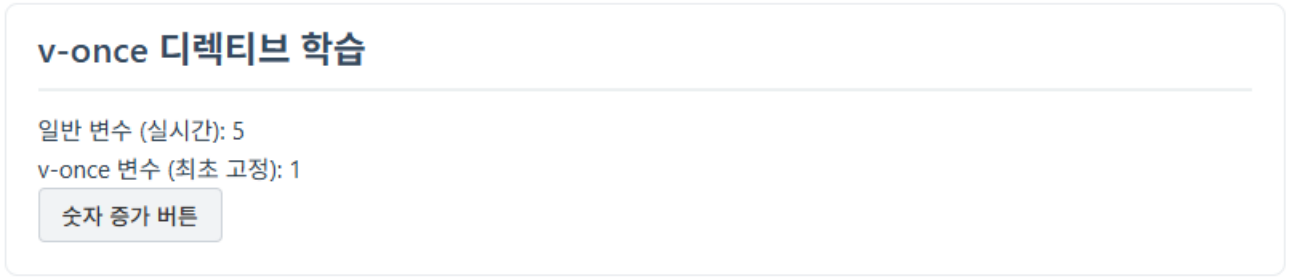
Vue Directive - v-once

- v-once
  - 해당 요소와 그 하위 요소는 최초에 한 번만 반응형으로 렌더링하고, 그 이후부터는 데이터가 변경되어도 DOM은 갱신되지 않음
  - Vue 엔진이 데이터를 실시간으로 감시하려면 메모리를 계속 소모해야 하는데, 소개글, 약관 내용처럼 처음 백엔드에서 한 번 받아온 이후로는 절대 바뀔 일이 없는 데이터에 v-once를 붙여 두면, Vue가 더 이상 감시하지 않아 메모리 부담이 준다.

```
<script setup>
import { ref } from 'vue'

const count = ref(1)
</script>

<template>
  <div class="practice-section">
    <h2>v-once 디렉티브 학습</h2>
    <p>일반 변수 (실시간): {{ count }}</p>
    <p v-once>v-once 변수 (최초 고정): {{ count }}</p>
    <button @click="count++">숫자 증가 버튼</button>
  </div>
</template>
```



Vue Directive - v-memo

- v-memo
  - 지정한 조건(변수)이 바뀔 때만 태그 내부를 업데이트하고, 그렇지 않으면 이전에 그려둔 화면(캐시)을 그대로 재사용해라
  - v-memo="[감시할변수1, 감시할변수2]"

```
<script setup>
import { ref } from 'vue'

const name = ref('홍길동')
const age = ref(20)
</script>

<template>
  <div class="practice-section">
    <h2>v-memo 디렉티브 학습</h2>
    <div v-memo="[name]" style="padding: 20px; border: 1px solid #42b883; margin-bottom: 10px">
      <p>📦 v-memo 적용 영역 (기준: name)</p>
      <p>이름: {{ name }}</p>
      <p>나이: {{ age }} (name이 바뀌어야 얘도 갱신됨)</p>
    </div>
    <button @click="name = '이순신'">1. 이름 변경 (이순신)</button>  
    <button @click="age++">2. 나이 한 살 추가 (age++)</button>
  </div>
</template>
```



Code Challenge

- Vue Directive
  - v-html / v-html XSS / v-text
  - v-bind (Basic, Class Binding, Style Binding, Shorthand)
  - v-if / v-else-if / v-else / v-show
  - v-for
  - v-pre / v-cloak / v-once / v-memo

Vue Event Handling - v-on

- v-on (@)
  - DOM 요소에 이벤트 리스너를 연결하여 이벤트를 감지하고 처리할 때 사용한다.
  - 주로 사용자 입력(클릭, 키보드 입력 등)에 반응하여 원하는 동작을 실행하는 데 사용한다.

```
<!-- 축약형 없이 사용 -->
<button v-on:click="doSomething">클릭</button>

<!-- 축약형 (@) 사용 -->
<button @click="doSomething">클릭</button>
```

- 주요 Events

주요 이벤트 이름	설명
click	클릭 이벤트
submit	폼 제출 이벤트
keyup	키보드 키를 뗐을 때
keydown	키보드를 눌렀을 때
input	입력 필드 변경 시
change	입력 값 변경 후 포커스 아웃 시
mouseenter	마우스가 요소 위로 올라올 때
mouseleave	마우스가 요소에서 벗어날 때

Vue Event Handling - v-on (Event Handler)

- Inline Handler
  - 태그 안에서 즉시 간단한 자바스크립트 연산을 처리할 때 사용 (숫자 증감, 스위치 토글 등)

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
</script>

<template>
  <button @click="count++">클릭 수: {{ count }}</button>
</template>
```

- Method Handler
  - 태그 안방이 지저분해지지 않도록, 복잡한 로직은 <script setup> 구역에 자바스크립트 함수(함수명)를 만들어서 연결

```
<script setup>
function handleClick() {
  alert("버튼이 클릭되었습니다!")
}
</script>

<template>
  <button @click="handleClick">클릭하세요</button>
</template>
```

[참고사항] 함수 참조(Function Reference) 전달 방식

- Vue 템플릿에서 @click="handleClick"와 같이 괄호 없이 함수 이름만 넘기면, Vue는 이 함수를 호출하는 것이 아니라 함수의 참조(주소) 자체를 이벤트 리스너로 등록한다.
- Vue의 @click="showMessage"와 동일한 동작 button.addEventListener('click', handleClick)

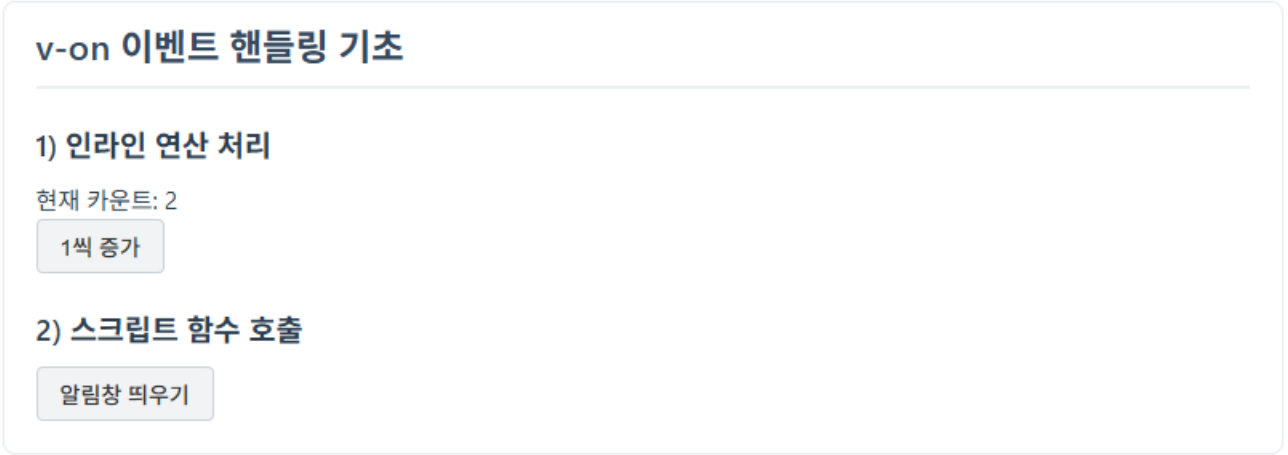
Vue Event Handling - v-on Event Handler Example

- v-on Event Handler Example

```vue
<script setup>
import { ref } from 'vue'
const count = ref(0)

// 메서드 핸들러 함수 정의
const showAlert = () => {
  alert('함수가 성공적으로 호출되었습니다!')
}
</script>

<template>
  <div class="practice-section">
    <h2>v-on 이벤트 핸들링 기초</h2>
    <h3>1) 인라인 연산 처리</h3>
    <p>현재 카운트: {{ count }}</p>
    <button @click="count++">1씩 증가</button>
    <br />
    <h3>2) 스크립트 함수 호출</h3>
    <button @click="showAlert">알림창 띄우기</button>
  </div>
</template>
```



Vue Event Handling - JavaScript Event Object

- Event Object란 사용자가 웹페이지에서 버튼을 클릭하거나, 키보드를 누르거나, 마우스를 움직이는 등의 '이벤트'를 발생시켰을 때 브라우저가 자동으로 생성하는 객체이다.
- Event Object 주요 Properties

분류	속성명 (Property)	데이터 타입	설명 및 실무 활용처
공통	**e.target**	HTMLElement	**이벤트를 발생시킨 HTML 태그.** e.target.value (입력값), e.target.tageName (BUTTON, INPUT..)
	e.currentTarget	HTMLElement	**이벤트 리스너가 걸려있는 HTML 태그** (부모 태그에 이벤트를 걸었을 때 e.target과 달라짐)
	e.type	String	**발생한 이벤트의 종류** (예: click, keyup, submit 등)
	e.timeStamp	Number	웹페이지가 로드된 후 이벤트를 실행하기까지 걸린 시간(ms)
마우스	**e.clientX / e.clientY**	Number	**브라우저 화면(Viewport) 기준** 마우스 커서의 X, Y 좌표
	e.pageX / e.pageY	Number	**전체 HTML 문서(Document) 기준** 마우스 커서의 X, Y 좌표
	e.screenX / e.screenY	Number	사용자의 **모니터 모니터 화면 기준** 마우스 커서의 X, Y 좌표
	e.button	Number	클릭한 마우스 버튼 번호 (0: 왼쪽, 1: 휠/가운데, 2: 오른쪽)
키보드	**e.key**	String	**사용자가 누른 키의 실제 문자 값** (예: Enter, ArrowUp, a, A, Escape)
	e.code	String	사용자가 누른 **물리적인 키보드 자판의 위치 값** (예: Enter, KeyA, Digit1, Escape)
	e.shiftKey	Boolean	이벤트를 유발할 때 **Shift 키를 같이 누르고 있었는지** 여부 (true/false)
	e.ctrlKey / e.altKey	Boolean	이벤트를 유발할 때 **Ctrl 또는 Alt 키를 같이 누르고 있었는지** 여부

Vue Event Handling - JavaScript Event Object

- Event Object 주요 Methods

Method	주요 역할 및 기능	실무 핵심 활용처 (예시)
e.preventDefault()	브라우저가 특정 태그에 대해 가지는 기본 고유 동작을 강제로 중단시킨다.	* <a> 태그를 눌렀을 때 페이지가 링크로 이동하는 것을 방지 * <form>의 submit 버튼을 눌렀을 때 페이지가 새로 고침되는 것을 방지
e.stopPropagation()	이벤트가 부모 태그로 타고 올라가는 이벤트 버블링(Bubbling)을 완전히 차단한다.	* 팝업창(자식) 내부를 클릭했을 때, 팝업창 뒤편의 배경(부모) 닫기 이벤트까지 동시에 실행되는 버블 버그 방지
e.stopImmediatePropagation()	현재 태그에 걸려있는 다른 이벤트 리스너들의 실행까지 전부 중단시키고, 버블링도 막는다.	* 하나의 버튼에 여러 개의 클릭 함수가 중복으로 엮여 있을 때, 첫 번째 함수만 실행하고 뒤는 싹 다 마비시키고 싶을 때

Vue Event Handling - Event Object 사용

- Vue에서 Event 객체($event)를 받는 Pattern 2가지
  - Method Handler에 함수 이름만 적어서 호출하면, JavaScript Engine이 첫 번째 인자로 이벤트 객체를 묵시적으로 전달한다.

    Syntax: @click="handleEvent" → 스크립트: const handleEvent = (e) => { ... }
  - 함수에 특정 데이터를 던지면서 이벤트 객체도 동시에 넘기고 싶을 때는, Vue가 제공하는 특별한 기호인 $event를 명시적으로 적어주어야 한다.

    Syntax: @click="handleEvent('홍길동', $event)"

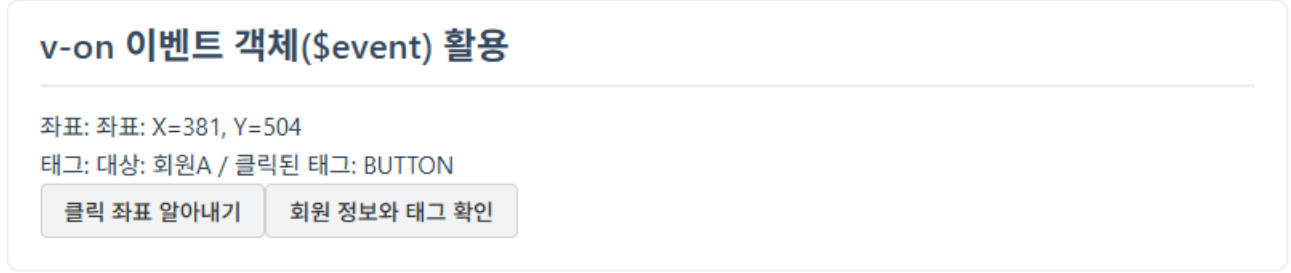
Vue Event Handling - Event Object 사용 Example

- Event Object Example

```
<script setup>
import { ref } from 'vue'

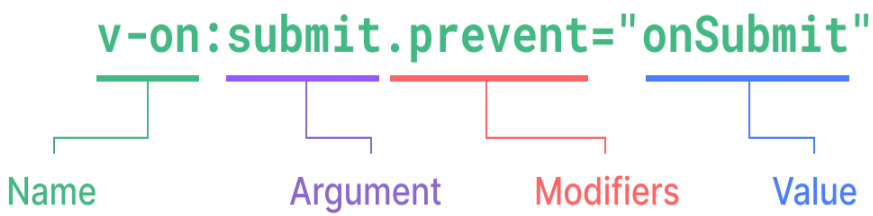
const position = ref('')
const tagName = ref('')
const getOnlyEvent = (e) => {
  position.value = `좌표: X=${e.clientX}, Y=${e.clientY}`
}
const getWithParam = (name, e) => {
  tagName.value = `대상: ${name} / 클릭된 태그: ${e.target.tagName}`
}
</script>

<template>
  <div class="practice-section">
    <h2>v-on 이벤트 객체($event) 활용</h2>
    <p>좌표: {{ position }}</p>
    <p>태그: {{ tagName }}</p>
    <button @click="getOnlyEvent">클릭 좌표 알아내기</button>
    <button @click="getWithParam('회원A', $event)">회원 정보와 태그 확인</button>
  </div>
</template>
```



Vue Event Handling - Event Modifier (이벤트 수식어)

- Event Modifier
  - 이벤트 리스너의 기본 동작을 보완하거나 제어하는 데 사용되는 특수 접미어



- 주요 Event Modifier

수식어	실제 매핑 되는 자바스크립트 스펙	주요 기능 및 활용처
.prevent	e.preventDefault()	**태그의 기본 동작 방지** (예: 폼 제출 시 새로고침 방지, 링크 이동 방지)
.stop	e.stopPropagation()	**이벤트 버블링 차단** (예: 자식 버튼 클릭 시 부모 박스 이벤트로 전염되는 것 방지)
.once	최초 1회 트리거 후 리스너 제거	**이벤트를 딱 한 번만 실행** (예: 설문조사 제출 버튼 중복 클릭 방지)
.self	e.target === e.currentTarget	**오직 자기 자신을 직접 클릭했을 때만** 이벤트 실행 (자식 태그 클릭 시 패스)

Vue Event Handling - Event Modifier Example

- Event Modifier Example

```vue
<script setup>
const handleLink = () => {
  alert('수식어 덕분에 네이버로 이동하지 않고 함수만 실행됩니다!')
}

const handleBox = () => {
  alert('부모 박스가 클릭되었습니다!')
}
</script>

<template>
  <div class="practice-section">
    <h2>이벤트 수식어(Modifiers) 학습</h2>
    <h3>1) .prevent (기본 동작 막기)</h3>
    <a href="https://www.naver.com" @click.prevent="handleLink">네이버 링크</a>
    <br />

    <h3>2) .stop (이벤트 버블링 막기)</h3>
    <div @click="handleBox" style="padding: 20px; background-color: #eee">
      <p>부모 영역 (클릭 시 alert 발동)</p>
      <button @click="window.alert('1번 자식 클릭!')">버블링 발생 버튼</button>
      <button @click.stop="window.alert('2번 자식(나만 켜짐) 클릭!')">버블링 차단 버튼</button>
    </div>
  </div>
</template>
```



Vue Event Handling - Event Modifier 종류

수식어	수식어	실제 자바스크립트 동작 / 기능	실무 활용 예시
공통	**.prevent**	e.preventDefault()	Form 제출 시 페이지 새로고침 방지, 링크 이동 방지
	.stop	e.stopPropagation()	자식 버튼 클릭 시 부모 박스로 이벤트가 퍼지는 버블링 차단
	.once	이벤트 리스너 실행 후 자동 제거	좋아요 버튼 중복 클릭 방지, 설문조사 단 1회 제출
	.self	e.target === e.currentTarget	회색 배경막(Dim)을 직접 클릭했을 때만 팝업창 닫기 구현
	.capture	캡처링 단계에서 이벤트 감지	버블링과 반대로 부모 이벤트가 자식보다 먼저 터지게 설정
	.passive	scroll 성능 최적화	모바일 화면에서 무거운 스크롤/터치 이벤트 부드럽게 처리
키보드	**.enter**	Enter 키	로그인, 댓글 입력 후 엔터 쳤을 때 즉시 전송
	.tab	Tab 키	다음 입력 칸으로 포커스가 넘어갈 때 사전 유효성 검사
	.delete	Delete 또는 Backspace 키	텍스트 칩(Tag)을 선택하고 지우기 키를 눌러 삭제할 때
	.esc	Escape 키	팝업창이나 모달 열린 상태에서 Esc 누르면 창 닫기
	.space	Space 키	체크박스 형태의 UI에서 스페이스바 누르면 토글 처리
	.up / .down	방향키 위 / 아래	자동완성 검색어 목록에서 화살표로 리스트 이동할 때
	.left / .right	방향키 왼쪽 / 오른쪽	이미지 슬라이더(캐러셀)에서 화살표 키로 사진 넘기기

Vue Event Handling - Event Modifier 종류

수식어	수식어	실제 자바스크립트 동작 / 기능	실무 활용 예시
시스템	**.ctrl**	Ctrl 키	Ctrl + 클릭으로 링크를 새 탭에서 열 때 전용 로직 처리
	.alt	Alt 키	일반 클릭과 Alt + 클릭의 동작을 다르게 분기할 때
	.shift	Shift 키	Shift + Enter를 누르면 전송되지 않고 인풋창 줄바꿈 처리
	.meta	윈도우 키 (Windows) / 커맨드 키 (Mac)	OS 전용 시스템 단축키 조합을 웹앱에 이식할 때
	.exact	**오직 지정한 키만** 눌렸을 때 지정	@click.ctrl.exact: 다른 키 안 섞이고 쌩 Ctrl만 누르고 클릭해야 함
마우스	**.left**	마우스 왼쪽 버튼 (기본값)	일반적인 버튼 클릭 처리
	.right	마우스 오른쪽 버튼	웹페이지 순정 메뉴 대신 개발자가 만든 **'커스텀 우클릭 컨텍스트 메뉴'**를 띄울 때
	.middle	마우스 휠 (가운데 버튼)	마우스 휠 클릭으로 빠른 탭 닫기나 특수 스크롤 기능을 넣을 때

Code Challenge

- Vue Event Handling
  - v-on Event Handler Example
  - Event Object Example
  - Event Modifier Example

Form Data Binding - Two-way Data Binding with v-model

- v-model
  - HTML의 입력 요소의 값과 JavaScript 데이터(Ref)를 묶어, 한쪽이 바뀌면 다른 한쪽도 실시간으로 똑같이 바뀌게 만드는 양방향 (Two-way) 바인딩 장치를 제공

```
<script setup>
import { ref } from 'vue'
const text1 = ref('') // v-model용 변수
const text2 = ref('') // 원리 이해용 변수
</script>

<template>
  <div class="practice-section">
    <h2>v-model 양방향 데이터 바인딩</h2>
    <h3>1) v-model 축약 문법 (양방향)</h3>
    <input type="text" v-model="text1" placeholder="여기에 입력하세요" />
    <p>
      입력된 값: <strong>{{ text1 }}</strong>
    </p>
    <h3>2) v-model의 내부 작동 원리 (단방향 + 이벤트)</h3>
    <input type="text" :value="text2" @input="(e) => (text2 = e.target.value)" placeholder="원리 파악용 입력창" />
    <p>
      입력된 값: <strong>{{ text2 }}</strong>
    </p>
  </div>
</template>
```



[참고사항]

- v-bind와 v-on:input을 결합하면 양방향 바인딩이 완성된다.
- 이를 간단하게 작성할 수 있도록 한 것이 v-model이다.

Form Data Binding - Declaring v-model Variables

- HTML Form 요소별 v-model 매핑 규칙

Form 태그 종류	연결된 ref 변수의 초기값 타입	v-model이 변수에 담아주는 실제 값
textarea (장문 입력)	ref('') (문자열)	사용자가 입력한 장문의 줄바꿈 포함 텍스트
input[type="checkbox"] (단일)	ref(false) (불리언)	체크하면 true, 해제하면 false
input[type="checkbox"] (다중)	ref([]) **(배열)**	체크된 항목들의 value 속성 값이 배열에 차곡차곡 쌓임
input[type="radio"] (단일선택)	ref('') (문자열)	여러 라디오 중 사용자가 최종 선택한 하나의 value 값
select (드롭다운)	ref('') (문자열)	사용자가 선택한 <option>의 value 값

  - v-model로 양방향 바인딩을 할 때는 HTML 요소의 특성 및 동작 방식과 일치하도록 ref 초기값을 선언해 두어야 예외나 의도치 않은 버그를 막을 수 있다.
- 내부 이벤트의 차이
  - 일반 텍스트 입력(input, textarea)은 타이핑할 때마다 반응하는 @input 이벤트를 기반으로 작동한다.
  - 선택형 요소(checkbox, radio, select)는 값이 확정되는 시점에 반응하는 @change 이벤트를 기반으로 작동한다.

Form Data Binding - Code Example

- Form Elements Handling Example

```
<script setup>
import { ref } from 'vue'
const comment = ref('')
const isAgreed = ref(false) // 단일 체크박스는 Boolean
const favoriteFruits = ref([]) // 다중 체크박스는 반드시 배열([])로 시작!
const gender = ref('')
const selectedCar = ref('')
</script>

<template>
  <div class="practice-section">
    <h2>모든 HTML Form 요소와 v-model 매핑</h2>
    <div>
      <h3>1) Textarea (장문 텍스트)</h3>
      <textarea v-model="comment" placeholder="의견을 남겨주세요"></textarea>
      <p>
        데이터 상태: <span>{{ comment }}</span>
      </p>
    </div>
    <div>
      <h3>2) 단일 Checkbox (동의 여부)</h3>
      <label> <input type="checkbox" v-model="isAgreed" /> 약관에 동의합니다. </label>
      <p>
        데이터 상태: <span>{{ isAgreed }}</span>
      </p>
    </div>
```



Form Data Binding - Code Example

- Form Elements Handling Example

```
    <div>
      <h3>3) 다중 Checkbox (복수 선택 -> 배열에 저장)</h3>
      <label><input type="checkbox" value="사과" v-model="favoriteFruits" /> 사과</label>  
      <label><input type="checkbox" value="바나나" v-model="favoriteFruits" /> 바나나</label>  
      <label><input type="checkbox" value="딸기" v-model="favoriteFruits" /> 딸기</label>
      <p>
        데이터 상태 (배열): <span>{{ favoriteFruits }}</span>
      </p>
    </div>
    <div>
      <h3>4) Radio (단일 선택)</h3>
      <label><input type="radio" value="남성" v-model="gender" /> 남성</label>  
      <label><input type="radio" value="여성" v-model="gender" /> 여성</label>
      <p>
        데이터 상태: <span>{{ gender }}</span>
      </p>
    </div>
    <div>
      <h3>5) Select (드롭다운 선택)</h3>
      <select v-model="selectedCar">
        <option value="">-- 선택하세요 --</option>
        <option value="tesla">테슬라</option>
        <option value="hyundai">현대자동차</option>
        <option value="bmw">BMW</option>
      </select>
      <p>
        데이터 상태: <span>{{ selectedCar }}</span>
      </p>
    </div>
  </div>
</template>
```

Form Data Binding - v-model Modifiers

- v-model Modifiers
  - v-model 수식어는 입력 요소의 동작 방식이나 수집되는 데이터 형태를 손쉽게 제어할 수 있도록 Vue가 제공하는 편의 기능 (Syntactic Sugar)이다.

수식어	기본 동작 이벤트	수식어 적용 후 동작	주요 사용 목적
.lazy	@input (타이핑할 때마다 반영)	@change (포커스를 잃거나 Enter 시 반영)	불필요한 실시간 상태 업데이트 및 API 요청 방지
.number	String 타입 수집	Number 타입으로 자동 형변환	숫자 데이터 입력 시 자동 타입 변환 처리
.trim	입력값 그대로 수집	양끝 공백(Whitespace) 제거 후 수집	공백 입력으로 인한 Validation 오류 예방

  - Modifier Chaining : 수식어는 필요한 만큼 이어 붙여서 사용할 수 있다.

Form Data Binding - Code Example

- v-model Modifiers Example

```vue
<script setup>
import { ref } from 'vue'

// v-model Modifiers 실습용 reactive 변수 선언
const lazyText = ref('')
const age = ref('')
const userEmail = ref('')
const price = ref('')
</script>

<template>
  <div class="practice-section">
    <h2>v-model 수식어 (Modifiers) 활용</h2>
    <!-- 1) .lazy 수식어 실습 -->
    <section style="margin-bottom: 20px">
      <h3>1) .lazy 수식어 (change 이벤트 시점 반영)</h3>
      <input type="text" v-model.lazy="lazyText" placeholder="입력 후 Enter 또는 외부 클릭" />
      <p>
        실시간이 아닌 확정된 값: <strong>{{ lazyText }}</strong>
      </p>
    </section>
    <!-- 2) .number 수식어 실습 -->
    <section style="margin-bottom: 20px">
      <h3>2) .number 수식어 (Number 타입 자동 형변환)</h3>
      <input type="text" v-model.number="age" placeholder="나이를 입력하세요" />
      <p>
        입력된 값: <strong>{{ age }}</strong>
      </p>
      <p>
        데이터 타입: <strong>{{ typeof age }}</strong>
      </p>
    </section>
```

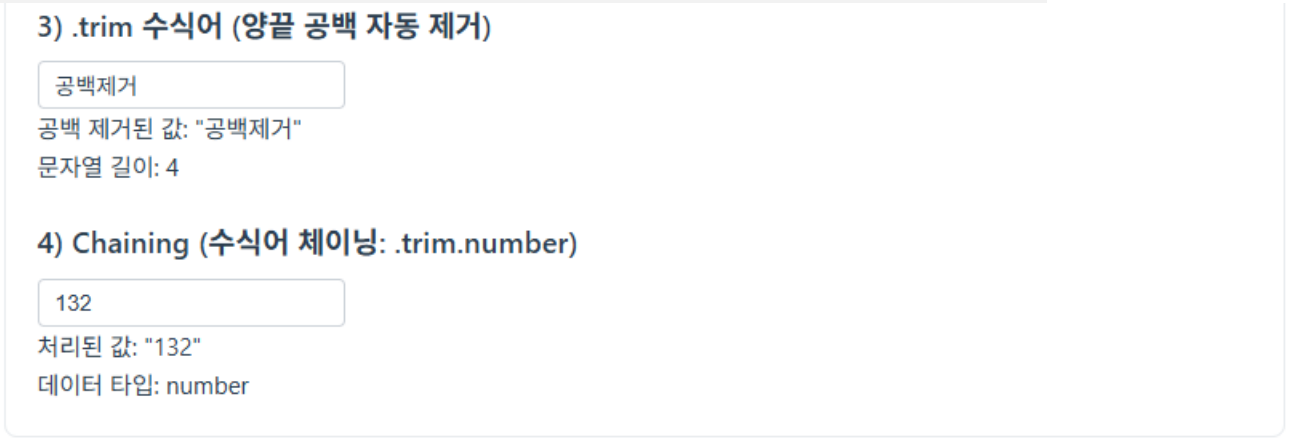


Form Data Binding - Code Example

- v-model Modifiers Example

```
    <!-- 3) .trim 수식어 실습 -->
    <section>
      <h3>3) .trim 수식어 (양끝 공백 자동 제거)</h3>
      <input type="text" v-model.trim="userEmail" placeholder="앞뒤 공백을 포함해 입력해 보세요" />
      <p>
        공백 제거된 값: <strong>"{{ userEmail }}"</strong>
      </p>
      <p>
        문자열 길이: <strong>{{ userEmail.length }}</strong>
      </p>
    </section>

    <!-- 4) 수식어 체이닝 (Chaining) 실습 -->
    <section>
      <h3>4) Chaining (수식어 체이닝: .trim.number)</h3>
      <input type="text" v-model.trim.number="price" placeholder="공백과 숫자를 섞어 입력해 보세요" />
      <p>
        처리된 값: <strong>"{{ price }}"</strong>
      </p>
      <p>
        데이터 타입: <strong>{{ typeof price }}</strong>
      </p>
    </section>
  </div>
</template>
```



Vue Style

- Scoped Style
  - SFC 파일의 구성 요소 중 <style>에 적용된 style은 모든 컴포넌트에 적용된다. 하지만,
  - <style scoped>내에 작성된 스타일은 현재 컴포넌트 내부에 선언된 HTML 태그에만 적용되고, 다른 컴포넌트에는 영향을 주지 않는다.

전역 스타일 정의

```
<template>
    <div class="header">Welcome to Vue</div>
</template>

<style>
.header {
    font-size: 24px;
    font-weight: bold;
    color: #42b983;
}
</style>
```

scoped 스타일 정의

```
<template>
    <div class="card">Styled Card</div>
</template>

<style scoped>
.card {
    background-color: lightblue;
    padding: 20px;
    border-radius: 8px;
}
</style>
```

- External Style
  - 공통 CSS나, 외부 라이브러리 CSS를 사용하는 방법
  - 프로젝트 전체에 적용할 공통 스타일은 src/main.js 에 등록한다.
  - 특정 컴포넌트에 외부 CSS파일을 적용할 때는 <style> 방 내부에서 자바스크립트의 @import 문법을 사용한다.

Vue Style - Example

- Vue Style Example

```vue
<script setup>
// 자바스크립트 방은 깨끗하게 비워둡니다.
</script>

<template>
  <div class="practice-section">
    <h2>Scoped 스타일 및 외부 CSS 활용</h2>
    <p class="title">이 글자는 이 컴포넌트 내부에서만 빨간색이 됩니다.</p>
    <button class="btn-external">외부 CSS에서 불러온 버튼 스타일</button>
  </div>
</template>

<style scoped>
/* 내 방 전용 타이틀 디자인 */
.title {
  color: #ff7675;
  font-weight: bold;
  font-size: 18px;
}
</style>
```




```vue
<style>
/* ⚠ 외부 스타일 파일(예: 버튼 디자인 뭉치)을 이 방 안으로 쏙 가리켜 가져옵니다 */
@import '@/assets/challenge.css';
</style>
```

Code Challenge

- Vue Form Handling
  - v-model 양방향 데이터 바인딩
  - HTML Form 요소와 v-model 매핑
  - v-model Modifiers
- Vue Style
  - Vue Style Example

[실습] 과제 - 날씨 Mockup

- 과제 요구사항

1. 배열 렌더링 (v-for)

   - 임의의 날씨 데이터 배열을 활용해 화면에 날씨 카드를 반복 출력한다.

```
const weatherList = ref([
  { id: 'city_01', name: '서울', temp: 28, status: '맑음' },
  { id: 'city_02', name: '수원', temp: 24, status: '비' },
  { id: 'city_03', name: '부산', temp: 26, status: '구름' },
])
```

   - :key에 id 바인딩 필수

2. 조건부 렌더링 (v-if)

   - 기온이 25도 이상인 도시는 "🔥 더움 (25도 이상)", 25도 미만인 도시는 "❄️ 선선함 (25도 미만)" 라벨을 붙힌다.

3. 양방향 바인딩 및 한글 처리 (:value, @input)

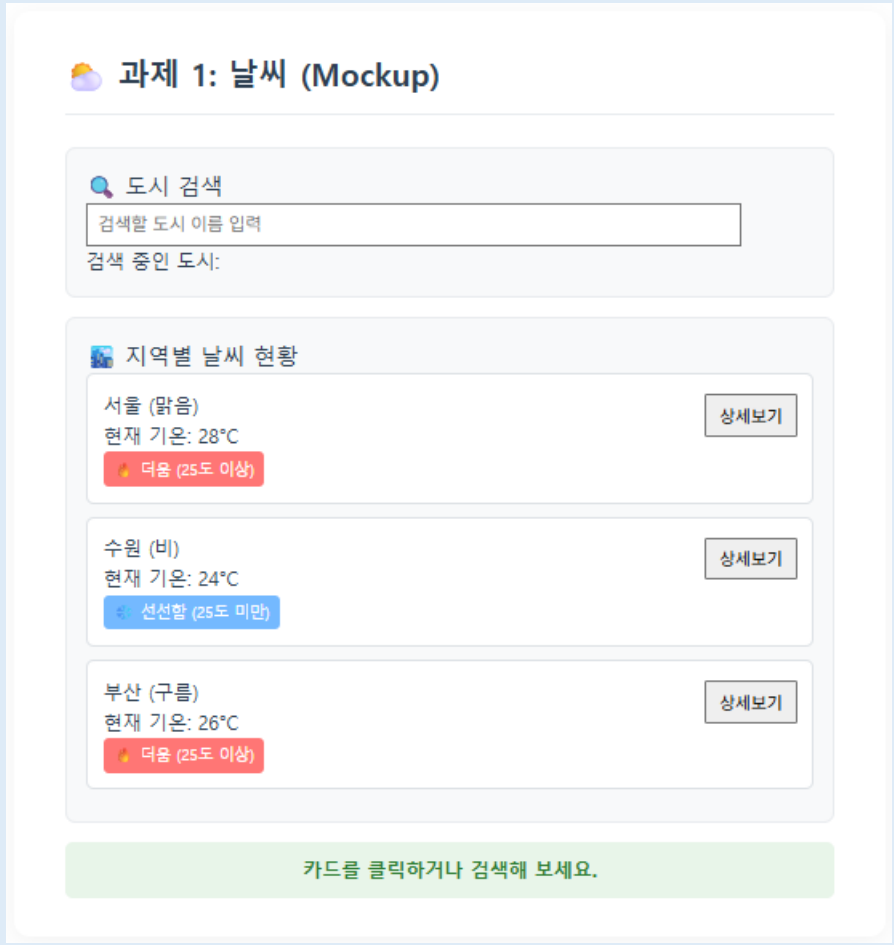
   - 도시 이름을 한글로 검색하는 input을 만든 후 한글 입력 후 입력 한 도시명을 출력한다.

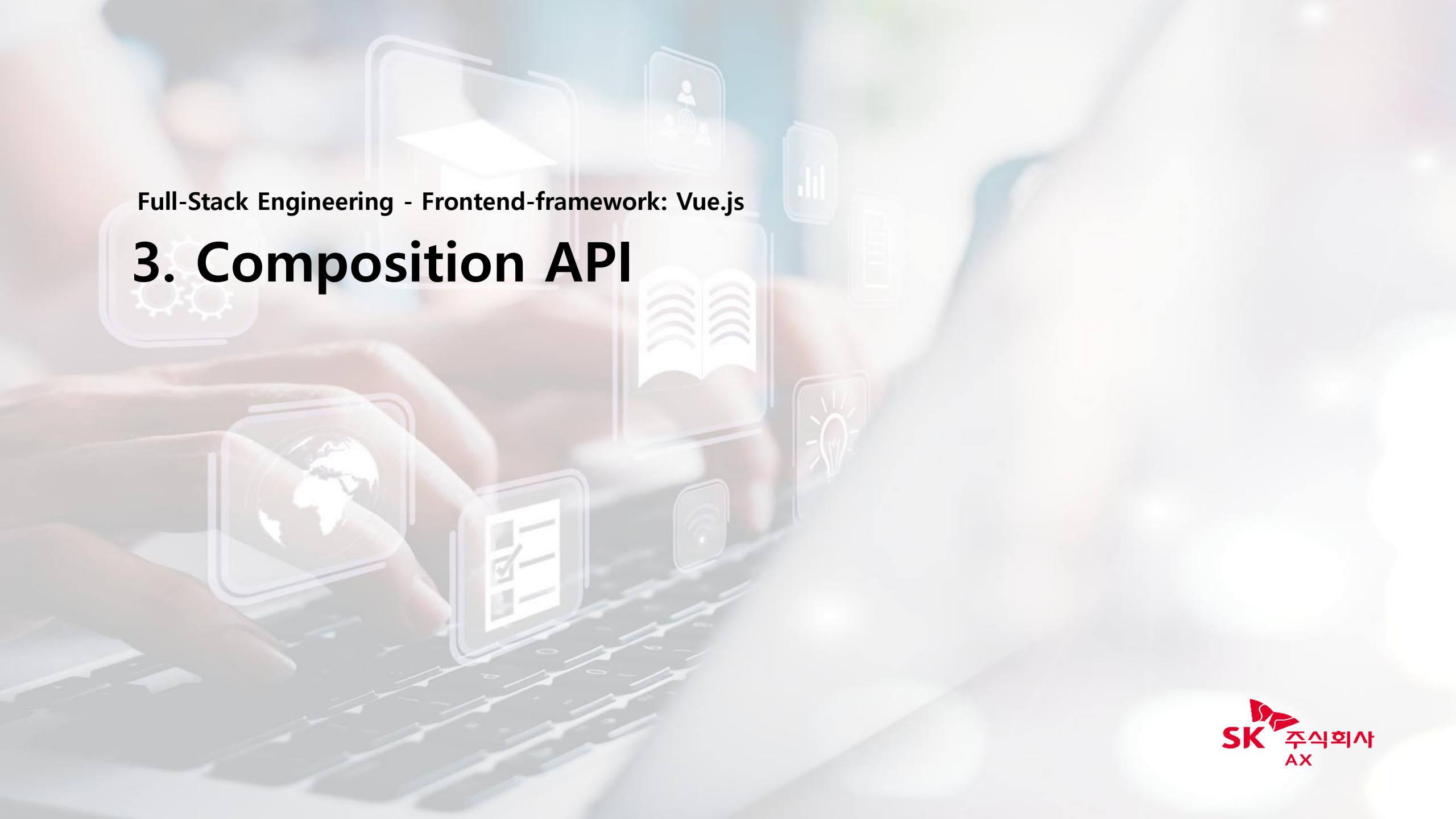
4. 이벤트 및 수식어

   - 지역별 날씨 현황 카드를 누르면 상태바에 "{도시}이 선택되었습니다." 표기

   - 지역별 날씨 현황 카드 내부의 [상세보기] 버튼을 누르면 버블링 없이 해당 도시의 날씨 내용을 window.alert로 띄운다.

```
const showDetail = (cityName, status) => {
  window.alert(`${cityName}의 현재 날씨는 [${status}] 상태입니다.`)
}
```





Full-Stack Engineering - Frontend-framework: Vue.js

3. Composition API

SK 주식회사 AX

Composition API Overview

- Composition API
  - Vue 3에서 도입된 현대적인 JavaScript 코딩 방식으로, Component의 로직(데이터, 함수, 계산된 속성 등)을 유연하고 깔끔하게 한곳에 조합(Composition)하여 작성하는 방법을 제공한다.
  - Vue 2 시절에는 Options API를 사용했는데 Options API는 데이터, 계산된 속성(Computed), 메서드(methods), Lifecycle Hook 등을 포함한 객체 기반 옵션 속성으로 구성된다.
  - Composition API는 로직(데이터, 함수, 계산된 속성)을 하나의 세트로 모아서 작성함으로 가독성이 좋다.
- Composition API 작성 방법
  - Vue 3.2 부터 <script setup> 안에 작성하면 된다.

Composition API Overview - Vue 3 내장함수

- Vue 프레임워크에서 제공하는 핵심 기능을 수행하는 함수들

카테고리	주요 함수
애플리케이션 (Application)	createApp, createSSRApp, app.*(), app.config.*
반응형 상태 관리 (Reactive State)	**ref, reactive,** readonly, , shallowRef, shallowReactive, shallowReadonly, toRef, toRefs, customRef, unref, toRaw, markRaw, isRef, isReactive, isReadonly
계산 및 감시 (Computed & Watchers)	**computed, watch, watchEffect**
라이프사이클 훅 (Lifecycle Hooks)	setup, onMounted, onUpdated, onUnmounted, onBeforeMount, onBeforeUpdate, onBeforeUnmount, onActivated, onDeactivated, onErrorCaptured, onRenderTracked, onRenderTriggered
컴포넌트 구성/메타 (Component Composition)	defineComponent, defineProps, defineEmits, useAttrs, defineExpose, useSlots, withDefaults, getCurrentInstance
렌더링 제어 (Rendering & DOM)	h, resolveComponent, withDirectives, renderList, renderSlot, mergeProps, nextTick, useCssModule, useCssVars
의존성 주입 (Dependency Injection)	provide, inject, hasInjectionContext

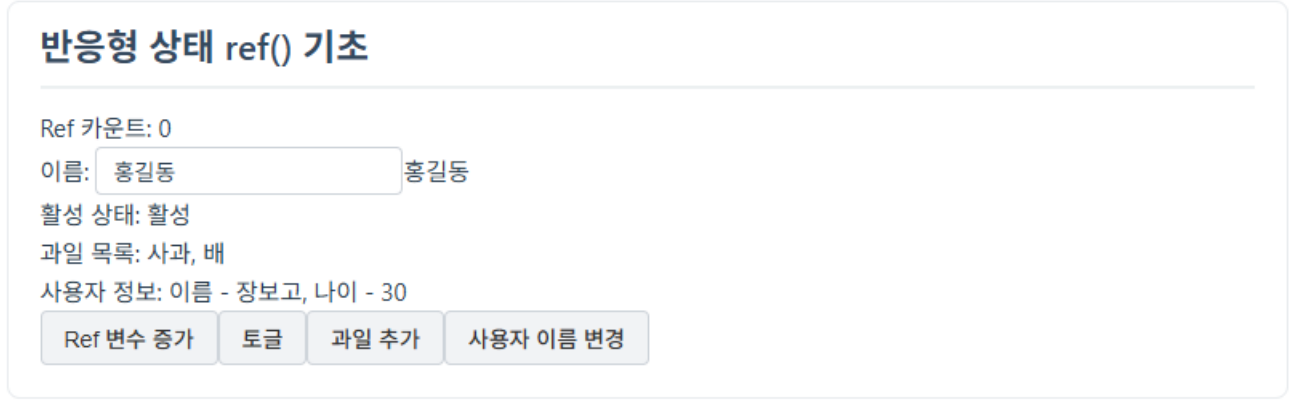
Reactive State - ref()

- ref()
  - 원시 타입 (Primitive Datatype)이나 참조 자료형 (Array, Object 등) 모든 값을 반응형 상태로 만든다.
  - <script setup> 상단에 ref()를 import 해야 한다.
  - <script setup>에서는 .value 로 접근하고, <template>에서는 .value 없이 사용

Reactive State - ref() Example

- ref() Example

```
<script setup>
import { ref } from 'vue'
const count = ref(0)
const name = ref('홍길동')
const isActive = ref(true)
const items = ref(['사과', '배'])
const user = ref({name: '이순신', age: 30,})
const increaseRef = () => {
  count.value++
}
const changeUserName = () => {
  user.value.name = '장보고'
}
</script>
<template>
  <div class="practice-section">
    <h2>반응형 상태 ref() 기초</h2>
    <p>Ref 카운트: <strong>{{ count }}</strong></p>
    <p>이름: <input v-model="name" />{{ name }}</p>
    <p>활성 상태: {{ isActive ? '활성' : '비활성' }}</p>
    <p>과일 목록: {{ items.join(', ') }}</p>
    <p>사용자 정보: 이름 - {{ user.name }}, 나이 - {{ user.age }}</p>
    <button @click="increaseRef">Ref 변수 증가</button>
    <button @click="isActive = !isActive">토글</button>
    <button @click="items.push('귤')">과일 추가</button>
    <button @click="changeUserName">사용자 이름 변경</button>
  </div>
</template>
```



Reactive State - reactive()

- reactive()
  - 참조 자료형(객체, 배열, Map, Set)데이터를 반응형 상태로 만드는 함수
  - <script setup> 상단에 reactive()를 import 해야 한다.
  - <script setup>에서 .value 없이 접근하고, <template>에서도 일반 객체처럼 사용한다.
- reactive()의 반응성 단절
  - 반응형 객체를 교체하거나, 구조를 분해해야 할당하면 반응성 연결이 끊어진다.

```
let state = reactive({ count: 0 })

// ❌ 통째로 새 객체를 갈아끼우면 반응성 연결이 끊어진다.
state = { count: 5 }
// 🟢 내부의 알맹이 속성만 조심스럽게 변경해야 한다.
state.count = 5
```

  - 이 약점 때문에 현업에서는 객체나 배열을 다룰 때도 그냥 안전하게 ref()로 통일해서 쓰는 추세가 강함.

Reactive State - reactive() Example

- reactive() Example

```vue
<script setup>
import { reactive } from 'vue'

const userReactive = reactive({name: '이순신', age: 30,})
const celebrateReactive = () => {userReactive.age++}

const items = reactive(['사과', '바나나'])
const addItem = () => {items.push(`과일 ${items.length + 1}`)}
const removeItem = (index) => {items.splice(index, 1)}
</script>

<template>
  <div class="practice-section">
    <h2>반응형 상태 reactive() 특징 및 주의점</h2>
    <h3>1) 객체(Object) reactive</h3>
    <p>이름: {{ userReactive.name }} / 나이: {{ userReactive.age }}세</p>
    <button @click="celebrateReactive">reactive 나이 한 살 추가</button>
    <h3>2) 배열(Array) reactive</h3>
    <ul>
      <li v-for="(item, index) in items" :key="index">
        {{ item }}
        <button @click="removeItem(index)" style="margin-left: 8px; padding: 2px 6px">삭제</button>
      </li>
    </ul>
    <button @click="addItem">과일 항목 추가</button>
  </div>
</template>
```



[참고사항]

- reactive로 선언된 배열은 items = ['a', 'b'] 처럼 전체를 새 배열로 재할당하면 반응형 연결이 깨진다
- 따라서, 배열 데이터를 변경할 할 때는 push/splice를 쓰거나 ref()를 사용하는 것이 권장된다.

Reactive State - 기타 함수들

- 반응형 상태 관리 함수 요약

분류	함수	설명
기본 반응형 선언	**ref**	기본형/객체 모두 감싸서 반응형으로 변환
	reactive	객체, 배열 등을 반응형으로 변환
읽기 전용 선언	readonly	반응형 객체를 읽기 전용으로
얕은 반응형/읽기전용	shallowRef	내부는 반응형 X, 객체의 참조만 감지
	shallowReactive	객체의 첫 레벨만 반응형
	shallowReadonly	첫 레벨만 읽기 전용
Ref ↔ Reactive 변환	toRef	reactive 객체의 속성 1개를 ref로 추출
	toRefs	reactive 객체의 모든 속성을 ref로 추출
Ref 해제 / 원본 추출	unref	ref에서 값만 꺼냄 (ref.value 대신)
	toRaw	reactive에서 원본 객체 추출
	markRaw	객체를 반응형 처리 제외
반응형 여부 확인	isRef	ref인지 확인
	isReactive	reactive인지 확인
	isReadonly	readonly인지 확인
커스텀 Ref 생성	customRef	사용자 정의 반응형 ref

Code Challenge

- Reactive State
  - ref() Example
  - reactive() Example

Computed & Watchers - computed()

- computed()
  - 의존하는 반응형 데이터(ref, reactive)가 변경될 때 자동으로 다시 계산된다.
  - 계산된 값은 메모리에 Caching되어 성능이 좋다. (일반 함수로 처리 시 화면이 바뀌면 함수를 다시 실행하는데, Computed는 다시 계산하지 않음으로 성능에 유리함)
  - 의존하는 반응형 데이터가 바뀔 때 새로 계산됨.
- Syntax

```
Import { computed } from 'vue';

const 식별자 = computed(() => { return 값 })
```

  - computed() 함수의 인자로는 Call Back 함수를 넣는다.
  - Call Back() 함수에서는 Vue 내부에서 정의된 Computed Ref 객체가 반환된다. (ref() 함수의 결과인 ref 객체와 비슷함)
  - 따라서, <script setup> 영역에서는 .value를 붙여서 값을 읽어야 한다.
  - computed() 함수로 생성한 계산된 속성(Computed Property)은 기본적으로 읽기 전용이므로 이 반수를 다른 값으로 재할당 할 수 없다.

Computed & Watchers - computed() Example

- computed() Example

```
<script setup>
import { ref, computed } from 'vue'

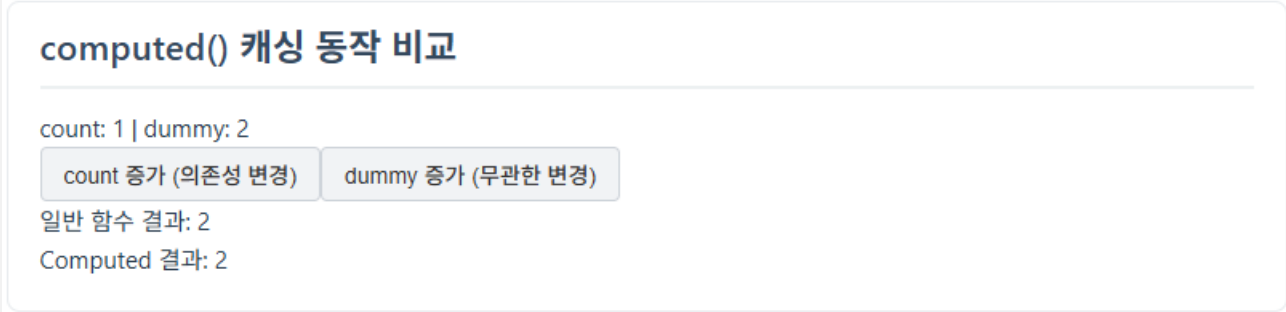
const count = ref(0)
const dummy = ref(0) // computed와 무관한 변수

// 1. 일반 함수: 화면이 조금이라도 리렌더링되면 무조건 재실행
const getMethodResult = () => {
  console.log('❌ 일반 함수 실행됨!')
  return count.value * 2
}

// 2. Computed: count가 바뀔 때만 재연산 (dummy가 바뀔 땐 이전 값 재사용)
const doubleCount = computed(() => {
  console.log('☑ Computed 연산 실행됨!')
  return count.value * 2
})
</script>

<template>
  <div class="practice-section">
    <h2>computed() 캐싱 동작 비교</h2>

    <p>count: {{ count }} | dummy: {{ dummy }}</p>
    <button @click="count++">count 증가 (의존성 변경)</button>
    <button @click="dummy++">dummy 증가 (무관한 변경)</button>
    <!-- dummy 버튼을 누를 때 콘솔 출력 차이를 확인 -->
    <p>일반 함수 결과: {{ getMethodResult() }}</p>
    <p>Computed 결과: {{ doubleCount }}</p>
  </div>
</template>
```



- Vue Component 재렌더링 동작원리
  - count 변수가 증가하면, Vue 반응형 시스템은 해당 컴포넌트의 DOM을 다시 그려야 할 필요성(Re-render)을 감지한다.
  - 템플릿을 다시 그릴 때 Vue는 <template> 내부의 모든 표현식과 메서드 호출을 처음부터 끝까지 다시 평가(Execute)한다.
  - 템플릿 안에 {{ getMethodResult() }}처럼 괄호()를 붙여 직접 호출한 일반 함수는, 컴포넌트가 다시 그려질 때마다 조건을 불문하고 무조건 다시 실행된다.
- Console 창 확인
  - count 증가 Click: 일반 함수 로그와 computed 로그가 둘 다 표시
  - dummy 증가 Click: 일반 함수 로그만 표시 (computed는 재 연산하지 않고 이전 결과를 Caching하여 재 사용)

Computed & Watchers - (참고) 함수정의방식

- <script setup> 내 함수 정의 방식 3종 비교

분류	1) 함수 선언문 (Function Declaration)	2) 함수 표현식 (Function Expression)	3) 화살표 함수 (Arrow Function)
Syntax	function 로직() { ... }	const 로직 = function() { ... }	const 로직 = () => { ... }
Hoisting	**작동** (태그/코드 어디서든 선언 전 호출 가능)	**작동 안 함** (선언된 아랫줄부터 호출 가능)	**작동 안 함** (선언된 아랫줄부터 호출 가능)
실무	가끔 사용 (전통적 방식)	거의 안 씀	**압도적 권장 (실무 표준 ✸)**

```
<script setup>
import { ref } from 'vue'
const message = ref('버튼을 누르세요')

// 1) 함수 선언문 (호이스팅 가능 - 하단에 있어도 위에서 호출 가능)
function 함수명() {
  message.value = '1번 함수 선언문 방식 발동!'
}

// 2) 함수 표현식 (호이스팅 불가 - 선언 후 호출 가능)
const 변수명 = function() {
  message.value = '2번 함수 표현식 방식 발동!'
}

// 3) 화살표 함수 (실무 국룰 ✸ - 호이스팅 불가, 가장 컴팩트함)
const 변수명 = () => {
  message.value = '3번 화살표 함수 방식 발동!'
}
</script>
```

Computed & Watchers - watch()

- watch()
  - 반응형으로 선언된 데이터의 값이 변경되었을 때, 후속 로직(비동기 통신, 데이터 저장 등)을 수행하도록 Call 함수를 지정
- Syntax

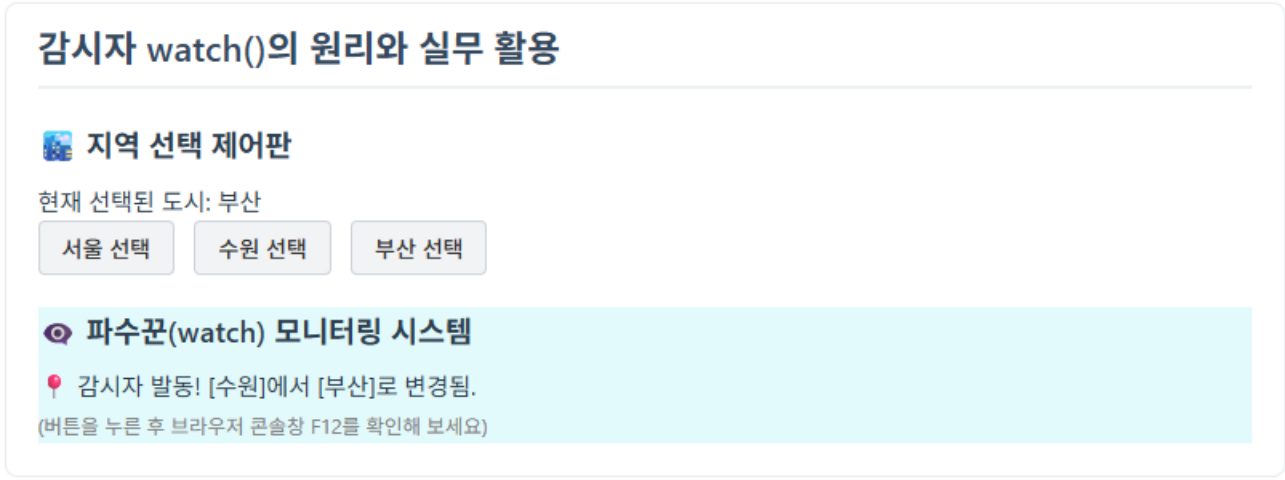
```
Import { watch } from ‘vue’;

watch(반응형데이터, (newVal, oldVal) => { 실행할 후속 로직 })
```

  - 첫 번째 인자로 감시할 데이터를 받고, 두 번째 인자로 변경 시 실행할 콜백함수를 지정한다.
  - 콜백 함수의 인자는 “새로 변경된 값(newVal)”과 “변경되기 전의 값(oldVal)” 이 전달된다.

Computed & Watchers - watch() Example

- watch() Example




```
<script setup>
import { ref, watch } from 'vue'

const currentCity = ref('서울')
const logMessage = ref('아직 감시 시스템이 작동하지 않았습니다.')

// currentCity 변수를 유심히 감시하는 watch 시스템 가동
watch(currentCity, (newValue, oldValue) => {
  // 값이 바뀌는 순간, 바뀐 알맹이(값) 두 개가 자동으로 주입됩니다.
  logMessage.value = `📍 감시자 발동! [${oldValue}]에서 [${newValue}]로 변경됨.`
  // 실무 활용처 시뮬레이션
  console.log(`🤖 [서버 요청 완료] 기상청 서버에서 ${newValue}의 날씨 API를 다시 조회합니다...`)
})
</script>

<template>
  <div class="practice-section">
    <h2>감시자 watch()의 원리와 실무 활용</h2>
    <h3>🌆 지역 선택 제어판</h3>
    <p>현재 선택된 도시: {{ currentCity }}</p>
    <button @click="currentCity = '서울'">서울 선택</button>  
    <button @click="currentCity = '수원'">수원 선택</button>  
    <button @click="currentCity = '부산'">부산 선택</button>
    <div class="monitor">
      <h3>👁️ 파수꾼(watch) 모니터링 시스템</h3>
      <p>{{ logMessage }}</p>
      <small style="color: gray">(버튼을 누른 후 브라우저 콘솔창 F12를 확인해 보세요)</small>
    </div>
  </div>
</template>
```

Computed & Watchers - watch() (Multi-Source Watch)

- Multi-Source Watch
  - 반응형으로 선언된 여러 데이터를 한꺼번에 감시할 때 쓰는 기법
- Syntax

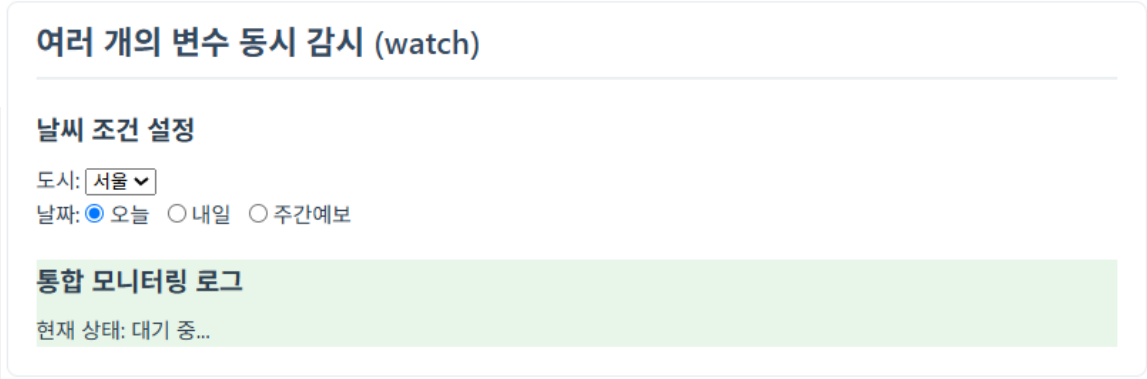
```
Import { watch } from 'vue';

watch([변수1, 변수2], ([새값1, 새값2], [옛값1, 옛값2]) => { 실행할 후속 로직 })
```

  - 첫 번째 인자에 감시할 대상을 배열[ ] 형태로 묶어 전달한다.
  - 첫 번째 인자 배열의 순서([변수1, 변수2])대로, 콜백 함수의 인자 배열 순서([새값1, 새값2], [옛값1, 옛값2])가 같다.
  - 둘 중 어느 하나라도 변하면 감시자 콜백 함수가 즉시 발동한다.

Computed & Watchers - watch() (Multi-Source Watch) Example

- Multi-Source Watch Example




```
<script setup>
import { ref, watch } from 'vue'
const city = ref('서울')
const dateType = ref('오늘')
const apiStatus = ref('대기 중...')

// 두 개의 ref 변수를 배열[] 형태로 묶어 동시에 감시합니다.
watch([city, dateType], ([newCity, newDate], [oldCity, oldDate]) => {
  apiStatus.value = `[변경 감지] ${oldCity}(${oldDate}) ➡️ ${newCity}(${newDate})`
  // 실무 활용: 두 옵션 중 하나만 바뀌어도 통합 API 요청을 보냅니다.
  console.log(`🤖 [통합 API 호출] ${newCity}의 ${newDate} 날씨를 불러옵니다...`)
})
</script>

<template>
  <div class="practice-section">
    <h2>여러 개의 변수 동시 감시 (watch)</h2>
    <h3>날씨 조건 설정</h3>
    <label>도시: </label>
    <select v-model="city">
      <option value="서울">서울</option><option value="수원">수원</option><option value="부산">부산</option>
    </select>
    <label>날짜: </label>
    <label><input type="radio" value="오늘" v-model="dateType" /> 오늘</label>  
    <label><input type="radio" value="내일" v-model="dateType" /> 내일</label>  
    <label><input type="radio" value="주간예보" v-model="dateType" /> 주간예보</label>
    <div class="monitor">
      <h3>통합 모니터링 로그</h3>
      <p>현재 상태: {{ apiStatus }}</p>
    </div>
  </div>
</template>
```

Computed & Watchers - watch() (Deep Watch)

- Deep Watch
  - ref()로 선언된 객체나 배열을 감시할 때 쓰는 기법
  - watch는 객체나 배열의 주소값(참조값)만 추적하고, 내부 속성의 변경은 감지하지 않음으로 값의 변화를 추적하려면 "{ deep: true }"를 명시 해야함
- Syntax

```
Import { watch } from 'vue';

watch(반응형데이터, (newValue) => { 실행할 후속 로직 },{deep:true})
```

  - 주의점: deep: true를 쓰면 newValue와 oldValue가 똑같이 최신 값으로 출력되어 과거 값을 추적할 수 없다. (주소값이 같기 때문)
- 객체의 특정 속성만 감시하기
  - watch(() => 변수.value.속성, (새값, 옛값) => {…})
  - 이 방식을 쓰면 newValue와 oldValue가 정상 수집이 된다.

Computed & Watchers - watch() (Deep Watch) Example

- Deep Watch Example

```
<script setup>
import { ref, watch } from 'vue'

const user = ref({
  name: '홍길동',
  age: 20,
})

const logDeep = ref('아직 반응 없음')
const logTarget = ref('아직 반응 없음')

// 실패하는 예시 (가장 많이 범하는 오류)
// watch(user, () => { console.log('이 로그는 영원히 안 찍힙니다.') })

// 해결책 1: deep 옵션을 켜서 객체 하위 속성 전체 감시하기
watch(user, (newVal) => {
    logDeep.value = `[deep 감지] 누군가 변경됨! 현재 이름: ${newVal.name}, 나이: ${newVal.age}`
  },
  { deep: true },
)

// 해결책 2: 화살표 함수로 특정 속성(age)만 콕 집어 감시하기 (★이전 값 추적 가능!)
watch(() => user.value.age, (newAge, oldAge) => {
    logTarget.value = `[타겟 감지] 나이가 ${oldAge}세 ➡ ${newAge}세로 변경됨!`
  },
)
</script>
```

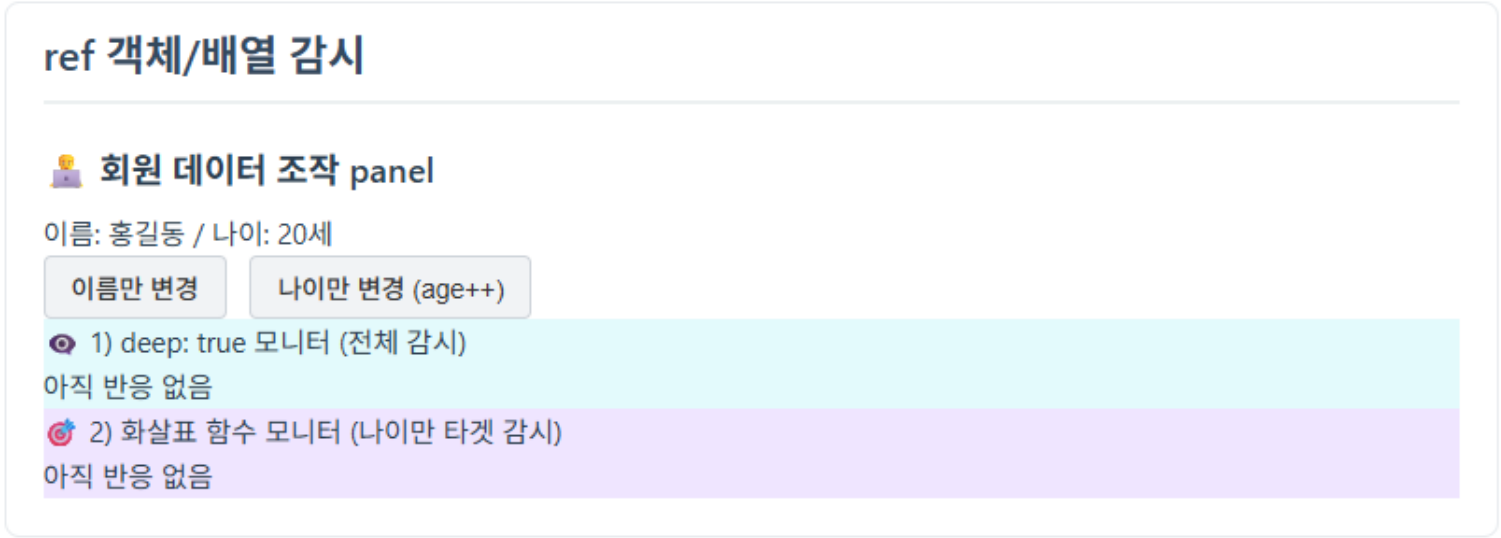
Computed & Watchers - watch() (Deep Watch) Example

- Deep Watch Example (Cont'd)

```
<template>
  <div class="practice-section">
    <h2>ref 객체/배열 감시</h2>
    <h3>👨‍💼 회원 데이터 조작 panel</h3>
    <p>이름: {{ user.name }} / 나이: {{ user.age }}세</p>
    <button @click="user.name = '이순신'">이름만 변경</button>  
    <button @click="user.age++">나이만 변경 (age++)</button>

    <div class="monitor">
      <p>👁 1) deep: true 모니터 (전체 감시)</p>
      <p>{{ logDeep }}</p>
    </div>

    <div class="monitor target">
      <p>🎯 2) 화살표 함수 모니터 (나이만 타겟 감시)</p>
      <p>{{ logTarget }}</p>
    </div>
  </div>
</template>
```



Computed & Watchers - watch() (reactive 반응형 데이터)

- reactive() 반응형 데이터 전체 감시
  - reactive() 함수로 생성된 반응형 데이터인 배열이나 객체는 내부 값이 변경되어서 감시가 가능함.
  - 하지만, deep 속성을 사용한 것처럼 콜백 함수의 이전 값을 알 수 없다.
- reactive() 반응형 데이터 특정 속성 감시
  - 객체 안의 수많은 속성 중 특정 속성만 바뀌는 타이밍을 낚아채고 싶다면 () => 객체.속성) 형태로 적어주어야 한다.
  - 이 방식으로 하면 이전 값을 추적할 수 있게 된다.

Computed & Watchers - watch() (reactive 반응형 데이터) Example

- Reactive 반응형 데이터 watch() Example

```
<script setup>
import { reactive, ref, watch } from 'vue'

// reactive로 선언한 묶음 상품 데이터
const state = reactive({
  productName: '노트북',
  price: 1000,
})

const logAutoDeep = ref('대기 중...')
const logTarget = ref('대기 중...')

// 🟢 1) 변수명 그대로 감시 (자동 deep: true 작동)
watch(state, (newVal, oldVal) => {
  // newVal.price와 oldVal.price가 똑같이 2000으로 나옵니다!
  logAutoDeep.value = `[자동 deep] 가격 변동! 이전가격인척하는:${oldVal.price}원 ➡️ 현재가격:${newVal.price}원`
})

// 🟢 2) 화살표 함수로 특정 속성만 감시 (이전 값 추적 가능!)
watch(
  () => state.price,
  (newPrice, oldPrice) => {
    // 💧 특정 알맹이 값만 추출했으므로 진짜 과거 가격(1000)이 정상 보존됩니다.
    logTarget.value = `[타겟 조준] 가격이 진짜 올랐음! 옛날값:${oldPrice}원 ➡️ 바뀐값:${newPrice}원`
  },
)
</script>
```

Computed & Watchers - watch() (reactive 반응형 데이터) Example

- Reactive 반응형 데이터 watch() Example (Cont'd)

```
<template>
  <div class="practice-section">
    <h2>reactive() 데이터 watch 감시 규칙</h2>
    <h3>🛒 상품 정보 관리 (reactive)</h3>
    <p>상품명: {{ state.productName }} / 가격: {{ state.price }}원</p>
    <button @click="state.price += 500">가격 500원 인상</button>

    <div class="monitor auto">
      <p>👁 1) state 변수 통째로 감시 (deep 자동화)</p>
      <p>{{ logAutoDeep }}</p>
      <small>※ 주의: 이전 값과 현재 값이 똑같이 찍힌다.</small>
    </div>

    <div class="monitor target">
      <p>🎯 2) () => state.price 콕 집어 감시 (과거 추적)</p>
      <p>{{ logTarget }}</p>
      <small>※ 성공: 과거의 원본 가격이 칼같이 보존된다.</small>
    </div>
  </div>
</template>
```



Computed & Watchers - watch() (Array)

- 기본형 배열의 특정 인덱스의 값 자체를 감시할 때
  - ref() 함수로 생성된 반응형 데이터

```
const teamMembers = ref(['홍길동', '이순신', '강감찬'])
```

```
watch(() => 배열.value[0], (새값, 옛값) => { ... })
```

  - reactive() 함수로 생성된 반응형 데이터

```
const todoList = reactive(['프로젝트 기획', '퍼블리싱', 'Vue 개발'])
```

```
watch(() => 배열[0], (새값, 옛값) => { ... })
```

- ref() 객체형 배열의 특정 객체 내부 속성을 감시할 때
  - ref() 함수로 생성된 반응형 데이터

```
const cityWeather = ref([
  { name: '서울', temp: 25 },
  { name: '수원', temp: 22 }
])
```

```
watch(() => 배열.value[0], (새객체) => { ... }, { deep: true })
```

  - reactive() 함수로 생성된 반응형 데이터

```
const favorCities = reactive([
  { name: '수원', temp: 21 },
  { name: '부산', temp: 24 }
])
```

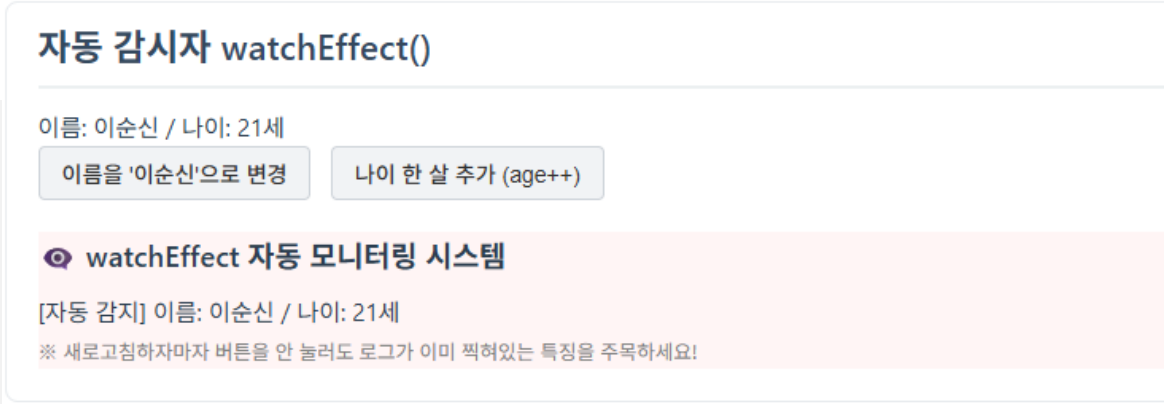
```
watch(() => 배열[0], (새객체) => { ... }, { deep: true })
```

Computed & Watchers - watchEffect()

- watchEffect()
  - **감시 대상을 명시하지 않아도** 반응형 데이터를 자동으로 추적해서 값이 바뀔 때마다 재실행되는 함수
  - 값이 바뀔 때 뿐만 아니라 컴포넌트가 처음 태어날 때(최초 1회)도 무조건 즉시 실행된다.
  - 오직 현재 시점의 코드만 자동 실행하므로 이전 값(oldValue)은 제공하지 않는다.
  - watchEffect() 함수는 함수 내부에서 접근한 반응형 데이터만 자동으로 감시한다.

Computed & Watchers - watchEffect() Example

- watchEffect() Example




```vue
<script setup>
import { ref, watchEffect } from 'vue'

const username = ref('홍길동')
const age = ref(20)
const logMessage = ref('대기 중...')

// watchEffect 가동: 감시 대상을 지정하는 파라미터가 없습니다!
watchEffect(() => {
  // Vue가 이 내부 코드를 읽고 'username'과 'age'를 자동으로 감시 리스트에 등록합니다.
  logMessage.value = `[자동 감지] 이름: ${username.value} / 나이: ${age.value}세`

  // 화면이 처음 켜질 때 1등으로 즉시 실행되는 증거를 콘솔에서 확인합니다.
  console.log('🤖 watchEffect가 내부 변수 변경을 감지하여 실행되었습니다.')
})
</script>

<template>
  <div class="practice-section">
    <h2>자동 감시자 watchEffect()</h2>
    <p>이름: {{ username }} / 나이: {{ age }}세</p>
    <button @click="username = '이순신'">이름을 '이순신'으로 변경</button>  
    <button @click="age++">나이 한 살 추가 (age++)</button>

    <div class="monitor">
      <h3>👁 watchEffect 자동 모니터링 시스템</h3>
      <p>{{ logMessage }}</p>
      <small style="color: gray">※ 새로고침하자마자 버튼을 안 눌러도 로그가 이미 찍혀있는 특징을 주목하세요!</small>
    </div>
  </div>
</template>
```

Computed & Watchers - 기타 함수들

- 반응형 상태 관리 함수 요약

분류	함수	설명
계산된 속성 선언	**computed**	기존 반응형 상태를 기반으로 가공된 값을 계산 (캐싱 최적화 지원, 읽기 전용이 기본)
지정형 감시자	**watch**	특정 반응형 데이터(들)를 지정하여 변경을 감시하고 후속 콜백(이전/현재 값 제공) 실행
자동 의존성 감시자	**watchEffect**	내부에 사용된 반응형 데이터를 자동으로 추적하여 감시, 컴포넌트 생성 시 즉시 최초 1회 실행
감시자 실행 타이밍 제어	watchPostEffect	watchEffect와 동일하나, DOM 업데이트(화면 렌더링)가 완료된 **후**에 콜백을 실행
	watchSyncEffect	watchEffect와 동일하나, 데이터가 변경되는 즉시 **동기적(Sync)**으로 콜백을 즉각 실행

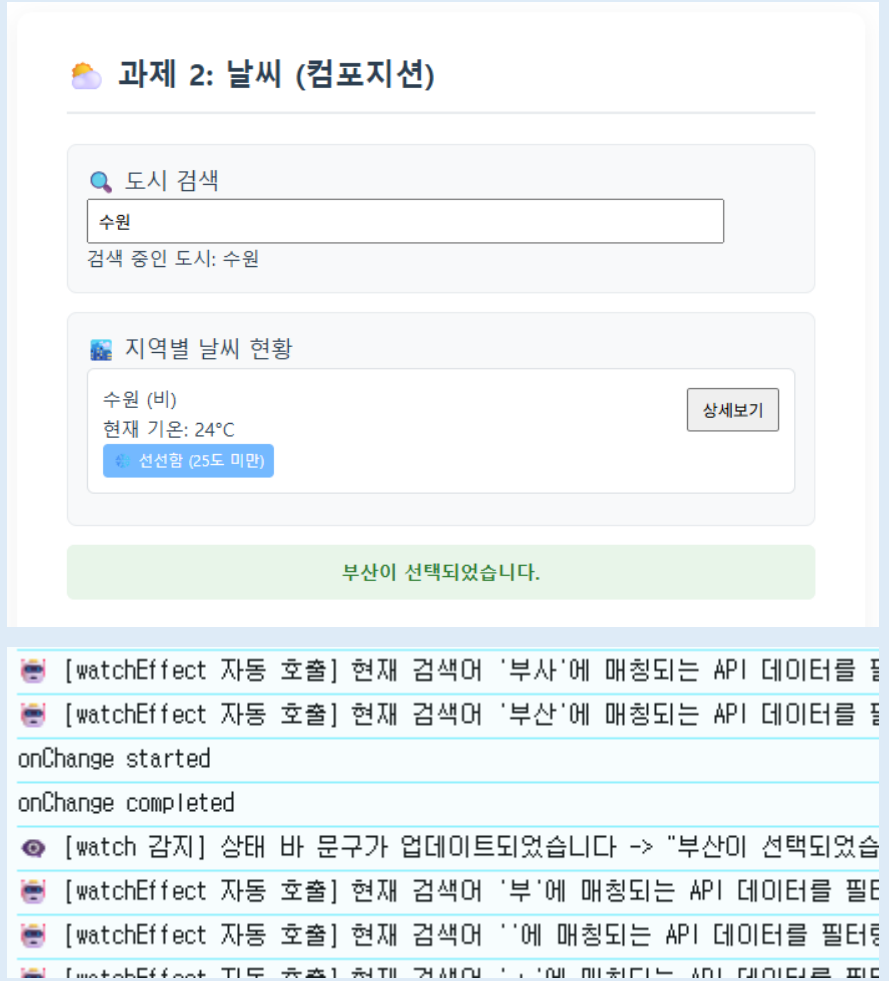
Code Challenge

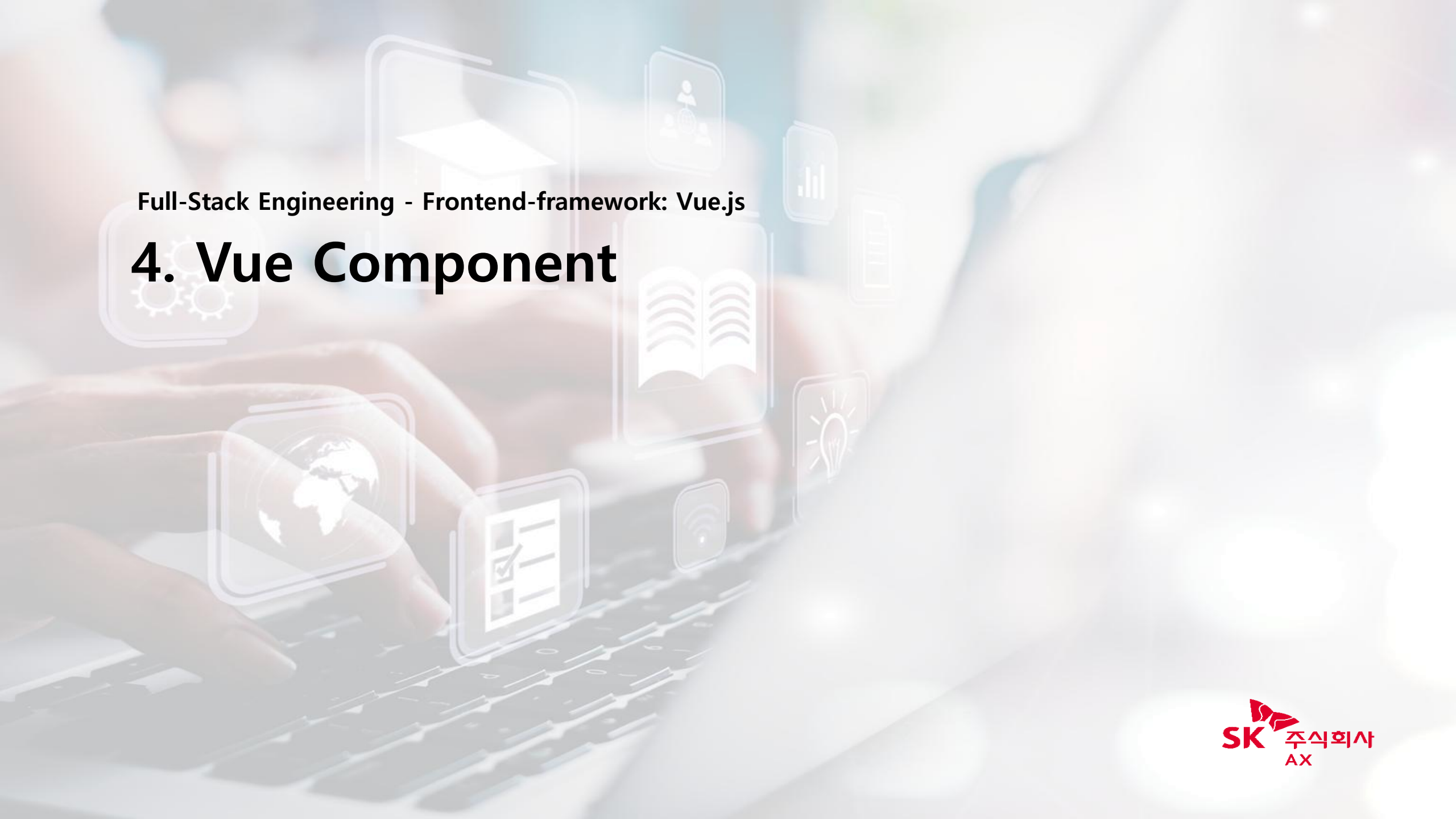
- Computed & Watchers
  - computed() Example
  - watch() Example
  - watch() (Multi-Source Watch) Example
  - watch() (Deep Watch) Example
  - watch() (reactive 반응형 데이터) Example
  - watchEffect() Example

[실습] 과제 - 날씨 (컴포지션)

- 과제 요구사항

1. 반응형 상태 관리: 검색어(searchQuery), 선택된 도시(selectedCityInfo), 그리고 지역별 날씨 데이터 배열(weatherList)을 반응형 상태로 정의. (1일차 동일)
2. 검색 도시 (computed 활용): 전체 날씨 리스트 중에서 사용자가 입력한 검색어가 도시 이름에 포함된 항목만 필터링하여 Computed 배열에 담아 놓는다. (filteredWeatherList)
3. 반응형 변수 변화 감시 (watch, watchEffect):
   - selectedCityInfo 감시 (watch 이용): 상태바 문구가 바뀔때 마다 콘솔로그를 작성
   - searchQuery 감시 (watchEffect 이용):
     도시 검색어를 타이핑할 때 마다 변하는 searchQuery를 추적하여 콘솔로그로 작성
4. 검색 결과 표시 (Template 영역)
   - 검색어가 비었을 때는 원본 데이트를 출력
   - 검색어와 일치하는 데이터가 있을 때는 해당 데이터 출력
   - 검색어와 일치하는 데이터가 없으면 검색 결과가 일치하는 도시가 없다고 안내





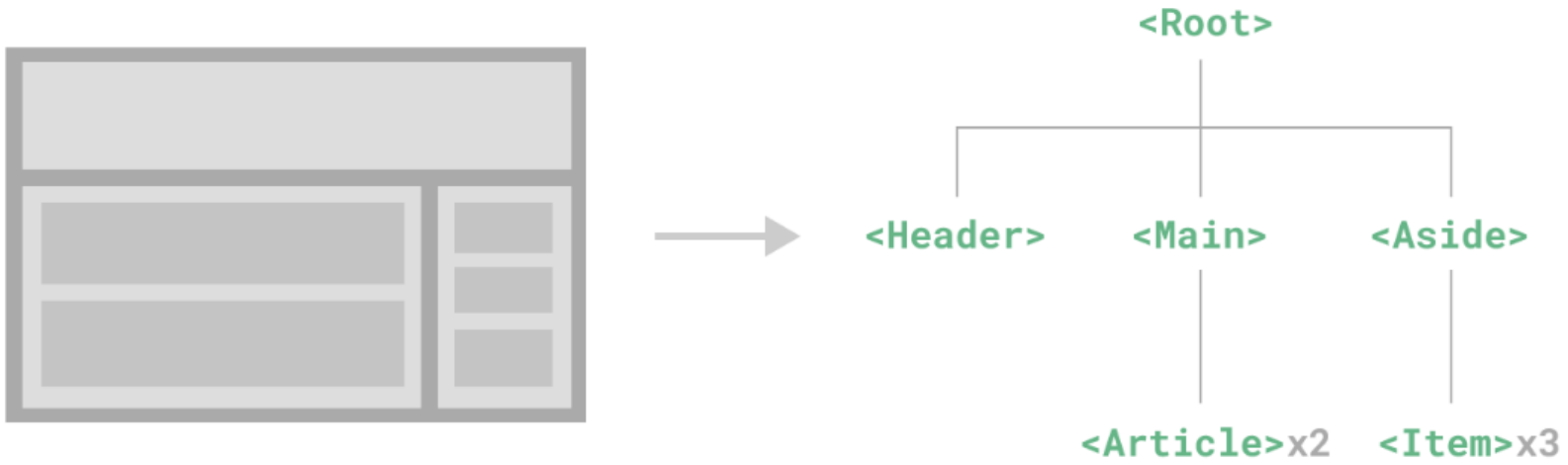
Full-Stack Engineering - Frontend-framework: Vue.js

4. Vue Component

SK 주식회사 AX

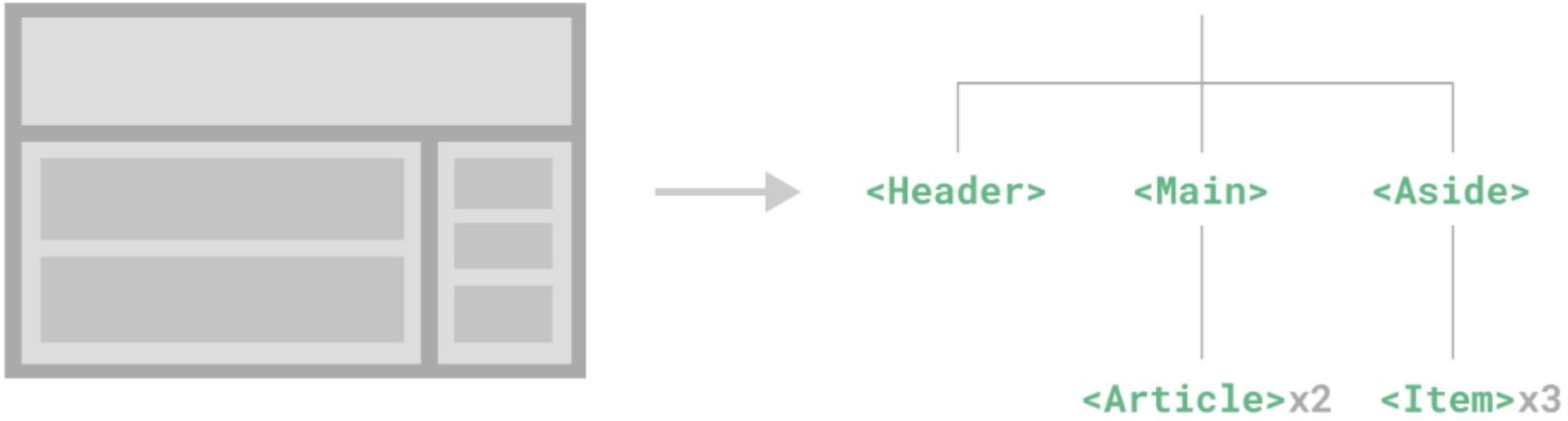
Component Overview

- Component란?
  - Software공학에서 Component는 "독립적인 기능을 수행하며, 언제든지 다른 부품으로 교체할 수 있고, 다른 프로그램과 연결할 수 있는 '표준화된 소프트웨어 모듈(부품)'을 뜻한다.
  - 가장 중요한 핵심은 독립성(Independency)과 교체 가능성(Replaceability)이다.
- Vue Component란?
  - 웹페이지를 구성하는 독립적이고 재사용이 가능한 '블록(부품)'을 말한다.
  - Vue에서는 HTML, CSS, JavaScript를 하나의 .vue 파일 안에 뭉쳐 놓은 하나의 화면 단위를 뜻한다(SFC).
  - Vue에서는 어플리케이션 하나를 완성하기 위해 여러 개의 컴포넌트를 사용한다.
  - 이 때, Component 일정한 형식을 가지고 내부적으로 Tree 구조로 연결된다.



Component Overview - Hierarchy

- Parent-Child (부모-자식 관계)
  - 다른 컴포넌트를 품고 있는 상위 블록이 **부모**, 그 안에 박혀서 작동하는 하위 블록이 **자식**이 된다.
  - 부모와 자식은 철저하게 독립되어 있어 자식은 부모의 변수를 가져다 쓸 수 없고, 부모 역시 자식의 내부 방을 들여다볼 수 없다.
- Sibling (형제 관계)
  - 동일한 부모 컴포넌트 아래에 나란히 조립된 자식 컴포넌트들끼리의 관계.
  - 형제끼리는 다이렉트로 대화하는 선이 없다. 형제에게 말을 걸고 싶다면, 반드시 부모를 거쳐서 올라갔다가 내려와야 한다
- Ancestors-Descendants (조상-후손 관계)
  - 컴포넌트가 거대해져서 자식의 자식, 그 자식의 자식까지 내려가는 다중 계층 구조를 말한다.



Component Overview - Component Local Registration

- Component 지역(Local) 등록
  - 부모 Component가 자식 Component를 import 하여 사용
  - 등록된 자식 컴포넌트는 <template> 영역에서 내장 태그처럼 쓸 수 있다.
  - 등록된 자식 컴포넌트는 PascalCase, kebab-case 스타일로 사용할 수 있다.

```
<script setup>
import BaseButton from './components/BaseButton.vue'
</script>

<template>
  <div class="box">
    <h3>컴포넌트 조립 테스트</h3>
    <hr />
    <BaseButton />
    <base-button></base-button>
  </div>
</template>
```

Component Overview - Component Global Registration

- Component 전역(Global) 등록
  - 전역 등록된 Component는 뷰 어플리케이션 모든 곳에서 별도 절차 없이 사용할 수 있다.
  - 전역 등록은 main.js파일에서 한다.

```
import { createApp } from 'vue'
import App from './App.vue'

import BaseButton from './components/BaseButton.vue'
import BaseInput from './components/BaseInput.vue'

const app = createApp(App)

// app.component()를 사용해 등록, 인자 (<template>에서 호출할 태그 이름, 위에서 import 한 컴포넌트 변수명)
App.component('BaseButton', BaseButton)
App.component('BaseInput', BaseInput)

// 팁: 아래처럼 체이닝(Chaining) 형태로 줄여 쓸 수도 있다.
// app.component('BaseButton', BaseButton).component('BaseInput', BaseInput)

app.mount('#app')
```

Component Overview - Vue 3 내장함수

- Vue 프레임워크에서 제공하는 핵심 기능을 수행하는 함수들

카테고리	주요 함수
애플리케이션 (Application)	createApp, createSSRApp, app.*(), app.config.*
반응형 상태 관리 (Reactive State)	**ref, reactive,** readonly, , shallowRef, shallowReactive, shallowReadonly, toRef, toRefs, customRef, unref, toRaw, markRaw, isRef, isReactive, isReadonly
계산 및 감시 (Computed & Watchers)	**computed, watch, watchEffect**
라이프사이클 훅 (Lifecycle Hooks)	**setup**, **onMounted**, **onUpdated**, **onUnmounted**, onBeforeMount, onBeforeUpdate, onBeforeUnmount, onActivated, onDeactivated, onErrorCaptured, onRenderTracked, onRenderTriggered
컴포넌트 구성/메타 (Component Composition)	defineComponent, **defineProps**, **defineEmits**, useAttrs, defineExpose, useSlots, withDefaults, getCurrentInstance
렌더링 제어 (Rendering & DOM)	h, resolveComponent, withDirectives, renderList, renderSlot, mergeProps, nextTick, useCssModule, useCssVars
의존성 주입 (Dependency Injection)	**provide**, **inject**, hasInjectionContext

Component Lifecycle

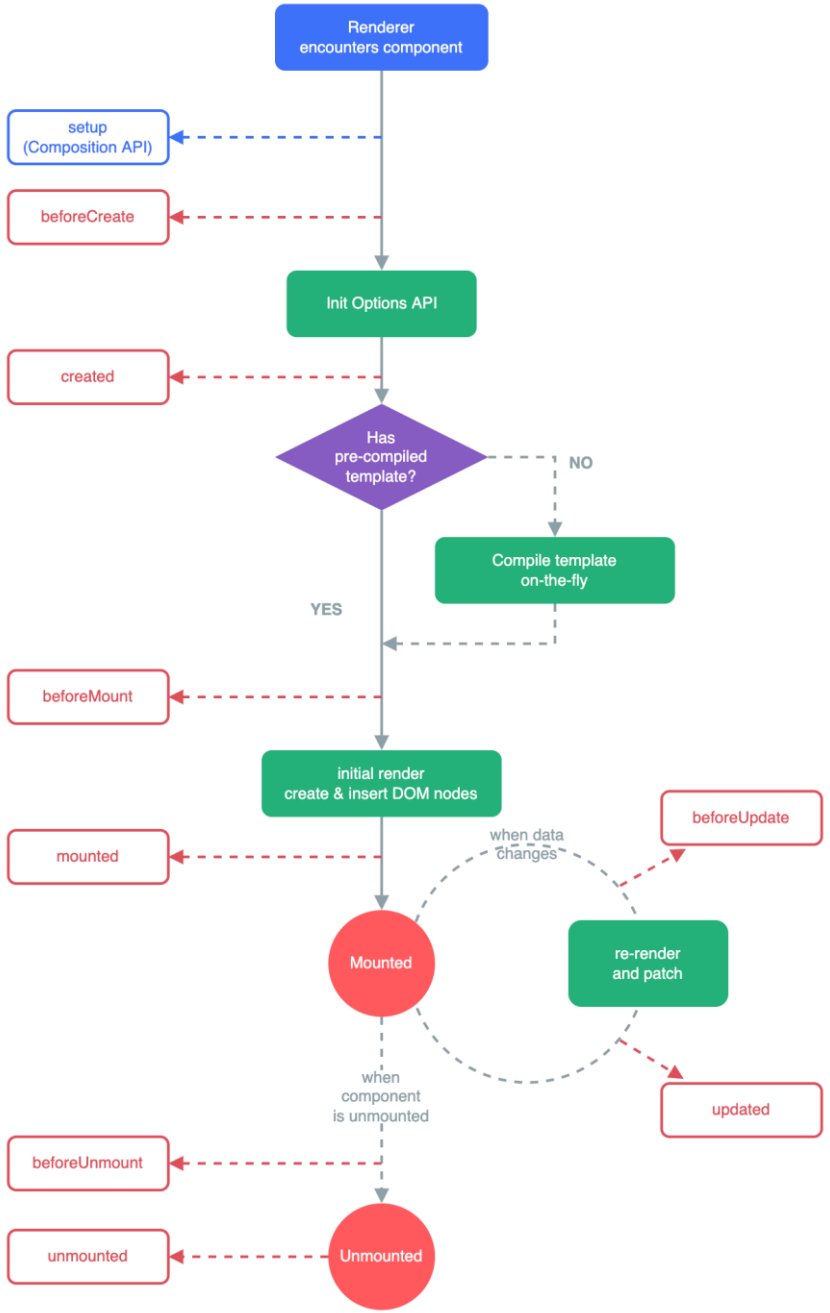
- Component Lifecycle은 컴포넌트가 생성되고 파괴되기까지의 여러 단계를 의미한다.

라이프사이클 단계	컴포넌트의 현재 상태	상세 내용
1. 생성 (Creation)	컴포넌트가 메모리상에 막 태어난 단계 (JavaScript 상에만 존재하고, 아직 HTML 에는 안 붙음)	데이터(ref, reactive)와 computed, watch 센서들이 초기화되어 가동을 시작함.
2. 부착 (Mounting)	가상으로 만들던 화면을 진짜 브라우저 HTML (DOM)에 붙인 단계	화면 요소(div, button 등)에 직접 접근할 수 있게 되며, **백엔드 API를 호출하여 초기 데이터를 받아오기에 가장 완벽한 타이밍.**
3. 갱신 (Updating)	반응형 데이터(ref 등)가 바뀌어서 화면을 싹 뜯어고치고 다시 그리는(Re-rendering) 단계	데이터가 변했을 때 화면이 새로 그려진 직후, 바뀐 HTML 요소들의 크기나 스크롤 위치를 다시 계산할 때 씀.
4. 소멸 (Unmounting)	v-if="false" 등으로 인해 컴포넌트가 화면에서 완전히 지워지고 파괴되는 단계	메모리 누수를 막기 위해 사용 중이던 실시간 타이머(setInterval)를 끄거나, 전역 이벤트 리스너를 청소(Clean-up)함.

Component Lifecycle - Lifecycle Hooks

- Component Lifecycle Hook
  - 컴포넌트의 생명주기 각 단계에 맞춰 Vue 엔진이 자동으로 실행해 주는 콜백 함수

단계	훅 함수	설명
생성 전	setup()	컴포넌트 생성 전 가장 먼저 실행. reactive 변수 및 함수 초기화
마운트 전	onBeforeMount()	DOM에 마운트되기 직전에 호출
마운트 완료	onMounted()	DOM에 마운트된 후 호출. 초기 DOM 조작, API 호출 등에 사용
업데이트 전	onBeforeUpdate()	반응형 데이터가 변경되어 DOM이 다시 업데이트되기 전
업데이트 후	onUpdated()	DOM 업데이트가 완료된 후 호출
언마운트 전	onBeforeUnmount()	컴포넌트가 DOM에서 제거되기 직전에 호출
언마운트 완료	onUnmounted()	컴포넌트가 DOM에서 제거된 후 호출
에러 처리 (선택)	onErrorCaptured()	자식 컴포넌트에서 에러가 발생했을 때 캡처
렌더 트리 캐시 복구용	onActivated()	<KeepAlive> 안에서 재활성화될 때 호출
렌더 트리 캐시 비활성화	onDeactivated()	<KeepAlive> 안에서 비활성화될 때 호출



Component Lifecycle - Lifecycle Hook Example

- Component Lifecycle Hook Example




```vue
<script setup>
import { ref, onMounted, onUpdated, onUnmounted } from 'vue'

const count = ref(0)
let timerId = null // 실시간 타이머 메모리 주소를 담을 변수

// 생성 (Creation) 단계 = <script setup> 본문 그 자체
console.log('1. [setup] 컴포넌트가 메모리에 생성되었습니다. (DOM 접근 불가능)')

// 부착 (Mounting) 단계
onMounted(() => {
  console.log('2. [onMounted] 화면에 완벽히 부착되었습니다! (API 호출/DOM 조작 적기)')
  // 🔥 실무 활용 시뮬레이션: 3초마다 숫자가 자동으로 올라가는 타이머 가동
  timerId = setInterval(() => {
    count.value++
  }, 3000)
})

// 갱신 (Updating) 단계 - count 변수가 바뀌어서 화면이 리렌더링(새로고침)될 때마다 매번 실행된다.
onUpdated(() => {
  console.log(`3. [onUpdated] 데이터가 변경되어 화면을 새로 그렸습니다. (현재 count: ${count.value})`)
})

// 소멸 (Unmounting) 단계 - v-if="false" 등으로 이 컴포넌트가 화면에서 완전히 파괴되어 사라질 때 실행된다.
onUnmounted(() => {
  // ❌ 주의: 여기서 타이머를 안 꺼주면 컴포넌트가 사라져도 백그라운드에서 영원히 타이머가 돕니다! (메모리 누수)
  clearInterval(timerId)
  console.log('4. [onUnmounted] 컴포넌트가 소멸했습니다. 타이머 청소 완료!')
})
</script>
```

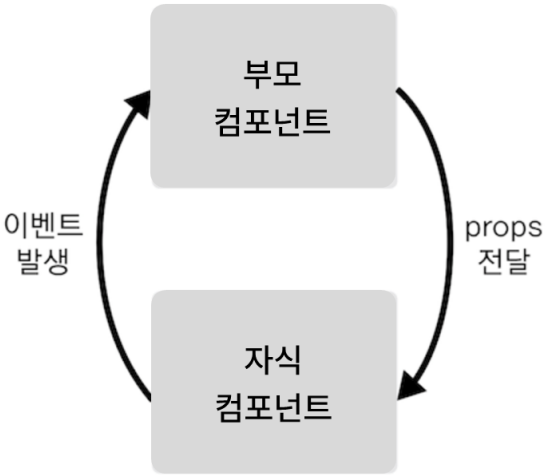
Code Challenge

- Component Lifecycle
  - Lifecycle Hook Example

Props & Emits

- Component 연동
  - Vue 3의 모든 컴포넌트 연동은 "데이터는 위에서 아래로 물려주고, 이벤트는 아래에서 위로 쏘아 올린다"는 구조를 따른다.

분류	▽ Props (하행선)	△ Emits (상행선)
개념 정의	부모가 자식에게 주는 **반응형 데이터 값**	자식이 부모에게 보고하는 **이벤트**
흐름 방향	**부모 → 자식** (위에서 아래로)	**자식 → 부모** (아래에서 위로)
데이터 권한	**읽기 전용.** 자식은 수정 불가	부모에게 변경 **요청 및 값 전달** 가능
Compiler Macro	defineProps({ ... })	defineEmits([...])
Parent Binding	자식 태그 속성에 콜론(:)으로 주입	자식 태그 이벤트에 골뱅이(@)로 청취



  - Compiler Macro란 Runtime 시점이 아닌 Build 시점에 Vue Compiler가 코드를 변환하는 특수 예약어로 defineProps(), defineEmits(), defineExpose() 같은 함수들은 <script setup>에서만 사용이 가능하다.

Props & Emits - defineProps()

- defineProps() - 속성 정의하기
  - 자식 컴포넌트 내부에서 "부모가 넘겨줄 데이터(속성)의 이름과 규격"을 선언하는 Vue 3 내장 컴파일러 매크로 함수
  - Compiler Macro이기 때문에 상단에 import할 필요 없이 <script setup> 안에서 즉시 호출할 수 있다
  - 간단한 배열 표기법과, 강력한 객체 표기법이 있다.

정의 형식	예시 유형	기본값 지정	타입 유효성
배열 형식	['name', 'age']	✕	✕
객체 형식	{ name: String, age: Number }	○ (default 속성)	○

[배열 형식]

```
const props = defineProps(['title', 'count'])
```

[객체 형식 - 기본값 지정]

```
defineProps({
  // 1. 타입만 간단히 지정하는 경우
  title: String,

  // 2. 필수 값과 기본값까지 꼼꼼하게 지정하는 경우
  likes: {
    type: Number,
    required: true // 부모가 이 값을 안 넘기면 에러발생.
  },
  status: {
    type: String,
    default: '대기 중' // 부모가 값을 안 주면 이 값이 기본으로 세팅.
  }
})
```

Props & Emits - defineProps()

- <template> 에서 사용하기
  - 정의된 변수를 그대로 사용하면 된다.

```
<template>
  <h1>{{ title }}</h1>
  <p>좋아요: {{ likes }}</p>
</template>
```

- <script setup> 에서 사용하기
  - defineProps가 반환하는 객체를 변수(보통 props라는 이름)에 받아서 .점 문법으로 접근해야 한다.

```
<script setup>
// 변수에 결과를 할당합니다.
const props = defineProps({
  title: String,
  likes: Number
})

// 내부 함수에서 쓸 때는 props.을 앞에 꼭 붙여야 합니다.
const checkPopularity = () => {
  if (props.likes > 100) {
    console.log(`${props.title}은 인기 게시글입니다.`)
  }
}
</script>
```

Props & Emits - defineProps()

- Readonly
  - defineProps로 Parent에서 전달된 값은 읽기 전용(ReadOnly)이다.
  - Child Component에서 이 값을 직접 바꾸려고 하면 에러가 발생된다.

```
const props = defineProps(['likes'])

const brokenFunction = () => {
  // ❌ 절대 금지! 콘솔에 ReadOnly 에러 발생.
  props.likes = 999
}
```

Props & Emits - defineProps()

- 부모 컴포넌트에서 자식 컴포넌트로 데이터 전달

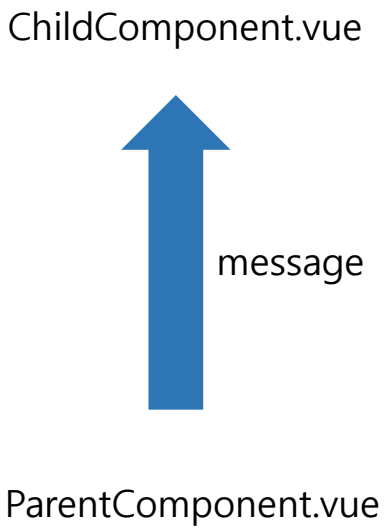
```
<script setup>
const props = defineProps({
    message: String
});
</script>

<template>
    <div>{{ message }}</div>
</template>
```

```
<script setup>
import { ref } from 'vue';
import ChildComponent from './ChildComponent.vue';

const parentMessage = ref('안녕하세요, 자식 컴포넌트!');
</script>

<template>
    <ChildComponent :message="parentMessage" />
</template>
```



Props & Emits - defineProps()

- Component의 Data는 camelCase로 Component의 속성은 kebab-case로 작성한다.
  - 이유: 원래 HTML 표준 마크업 언어는 대소문자를 구분하지 못하고 전부 소문자로 인식하기 때문.
  - Vue 엔진이 JavaScript의 camelCase를 HTML의 kebab-case로 자동 매핑해 준다.

[Parent Component]

```
<script setup>
import { ref } from 'vue'
// 컴포넌트 파일 가져올 때는 파스칼 케이스(PascalCase)
import WeatherCard from './components/WeatherCard.vue'

// 내부 반응형 데이터는 카멜 케이스(camelCase)
const selectCityName = ref('수원시 영통구')
const areaCode = ref(103)
</script>

<template>
  <div>
    <WeatherCard
      :city-name="selectCityName"
      :area-code="areaCode"
    />
  </div>
</template>
```

[Child Component]

```
<script setup>
// defineProps 내부 규격 선언은 카멜 케이스(camelCase)
defineProps({
  cityName: String,
  areaCode: Number
})
</script>

<template>
  <div class="card">
    <p>지역명: {{ cityName }}</p>
    <p>지역 코드: {{ areaCode }}</p>
  </div>
</template>
```

Props & Emits - defineProps()

- Prop Validation (유효성 검사)
  - 부모가 넘긴 데이터에 대한 유효성 검사를 통해 에러 추적의 용이성을 높여 견고한 컴포넌트를 만들 수 있다.

```
defineProps({
  // 1. 가장 기본: 타입(Type)만 단독 검사
  cityName: String,

  // 2. 다중 타입 허용: 문자열 혹은 숫자 둘 다 괜찮을 때
  areaId: [String, Number],

  // 3. 필수 여부(Required): 부모가 무조건 넘겨야 하는 필수 데이터일 때
  temperature: {
    type: Number,
    required: true // 안 넘기면 콘솔에 [Vue warn]: Missing required prop 경고가 뜹니다.
  },

  // 4. 기본값(Default): 부모가 깜빡하고 안 넘겼을 때 채워줄 디폴트 값
  status: {
    type: String,
    default: '맑음' // 부모가 값을 생략하면 '맑음'으로 자동 세팅
  },

  // 5. 커스텀 유효성 검사기(Validator): 내 입맛대로 세부 조건 필터링
  score: {
    type: Number,
    validator(value) {
      // 값이 0부터 100 사이일 때만 합격(true)
      return value >= 0 && value <= 100
    }
  }
})
```

Props & Emits - defineProps()

- Vue Props 지원 자료형(Type) 종류

유형	defineProps 기입 법	실제 넘어오는 데이터 예시	수강생들이 기억할 특징
문자열	String	"서울", "25°C", "Sunny"	따옴표로 감싸진 모든 텍스트
숫자	Number	25, -5, 3.14	소수점, 음수 포함 모든 숫자
논리형	Boolean	true, false	
배열	Array	['서울', '수원', '부산']	대괄호로 묶인 데이터 목록. default 지정 시 반드시 함수 형태() => []로 작성
객체	Object	{ name: '수원', temp: 24 }	키-값 쌍으로 이루어진 데이터. default 지정 시 반드시 함수 형태() => ({})로 작성
함수	Function	() => console.log('클릭')	자식 컴포넌트가 실행할 콜백 함수 자체를 통째로 넘겨줄 때 씁니다. (사용빈도 낮음)
기타	Date, Symbol, Promise	new Date() 등	특정 날짜 객체나 비동기 객체 검증도 지원

Props & Emits - defineProps()

- Vue Props 지원 자료형(Type) Example

```
defineProps({
  // 1. 문자열 (String)
  cityName: String,

  // 2. 숫자 (Number)
  temperature: Number,

  // 3. 논리형 (Boolean)
  isActive: {
    type: Boolean,
    default: false
  },

  // 4. 배열 (Array)
  weeklyForecast: {
    type: Array,
    // 중요: 배열의 기본값은 무조건 '새 바구니를 구워내는 화살표 함수' 형태로!
    default: () => []
  },

  // 5. 객체 (Object)
  coordinates: {
    type: Object,
    // 중요: 객체의 기본값도 무조건 화살표 함수 형태로 반환!
    default: () => ({ lat: 37.5, lng: 126.9 })
  }
})
```

Props & Emits - defineEmits()

- defineEmits() - 이벤트 정의하기
  - 자식 컴포넌트에서 부모에게 사용자 정의 이벤트를 전달하기 위해 사용하는 Vue 3 내장 컴파일러 매크로 함수
  - Compiler Macro이기 때문에 상단에 import할 필요 없이 <script setup> 안에서 즉시 호출할 수 있다
  - 자식 컴포넌트 내부의 브라우저 표준 이벤트(클릭, 키보드 입력 등) 또는 내부 로직 변화를 트리거로 삼아, 부모 컴포넌트가 등록한 Custom Event Listener를 호출하여 콜백 함수를 가동한다.
  - 데이터 전달(Payload): 자식 컴포넌트는 emit() 함수의 첫 번째 인자로 이벤트 식별 문자열을, 두 번째 인자부터는 부모 컴포넌트의 콜백 함수로 인계할 Payload(데이터)를 Argument로 바인딩하여 전달한다.

Props & Emits - defineEmits()

- defineEmits() Example 1

```
<script setup>
const emit = defineEmits(['childEvent']);

const sendToParent = () => {
    emit('childEvent', '안녕하세요, 부모 컴포넌트!');
};
</script>

<template>
    <button @click="sendToParent">부모에게 메시지 보내기</button>
</template>
```

```
<script setup>
import { ref } from 'vue';
import ChildComponent from './ChildComponent.vue';

const handleChildEvent = (message) => {
    console.log('자식으로부터 받은 메시지:', message);
};
</script>

<template>
    <ChildComponent @childEvent="handleChildEvent" />
</template>
```

ChildComponent.vue

- defineEmits로 이벤트타입 등록
- emit(‘이벤트타입’,보낼데이터)으로 이벤트 발생



childEvent

ParentComponent.vue

- @이벤트타입=“이벤트핸들러”로 수신
- 이벤트핸들러에서 데이터처리

Props & Emits - defineEmits()

- defineEmits() Example 2

```
<script setup>
// 부모에게 물려받을 Props
defineProps({
  cityName: String,
  status: String
})

// 부모에게 전달할 커스텀 이벤트 타입을 배열로 등록.
// (관례상 변수명은 'emit'이라고 똑같이 지어준다
const emit = defineEmits(['select-city'])

// 카드가 클릭되었을 때 실행될 함수
const handleCardClick = (name) => {
  // 부모에게 'select-city' 이벤트 발생하면서, 선택된 도시 이름 전달
  emit('select-city', name)
}
</script>

<template>
  <div class="weather-card" @click="handleCardClick(cityName)">
    <h4>{{ cityName }} ({{ status }})</h4>
    <p>클릭하면 부모에게 신호를 보냅니다.</p>
  </div>
</template>
```

- 이벤트 타입명은 kebab-case로 작성

```
<script setup>
import { ref } from 'vue'
import WeatherCard from './components/WeatherCard.vue'

const selectedCityInfo = ref('카드를 클릭해 보세요.')

// 자식이벤트 발생 시 이벤트 핸들러
const receiveCitySignal = (cityName) => {
  selectedCityInfo.value = `${cityName}이(가) 성공적으로 선택되었습니다.`
}
</script>

<template>
  <div class="parent-box">
    <h2>📡 부모 관제탑 (Emits 수신 패널)</h2>
    <hr />

    <WeatherCard cityName="서울" status="맑음" @select-city="receiveCitySignal"/>
    <WeatherCard cityName="수원" status="비" @select-city="receiveCitySignal"/>

    <div class="status-bar">
      {{ selectedCityInfo }}
    </div>
  </div>
</template>
```

Props & Emits - Props & Emits Example

- Props & Emits Example

[PropsEmitsParent.vue]

```vue
<script setup>
import { ref } from 'vue'
import PropsEmitsChild from './PropsEmitsChild.vue'

// 1. 상위 컴포넌트의 로컬 반응형 상태 정의
const message = ref('Parent 초기 메시지')

// 2. 하위 컴포넌트의 커스텀 이벤트를 수신했을 때 실행될 핸들러 함수
// 인자(newValue)로 하위 컴포넌트가 보낸 페이로드가 자동 주입됩니다.
const handleUpdateRequest = (newValue) => {
  message.value = newValue
}
</script>

<template>
  <div class="practice-section">
    <h2>Props & Emits</h2>
    <div class="parent-container">
      <h2>상위 컴포넌트 (Parent)</h2>
      <p>
        현재 로컬 데이터(State): <strong>{{ message }}</strong>
      </p>
      <br />
      <PropsEmitsChild :parent-data="message" @update-
request="handleUpdateRequest" />
    </div>
  </div>
</template>
```

[PropsEmitsChild.vue]

```vue
<script setup>
// 1. 상위 컴포넌트로부터 주입받을 데이터의 자료형 및 필수 여부 정의
defineProps({
  parentData: {
    type: String,
    required: true,
  },
})

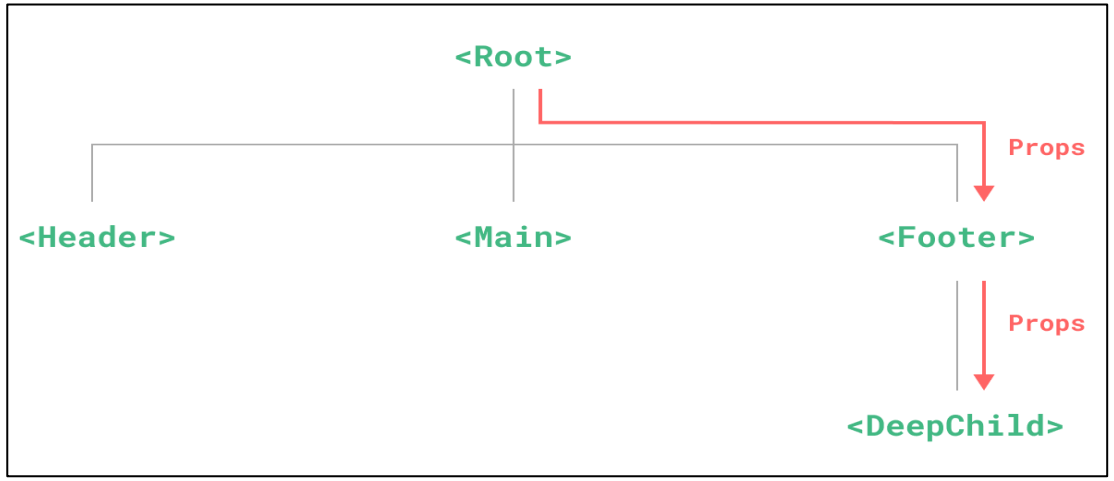
// 2. 상위 컴포넌트로 송신할 커스텀 이벤트 식별자 등록
const emit = defineEmits(['update-request'])

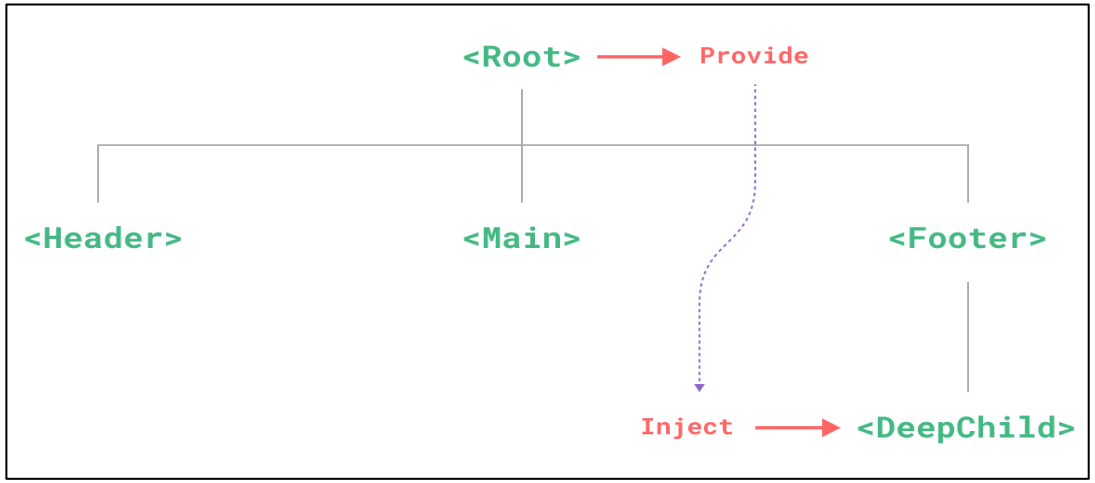
// 3. 내부 이벤트 발생 시 페이로드를 실어 상위로 이벤트를 디스패치하는 함수
const sendNotification = () => {
  const payload = 'Child에서 가공한 새로운 데이터'
  emit('update-request', payload)
}
</script>

<template>
  <div class="child-container">
    <h2>하위 컴포넌트 (Child)</h2>
    <p>
      수신된 Props 데이터: <strong>{{ parentData }}</strong>
    </p>
    <br />
    <button @click="sendNotification">상위 컴포넌트로 갱신 요청 (Emit)</button>
  </div>
</template>
```

Provide & Inject

- Provide / Inject 개념
  - Props Drilling 현상: 컴포넌트 아키텍처의 계층이 깊어질 때(예: 상위 → 하위 → 최하위), 중간에 위치한 컴포넌트들은 해당 데이터가 필요 없음에도 오직 최하위 컴포넌트로 전달하기 위해 Props를 받아 아래로 토스하는 과정을 반복해야 하는 현상
  - 중간 계층의 컴포넌트들을 완전히 건너뛰고, 조상 컴포넌트가 선언한 반응형 상태를 하위 컴포넌트에서 inject 함수를 통해 다이렉트로 결합하여 사용하는 방식
  - 중첩된 컴포넌트 계층에서도 중간 컴포넌트가 해당 값을 몰라도 전달 가능
  - [참고] 전역 상태 관리 라이브러리 (Pinia)로 인해 Provide& Inject 사용빈도는 높지 않다.





Provide & Inject - 핵심 코드

- 핵심 코드

[GrandParent.vue]

```
import { ref, provide } from 'vue'
const themeColor = ref('dark-mode')

// 주입할 키(Key) 이름과 실제 데이터(Value)를 등록
provide('globalTheme', themeColor)
```

[GrandChild.vue]

```
import { inject } from 'vue'

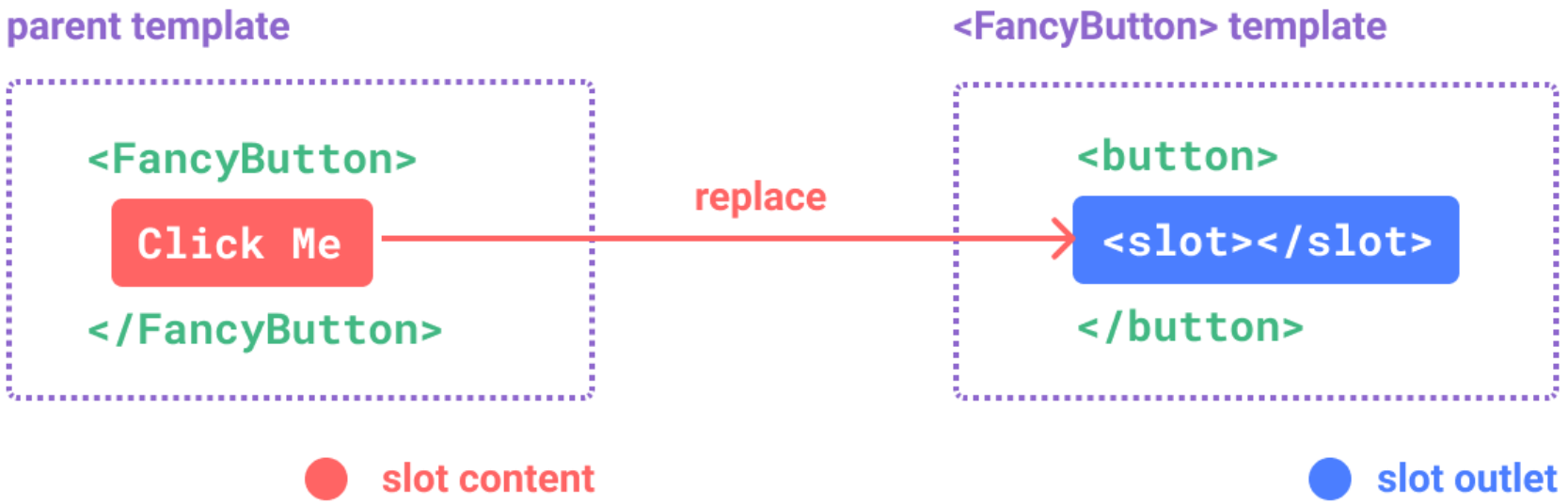
// 상위 조상이 provide한 키 이름을 지정하여 직접 인젝션 유도
const theme = inject('globalTheme')
```

Code Challenge

- Props & Emits
  - Props & Emits Example

Component Slot

- Component Slot
  - 자식 컴포넌트의 특정 구역을 비워두고, 부모 컴포넌트가 자식 컴포넌트를 호출할 때 내장할 HTML 마크업 및 템플릿 콘텐츠를 동적으로 주입 받아 렌더링할 수 있도록 해주는 기능.
  - Props가 부모가 자식에게 데이터를 주입한다면, Slot은 HTML 마크업 태그나 디자인 레이아웃 자체를 주입한다.
- Slot 유형
  - Default Slot: 자식 컴포넌트에서 slot 태그를 사용하여 부모 컴포넌트로부터 전달받은 콘텐츠를 표시
  - Named Slot: 여러 슬롯이 필요한 경우 v-slot:name 또는 #name을 사용해 위치에 지정
  - Scoped Slot: 자식 컴포넌트에서 부모 컴포넌트로 데이터를 전달할 때 사용(v-slot 바인딩)



Component Slot - Default Slot

- Default Slot (= 이름없는 Slot)
  - 별다른 속성이 없이 단순하게 <slot> 태그만 사용하는 Slot

[SlotDefaultParent.vue]

```vue
<script setup>
import SlotDefaultChild from '@/components/practices/component/SlotDefaultChild.vue'
</script>

<template>
  <div class="practice-section">
    <h2>Default Slot 레이아웃 주입 실습</h2>
    <SlotDefaultChild>
      <p>단순한 텍스트 문장을 주입합니다.</p>
    </SlotDefaultChild>
    <SlotDefaultChild>
      <h2 style="color: #e74c3c">🔥 경고 상태</h2>
      <button>확인</button>
    </SlotDefaultChild>
    <SlotDefaultChild> </SlotDefaultChild>
  </div>
</template>
```

[SlotDefaultChild.vue]

```vue
<template>
  <div class="base-card">
    <slot>
      <p>기본 콘텐츠 영역입니다.</p>
    </slot>
  </div>
</template>
```

- 만약 부모로부터 Slot에 교체할 템플릿 조각이 넘어오지 않으면 <slot></slot> 내 Contents가 기본 값이 된다.



Component Slot - Named Slot

- Named Slot (이름있는 Slot)
  - 자식 Component 내에서 여러 개의 Slot을 사용할 때 <slot> 태그에 name 속성으로 이름을 지정 <slot name="값"></slot>
  - 부모 Component 내에서는 v-slot directive를 지정해서 사용 <template v-slot:값>*Code*</template>

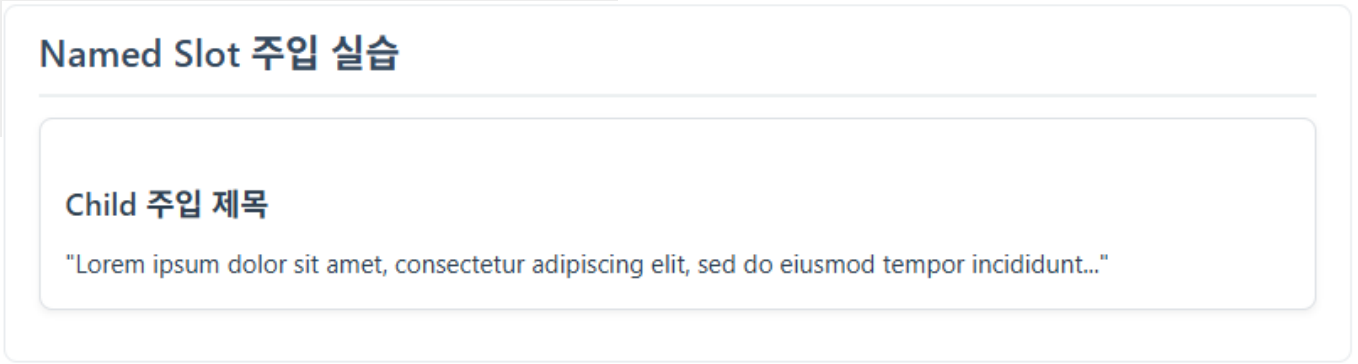
[SlotNamedParent.vue]

```
<script setup>
import SlotNamedChild from './SlotNamedChild.vue'
</script>

<template>
  <div class="practice-section">
    <h2>Named Slot 주입 실습</h2>
    <SlotNamedChild>
      <template v-slot:header>
        <h3>Child 주입 제목</h3>
      </template>
      <p>"Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor
incididunt..."</p>
    </SlotNamedChild>
  </div>
</template>
```

[SlotNamedChild.vue]

```
<template>
  <div class="base-card">
    <header>
      <slot name="header"></slot>
    </header>
    <main>
      <slot></slot>
    </main>
  </div>
</template>
```



Component Slot - Scoped Slot

- Scoped Slot
  - 자식 컴포넌트에서 부모 컴포넌트로 데이터를 전달할 때 사용한다.
  - 자식 Component 내에서 로컬 데이터를 <slot> 태그의 속성 바인딩(:이름="변수명")을 통해 데이터를 상위로 전달한다.
  - 부모 Component 내에서는 v-slot="변수주머니이름"을 통해 자식이 보낸 데이터를 수신하고, 변수주머니이름에서 변수를 꺼내 HTML 코드안에 배치한다.

[SlotScopedParent.vue]

```vue
<script setup>
import SlotScopedChild from './SlotScopedChild.vue'
</script>

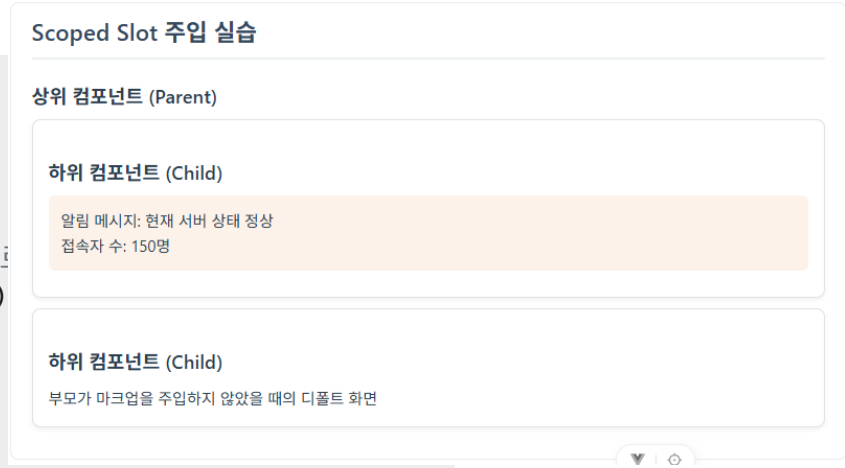
<template>
  <div class="practice-section">
    <h2>Scoped Slot 주입 실습</h2>
    <h3>상위 컴포넌트 (Parent)</h3>
    <SlotScopedChild v-slot="slotBag">
      <div class="display-panel">
        <p>알림 메시지: {{ slotBag.text }}</p>
        <p>접속자 수: {{ slotBag.count }}명</p>
      </div>
    </SlotScopedChild>
    <SlotScopedChild> </SlotScopedChild>
  </div>
</template>
```

[SlotScopedChild.vue]

```vue
<script setup>
import { ref } from 'vue'

// 하위 컴포넌트 내부에서 관리하는 2개의 서
const message = ref('현재 서버 상태 정상')
const userCount = ref(150)
</script>

<template>
  <div class="base-card">
    <h3>하위 컴포넌트 (Child)</h3>
    <slot :text="message" :count="userCount">
      <p>부모가 마크업을 주입하지 않았을 때의 디폴트 화면</p>
    </slot>
  </div>
</template>
```



Code Challenge

- Component Slot
  - Default Slot Example
  - Named Slot Example
  - Scoped Slot Example

[실습] 과제 - 날씨 (컴포넌트)

- 과제 요구사항 : 기능 변경없이 4개의 Component 파일로 분리

1. WeatherParent.vue
   - 모든 반응형 데이터 유지
2. BaseDashboardCard.vue
   - 검색박스와 리스트박스의 디자인을 공통화.
   - <slot> 배치하여 부모 컴포넌트가 도시 검색, 날씨 현황 주입
3. SearchBar.vue
   - 부모로 부터 검색도시 반응형 데이터를 전달받아 표시 (props)
   - 도시 검색 시 update-query 이벤트를 발생하면서 검색어를 부모에게 전달 (emits)
4. WeatheCard.vue
   - 선택된 도시 객체를 전달 받아 표시 (props)
   - 카드를 선택하는 것(select-card 이벤트)과 상세보기(click-detail 이벤트)를 부모에게 전달 (emits)
5. 각 컴포넌트로 분리하면서 Component에 해당되는 디자인은 <style scoped>로 각각 분리
6. [참고] Slot으로 전달되는 자식 컴포넌트(SearchBar, WeatherCard)는 시각적으로는 BaseDashboardCard 내부에 위치하지만, 스크립트적으로는 부모 컴포넌트의 스코프에서 컴파일되고 평가되므로, WeatherParent에서 SearchBar와 WeatherCard와 직접 바인딩/통신이 가능하다.





Full-Stack Engineering - Frontend-framework: Vue.js

5. Vue Router

SK 주식회사 AX

Vue Router Basic

- Vue Router
  - 전통적인 웹사이트는 페이지 이동 시마다 서버에 새 HTML을 요청하여 화면 전체를 새로고침 했다.
  - 반면, Vue는 최초 접속 시 하나의 HTML만 다운로드하는 SPA(Single Page Application) 구조이다.
  - Vue Router는 브라우저의 URL 변화를 JavaScript 엔진이 가로채서, 서버에 새 페이지를 요청하지 않고 현재 주소(Path)에 매칭되는 미리 정의된 컴포넌트만 가상 DOM 상에서 실시간으로 교체해 주는 공식 라이브러리이다.

{} package.json > ...

```
{
  "name": "skala-vue",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "run-s lint:*",
    "lint:oxlint": "oxlint . --fix",
    "lint:eslint": "eslint . --fix --cache",
    "format": "prettier --write --experimental-cli src/"
  },
  "dependencies": {
    "pinia": "^3.0.4",
    "vue": "^3.5.32",
    "vue-router": "^5.0.4"
  },
```

Vue Router Basic - Router Setup Steps

- Step 1 : Vue Router 설정하기 (src/router/index.js)
  - vue-router 패키지에서 제공하는 createRouter()를 사용해 Router configuration object를 생성
  - history: createWebHistory()는 전통적인 웹 어플리케이션에서 사용하는 방식으로 슬래시(/)를 사용해 URL을 관리한다.
    예) /user/profile
  - routes: 배열로 된 routes object를 지정한다.

속성명	값 자료형	역할 및 설정 방법
path	String	필수, 브라우저 URL 경로
component	Component / Function	필수, path에 매핑되는 Vue 컴포넌트
name	String	고유한 식별 이름입니다.
redirect	String / Object	강제 강제 리다이렉션 시킬 경로 지정

src > router > index.js > ...

```
1   import { createRouter, createWebHistory } from 'vue-router'
2   import HomeView from '../views/HomeView.vue' (1)
3
4   const router = createRouter({
5     history: createWebHistory(import.meta.env.BASE_URL),
6     routes: [
7       {
8         path: '/',
9         name: 'home',
10        component: HomeView, (1)
11      },
12      {
13        path: '/about',
14        name: 'about',
15        // route level code-splitting
16        // this generates a separate chunk (About.[hash].js) for this route
17        // which is lazy-loaded when the route is visited.
18        component: () => import('../views/AboutView.vue'), (2)
19      },
20    ],
21  })
22
23  export default router
```

- component 속성 지정 방식
  ① 정적 import (static import) - 어플리케이션 시작 시점에 컴포넌트를 메모리에 로드
  ② 동적 import (dynamic import) - 해당 컴포넌트가 필요한 순간에 로드 (Lazy Loading)

Vue Router Basic - Router Setup Steps

- Step 2 : Vue Router 등록하기 (src/main.js)
  - 생성한 라우터 설정을 Vue 어플리케이션에 등록
  - 등록할 때는 어플리케이션 인스턴스의 use() 메서드를 사용
- src/main.js
  - ① ./router/는 router 폴더의 index.js 파일을 불러오라는 의미
  - ② use()메소드에 전달된 router 설정 객체를 등록

src > JS main.js > ...

```
1   import './assets/main.css'
2
3   import { createApp } from 'vue'
4   import { createPinia } from 'pinia'
5
6   import App from './App.vue'
7   import router from './router'   (1)
8
9   const app = createApp(App)
10
11  app.use(createPinia())
12  app.use(router)   (2)
13
14  app.mount('#app')
```

Vue Router Basic - Router Setup Steps

- Step 3 : Vue Router 사용하기 (<RouterVue>, <RouterLink>)
  - ① <RouterLink to="…"> - 링크를 생성
  - ② <RouterView /> - 경로와 일치하는 컴포넌트를 배치
- HTML <a> 링크 사용금지
  - About는 에러 유발
  - <a> 태그는 브라우저를 강제로 새로고침 시켜 메모리에 들고 있던 모든 반응형 데이터(ref, computed)를 초기화한다.

src > App.vue > { } style scoped > nav a.router-link-exact-active:hover

```vue
<script setup>
import { RouterLink, RouterView } from 'vue-router'
import HelloWorld from './components/HelloWorld.vue'
</script>

<template>
  <header>
    <img alt="Vue logo" class="logo" src="@/assets/logo.

    <div class="wrapper">
      <HelloWorld msg="You did it!" />

      <nav>
        <RouterLink to="/">Home</RouterLink> ②
        <RouterLink to="/about">About</RouterLink>
      </nav>
    </div>
  </header>

  <RouterView /> ①
</template>
```

Vue Router Basic - Summary

- Vue Router 핵심 요소 정리

핵심 요소	기술적 본질	가동되는 위치 (파일 스코프)	런타임 주요 역할 및 핵심 기능
route	JavaScript 객체	각 컴포넌트 내부의 <script setup>	현재 브라우저 주소창에 활성화된 페이지의 상세 정보 목록
router	JavaScript 객체	src/router/index.js, src/main.js	앱 전체의 라우팅 시스템을 총괄
RouterView	Vue 내장 컴포넌트	레이아웃 컴포넌트	현재 URL 주소(route.path)에 매칭된 페이지 컴포넌트가 가상 DOM 상에서 치환되어 출력되는 **동적 렌더링 위치** 역할을 수행
RouterLink	Vue 내장 컴포넌트	메뉴, 내비게이션 바 등 링크 컴포넌트	브라우저의 전면 새로고침(Page Refresh)을 차단하고, URL 주소만 안전하게 변경하여 SPA 내부의 화면 전환을 트리거하는 **전용 내비게이션 태그**입니다.

```
4  const router = createRouter({                          [ROUTER]
5    history: createWebHistory(import.meta.env.BASE_URL),
6    routes: [
7      {                                                  [ROUTE]
8        path: '/',
9        name: 'home',
10       component: HomeView,
11     },
12     {
13       path: '/about',
14       name: 'about',
15       // route level code-splitting
16       // this generates a separate chunk (About.[hash].js) for this route
17       // which is lazy-loaded when the route is visited.
18       component: () => import('../views/AboutView.vue'),
19     },
20   ],
21 })
22
23 export default router
```

```
6  <template>
7    <header>
8      <img alt="Vue logo" class="logo" src="@/assets/logo.
9
10     <div class="wrapper">
11       <HelloWorld msg="You did it!" />
12
13       <nav>
14         <RouterLink to="/">Home</RouterLink>              [RouterLink]
15         <RouterLink to="/about">About</RouterLink>
16       </nav>
17     </div>
18   </header>
19                                                         [RouterView]
20   <RouterView />
21 </template>
```

Vue Router Basic - views

- views 폴더
  - views 폴더에 위치한 컴포넌트들은 <RouterView/> 영역에 직접 Rendering되는 "페이지 단위의 최상위 컴포넌트"이다.
  - routes 배열에서 component 속성에 직접 mapping 되어 URL Path와 1:1로 대응된다. (예, /dashboard → DashboardView.vue)
  - Vue.js 공식 Style Guide 및 커뮤니티 권장사항에 따르면, RouterView에 의해 직접 호출되는 최상위 페이지 컴포넌트에는 접미사로 View를 붙이는 것을 강력히 추천한다.
- views vs components

구분	views 폴더	components 폴더
주요 역할	페이지(화면) 단위 컴포넌트	재사용 가능한 UI/기능 컴포넌트
Router 매핑	RouterView에 직접 매핑됨 (routes에 등록)	RouterView에 직접 매핑되지 않음
재사용성	낮음 (특정 URL 화면만을 위해 독립적으로 존재)	높음 (여러 views나 다른 components에서 반복 사용)
예시 컴포넌트	DashboardView.vue, UserProfileView.vue	AppButton.vue, SearchBar.vue, BaseDashboardCard.vue

useRoute()

- useRoute()
  - Script Setup 환경에서 현재 활성화된 라우트(Active Route) 정보에 접근하기 위한 Composable 함수*
  - URL 경로, 파라미터, 쿼리 스트링, 메타 데이터 등 현재 페이지의 모든 상태 정보를 reactive 객체 형태로 제공
- route 객체의 주요 프로퍼티
  - route.params: 동적 경로 파라미터 객체 (예, /user/:id → { id: '42' })
  - route.query: URL 쿼리 스트링 객체 (예, /search?q=vue → { q: 'vue' })
  - route.path: 현재 요청된 URL의 순수 경로 (예, /user/42)
  - route.name: 해당 라우트 설정에 지정된 고유 이름 (예, UserDetail)
- Code Snippet (View Component)

```
<script setup>
import { useRoute } from 'vue-router'

const route = useRoute() // Active Route Object

console.log('현재 접근한 경로:', route.path)
console.log('전달된 ID 파라미터:', route.params.id)
</script>

<template>
  <!-- template에서 route 객체 프로퍼티 직접 접근 -->
  <p>사용자 ID: {{ route.params.id }}</p>
  <p>검색 키워드: {{ route.query.q }}</p>
</template>
```

- useRoute()로 추출한 route 객체는 반응성을 유지하므로 template 및 script 내부에서 즉시 활용 가능

* Composable: Vue의 반응형 상태(Ref, Reactive)와 로직을 묶어 재사용할 수 있도록 많든 함수

useRoute() - Dynamic Route Matching

- Dynamic Route Matching(동적 경로 매칭) 설정
  - URL의 일부분이 동적으로 변경되는 패스를 다뤄야 할 때 사용
  - 예) 날씨 View에서 각 도시별 상세 페이지 (/weather/seoul, /weather/suwon)를 만들 때, 모든 도시의 Route 객체를 선언하는 비효율을 방지
  - 주소창 뒤에 콜론(:) 식별자를 붙여 변수화(/weather/:cityId)하며 이 부분을 동적 세그먼트(Dynamic Segment)라 한다.

```
// router/index.js
{
  path: '/weather/:cityId',.                                    [동적 세그먼트]
  name: 'WeatherDetail',
  component: WeatherDetail
}
```

- Dynamic Route Matching 수신
  - Component 내부에서는 useRoute()로 route 객체를 수신하여 route.params.cityId로 값을 확인할 수 있다.

```
<script setup>
import { useRoute } from 'vue-router'

const route = useRoute()

console.log(route.path)           // 현재 경로: '/weather/seoul'
console.log(route.params.cityId)  // 동적 파라미터: 'seoul'
</script>
```

useRoute() - Dynamic Route Matching

- Multiple Dynamic Segments (다중 동적 세그먼트)
  - URL 경로 내에 2개 이상의 동적 파라미터를 조합하여 복잡한 계층 구조 표현
  - 예) 카테고리별 상품 상세 페이지 (/category/electronics/product/101)
  - 모든 동적 파라미터는 route.params 객체의 속성으로 각각 매핑 됨

```
// router/index.js
{
  path: '/category/:categoryId/product/:productId', // 💡 :categoryId, :productId 2개 지정
  name: 'ProductDetail',
  component: ProductDetailView
}
```

- Inline Dynamic Segment (중간 위치 동적 세그먼트)
  - 경로의 중간에 콜론(:) 식별자를 배치하여 하위 리소스나 특정 액션 구분
  - 예) 특정 사용자의 게시글 리스트 조회 (/user/42/posts)

```
<script setup>
import { useRoute } from 'vue-router'

const route = useRoute()

console.log(route.path)           // 현재 경로: '/weather/seoul'
console.log(route.params.cityId)  // 동적 파라미터: 'seoul'
</script>
```

useRoute() - Query String Routing

- Query String Routing 설정
  - URL 주소창 뒤에 물음표(?)에 이어 key=value 형태의 쌍으로 붙는 Query String을 Vue Router와 동기화하는 라우팅 기법.
  - 예) /weather?search=수원&page=2
  - 라우터 설정 파일에는 별도의 변수 처리를 명시하지 않아도 자유롭게 확장 가능하다
- Query String 수신
  - Component 내부에서는 useRoute()로 route 객체를 수신하여 route.query.search로 값을 확인할 수 있다.

```
<script setup>
import { useRoute } from 'vue-router'

const route = useRoute()

// 컴포넌트 마운트 시점: 주소창에 ?search=값 이 이미 있다면 해당 값으로 내부 상태 복원
onMounted(() => {
  if (route.query.search) {
      searchQuery.value = route.query.search
  }
})
</script>
```

useRouter()

- useRouter()
  - Script Setup 환경에서 라우터 인스턴스(Router Instance)에 접근하기 위한 Composable 함수*
  - <RouterLink>와 같은 태그 클릭 외에, 자바스크립트 코드(이벤트 핸들러, 비동기 로직 등)로 페이지를 이동할 때 활용 → Programmatic Navigation
- router 객체의 주요 메소드
  - router.push(): 새로운 히스토리 항목을 스택에 추가하며 페이지 이동 (뒤로 가기 가능)
  - router.replace(): 현재 히스토리 항목을 대체하며 페이지 이동 (뒤로 가기 불가)
  - router.go(n): 히스토리 스택에서 n단계만큼 앞/뒤로 이동 (router.go(-1) 은 뒤로 가기)

useRouter() - Programmatic Navigation

- Programmatic Navigation (프로그래밍 방식의 네비게이션)
  - <RouterLink to="…">가 아닌 Script 내부에서 페이지를 전환하는 방법
  - 예) 로그인 성공 후 메인으로 이동, 상세 페이지에서 버튼을 눌러 스크립트 명령어로 페이지를 이동
  - 라우터 인스턴스(router)의 내장 메소드를 직접 호출하여 페이지를 전환하는 기법.
  - router.push()를 호출하면 Vue Router가 URL(경로)을 변경하고, 그 경로에 매핑된 컴포넌트를 <RouterView/>가 위치한 영역에 동적으로 교체(렌더링)해 준다.
- Method Example

Method	Example	설명
.push(path)	**router.push('/about')**	/about
	router.push({ name: 'user', params: { id: 1 } })	/user/1
	router.push({ name: 'search', query: { q: 'vue' } })	/search?q=vue
	router.push({ name: 'user', params: { id: 1 }, query: { tab: 'info' } })	/user/1?tab=info
	router.push({ path: 'details' })	상대 경로: 현재 /user → /user/details
.replace(path)	router.replace('/login'), *router.replace() : 이전 페이지 기록 제거*	현재 히스토리 항목을 대체하며 페이지 이동 (뒤로 가기 불가)
.go(n)	router.go(n)	앞/뒤 이동: 이전 페이지(-1), 다음 페이지(1)
.back()	router.back()	이전 페이지로
.forward()	router.forward()	다음 페이지로

useRouter() - Code Example

- Programmatic Navigation Example

```
<script setup>
import { useRouter } from 'vue-router’

const router = useRouter()

const handleGoHome = () => {
  router.push('/')
}
const handleLoginRedirect = () => {
  router.replace('/')
}
const handleAdvancedMove = () => {
  router.push({
    name: 'WeatherDetail',       // 라우터 설정에 등록된 고유 Name 호출
    params: { cityId: 'city_02' }, // 주소창 :cityId 구역에 변수 매핑
    query: { search: '수원' }      // 주소창 뒤에 ?search=수원 쿼리 추가
  })
}
const handleGoBack = () => {
  router.go(-1)  // 💡 1단계 이전 주소 기록으로 Back
}
</script>

<template>
  <button @click="handleGoHome">일반 홈 이동 (push)</button>
  <button @click="handleLoginRedirect">인증 만료! 강제 리다이렉트 (replace)</button>
  <button @click="handleAdvancedMove">고급 파라미터 이동</button>
  <button @click="handleGoBack">이전 화면으로</button>
</template>
```

Navigation Guard

- Navigation Guard
  - 특정 라우트로 진입하기 직전, 중간에 가로채서 접근 권한 검사 및 페이지 리다이렉션 같은 사용자 정의 로직을 실행할 수 있게 함
  - 활용 Case: 관리자 페이지나 마이페이지 처럼 로그인 한 회원만 들어가야 하는 주소에 비로그인 사용자가 진입하는 경우 로그인 페이지로 전환한다.
  - Navigation Guard는 사용 방식에 따라 전역 가드(Global Guard), 라우터별 가드(Per-route Guard), 컴포넌트 내 가드(In-component Guard)로 구분된다.

Navigation Guard - Global Guard

- Navigation Guard - Global Guard
  - 모든 Route 전환에서 사용자 정의 로직이 실행된다.
  - Navigation Guard 실행되는 시점

Hook Method	트리거 시점	주요 Use Cases
router.beforeEach	새로운 라우트로의 이동이 **시작되기 직전** 트리거	**접근 권한 통제 및 보안** - 비로그인 사용자 차단, 관리자 권한 검증
router.beforeResolve	라우트 진입 전, 컴포넌트 내부 가드와 비동기 라우트 컴포넌트 분석이 모두 완료된 직후 트리거	**최종 데이터 검증 및 승인** - 페이지 진입 직전 사용자의 필수 토큰/데이터 최종 확인
router.afterEach	네비게이션이 완전히 종결되어 **화면 전환이 완료된 후**	**후속 처리 및 로그 기록** - Google Analytics 등 사용자 분석 로그 송신 등

```
// 라우터 인스턴스 하단에 배치 (to: 이동할 목적지, from: 현재 출발지, next: 이동을 허가하는 종결 함수)
router.beforeEach((to, from, next) => {

  const isAuthenticated = false // 실제로는 쿠키나 로컬스토리지의 토큰 검사

  // 권한이 필요한데 로그인이 안 된 경우
  if (to.meta.isAuth && !isAuthenticated) {
    alert('로그인이 필요한 서비스입니다.')
    next('/') // 통과를 불허하고 메인 홈 주소로 강제 이동
  } else {
    next() // 일반 통과 허가
  }
})
```

Unmatched Route Handling

- Route 미매핑 시 발생 상황
  - 정의되지 않은 경로(예: /unknown-page)로 접속할 경우, Vue Router는 에러를 던지는 대신 단지 매핑되는 컴포넌트를 찾지 못함
  - 그 결과 <RouterView/> 영역에 아무것도 렌더링되지 않아 화면이 하얗게 비어 보이는 현상 발생
- Catch-all Route
  - Vue Router의 Dynamic Route Matching과 Catch-all Regex(path: '/:pathMatch(.*)*') 패턴을 활용하여 구현 → Catch-all Route
  - 라우트 등록 목록의 가장 마지막에 Catch-all Route를 배치

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '@/views/HomeView.vue'
import NotFoundView from '@/views/NotFoundView.vue'

const routes = [
  {
    path: '/',
    name: 'Home',
    component: HomeView
  },
  // ... 기타 정의된 라우트들 ...

  // 상단 라우트와 매칭되지 않는 모든 경로를 NotFoundView로 리다이렉트
  {
    path: '/:pathMatch(.*)*',
    name: 'NotFound',
    component: NotFoundView
  }
]
…
```

[실습] 과제 - Router 활용

- 프로젝트 폴더 트리

```
src/
├── main.js                # 라우터 인스턴스 전역 주입 (.use(router))
├── App.vue                # 내비게이션 바 (<RouterLink>) 및 메인 수문장 (<RouterView />) 배치
├── router/
│   └── index.js           # 라우트 규칙(routes 배열) 정의 및 Lazy Loading 설정
├── components/
│   └── exercise/          # ⭐ 실습용 부품 컴포넌트 격리 폴더
│       ├── BaseDashboardCard.vue
│       ├── SearchBar.vue
│       └── WeatherCard.vue
└── views/                 # 페이지 단위 컴포넌트 보관 폴더
    ├── WeatherHomeView.vue   # 메인 날씨 대시보드 화면
    ├── WeatherAboutView.vue  # 서비스 소개용 정적 페이지
    ├── WeatherDetailView.vue # :cityId 패턴을 수신하는 동적 상세 페이지
    └── NotFoundView.vue      # 정의되지 않은 경로 접근 시 (Catch-all Route)
```

- 과제 요구사항

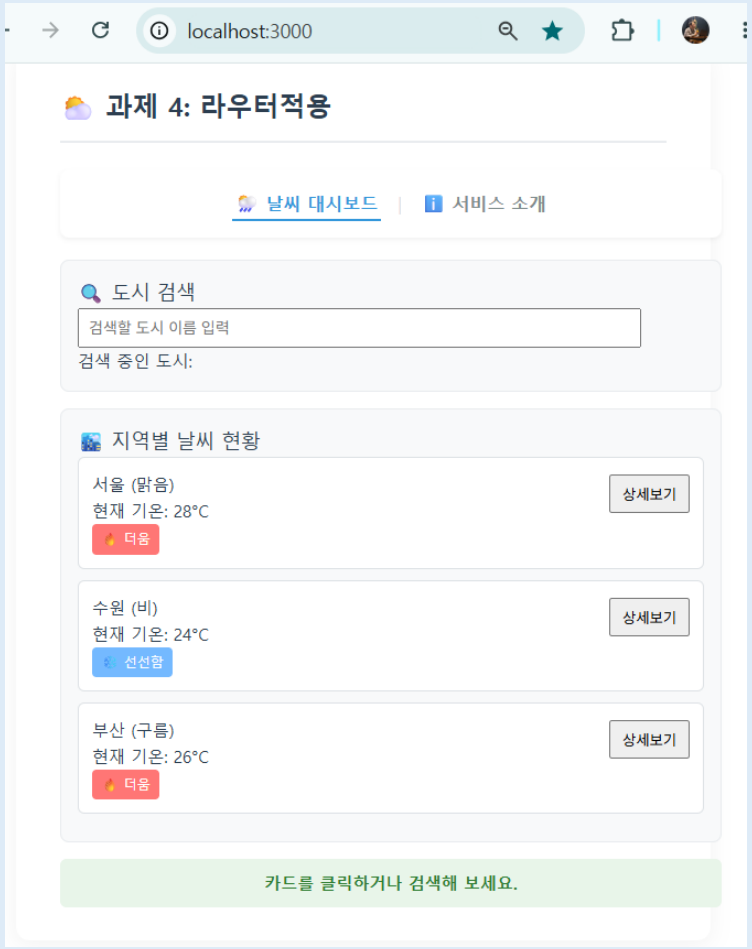
1. Vue Router 설정 : 라우터 지연 로딩 적용, Catch-all Route 적용
2. App.vue : Navigation Bar 추가 (RouterLink) 및 메인 콘텐츠 영역 배치(RouterView)
3. WeatherHomeView.vue : WeatherParent 대체 (WeatherParent를 참고하여 / 경로에 맞게 작성)

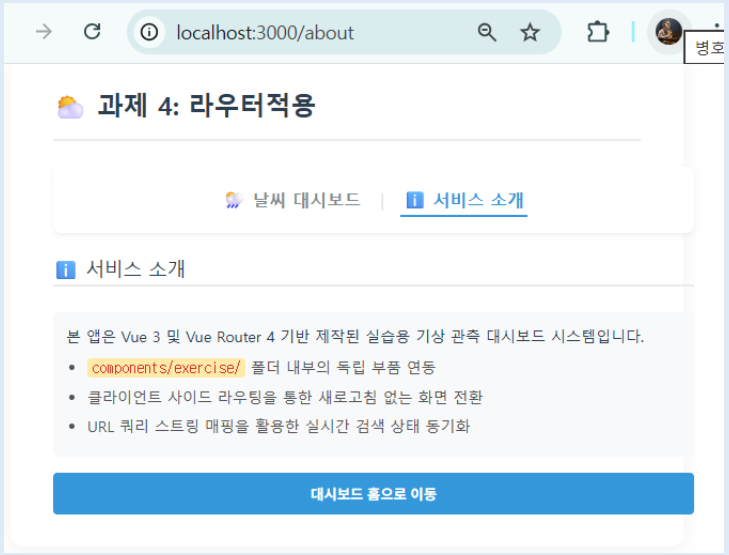
   - 상세보기 버튼 클릭 시 window.alert()를 제거하고, Programmatic Navigation 처리 (router.push('/weather/' + id)
4. WeatherDetailView.vue : 지역별 상세 기상관측 정보를 보여주는 페이지

   - 도시 코드에 해당하는 Mock Data를 임시로 활용

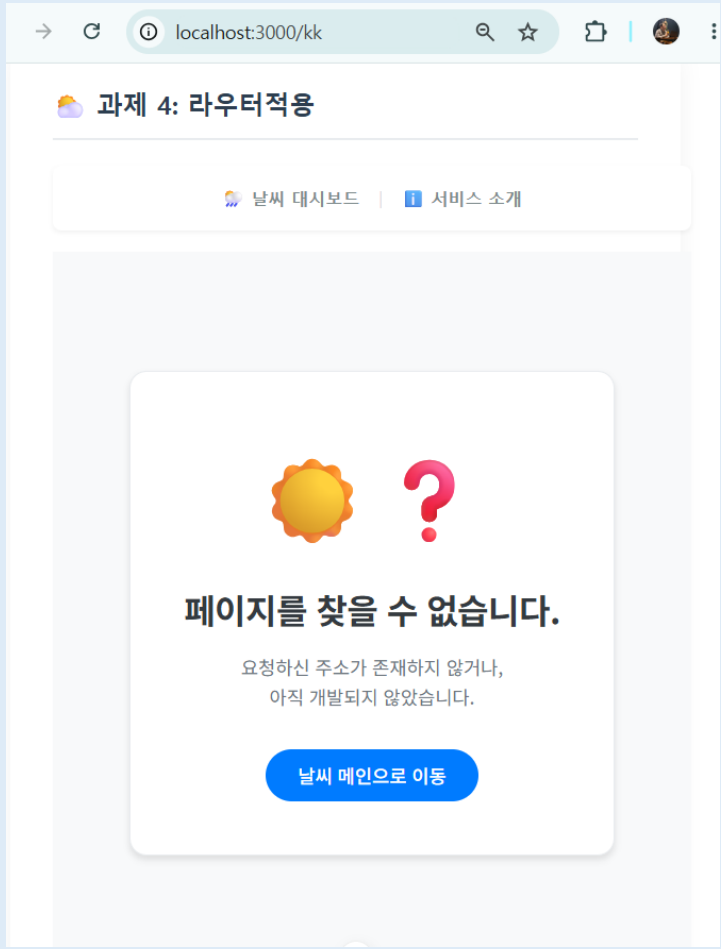
   - Router 동적 경로 매칭에 해당되는 도시ID (cityId)를 기반으로 Mount 시점에 Mock Data에서 도시 객체 선택
5. WeatherAboutView.vue : 적당한 내용 작성 및 메인 대시보드로 돌아가기 작성

[실습] 과제 - Router 활용



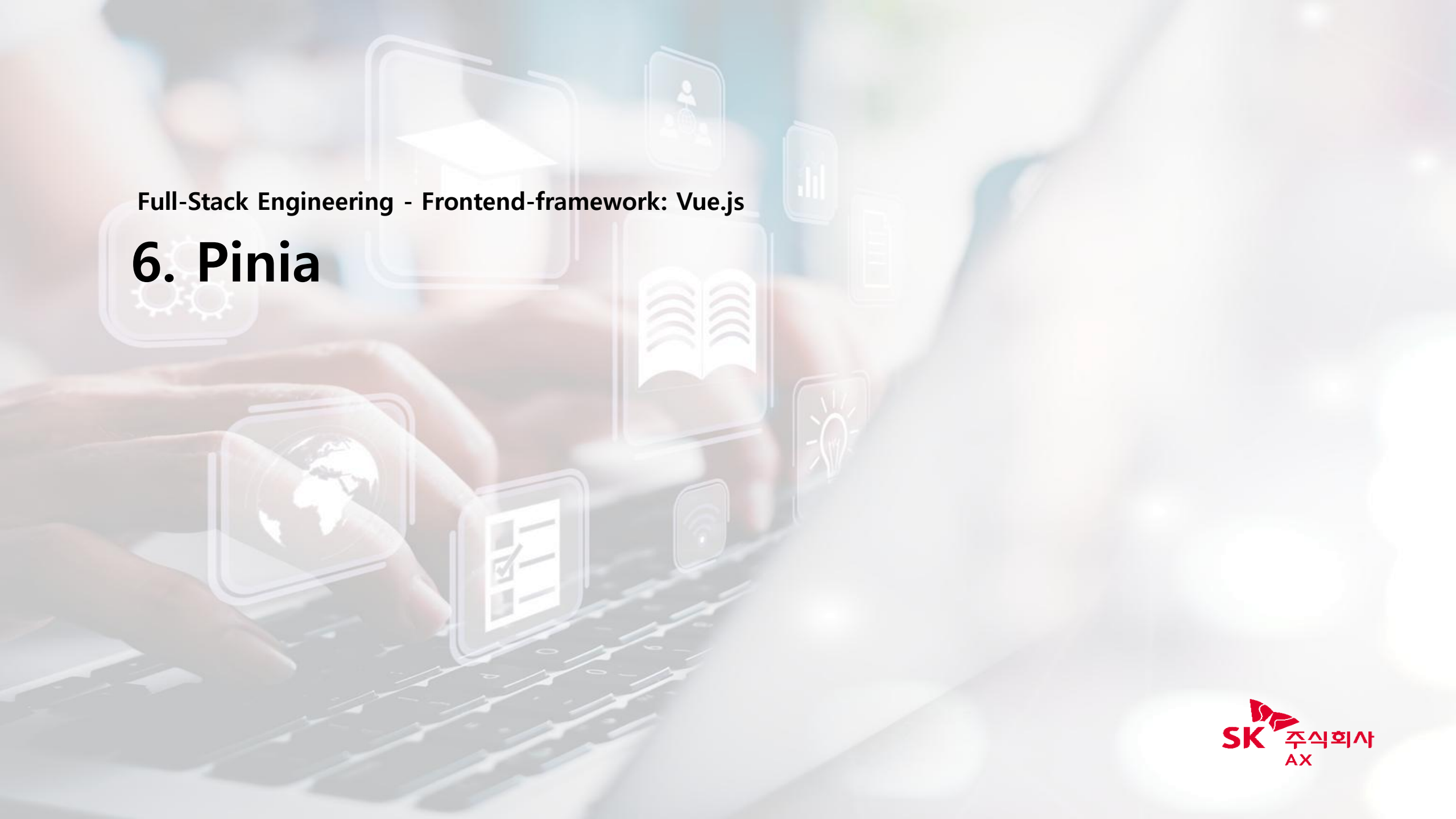






Full-Stack Engineering - Frontend-framework: Vue.js

6. Pinia



SK 주식회사 AX

Pinia Basic

- Pinia
  - Vue Application이 크고 복잡해 질수록 Component간 데이터 전달하는 것은 어렵다.
  - Pinia는 Component 계층 구조와 상관없이 별도의 전역 데이터 저장소(**Store**)를 개설하여 반응형 데이터를 관리한다.
  - **Pinia는 Vue3의 상태(state)관리 라이브러리**이며, Vue2에서는 Vuex라는 상태관리 라이브러리를 사용했다
  - 상태(State)란 웹 어플리케이션을 렌더링하는 과정에 영향을 줄 수 있는 값을 의미하며, 상태관리란 이러한 값을 관리하는 방법을 의미한다.

package.json > ...

```
{
  "name": "skala-vue",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  ▷Debug
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "run-s lint:*",
    "lint:oxlint": "oxlint . --fix",
    "lint:eslint": "eslint . --fix --cache",
    "format": "prettier --write --experimental-cli src/"
  },
  "dependencies": {
    "pinia": "^3.0.4",
    "vue": "^3.5.32",
    "vue-router": "^5.0.4"
  },
```

Pinia Basic - Store

- Store
  - Store는 여러 파일로 구성될 수 있으며, 일반적으로 의미가 있는 상태끼리 파일 하나로 작성한다.
  - 예) 인증스토어(authStore.js), UI스토어(uiStore.js), 알림스토어(alertStore.js), 공통코드스토어 (commonStore.js) 등
- Store 핵심 개념

용어	기술적 본질	Vue 3 내장 문법 매핑	주요 역할
state	반응형 데이터 변수	ref() / reactive()	**전역**으로 공유할 **상태 데이터 객체**를 정의한다.
getters	읽기 전용 계산된 변수	computed()	원본 state를 기반으로 실시간 가공한다.
actions	상태 변경 및 통신 함수	function()	1. state 값을 변경하는 핸들러 로직 2. 서버 비동기 API 통신(axios 등) 작업 수행

Pinia Basic - Pinia 구축 3단계

- Step 1: Pinia 등록하기 (src/main.js)
  - ① createPinia() 함수를 통해 pinia 인스턴스 생성
  - ② 어플리케이션 인스턴스의 use() 함수로 등록

src > JS main.js > ...

```
1   import './assets/main.css'
2
3   import { createApp } from 'vue'
4   import { createPinia } from 'pinia'
5
6   import App from './App.vue'
7   import router from './router'
8
9   const app = createApp(App)
10
11  app.use(createPinia()) ① ②
12  app.use(router)
13
14  app.mount('#app')
```

Pinia Basic - Pinia 구축 3단계

- Step 2: Store 생성하기 (src/stores/*스토어명*.js)
  - ① Pinia 패키지에서 제공하는 defineStore() 함수로 생성
  - ② defineStore() 함수로 생성한 Store Instance를 할당하는 변수의 식별자는 **use+파일명+Store** 규칙에 따라 작성한다.

src > stores > counter.js

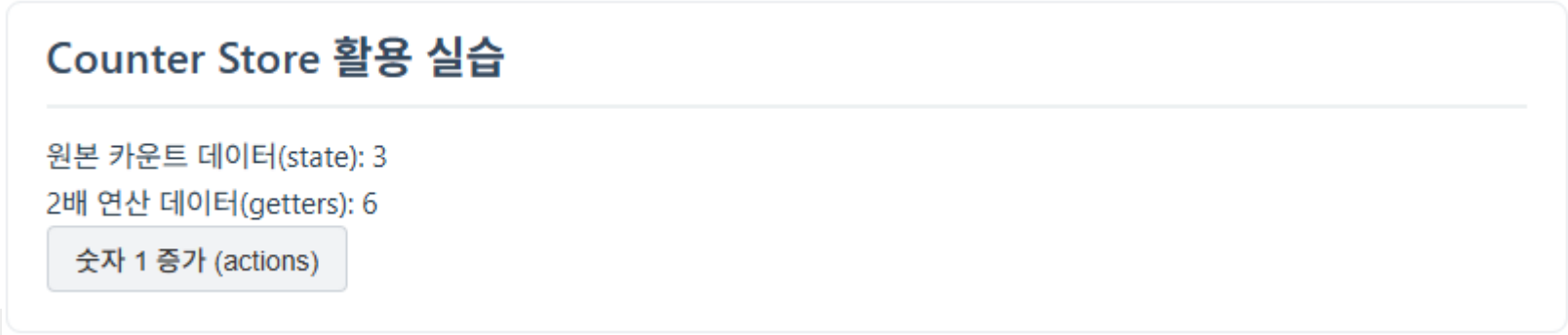
```
1  import { ref, computed } from 'vue'
2  import { defineStore } from 'pinia'
3                          ②              ①
4  export const useCounterStore = defineStore('counter', () => {
5    const count = ref(0)
6    const doubleCount = computed(() => count.value * 2)
7    function increment() {
8      count.value++
9    }
10
11   return { count, doubleCount, increment }
12 })
```

- Sample Code 설명

코드	Vue 3 내장 기술 스펙	Pinia 관점의 명칭	실제 런타임 역할 및 의미
count	ref() 반응형 상태 변수	**state** (상태 데이터)	전역에서 공유할 **원본 숫자 데이터**의 저장소(기본값은 0)
doubleCount	computed() 계산된 변수	**getters** (연산 상태)	count 데이터가 변경될 때만 실시간으로 연동되어 계산 **(Read-Only) 변수**
increment()	일반 JavaScript 함수	**actions** (실행 액션)	전역 state인 count 값을 안전하게 1씩 증가하는 함수
return { ... }	객체 반환 문법	**Expose (외부 개방 API)**	이 스토어를 임포트할 외부 컴포넌트가 전역 데이터에 접근할 수 있도록 선언

Pinia Basic - Pinia 구축 3단계

- Step 3: Store 사용하기
  - ① Import Store
  - ② Instance 가동
  - ③ state/getter/action 사용




```
<script setup>
// 1. 정의한 카운터 스토어 플러그인 import
import { useCounterStore } from '@/stores/counter.js'

// 2. 인스턴스 가동 (전역 저장소 포인터 확보)
const counterStore = useCounterStore()
</script>

<template>
  <div class="practice-section">
    <h2>Counter Store 활용 실습</h2>

    <p>
      원본 카운트 데이터(state): <strong>{{ counterStore.count }}</strong>
    </p>
    <p>
      2배 연산 데이터(getters): <span>{{ counterStore.doubleCount }}</span>
    </p>

    <button @click="counterStore.increment">숫자 1 증가 (actions)</button>
  </div>
</template>
```

Pinia Basic - Frequent Mistakes

- Store의 데이터 속성(State, Getters)은 구조분해할당(Destructuring Assignment)시 반응형이 유실될 수 있다.
  - 오류 유발 코드

```
// 이렇게 구조 분해 할당을 하면 Vue 3 반응형 시스템(Proxy 주소)이 단절되어 화면이 갱신되지 않습니다.
const { count, increment } = counterStore
```

  - 데이터(State, Getters) 속성은 Pinia 내장 함수인 storeToRefs로 감싸 호출해야만 반응형 연결 고리가 보존된다.
    (단, 함수인 Actions는 일반 구조 분해 할당을 해도 무방하다.)

```
import { storeToRefs } from 'pinia'

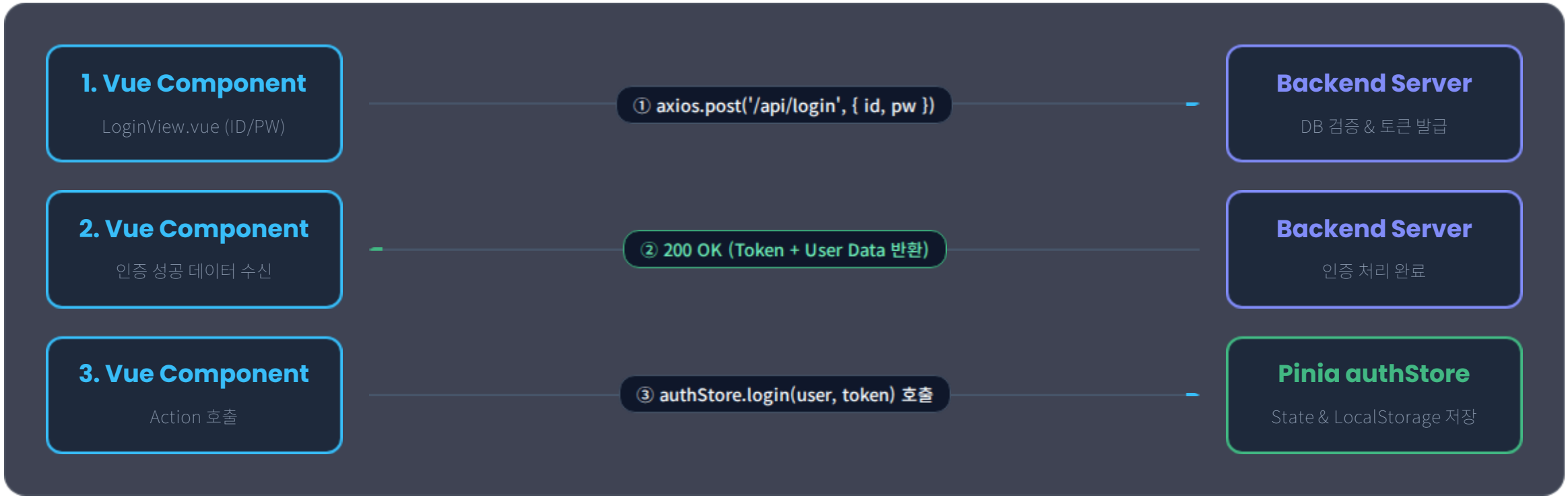
const { count, doubleCount } = storeToRefs(counterStore)
const { increment } = counterStore
```

※ 구조분해할당(Destructuring Assignment)

- Array나 Object의 구조를 분해하여, 내부의 값들을 별도의 독립된 개별 변수에 각각 직접 할당하는 모던 JavaScript 표현식

(사례연구) authStore - Login Flow

- Login 전체 Sequence Flow



(사례연구) authStore - Login Flow

- Backend login() vs Pinia authStore.login()

검증 & 발급 (Authentication)

Node.js / Spring / Python

Backend API login()

✓ 사용자가 입력한 ID/PW를 DB에 저장된 값과 비교

✓ 인증 성공 시 보안 **JWT Access Token** 생성

✓ 클라이언트에 토큰 및 유저 프로필 객체 응답

```
POST /api/login
➔ Response: { token: "eyJhb..." , user: { ... } }
```

저장 & 유지 (State Management)

Vue 3 State Management

Pinia authStore.login()

✓ 백엔드가 응답한 토큰과 유저 정보를 **반응형 State**에 저장

✓ 새로고침 시에도 유지되도록 **localStorage** 동기화

✓ Navigation Guard에서 접근 권한 검사 용도로 상태 공유

```
authStore.login (userData, authToken)
➔ token.value = authToken;
```

(사례연구) authStore - JWT

- JWT(JSON Web Token)은 정보를 안전하게 JSON 객체 형태로 주고받기 위해 정의된 표준 규격으로 Backend가 발급
- JWT의 구조

```
eyJhbGci... . eyJzdWIi... . d3g4eT...
```

  - 점(.) 2개로 구분된 3개의 긴 암호문(Base64) 문장으로 이루어져 있다. (Header, Payload, Signature)
  - Base64로 누구나 쉽게 복호화 할 수 있으므로 Payload에 들어가는 사용자 정보에는 민감정보를 넣으면 안된다.
- JWT vs Session

구분	기존 세션(Session) 방식	JWT (Token) 방식
로그인 정보 저장소	**서버 메모리/DB**	**클라이언트 (브라우저 State / LocalStorage)**
서버 부하	동시 접속자가 많으면 서버 메모리 부담 증가	서버에 저장하지 않으므로 부하가 매우 적음 (Stateless)
서버 확장성	서버를 여러 대 늘릴 때 세션 공유 설정(Redis 등) 필요	서버가 여러 대여도 토큰 서명만 검증하면 되므로 확장 용이
적합한 아키텍처	전통적인 서버 사이드 렌더링 (SSR)	**SPA (Vue, React) + REST API 서버**

- Pinia에 저장된 JWT Token은 HTTP 요청 헤더(Authorization)에 포함하여 Backend로 전달해야 한다.
  - 웹 브라우저가 백엔드 서버로 API를 요청할 때, 토큰은 주로 Authorization 헤더에 Bearer 타입으로 실어 보낸다.

```
GET /api/user/profile HTTP/1.1
Host: api.skala.com
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Content-Type: application/json
```

  - 실무에서는 Axios Request Interceptor를 사용해 자동으로 토큰을 주입한다.

(사례연구) authStore - Source Code

- 전역 인증 스토어 (stores/authStore.js)
  - 사용자의 토큰(JWT), 사용자 정보, 로그인 상태 등을 앱 전체에서 공유하기 위한 전역 인증 스토어 파일.
  - 파일명은 authStore.js로 지정하고, 외부에서 불러올 함수명은 Vue Composable 관례에 따라 useAuthStore로 내보낸다.(export).

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useAuthStore = defineStore('auth', () => {
  // 1. State: 로그인 토큰 및 사용자 정보
  const token = ref(localStorage.getItem('accessToken') || null)
  const user = ref(JSON.parse(localStorage.getItem('userInfo') || 'null'))
  // 2. Getters: 로그인 여부 확인 및 사용자이름
  const isLoggedIn = computed(() => !!token.value)
  const username = computed(() => user.value?.name ||  '게스트' )
  // 3. Actions: 로그인 / 로그아웃 로직
  function login(userData, authToken) {
    user.value = userData
    token.value = authToken
    // 브라우저 재접속 시 유지용
    localStorage.setItem('accessToken', authToken)
    localStorage.setItem('userInfo', JSON.stringify(userData))
  }
  function logout() {
    user.value = null
    token.value = null
    localStorage.removeItem('accessToken')
    localStorage.removeItem('userInfo')
  }

  return { token, user, isLoggedIn, userName, login, logout }
})
```

(사례연구) authStore - Navigation Guard와 연동

- router/index.js에서 store/authStore.js를 import 하여 이동 직전 접근 권한 검사

```
import { createRouter, createWebHistory } from 'vue-router'
import { useAuthStore } from '@/stores/authStore'

const routes = [
  … 생략 …
]

Const router = createRouter({
  history: createWebHistory(),
  routes
})

// Navigation Guard 연동
router.beforeEach((to, from) => {
  // Guard 내부에서 authStore 호출
  const authStore = useAuthStore()

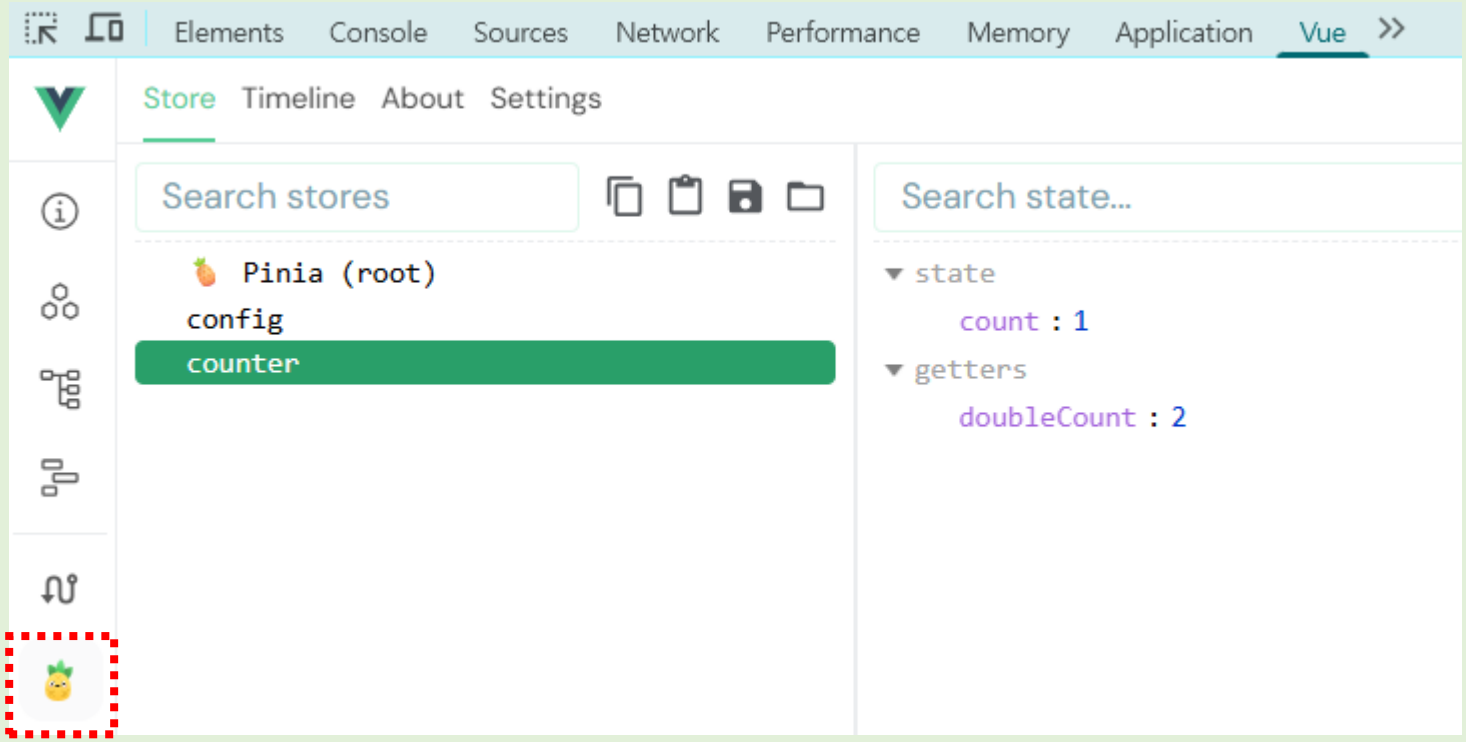
  // 1. 인증이 필요한 페이지 접근 시 로그인 여부 체크
  if (to.meta.requiresAuth && !authStore.isLoggedIn) {
    alert('로그인이 필요한 서비스입니다.')
    return { name: 'Login', query: { redirect: to.fullPath } } // 로그인 후 돌아올 위치 전달
  }

  // 2. 이미 로그인한 사용자가 로그인 페이지 접근 시 메인으로 이동
  if (to.name === 'Login' && authStore.isLoggedIn) {
    return { name: 'Dashboard' }
  }
})

export default router
```

Code Challenge

- Store (counter.js) 작성하기
  - Pinia 등록하기 (src/main.js)
  - Store 생성하기 (src/stores/counter.js)
  - Store 사용하기 (src/components/practices/library/StoreCounter.vue)
- Vue Devtools에서 Pinia 확인



[실습] 과제 - Store 활용

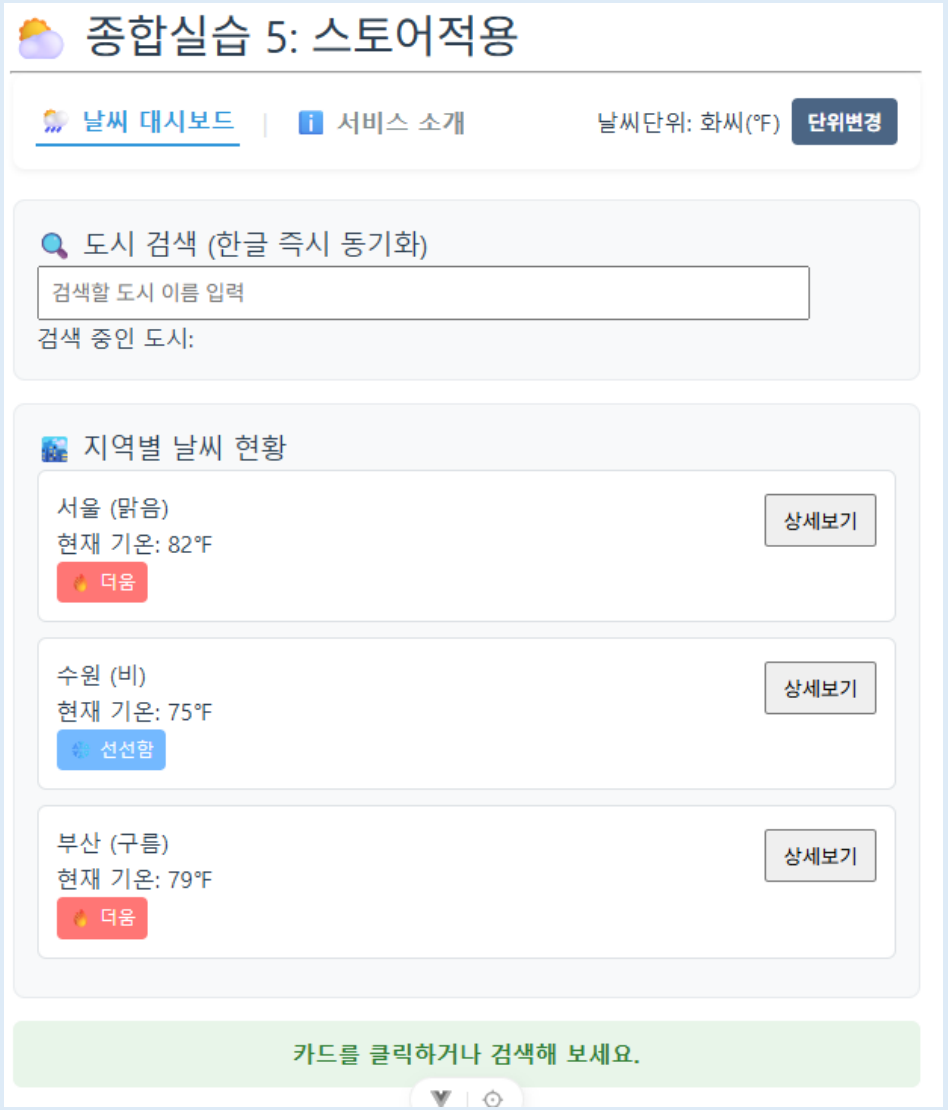
- 날씨 단위를 세팅하는 stores/configStore.js 작성

state	unit	단위를 저장하는 변수 (초기값: celsius)
getters	unitSymbol	현재 단위 상태에 맞는 기호 (℃ / ℉)
actions	toggleUnit	'celsius'와 'fahrenheit'를 토글(스위칭)하는 함수

- 과제 요구사항
  1. UnitToggler.vue : 대시보드 상단에 배치되어 단위 설정을 변경하는 UI 버튼과 영역
  2. Navigation Bar 옆에 UnitToggler.vue 배치
  3. 메인과 상세 날씨에 단위 설정 변경 적용

```
const displayTemp = computed(() => {
  const rawTemp = props.cityItem.temp // 기본 원본 데이터는 섭씨 숫자
  if (configStore.unit === 'fahrenheit') {
    return Math.round((rawTemp * 9) / 5 + 32) // 화씨 변환 연산
  }
  return rawTemp // 'celsius'일 때는 원본 그대로 반환
})
```

- 참고사항) 메인/상세 날씨에 단위 설정을 변경을 적용할 경우 유사한 코드가 중복됨 → Composable 로 해결 가능함 (범위 제외)



Full-Stack Engineering - Frontend-framework: Vue.js

7. Axios



SK 주식회사 AX

Data Communication - HTTP

- HTTP (HyperText Transfer Protocol)
  - 웹 브라우저와 웹 서버가 인터넷상에서 데이터를 주고받기 위해 세계적으로 약속한 표준 통신 규약(Protocol)
  - 일반적으로 Client에서 Server로 HTTP Request를 보내고 서버는 요청에 대해 HTTP Response를 보낸다.
- HTTP Methods
  - Client 가 Server 요청하는 작업이 무엇인지를 나타낸다.
  - 데이터베이스 CRUD 연산과의 매핑

HTTP 메서드	CRUD 연산 기능 (Database)	실제 수행하는 행위의 의미
GET	**Read** (조회)	서버의 데이터를 바꾸지 않고 오직 읽어오기만 함
POST	**Create** (생성)	서버에 새로운 데이터를 밀어 넣어 등록함
PUT / PATCH	**Update** (수정)	서버에 이미 있는 데이터를 찾아서 값을 변경함
DELETE	**Delete** (삭제)	서버에 있는 특정 데이터를 타격하여 파괴함

  - 각 Method의 역할은 강제 규칙은 아니다. (POST로 데이터를 삭제하거나 변경해도 문제 없다)

Data Communication - API

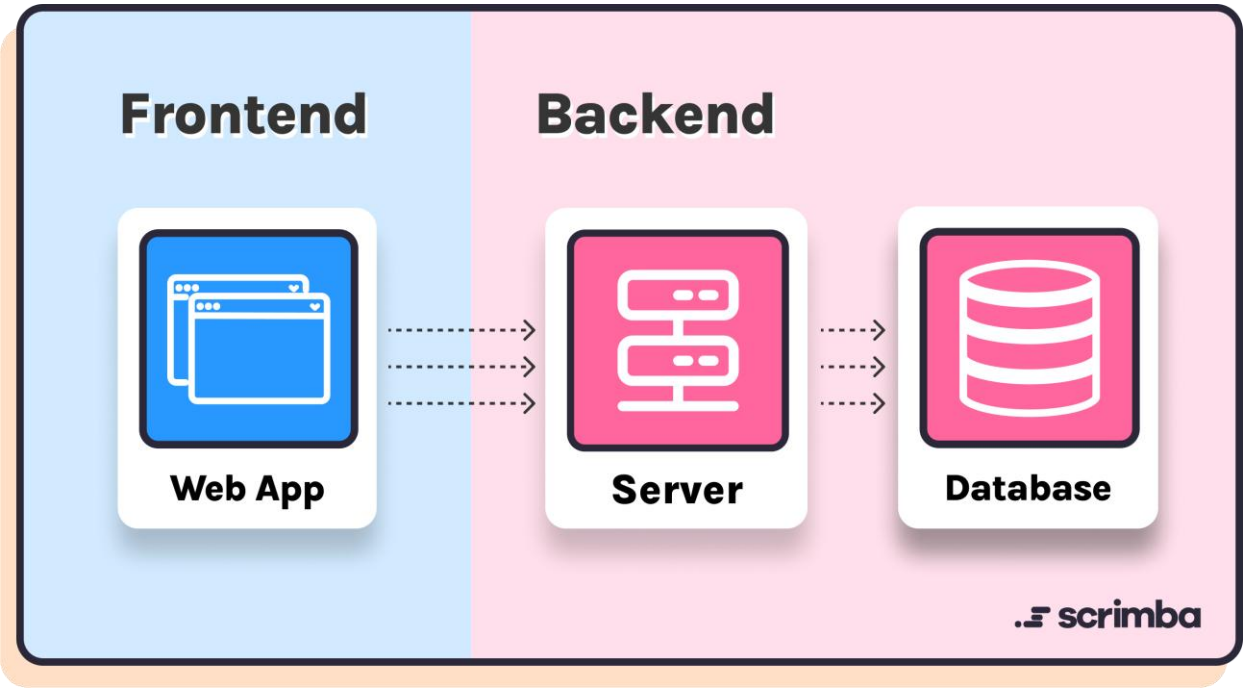
- API (Application Programming Interface)
  - 서로 다른 소프트웨어 애플리케이션이 자신들의 기능이나 데이터를 상대방이 안전하고 쉽게 가져다 쓸 수 있도록 열어놓은 규칙
  - Web에서의 API는 Browser와 Server간에 HTTP를 사용해 데이터를 주고 받는 약속을 의미한다.
- REST(REpresentational State Transfer) API
  - 웹의 HTTP를 활용하면서, 자원을 이름으로 구분하여 해당 자원의 데이터를 주고 받는 방식에 웹 인터페이스 스타일
  - HTTP Method (GET, POST, DELETE, PUT)를 활용하여 자원에 대한 CRUD 작업을 적용하는 것을 의미한다.
  - 오늘날 대부분의 인터페이스에 활용
- REST API 설계 원칙
  - 주소(URI)는 오직 명사(자원)로만 구성한다.

    ❌ 나쁜 예 (Non-REST): /getWeather, /deleteUser, /update_city (주소에 동사가 포함됨)

    ◍ 바른 예 (REST): /weather, /users, /cities (오직 깔끔한 명사만 남김)
  - 행위(동사)는 HTTP Method로 대체한다.

Data Communication - Frontend vs. Backend

- Frontend
  - 사용자가 웹 브라우저를 통해 눈으로 보고, 클릭하고, 입력하는 모든 **화면(UI/UX) 영역**
  - 사용자의 행동(Event)을 감지하고, Backend API를 통해 받은 데이터를 보기 좋게 시각화(렌더링)한다.
- Backend
  - 시스템의 핵심 비즈니스 로직, 데이터 가공, 보안, 데이터베이스(DB) 관리를 전담하는 서버 영역
  - 데이터베이스를 안전하게 제어하고, 로직과 규칙을 거쳐 Frontend가 필요한 데이터를 API를 통해 전달



https://scrimba.com/articles/frontend-backend-or-fullstack/

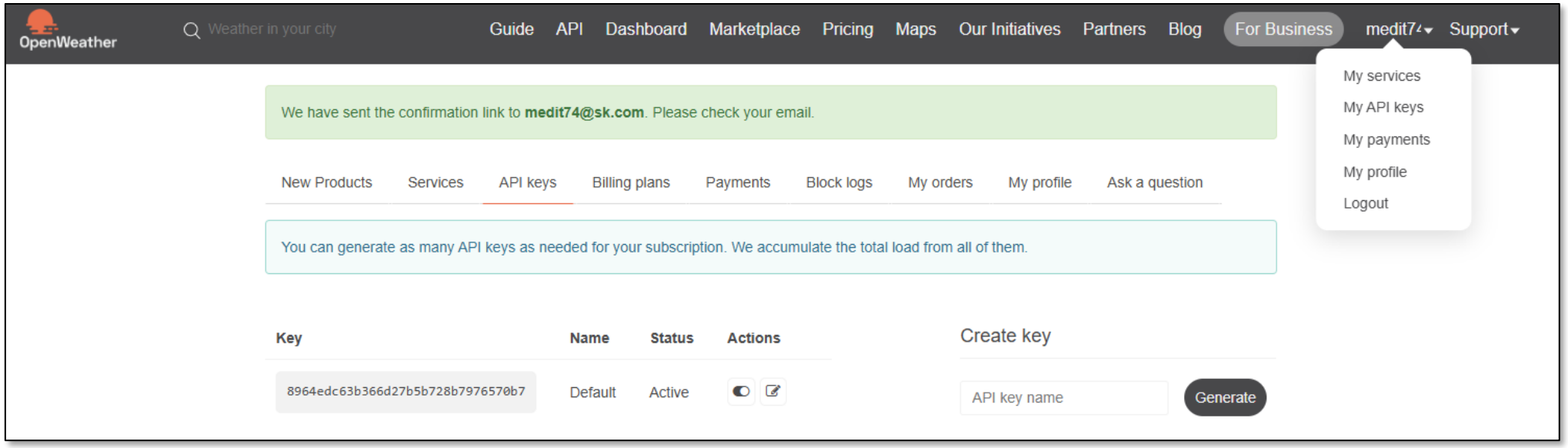
API - JSON Placeholder

- JSON Placeholder
  - https://jsonplaceholder.typicode.com/
  - 전 세계 프론트엔드 개발자들이 통신 및 CRUD 코드를 테스트할 때 사용하는 무료 가상 REST API 서비스
- JSON Placeholder Test API

Method	**타격할 API 엔드포인트 주소**	**전달할 데이터 (Body)**	**Response**
GET	https://jsonplaceholder.typicode.com/posts	없음	서버에 등록된 전체 가상 데이터 목록 조회 (배열)
GET	https://jsonplaceholder.typicode.com/posts/1	없음	1번 고유 ID를 가진 데이터 상세 조회
POST	https://jsonplaceholder.typicode.com/posts	{ title: '도시명', body: '날씨현황', userId: 1 }	새로운 정보 **등록** (보낸 데이터와 함께 id: 101을 추가해서 객체 반환)
PUT	https://jsonplaceholder.typicode.com/posts/1	{ title: '수정도시', body: '수정현황' }	새 내용으로 **교체 (보낸데이터와 함께 보상 id:1 추가해서 반환 반환)**
DELETE	https://jsonplaceholder.typicode.com/posts/1	없음	1번 데이타 **삭제** (성공 시 공백 반환)

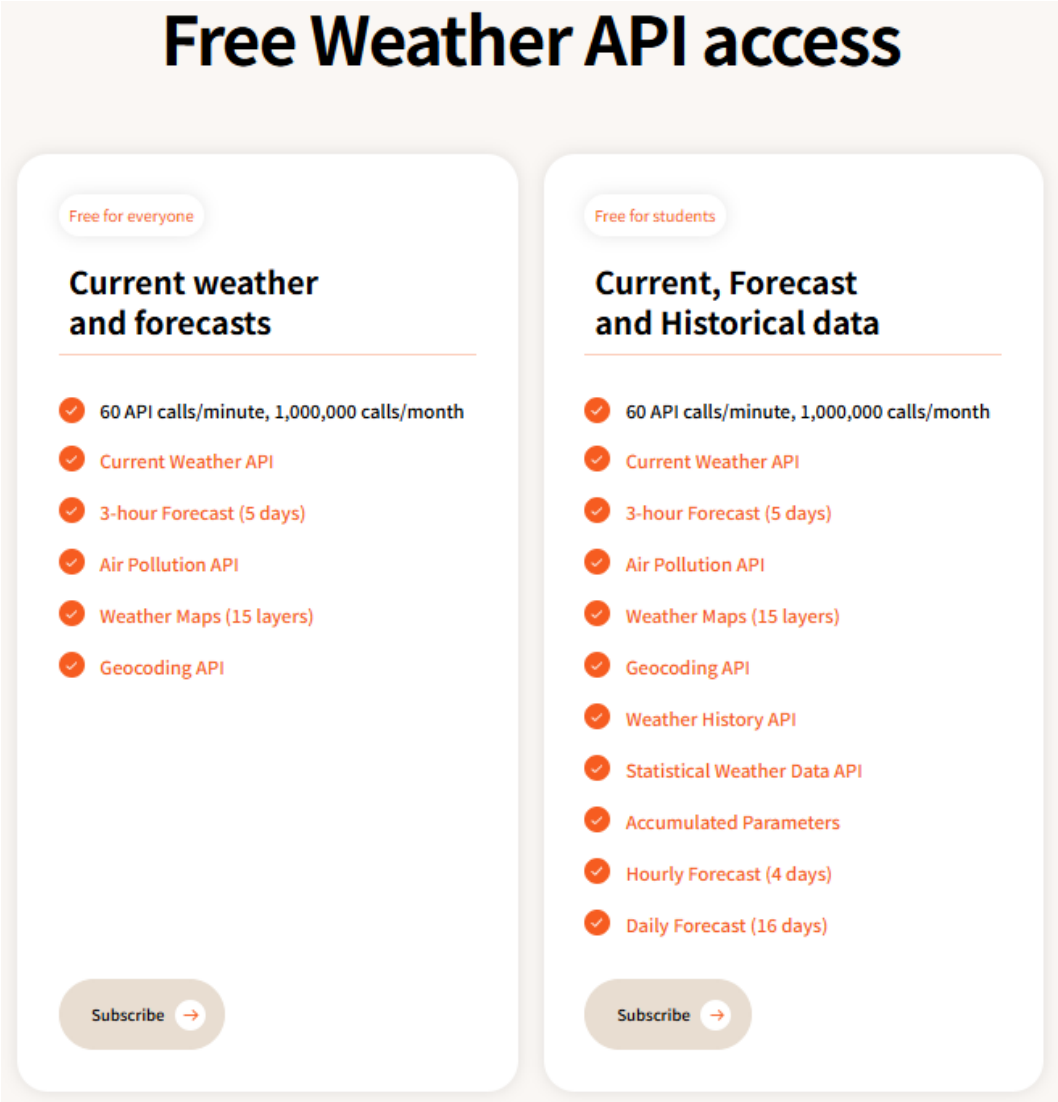
API - Open Weather

- Open Weather API 활용
  - 전 세계 20만 개 이상의 도시 데이터를 일관된 규격으로 제공하는 가장 대중적인 REST API 서비스
  - 월 1,000,000건, 분당 60건의 호출을 무료로 제공
  - 데이터 응답 결과가 완벽한 JSON 형식으로 전달됨.
- Open Weather (https://openweathermap.org/)
  - Sign Up
  - My API Keys 확인



API - Open Weather

- Free Tier 요금제 (https://openweathermap.org/price Scroll Down)

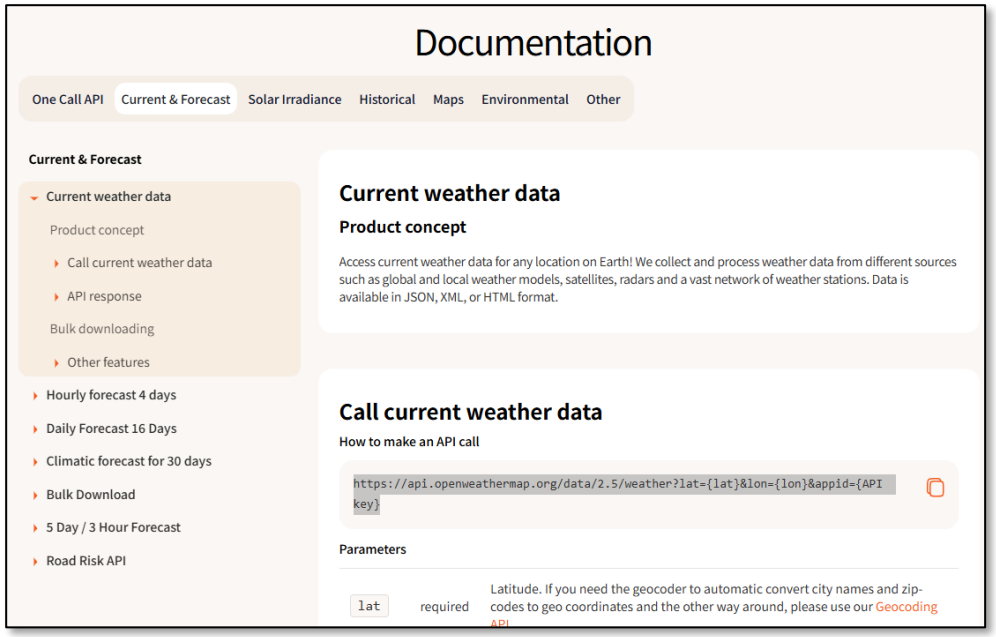


API - Open Weather

- Current Weather API
  - https://openweathermap.org/api/current?collection=current_forecast
  - Call current weather data

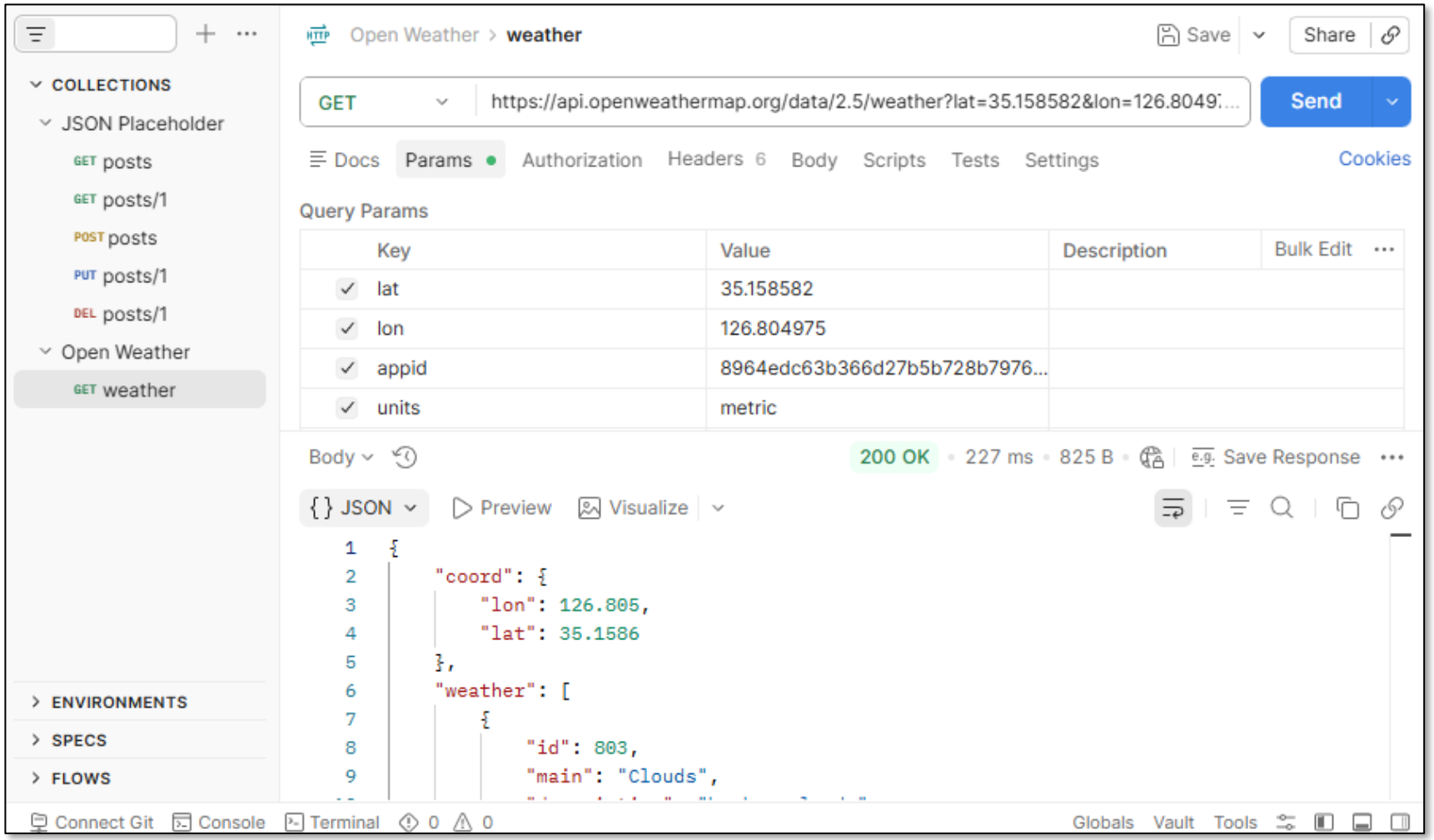
    https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key} }&units=metric&lang=kr
  - (Geocoding API) Build-in API request by city name

    https://api.openweathermap.org/data/2.5/weather?q=${targetCity.english}&appid=${API_KEY}&units=metric&lang=kr



API - API Test (Postman)

- Postman
  - 서버에서 제공하는 API를 테스트하는 도구



API - Fetch API vs. Axios

- HTTP API 호출 방법 비교

비교 항목	Fetch API	Axios
설치 여부	◍ 없음 (브라우저 기본 내장 빌트인)	✕ 필수 (npm i axios로 패키지 설치 필요)
JSON 변환	✕ 수동 (반드시 res.json() 파싱 단계 거침)	◍ 자동 (내부적으로 JSON을 object로 자동 변환)
에러 핸들링	✕ 수동처리	◍ 자동처리
실무 선호도	◍ 중간	◍ 매우 높음
BaseURL 설정	✕ 지원 안 함	◍ 지원 (axios.create)
요청/응답 인터셉터	✕ 지원 안 함	◍ 지원 (요청직전 로그인 토큰 자동 탑승, 에러 발생 시 공통 팝업 가로채기 등)

Axios - Installation

- Installation

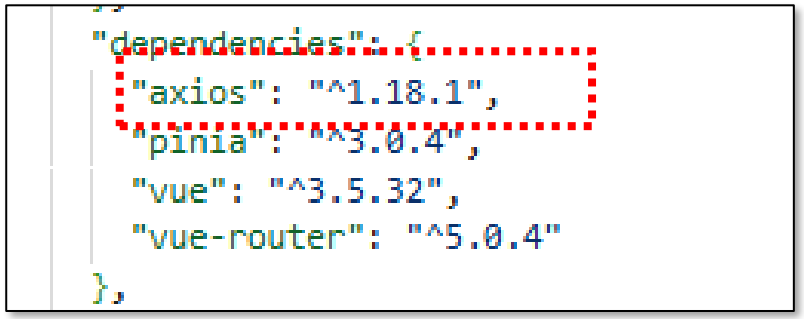
```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm install axios

added 25 packages, and audited 288 packages in 2s

78 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- 설치 확인 (package.json)



Axios - API Example

- Open Weather Map 호출 예제를 통한 기본 동작 원리 파악

```vue
<script setup>
import { ref } from 'vue'
import axios from 'axios'

const weatherData = ref(null)
const isLoading = ref(false)

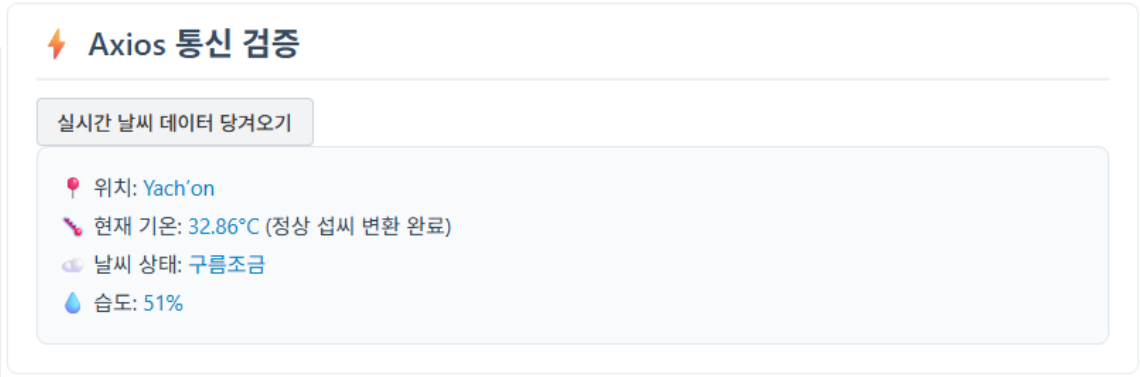
const handleFetchWeather = async () => {
  isLoading.value = true

  const API_KEY = '8964edc63b366d27b5b728b7976570b7'
  const URL = `https://api.openweathermap.org/data/2.5/weather?lat=35.158582&lon=126.804975&appid=${API_KEY}&units=metric&lang=kr`

  try {
    // 비동기 통신: 서버에서 데이터를 다 가져올 때까지 await로 기다린다.
    const response = await axios.get(URL)
    // fetch()는 응답 String을 Json으로 변환해야 하지만(.json()) Axios에서는 응답 String(response.data)가 자동으로 JSON 파싱 됨.
    console.log('Axios 통신 응답 전체 객체:', response)
    console.log('백엔드가 준 핵심 날씨 데이터(JSON):', response.data)
    weatherData.value = response.data
  } catch (error) {
    // 4xx, 5xx 에러나 네트워크 오프라인 시 자동으로 reject되어 catch 영역에서 처리 한다.
    console.error('통신 중 에러가 발생했습니다:', error)
    alert('데이터를 가져오지 못했습니다. API 키 활성화 여부나 주소를 확인하세요.')
  } finally {
    isLoading.value = false
  }
}
</script>
```

Axios - API Example

- Open Weather Map 호출 예제를 통한 기본 동작 원리 파악




```
<template>
  <div class="practice-section">
    <h2> Axios 통신 검증</h2>
    <button @click="handleFetchWeather" :disabled="isLoading">
      {{ isLoading ? '데이터 로딩 중...' : '실시간 날씨 데이터 당겨오기' }}
    </button>
    <div v-if="weatherData" class="result-card">
      <p>
         위치: <strong>{{ weatherData.name }}</strong>
      </p>
      <p>
         현재 기온: <strong>{{ weatherData.main.temp }}°C</strong> (정상 섭씨 변환 완료)
      </p>
      <p>
         날씨 상태: <strong>{{ weatherData.weather[0].description }}</strong>
      </p>
      <p>
         습도: <strong>{{ weatherData.main.humidity }}%</strong>
      </p>
    </div>
    <div v-else>
      <p>아직 가져온 데이터가 없습니다. 버튼을 눌러 통신을 가동하세요.</p>
    </div>
  </div>
</template>
```

Axios - Method List

- Axios 제공 함수 목록

구분	함수명 / 호출 형태	주요 특징 및 용도	예시 코드
인스턴스 생성	axios.create([config])	사용자 정의 설정을 가진 독립된 Axios 인스턴스를 생성한다.	const api = axios.create({ baseURL: '/api' })
기본 객체 호출	axios(config)	Config 객체 하나만 전달하여 요청을 보낸다.	axios({ method: 'get', url: '/users' })
URL+설정 호출	axios(url, [config])	첫 번째 인자로 URL, 두 번째 인자로 설정 객체를 전달하여 호출한다.	axios('/users', { method: 'post', data: {} })
HTTP 단축 메서드	axios.get(url, [config])	데이터 조회 (Body 없음, Query String 사용)	axios.get('/users', { params: { id: 1 } })
	axios.post(url, [data], [config])	데이터 생성 (Body 데이터 포함, JSON 자동 변환)	axios.post('/users', { name: 'Kim' })
	axios.put(url, [data], [config])	데이터 전체 수정 (Body 데이터 포함)	axios.put('/users/1', { name: 'Lee' })
	axios.patch(url, [data], [config])	데이터 일부 수정 (Body 데이터 포함)	axios.patch('/users/1', { age: 20 })
	axios.delete(url, [config])	데이터 삭제	axios.delete('/users/1')
인터셉터	axios.interceptors.request	요청 전송 직전 토큰(JWT) 삽입 등 공통 전처리를 수행한다.	api.interceptors.request.use(config => ...)
	axios.interceptors.response	응답 수신 직후 에러 공통 처리, 데이터 가공 등 후처리를 수행한다.	api.interceptors.response.use(res => ...)
병렬 요청	axios.all([...])	여러 개의 Axios Promise를 배열로 받아 동시에 실행한다.	axios.all([getA(), getB()])
	axios.spread(callback)	axios.all의 결과를 각 변수로 분해하여 수신한다.	.then(axios.spread((resA, resB) => ...))

- 배경색이 들어간 axios 통신 메서드들은 호출 후 자바스크립트의 Standard Promise 객체를 반환한다.

Axios - 비동기 호출 방식

- Promise…Then vs. Async/Await
  - Axios가 Promise를 리턴하기 때문에, 아래의 두 가지 비동기 처리 방식을 사용할 수 있다.

비교 항목	**Promise (.then()) 방식**	**async / await 방식**
문법적 기반	ES6 (2015년) 도입 표준 객체 기반	ES8 (2017년) 도입, Promise를 감싼 문법적 설탕
코드 스타일	**체이닝 방식** (콜백 함수를 뒤에 줄줄이 엮어 나감)	**동기식 스타일** (위에서 아래로 순차적으로 읽히는 일반 코드 형태)
에러 핸들링	.catch(error => { ... }) 메서드로 제어	자바스크립트 정석 문법인 try { ... } catch (e) { ... } 제어
실무 선호도	보통 (간단한 단발성 통신에 가끔 사용)	**매우 높음** (가독성과 유지보수성이 매우 유리)

```
const fetchWeatherPromise = () => {
  console.log('1. 통신 시작 구역')

  axios.get(URL)
    .then((response) => {  // 통신이 성공했을 때
      console.log('3. 데이터 도착:', response.data)
    })
    .catch((error) => {    // 에러가 났을 때
      console.error('에러 발생:', error)
    })

  console.log('2. 통신 요청 직후 라인')
  // 백엔드 데이터가 오기 전에 '2번 로그'가 콘솔창에 먼저 기록된다
}
```

```
const fetchWeatherAsync = async () => {
  console.log('1. 통신 시작 구역')

  try {
    // 서버 데이터가 도착할 때까지 대기하고 함수를 호출한 바깥 부분 먼저 실행
    const response = await axios.get(URL)
    // 데이터가 도착하면 실행
    console.log( ' 2. 데이터 도착: ' , response.data)
  } catch (error) {
    console.error('에러 발생:', error)
  }

  console.log('3. 모든 통신 프로세스 완전히 끝난 후 라인')
  // '1번 -> 2번 -> 3번' 순서대로 기록된다.
}
```

Axios - API Example

- JSONPlaceholder 예제를 통해 REST API CRUD 처리 코드 확인 (AxiosJson.vue)

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

// 💡 1. 백엔드 공용 주소
const BASE_URL = 'https://jsonplaceholder.typicode.com/posts'

// 💡 2. 반응형 상태 데이터
const items = ref([]) // 서버에서 받아온 데이터 배열 박스
const textInput = ref('') // 입력창과 연결된 글자 데이터 박스

// ------------------------------------------------
// [READ] GET : 데이터 가져오기
// ------------------------------------------------
const handleRead = async () => {
  try {
    // 공부용으로 딱 3개만 들고 옵니다.
    const response = await axios.get(BASE_URL, { params: { _limit: 3 } })
    items.value = response.data
    console.log('GET 성공:', response.data)
  } catch (error) {
    console.error('GET 실패:', error)
  }
}
…
```



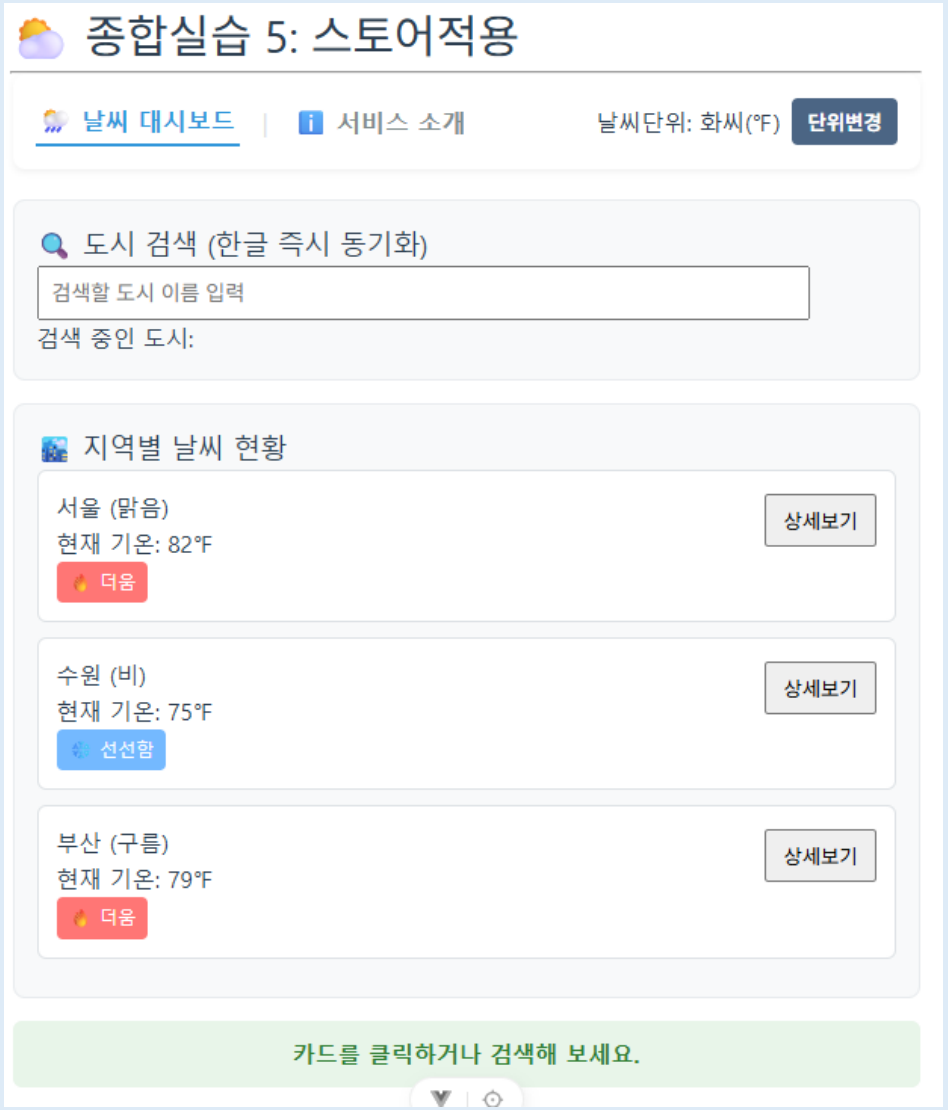
Code Challenge

- Axios 설치
- Axios Weather Example
  - Open Weather API 가입 및 키 발급
  - Pinia 등록하기 (src/main.js)
  - Store 생성하기 (src/stores/counter.js)
  - Store 사용하기 (src/components/practices/library/StoreCounter.vue)
- Axios JSON Example

[실습] 과제 - 날씨 데이터 연동

- Axios 활용 준비
  1. Axios 라이브러리 설치
  2. OpenWeatherMap API 가입 및 Key 발급
- 과제 요구사항
  1. UnitToggler.vue : 대시보드 상단에 배치되어 단위 설정을 변경하는 UI 버튼과 영역
  2. Navigation Bar 옆에 UnitToggler.vue 배치
  3. 메인과 상세 날씨에 단위 설정 변경 적용

```
const displayTemp = computed(() => {
  const rawTemp = props.cityItem.temp // 기본 원본 데이터는 섭씨 숫자
  if (configStore.unit === 'fahrenheit') {
    return Math.round((rawTemp * 9) / 5 + 32) // 화씨 변환 연산
  }
  return rawTemp // 'celsius'일 때는 원본 그대로 반환
})
```





Full-Stack Engineering - Frontend-framework: Vue.js

8. UI 라이브러리

SK 주식회사 AX

UI Library

- UI 라이브러리
  - 웹 애플리케이션 UI(User Interface) 구축에 필요한 공통 컴포넌트(Button, Input, Form, Dialog, Table 등)를 Vue 3 컴포넌트 단위로 모듈화하여 제공하는 오픈소스 소프트웨어 패키지
- 사용 효과
  - 개발 리소스 절감: CSS 스타일시트 및 HTML 마크업 코드 작성을 생략하고 완성된 컴포넌트 태그를 호출하므로 UI 구현 속도가 향상된다.
  - 크로스 브라우징 및 반응형 대응: 다양한 브라우저 환경(Chrome, Safari, Firefox 등)과 디바이스 해상도(Mobile, Tablet, Desktop) 별 미디어 쿼리가 내부적으로 최적화되어 있다.
  - 웹 표준 및 접근성(WAI-ARIA) 준수: 스크린 리더 인식, 키보드 포커스 제어 등 웹 접근성 가이드라인이 컴포넌트 레벨에서 사전 구현되어 있다.

UI Library

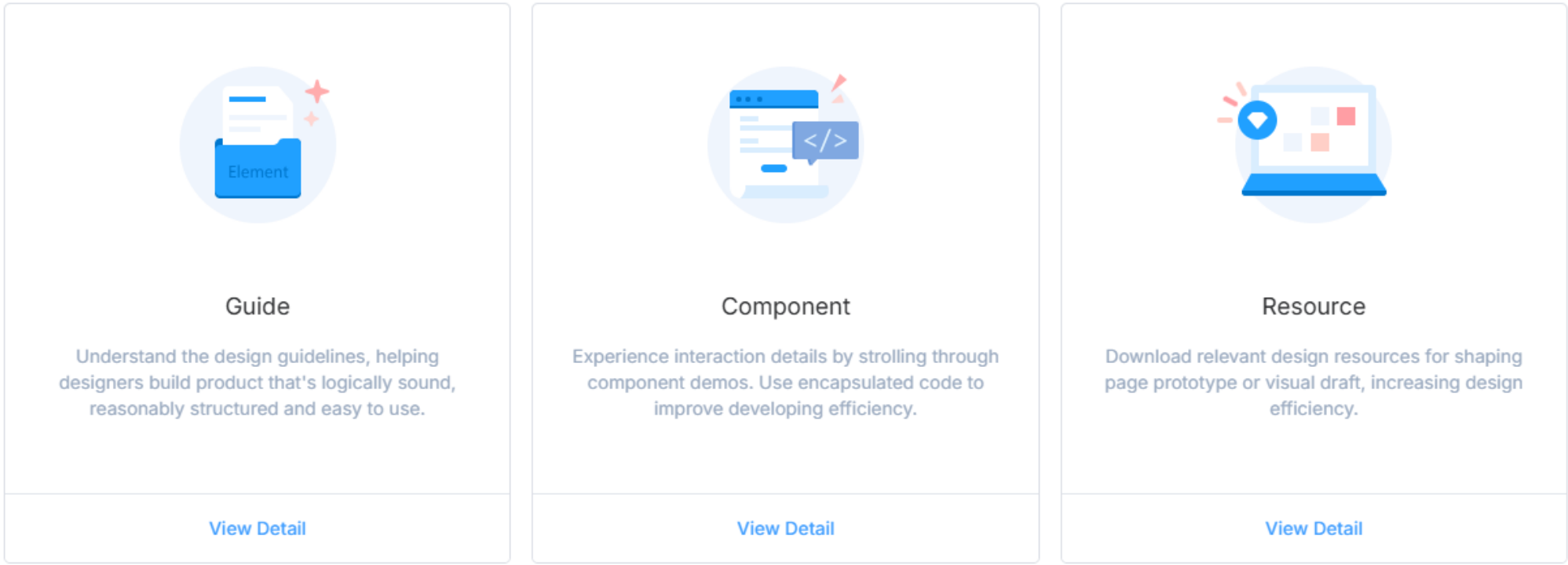
- Vue 3 생태계 주요 UI 라이브러리 비교

비교 항목	**Vuetify**	**Element Plus**	**PrimeVue**
기반 디자인 명세	Google Material Design	Enterprise Desktop View	Multi-Theme & Flex CSS
TypeScript 지원	지원 완료	**완전 기본 내장 (최상)**	지원 완료
스타일 격리/커스텀	SASS 변수 수정 방식	CSS 변수 수정 방식	**Unstyled 모드 지원 (최상)**
특화 컴포넌트	Mobile V-App Layout	**Data Table, Form Validation**	Advanced Chart, Tree Table
전용 컴포넌트 태그 예시	<v-btn>	<el-button>	<Button>

  - Global 시장에서는 PrimeVue와 Vuetify가 점유율이 높으며, 국내에서는 Element Plus의 점유율이 높다.
  - Element Plus의 학습 난이도가 가장 낮다.

Element Plus - 둘러보기

- https://element-plus.org/



Element Plus - Installation

- Installation

```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm install element-plus

added 21 packages, and audited 309 packages in 10s

82 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

- 설치 확인 (package.json)

```
"dependencies": {
  "axios": "^1.18.1",
  "element-plus": "^2.14.2",
  "pinia": "^3.0.4",
  "vue": "^3.5.32",
  "vue-router": "^5.0.4"
},
```

https://element-plus.org/en-US/guide/installation

Element Plus - Quick Start

- 전역 설정 주입 (src/main.js)

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import { createPinia } from 'pinia'

// 🟢 Element Plus 모듈 및 필수 CSS 장부 파일 Import
import ElementPlus from 'element-plus'
import 'element-plus/dist/index.css'

const app = createApp(App)

app.use(createPinia())
app.use(router)
app.use(ElementPlus) // 🟢 Vue 앱에 Element Plus 사용 등록

app.mount('#app')
```

https://element-plus.org/en-US/guide/quickstart

Element Plus Component - Basic

- 화면 구조를 잡고, Text나 Button 등 가장 기본적인 Component 모음.

컴포넌트명	전용 태그	기능 설명
Button	<el-button>	다양한 색상, 크기, 비활성화 등을 지원하는 실무 표준 버튼 컴포넌트
Border	*(CSS Utility)*	컴포넌트의 테두리 둥글기(Radius)와 두께 표준 디자인 시스템 규격
Color	*(CSS Utility)*	브랜드 메인 컬러, 성공, 실패 등 Element Plus가 규정한 색상 세트
Container	<el-container>	레이아웃 외곽을 잡는 부모 컨테이너 (<el-header>, <el-aside> 등과 결합)
Icon	<el-icon>	시스템 아이콘(화살표, 검색 등)을 손쉽게 주입하는 전용 아이콘 시스템
Layout	<el-row>, <el-col>	24분할 기반의 정밀한 반응형 그리드(Grid) 레이아웃 배치 시스템
Link	<el-link>	스타일 처리와 밑줄, 아이콘 결합이 내장된 텍스트 하이퍼링크 부품
Text	<el-text>	크기, 두께, 말줄임표(Truncate) 처리가 손쉬운 표준 텍스트 컴포넌트
Scrollbar	<el-scrollbar>	브라우저 기본 스크롤바 대신 세련된 커스텀 스크롤바를 씌우는 부품
Space	<el-space>	자식 컴포넌트들 간의 가로/세로 여백(Gap)을 균일하게 통제하는 인프라
Splitter	<el-splitter>	화면을 좌우/상하로 분할하고 마우스 드래그로 너비를 조절하는 레이아웃 부품
Typography	*(CSS Utility)*	웹 서비스 전체의 표준 글자 폰트, 크기, 행간 명세 장부

Element Plus Component - Configuration

- 애플리케이션 전체의 글로벌 설정을 중앙 통제하는 레이어이다.

컴포넌트명	전용 태그	기능 설명
Config Provider	<el-config-provider>	프로젝트 전체의 다국어 언어팩(Locale), 컴포넌트 기본 크기, z-index를 일괄 제어

Element Plus Component - Form

- 회원가입, 조건 검색 등 데이터를 입력하고 검증하는 컴포넌트이다.

컴포넌트명	전용 태그	기능 설명
Autocomplete	<el-autocomplete>	사용자가 입력 시 백엔드 추천 검색어 목록을 아래에 즉시 띄워주는 인풋창
Cascader	<el-cascader>	시/도 -> 구/군 -> 동 처럼 계층 구조의 데이터를 단계별로 선택하는 창
Checkbox	<el-checkbox>	다중 선택(체크박스) 및 그룹화 기능 컴포넌트
Color Picker	<el-color-picker>	마우스 클릭으로 웹 표준 컬러차트에서 색상을 선택하는 부품
Date Picker	<el-date-picker>	실무 빈도 최상. 달력이 팝업되어 날짜 또는 기간 범위를 선택하는 부품
DateTime Picker	<el-date-time-picker>	날짜 선택과 동시에 몇 시 몇 분 초(시간)까지 통합 정밀 선택하는 달력
Form	<el-form>, <el-form-item>	입력창들을 감싸서 실시간 데이터 검증(Validation) 및 경고 메시지를 뿜는 통제소
Input	<el-input>	기본 텍스트, 비밀번호(눈 아이콘), 한 번에 지우기(X) 등을 지원하는 필수 인풋
Input Number	<el-input-number>	오직 숫자만 입력받으며 +, - 버튼으로 수량을 조절하는 전용 인풋
Input Tag	<el-input-tag>	키워드를 입력하고 엔터를 치면 블록 태그 형태로 칩을 생성해 주는 입력창
Input OTP	<el-input-otp>	금융 앱이나 인증 번호 4~6자리를 한 칸씩 입력하도록 쪼개놓은 보안 인증 박스
Mention	<el-mention>	@나 #를 타이핑하면 슬랙이나 디스코드처럼 사용자 멘션 창을 띄워주는 부품

Element Plus Component - Form

- 회원가입, 조건 검색 등 데이터를 입력하고 검증하는 컴포넌트이다. (Cont'd)

컴포넌트명	전용 태그	기능 설명
Radio	<el-radio>	여러 단일 선택지 중 무조건 1개만 고르도록 통제하는 라디오 버튼
Rate	<el-rate>	쇼핑몰 별점 평점(★)을 마우스 드래그나 클릭으로 입력하는 부품
Select	<el-select>	화살표를 누르면 하부 옵션 목록이 슬라이딩 드롭다운되는 표준 선택 상자
Virtualized Select	<el-select-v2>	셀렉트 박스 안에 옵션이 수만 개일 때 화면 버벅임 없이 렌더링하는 가상 스크롤 버전
Slider	<el-slider>	바(Bar) 위의 볼륨 조절 슬라이더를 마우스로 밀어서 수치 범위를 지정하는 부품
Switch	<el-switch>	ON/OFF, 토글 모드, 다크모드 스위칭을 시각적으로 변환하는 버튼
Time Picker	<el-time-picker>	특정 시각을 시:분:초 단위로 롤링하여 정밀 선택하는 스크롤 피커
Time Select	<el-time-select>	09:00, 09:30 등 미리 지정된 정시 타임라인 목록 중 하나를 선택하는 박스
Transfer	<el-transfer>	왼쪽 바구니의 대량 목록 중 선택한 아이템들을 오른쪽 바구니로 이동시키는 검수기
TreeSelect	<el-tree-select>	조직도나 폴더 트리 구조 형태를 품고 있는 고급형 드롭다운 선택 상자
Upload	<el-upload>	마우스 클릭 혹은 드래그 앤 드롭으로 파일을 첨부해 백엔드로 전송하는 인프라

Element Plus Component - Data

- 백엔드에서 전달받은 데이터 등을 표나 리스트로 가공해 뿌리는 컴포넌트이다.

컴포넌트명	전용 태그	기능 설명
Avatar	<el-avatar>	유저 프로필 사진을 동그라미 혹은 사각형으로 이쁘게 크롭해 주는 부품
Badge	<el-badge>	알림 아이콘 우측 상단에 빨간색 숫자 [9+] 배지를 달아주는 알림 카운터
Calendar	<el-calendar>	화면 전체에 큼직한 월간 달력 판을 깔아서 스케줄을 기록 및 렌더링하는 컴포넌트
Card	<el-card>	외곽 섀도우 펜스를 치는 만능 레이아웃 블록
Carousel	<el-carousel>	광고 배너나 이미지들이 좌우로 슬라이딩 서커스하며 롤링되는 슬라이더 뷰어
Collapse	<el-collapse>	질문을 누르면 하부 답변 내용이 아코디언처럼 아래로 슥 펼쳐지는 접이식 메뉴
Countdown	<el-countdown>	타임세일, 시험 종료 시간 등을 실시간 초 단위로 마이너스 카운팅하는 시계
Descriptions	<el-descriptions>	회원 정보 조회 화면처럼 이름 : 홍길동 / 나이 : 20 구조의 정갈한 명세서 표
Empty	<el-empty>	"조회된 검색 결과가 없습니다" 라는 안내 이미지와 안내 문구를 자동 배치
Image	<el-image>	로딩 실패 시 대체 이미지 처리 및 클릭 시 확대(Viewer) 기능이 내장된 이미지 태그
Infinite Scroll	v-infinite-scroll	인스타그램처럼 스크롤을 맨 아래로 내리면 다음 데이터를 알아서 계속 이어 붙이는 장치

Element Plus Component - Data

- 백엔드에서 전달받은 데이터 등을 표나 리스트로 가공해 뿌리는 컴포넌트이다. (Cont'd)

컴포넌트명	전용 태그	기능 설명
Pagination	<el-pagination>	데이터가 많을 때 [1] [2] [3] ... [다음] 구조로 페이지를 쪼개주는 네비게이터
Progress	<el-progress>	진행률이나 다운로드 게이지 바를 퍼센트(%) 애니메이션 그래프로 보여주는 바
Result	<el-result>	결제 성공(Green 체크), 실패(Red 엑스) 화면을 아이콘과 함께 통째로 그려주는 완성판
Skeleton	<el-skeleton>	실제 데이터가 오기 전, 회색빛 유령 레이아웃을 띄워 유저의 체감 속도를 높이는 버퍼
Table	<el-table>	**실무 점유율 1위.** 정렬, 필터, 열 고정, 합계 연산이 탑재된 끝판왕 그리드 표
Statistic	<el-statistic>	매출액 1,500,000 처럼 숫자에 콤마를 달고 강조해 주는 통계 전용 텍스트 부품
Tag	<el-tag>	키워드나 상태값(맑음, 비, 완료)을 색상 배지 형태로 강조하는 칩 태그
Timeline	<el-timeline>	1일차 -> 2일차 -> 3일차 처럼 시간 흐름순 이력을 수직선 그래프로 정렬하는 카드
Tour	<el-tour>	신규 유저 진입 시 "여기를 클릭하세요" 하고 가이드를 돌며 팝업 팁을 안내하는 튜토리얼
Tree	<el-tree>	폴더 구조나 부서 조직도 데이터를 계층형 아코디언 구조로 트리화하는 컴포넌트
Watermark	<el-watermark>	기업 대외비 문서 화면 뒤쪽에 로그인 유저 이메일을 투명하게 도배하는 보안 부품
Upload	<el-upload>	마우스 클릭 혹은 드래그 앤 드롭으로 파일을 첨부해 백엔드로 전송하는 인프라

Element Plus Component - Navigation

- 화면 경로 이동 및 조종과 관련된 컴포넌트 컴포넌트이다.

컴포넌트명	전용 태그	기능 설명
Anchor	<el-anchor>	긴 스크롤 문서의 목차를 우측에 띄워 클릭 시 해당 세션 위치로 고속 워프하는 링크
Backtop	<el-backtop>	스크롤을 한참 내렸을 때 화면 우측 하단에 뜨는 "맨 위로 이동(▲)" 마법 버튼
Breadcrumb	<el-breadcrumb>	현재 유저 위치 경로를 Dashboard > Weather > Suwon 형태로 정렬해 주는 텍스트 바
Dropdown	<el-dropdown>	마우스를 올리거나 클릭하면 하부 액션 메뉴 목록이 주르륵 내려오는 드롭다운 메뉴
Menu	<el-menu>	사내 시스템 좌측에 들어가는 정석적인 아코디언 사이드바 내비게이션 메뉴 시스템
Page Header	<el-page-header>	페이지 상단 뒤로가기 버튼과 현재 서브 타이틀을 잡아주는 표준 헤더 가이드라인
Steps	<el-steps>	정보입력 -> 본인인증 -> 가입완료 단계 진행 상태를 숫자로 시각화하는 바
Tabs	<el-tabs>	한 화면 안에서 1번 탭, 2번 탭을 누를 때마다 하부 본문만 스위칭해 주는 전환 탭

Element Plus Component - Feedback

- 사용자에게 알림, 경로, 로딩, 확인 등의 신호를 보내는 컴포넌트이다.

컴포넌트명	전용 태그	기능 설명
Alert	<el-alert>	화면 상단 고정 공간에 경고나 성공 공지사항 문구를 알림 띠 형태로 박아놓는 부품
Dialog	<el-dialog>	화면 중앙에 어두운 딤드(Dimmed) 처리를 하고 팝업창을 띄우는 정석 모달창
Drawer	<el-drawer>	스마트폰 앱 메뉴처럼 화면 우측이나 좌측에서 거대한 서랍장이 스르륵 열리는 슬라이드 창
Loading	v-loading *(Directive)*	태그에 이 지시어만 적어주면 데이터 수신 중일 때 예쁜 회전 스피너와 막을 쳐주는 사양
Message	ElMessage *(JS Call)*	화면 상단 중앙에 2초간 "저장 완료" 토스트 알림을 띄우고 하늘로 사라지는 일시 알림창
MessageBox	ElMessageBox	브라우저 구식 alert(), confirm()을 완벽히 대체하는 세련된 최종 확인 팝업창
Notification	ElNotification	화면 우측 구석탱이에서 윈도우 알림처럼 상세 메시지 카드를 띄워주는 고급 알림창
Popconfirm	<el-popconfirm>	삭제 버튼을 누르면 버튼 바로 위에 조그맣게 "진짜 지울래?" 말풍선 팝업을 띄우는 팝업 확인창
Popover	<el-popover>	마우스를 특정 단어에 올리면 상세 가이드 백과사전 설명창을 풍선처럼 띄워주는 부품
Tooltip	<el-tooltip>	아이콘에 마우스를 대면 위/아래에 "검색하기" 같은 툴팁 힌트 글자를 출력해 주는 기능

Element Plus Component - Others

- 기타 특수 유틸리티 시각 요소 및 기능과 관련된 컴포넌트이다.

컴포넌트명	전용 태그	기능 설명
Affix	<el-affix>	스크롤을 내려도 특정 메뉴나 버튼이 화면 최상단에 껌딱지처럼 계속 고정되어 따라오는 장치
Animate	*(CSS Utility)*	컴포넌트들이 화면에 나타나거나 사라질 때 주는 내장 애니메이션 효과 세트
Segmented	<el-segmented>	라디오 버튼을 가로형 슬라이딩 스위치 바 탭 형태로 모던하게 진화시킨 제어 컨트롤러

Code Challenge

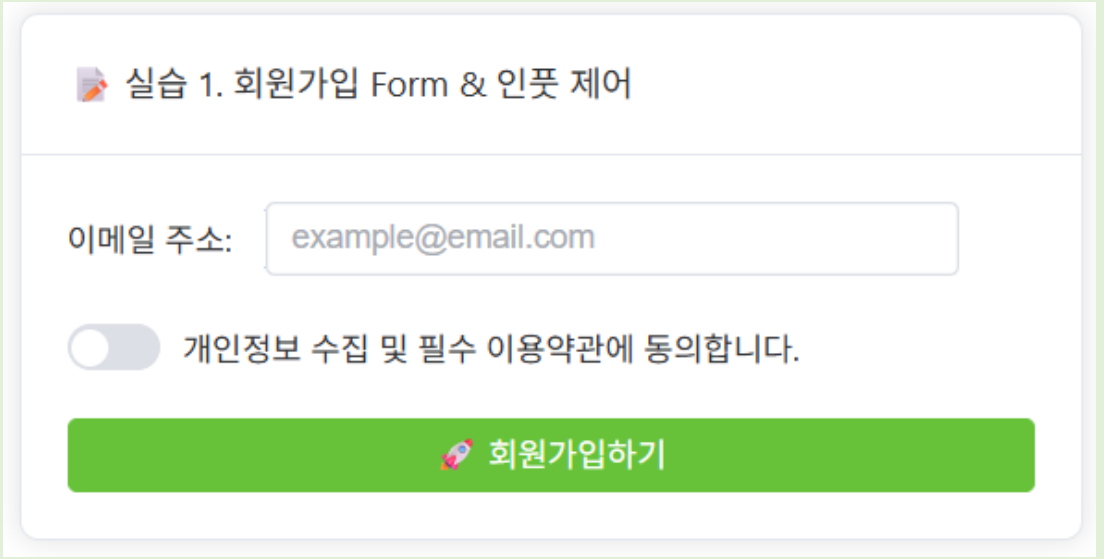
- userForm Object

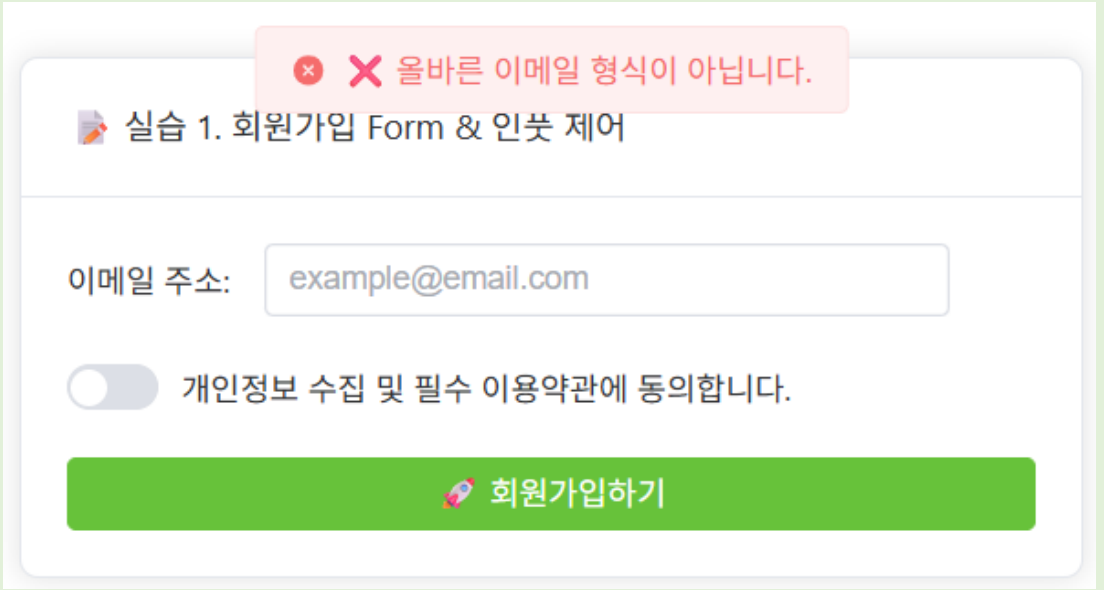
```
const userForm = ref({
  email: '',
  agree: false,
})
```

- userForm Validation

```
const handleRegister = () => {
  if (!userForm.value.email.includes('@')) {
    ElMessage.error('❌ 올바른 이메일 형식이 아닙니다.')
    return
  }
  if (!userForm.value.agree) {
    ElMessage.warning('⚠ 이용약관에 동의하셔야 합니다.')
    return
  }
  ElMessage.success('🎉 가입 신청이 정상적으로 완료되었습니다!')
}
```

- Template
  - <el-card> : v-slot 및 기타 속성 사용
  - <el-input> : 이메일주소 입력 창
  - <el-switch> : 동의 Switch
  - <el-button> : 회원가입 버튼





Code Challenge

- 반응형 Data

```
const productQuantity = ref(1) // 수량 카운터 기본값
const productRate = ref(4) // 별점 기본값 (별 4개)
```

- Template
  - <el-card> : v-slot 및 기타 속성 사용
  - <el-input-number> : 구매수량입력



Code Challenge

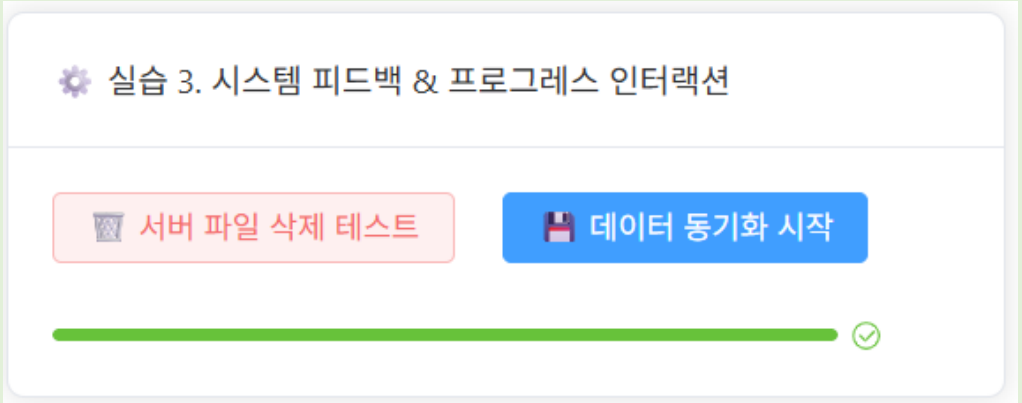
- 반응형 Data

```
const downloadProgress = ref(0)
const isDownloading = ref(false)
```

- 파일삭제 Confirm

```
const confirmDelete = () => {
  ElMessageBox.confirm('서버에서 해당 파일을 영구히 삭제하시겠습니까?',
'🩸 최종 경고', {
    confirmButtonText: '네, 삭제합니다',
    cancelButtonText: '취소',
    type: 'danger',
  })
    .then(() => {
      ElMessage.success('🗑️ 파일이 안전하게 파쇄되었습니다.')
    })
    .catch(() => {
      ElMessage.info('❌ 삭제 작업이 취소되었습니다.')
    })
}
```

- Template
  - <el-card> : v-slot 및 기타 속성 사용
  - <el-button> : 삭제 테스트 버튼, 동기화 시작 버튼
  - <el-progress> : 진행률 표시



- 게이지 바 애니메이션

```
const startDownload = () => {
  if (isDownloading.value) return (isDownloading.value = true)
  downloadProgress.value = 0

  const interval = setInterval(() => {
    downloadProgress.value += 20
    if (downloadProgress.value >= 100) {
      clearInterval(interval)
      isDownloading.value = false
      ElMessage.success('💾 대용량 데이터 로드가 완료되었습니다!')
    }
  }, 400)
}
```

[실습] 과제 - Element Plus 활용

- 3일차 과제에 Element Plus를 자유롭게 적용해 본다.



Full-Stack Engineering - Frontend-framework: Vue.js

9. Modern JavaScript

SK 주식회사 AX

Modern JavaScript - History

- 1세대: ECMAScript 탄생 (1995~1999)
  - Netscape (1995) - Web Browser의 동적인 기능을 넣기 위해 설계된 언어가 JavaScript의 시초
  - ECMAScript (1997) - Microsoft Internet Explorer에서 JavaScript와 유사한 Jscript 탑재하자 Browser 마다 코드가 다르게 동작하여 이를 막기 위해 ECMA 국제 표준기구에서 JavsScript 표준 규격을 정의 (ECMAScript = ES)
- 2세대: Web의 한계와 jQuery (2000~2008)
  - Web의 한계 - Internet Explorer의 독점으로 표준화 작업(ES4)이 무산. 이 시기 JavaScript는 간단한 팝업창을 띄우는 것 같은 가벼운 Visual Script 취급.
  - jQuery 등장 - Browser마다 JavaScript 해석 방식이 달라 Browser 별로 코드를 다르게 작성할 때, Cross-Browsing을 해결한 JavaScript Library인 jQuery가 등장하여 시장을 지배
- 3세대: ES5의 표준 (2009~2014)
  - Google V8 엔진과 Node.js (2009) - Chrome이 탄생하며 V8 JavaScript 엔진을 공개. 이를 기반으로 JavaScript가 Browser가 아닌 Server 환경에서도 실행할 수 있게 됨 (Node.js의 탄생)
  - ES5의 표준 정착 (2009) - ES5가 표준으로 제정되며, 안전한 코딩을 위한 'use strict', 배열 전용 고차함수 (forEach, map, filter) 등이 도입
- 4세대: Modern JavaScript 시대 (2015~)
  - ES6 (ECMAScript 6, ECMAScript 2015) - 문법, 기능, 개선사항이 많이 도입하여 언어를 혁신적으로 뜯어고침 (대규모 개편)
  - 연례 업데이트 정착 : ES6이후 매년 소규모 문법을 발표함. 이를 통칭하여 Modern JavaScript라고 함.

Modern JavaScript - Browser Support

- 최신 Browser의 ECMAScript 반영 현황 (2026년 기준)
  - ES6 (2015) ~ ES11 (2020) 규격: let/const, Arrow Function, Promise, async/await, Optional Chaining(?.) 등은 현재 데스크톱 및 모바일 브라우저를 가리지 않고 100% 네이티브로 지원
  - ES12 (2021) ~ ES15 (2024) 규격: 비교적 최신 기능인 replaceAll(), Logical Assignment (&&=, ||=), Array.prototype.toReversed() 등 배열 변경 방지 메서드들도 현재 모던 브라우저 점유율 기준 96% 이상 지원.
  - 최신 ES16+ (2025~2026) 신규 후보 문법: 현재 제안 단계(Stage 3~4)에 있거나 막 통과된 극 최신 문법들은 크롬 등 카나리 버전에 선반영되어 실험적 기능으로 작동하며, 브라우저 버전업에 따라 순차적으로 자동 탑재된다.
- 브라우저가 지원하지 않는 최신/과거 문법을 해결하는 기술
  - 실무 Frontend 생태계는 브라우저의 반영을 기다리지 않고, 개발자가 최신 문법으로 코딩하면 컴퓨터가 알아서 구형 문법으로 번역해 주는 도구 체인을 구축해 두었음
  - Babel: 개발자가 2026년형 최고급 ES15+ 문법으로 코딩을 해도, 빌드 시스템이 배포 직전에 구형 브라우저도 알아들을 수 있는 안전한 ES5(2009년형) 문법으로 코드를 다운그레이드 번역
  - Polyfill: 구형 브라우저 엔진에 아예 존재하지 않는 최신 객체나 메서드(예: Promise, Array.prototype.includes)가 실행되면 에러가 난다. Polyfill은 브라우저가 켜질 때 해당 부분을 JavaScript로 직접 구현해서 엔진에 임시로 제공하는 Script Code이다.
  - **우리가 쓰고 있는 Vite 번들러 내부에는 이 Babel과 Polyfill 기반의 변환 엔진이 기본 내장되어 있으므로 개발자는 브라우저 호환성을 전혀 걱정하지 않고 가장 트렌디한 최신 문법으로 코딩할 수 있다**

Core Syntax - let & const

- let vs const vs var

특성	var (구시대의 유산)	let (변수)	const (상수)
스코프 (Scope)	**함수** 레벨 스코프	**블록** 레벨 스코프 ({})	**블록** 레벨 스코프 ({})
재선언	가능 (버그의 원인)	불가능	불가능
재할당	가능	가능	**불가능**
호이스팅 (Hoisting)	발생 (undefined로 초기화)	발생 (TDZ로 인해 에러 발생)	발생 (TDZ로 인해 에러 발생)

- 재선언/재할당 Example

```
var name = “철수”;
var name = “영희”;
console.log(name);  // 출력결과: 영희
```

```
const name = “철수”;
const name = “영희”; // 에러 발생: Identifier ‘name’ has already been declared
```

```
let name = “철수”;
name = “영희”;
console.log(name);  // 출력결과: 영희
```

```
const name = “철수”;
name = “영희”;      // 에러 발생: Assignment to constant variable
```

Core Syntax - Arrow Function

- JavaScript 함수 선언 방식 비교

비교 항목	Function Declaration	Function Expression	Arrow Function

Syntax	function foo(arg1, arg2,…, arg n) {
	  return;
	}
		const foo = function(arg1, arg2,…, arg n) {
		  return;
		}
			const foo = (arg1, arg2,…, arg n) => {
			  return;
			}

Hoisting	**함수 전체가 호이스팅됨** (선언문 전에도 호출 가능)	**변수만 호이스팅됨** (초기화 전 호출 시 에러 발생)	**변수만 호이스팅됨** (초기화 전 호출 시 에러 발생)
주요 용도	전통적인 전역 / 유틸리티 함수 정의	클로저(Closure) 구현, 콜백 함수	**모던 프레임워크(Vue/React)**, 메서드 내부 비동기 콜백 함수

- Arrow Function 특징
  - 함수 내부에 코드가 한 줄이면 return 문 생략 가능

```
const sum = (num1, num2) => num1 + num2;
```

  - 매개변수가 1개이면 소괄호 생략 가능

```
const pow = x => x * x;
```

  - 화살표 함수를 매개변수로 전달 가능

```
const calculate = (num1, num2, operation) => {
  return operation(num1, num2); // 배달받은 화살표 함수를 여기서 대신 실행(실행 대행)
};
const addResult = calculate(10, 5, (a, b) => a + b);
console.log(`➕ 더하기 결과: ${addResult}`); // 출력: 15
const multiplyResult = calculate(10, 5, (a, b) => a * b);
console.log(`✖ 곱하기 결과: ${multiplyResult}`); // 출력: 50
```

Core Syntax - Template Literals

- Template Literal
  - ES6에서 도입된 문자열 표기법으로, 문자열 내부에 변수나 연산 결과를 동적으로 주입하고 줄바꿈을 자유롭게 허용하는 문법
  - 일반 따옴표(')나 쌍따옴표(")가 아닌, 키보드 숫자 1 왼쪽에 있는 **Backtick(`)** 기호를 사용하여 문자열 값을 선언할 수 있다.

```
let str = `he”l’lo`;
console.log(str);  // 출력결과 he”l’lo
```

  - 문자열 중간에 ${변수명} 혹은 ${연산식}을 적어주면 자바스크립트가 이를 자동으로 계산해서 문자열로 합쳐준다.

```
const city = '수원’;
const temp = 24;
const message = '현재 ' + city + '의 기온은 ' + temp + '도입니다.';
```



```
const city = '수원’;
const temp = 24;
// 백틱 안에서 ${}로 해결
const message = `현재 ${city}의 기온은 ${temp}도입니다.`;
```

  - 기존에는 문자열 안에서 줄바꿈을 하려면 개행 문자(□n)를 강제로 집어넣고 더해줘야 했지만, 템플릿 리터럴은 백틱 안에서 Enter를 치는 대로 줄바꿈이 그대로 반영된다.

```
const htmlTemplate = '<div>\n' +
                     '  <h1>Hello</h1>\n' +
                     '</div>';
```



```
// 엔터 치는 모양 그대로 문자열이 보존됨
const htmlTemplate = `
  <div>
    <h1>Hello</h1>
  </div>
`;
```

Core Syntax - Destructuring Assignment

- Destructuring Assignment (비구조화 할당, 구조분해 할당)
  - Array나 Object의 구조를 분해하여, 내부의 값들을 별도의 독립된 개별 변수에 각각 직접 할당하는 모던 JavaScript 표현식
  - 순수 인덱스 접근(arr[0])이나 점 표기법(obj.key)을 반복해서 치던 번거로움을 제거하고 코드 라인 수를 획기적으로 단축한다.
- Object Destructuring Assignment
  - 객체의 내부 key를 기준으로 매칭하여 데이터를 추출한다. 순서는 상관없으며, 이름만 맞으면 일치하는 변수에 값이 할당된다.

```
const user = { name: '홍길동', age: 20, role: 'admin' };

const name = user.name;
const age = user.age;
```



```
const user = { name: '홍길동', age: 20, role: 'admin' };

const { name, age } = user;
```

- Array Destructuring Assignment
  - Index 위치(순서)를 기준으로 대칭 할당된다.

```
const coords = [37.5, 127.0];

const latitude = coords[0];
const longitude = coords[1];
```



```
const coords = [37.5, 127.0];

// ◍ 대괄호 안의 순서대로 각각 0번째, 1번째 값이 배정됨
const [latitude, longitude] = coords;
```

  - 특정 위치를 건너뛰고 싶을 때 (쉼표 공백 배치)

```
const colors = ['red', 'green', 'blue'];
const [first, , third] = colors; // green은 건너뛰고 'red'와 'blue'만
```

Core Syntax - Spread Operator (...)

- Spread Operator (스프레드 연산자)
  - 배열이나 객체앞에 마침표 3개(...)를 붙여, 그 내부의 요소를 **하나하나 낱개로 순서대로 '펼쳐서(전개하여)** 흩뿌려주는' 연산자
  - 복잡한 반복문 없이 대량의 데이터를 복사하거나 결합(병합)할 수 있다.
- Array에서의 활용
  - 배열 병합 (기존 배열의 괄호를 풀어서 새로운 배열 안에 순서대로 흩뿌려 병합)

```
const frontEnd = ['HTML', 'CSS', 'Vue'];
const backEnd = ['Java', 'Spring'];
const fullStack = [...frontEnd, ...backEnd, 'Git'];
console.log(fullStack); // ['HTML', 'CSS', 'Vue', 'Java', 'Spring', 'Git']
```

  - 배열 복사 (얕은 복사)

```
const original = [1, 2, 3];

const cloneWrong = original;      // ❌ 단순 대입(=)은 주소값만 복사되어 복사본을 고치면 원본도 깨짐
const cloneRight = [...original]; // 🟢 스프레드를 쓰면 원본과 주소가 완전히 분리된 독립적인 새 복사본이 생성됨

cloneRight.push(99);
console.log(original);   // [1, 2, 3] (원본 안전 보존)
console.log(cloneRight); // [1, 2, 3, 99]
```

Core Syntax - Spread Operator (...)

- Object에서의 활용
  - 기존 객체의 속성을 그대로 유지하면서 특정 데이터만 수정하거나 추가된 새로운 객체를 리턴할 때 사용.
  - Vue의 상태 관리나 백엔드 요청 바디를 만들 때 필수 문법

```
const baseConfig = { theme: 'dark', language: 'ko', version: 1.0 };

// 🟢 기본 설정을 복사해 오되, version만 2.0으로 덮어쓰고 새로운 속성 추가하기
const newConfig = {
  ...baseConfig,
  version: 2.0,    // 동일한 key가 있으면 뒤에 적힌 녀석이 앞의 값을 덮어씁니다 (Override)
  author: 'Graves' // 새로운 key-value 추가
};

console.log(newConfig);
// { theme: 'dark', language: 'ko', version: 2.0, author: 'Graves' }
```

- 문자열 전개
  - 문자열을 하나씩 쪼개서 전개할 수 있다.

```
const str = `Hello`;
const charArry = [...str];
console.log(charArray);     // [`H`, `e`, `l`, `l`, `o`]
```

- 함수 인수 전개

```
Function sum(a, b, c) {
  return a+b+c;
}
const numbers = [1, 2, 3];
const result = sum(...numbers);  // 6이 할당됨
```

Core Syntax - Rest 문법 (...)

- Rest 문법
  - **똑같이 ... 기호를 쓰지만** Spread가 주로 값을 전개하는데 활용하지만 Rest는 나머지 값을 처리하는데 주로 사용된다.
- Array/Object Destruncturing Assignment에서의 REST
  - 몇 개만 변수로 빼고 남은 속성을 한데 묶어 별도의 객체나 배열로 보존

```
const employee = {
  name: 'Graves',
  age: 35,
  role: 'Instructor',
  team: 'Edu-Tech',
  location: 'Seoul'
};

const { name, age, ...restInfo } = employee; // 🟢 name과 age만 개별 변수로 꺼내고, 나머지 속성들은 restInfo 객체에 담아라!
console.log(name);     // 'Graves'
console.log(age);      // 35
console.log(restInfo); // { role: 'Instructor', team: 'Edu-Tech', location: 'Seoul' }
```

- 함수 매개변수에서의 Rest (나머지 매개변수)

```
// 🟢 앞의 두 개는 1등, 2등 변수에 담고, '나머지(Rest)' 참가자들은 한꺼번에인 others 배열 주머니에 수집해라!
const printMedalList = (gold, silver, ...others) => {
  console.log(`🥇 금메달: ${gold}`);            // 🥇 금메달: 수원
  console.log(`🥈 은메달: ${silver}`);          // 🥈 은메달: 서울
  console.log(`🏃 나머지 참가자 명단:`, others); // 🏃 나머지 참가자 명단: ['부산', '대구', '제주', '광주']
};

printMedalList('수원', '서울', '부산', '대구', '제주', '광주');
```

Core Syntax - Promise

- JavaScript Promise Object
  - JavaScript 엔진에서 비동기 연산의 최종 완료 또는 실패와 그 결과 가치를 나타내는 표준 객체(Object)이다.
  - Promise는 비동기 작업이 성공하거나 실패할 때 그 결과를 알려주겠다고 "약속"하는 객체
  - ES6 이전에는 주로 비동기 처리가 끝난 후 실행할 로직을 함수 인자로 전달하는 Callback 함수를 이용 → Callback Hell 유발
- Promise의 3가지 상태(State)

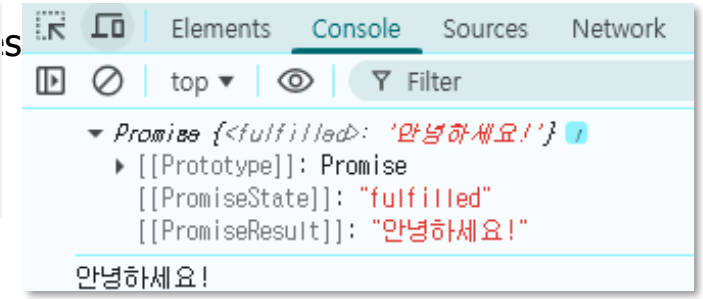
Pending (대기)	The initial state; the operation has started but is neither fulfilled nor rejected.
Rejected (거부/실패)	The operation failed, and a reason (error) is available
Fulfilled (이행/성공)	The operation completed successfully, and a value is available.

- Promise Chains (.then / .catch)
  - Promise 객체 뒤에 Method를 체인 형태로 엮어서 성공과 실패 파이프라인을 통제한다.

```
fetchWeatherData()                  // fetchWeatherData는 Promise 객체를 return;
  .then((data) => {
    console.log('서버 통신 성공:', data);        // 🟢 Fulfilled 상태일 때 실행: 성공 데이터 가공
  })
  .catch((error) => {
    console.error('네트워크 에러 발생:', error);  // 🔴 Rejected 상태일 때 실행: 에러 예외 처리
  })
  .finally(() => {
    console.log('비동기 통신 프로세스 완전 종료'); // 🟡 성공/실패 여부와 무관하게 무조건 마지막에 실행 (예: 로딩 스피너 종료)
  });
```

Core Syntax - async / await

- ES8(2017)에서 도입, Promise를 기반으로 작동하되 비동기 흐름(Chaining)대신, 동기식 코드 구조로 작성할 수 있게 준다.
- Async
  - 이 함수 안에서 비동기 처리를 할 거야!"라고 선언하는 키워드
  - async 함수는 항상 Promise를 반환한다.
  - ```
    async function hello() {
      return "안녕하세요!";
    }
    console.log(hello());
    hello().then(res => console.log(res));
    ```



- Await
  - "비동기 작업이 끝날 때까지 다음 줄로 넘어가지 말고 기다려!"라고 명령하는 키워드
  - ```
    async function handleData() {
      try {
        const result = await fetchData();       // 첫 번째 비동기 기다림
        const saveResult = await saveToDatabase(result); // 두 번째 비동기 기다림
        console.log("저장 성공!", saveResult);
      } catch (error) {
        console.log("에러 발생!", error);
      }
    }
    ```

Core Syntax - Array Methods

- ES6 이후 도입된 Array Methods

도입 버전	기능 및 메서드명	전용 문법 / 예시 코드	기술적 핵심 요약 및 실무 용도
ES6 (2015)	**Array.from()**	Array.from(arguments);	유사 배열 객체(Arguments, DOM NodeList)를 순수 배열로 변환하여 배열 내장 고차 함수를 쓸 수 있게 유도.
ES6 (2015)	**find()**	arr.find(item => item.id === 3);	조건식 화살표 함수를 판별하여, 만족하는 **최초의 '아이템 알맹이 자체'**를 반환 (없으면 undefined).
ES6 (2015)	**findIndex()**	arr.findIndex(item => item.id === 3);	조건식을 만족하는 **최초의 '방 번호(인덱스 숫자)'**를 반환 (없으면 -1).
ES7 (2016)	**includes()**	arr.includes('수원');	배열 내 특정 값이 박혀있는지 여부를 판별해 오직 **true / false**만 반환. 구식 indexOf 조건식 전면 대체.
ES10 (2019)	**flat()**	[1, [2, 3]].flat();	[[1, 2], [3, 4]] 처럼 다차원으로 중첩된 다층 배열을 지정한 깊이만큼 평평하게 1차원 배열로 펴주는 부품.
ES13 (2022)	**at()**	arr.at(-1);	대괄호 인덱서(arr[arr.length - 1]) 대신, **음수 인덱스(-1)를 지원**하여 맨 뒤의 요소를 쉽게 역추적하는 메서드.
ES14 (2023)	**toReversed()** **toSorted()** **toSpliced()**	const newArr = arr.toSorted();	**[최신 실무 표준]** 원본 배열을 강제로 변형(Mutate)시키던 구형 메서드들과 달리, **원본을 안전하게 보존하면서 정렬/반전된 '새로운 복사본 배열'을 리턴**하는 불변성 메서드 세트.

Core Syntax - Object

- ES6 이후 도입된 Object 기능 명세

도입 버전	기능 및 메서드명	전용 문법 / 예시 코드	기술적 핵심 요약 및 실무 용도
ES6 (2015)	**단축 속성명** (Property Shorthand)	const user = { name, age };	객체를 만들 때 key 이름과 대입할 변수명이 같다면, name: name처럼 중복해서 쓰지 않고 **이름만 한 번만 적어줘도 자동으로 매핑**되는 축약 문법.
ES6 (2015)	**계산된 속성명** (Computed Property)	const obj = { [keyName]: value };	객체의 key(속성명) 자리에 고정된 문자열이 아니라, **대괄호[]를 쳐서 변수나 연산식의 결과물을 실시간 key 명으로 박아넣는** 기술.
ES6 (2015)	**메서드 축약 표현** (Method Shorthand)	const obj = { greet() {} };	객체 내부에서 함수(메서드)를 정의할 때 : function 키워드를 완전히 생략하고 **함수명() {}** 형태로 담백하게 선언하는 문법.
ES6 (2015)	**Object.assign()**	Object.assign(target, src);	여러 객체의 속성들을 하나의 대상 객체로 병합하거나 복사할 때 쓰던 객체 병합 메서드. (현재는 스프레드 연산자에 밀려 빈도 감소)
ES8 (2017)	**Object.keys()** **Object.values()** **Object.entries()**	Object.values(user); Object.entries(user);	객체를 다루기 힘든 단점을 극복하기 위해, 객체의 **[key 배열]**, **[value 배열]**, **[[key, value] 쌍의 2차원 배열]]**을 각각 추출하여 배열로 변환하는 메소드.
ES11 (2020)	**Optional Chaining**	user?.profile?.address	**[실무 빈도 극상]** 깊숙한 객체를 참조할 때 중간에 데이터가 비어 있어도 (null/undefined) 에러로 뻗지 않고, 안전하게 **undefined를 뱉으며 프로그램을 보호**하는 문법.

Core Syntax - Object Example

- Property Shorthand & Method Shorthand

```
const title = 'Vue 3 특강';
const price = 99000;

// 구식 방식 (ES5)
const courseOld = {
  title: title,
  price: price,
  getInfo: function() { return 'ES5 스타일'; }
};

// 모던 방식 (ES6+) - 변수명과 key가 같으면 퉁치고, function도 걷어냅니다.
const courseModern = {
  title,
  price,
  getInfo() { return 'ES6 스타일'; }
};
```

- Computed Property Name (동적인 Key 입력이 필요할 때)

```
const inputType = 'email';

const userForm = {
  name: '홍길동',
  // 🟢 변수 inputType의 값인 'email'이 이 객체의 진짜 key 이름으로 실시간 이식됩니다.
  [inputType]: 'hong@email.com'
};

console.log(userForm.email); // 'hong@email.com' 출력
```

Core Syntax - Object Example

- Object.entries() : 객체를 [[key, value] 쌍의 2차원 배열]]로 전환

```
const scoreBoard = { math: 90, english: 80, science: 100 };

// 객체를 [['math', 90], ['english', 80], ['science', 100]] 이라는 2차원 배열로 쪼개준다.
const entries = Object.entries(scoreBoard);

// 배열이 되었으니 모던 배열 메서드로 자유롭게 순회 및 비구조화 할당 가동!
entries.forEach(([subject, score]) => {
  console.log(`📑 과목: ${subject}, 💯 점수: ${score}`);
});
```

Core Syntax - Object Example

- Optional Chaining (?.)
  - .왼쪽에 있는 대상이 null이거나 undefined이면, 다음 하위 속성으로 진입하지 않고 에러 없이 undefined를 반환

```
// 📦 1번 유저 (모든 정보가 완벽함)
const user1 = {
  name: 'Graves',
  profile: {
    address: { city: 'Suwon' }
  }
};

// 📦 2번 유저 (프로필 정보가 아예 누락되어 빈 객체로 옴)
const user2 = {
  name: '홍길동'
  // profile 속성이 물리적으로 존재하지 않는 상태 (undefined)
};

// 🟢 모던 자바스크립트 표준 방어법
// "탐색하다가 도중에 null이나 undefined를 만나면 에러 내지 말고 거기서 즉시 중단하고 덤덤하게 undefined 뱉어라"
const cityModern1 = user1?.profile?.address?.city;
const cityModern2 = user2?.profile?.address?.city;

console.log(cityModern1); // 출력: 'Suwon'
console.log(cityModern2); // 출력: undefined (✨ 에러 없이 안전하게 통과!)

// 옵셔널 체이닝(?.) 뒤에 Null 병합 연산자(??)를 붙여 안전장치 이중 락을 겁니다.
const finalCity = user2?.profile?.address?.city ?? '등록된 주소 없음';

console.log(finalCity); // 출력: '등록된 주소 없음' (유저 친화적 UI 구현 완벽 가능)
```

Core Syntax - Nullish Coalescing Operator (??)

- Nullish Coalescing Operator (??) - Null 병합 연산자
  - ES11(ECMAScript 2020)에 도입된 모던 문법
  - 좌항의 피연산자가 null이거나 undefined일 때만 우항의 기본값을 반환하고, 그 외의 값이면 좌항의 값을 그대로 유지하는 연산자
  - 과거에는 기본값을 줄 때 논리합(||) 연산자를 썼다. 하지만 || 연산자는 null/undefined뿐만 아니라 자바스크립트가 false으로 취급하는 모든 Falsy 값(0, “”, false)까지 전부 카운트하여 우항의 기본값으로 덮어버리는 버그를 유발했다.

```
const userSetting = {
  alertCount: 0,
  nickname: "" // 닉네임을 아직 입력 안 해서 빈 문자열인 상황
};

// ❌ 1. 구식 OR (||) 방식의 버그
const countOld = userSetting.alertCount || 10;
const nameOld = userSetting.nickname || '익명';
console.log(countOld); // 출력: 10 (💥 버그: 유저는 분명 0회 알림을 원했는데 10회로 조작됨)
console.log(nameOld);  // 출력: '익명' (💥 버그: 유저가 닉네임을 공백으로 비워두고 싶었어도 강제 변환됨)

// 🟢 2. 모던 Nullish (??) 방식
// 오직 null이나 undefined일 때만 우항으로 넘어간다. 숫자 0이나 빈 문자열은 '데이터'로 인정한다.
const countModern = userSetting.alertCount ?? 10;
const nameModern = userSetting.nickname ?? '익명';

console.log(countModern); // 출력: 0  (🟢 안전 보존: 유저가 의도한 0이 정확히 살아남음)
console.log(nameModern);  // 출력: "" (🟢 안전 보존: 빈 문자열 데이터 유지)
```

Code Challenge

- 다음 미션을 수행하시오

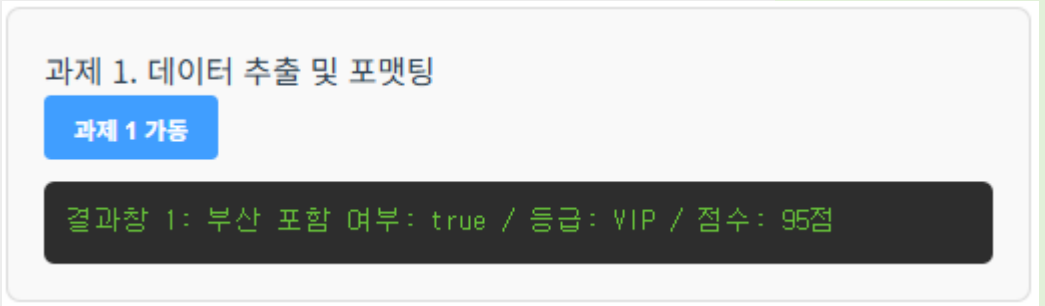
```vue
<script setup>
import { ref } from 'vue'

// 실시간 화면 출력을 위한 Vue 상태값 (수정 금지)
const result1 = ref('')

// ================================================================
// 📝 [과제 1] 회원 명단 가공 및 VIP 추출 핸들러
// ================================================================
const runTask1 = () => {
  const members = ['김수원', '이서울', '박부산', '최대전']
  const rawData = { id: 101, grade: 'VIP', details: { score: 95 } }

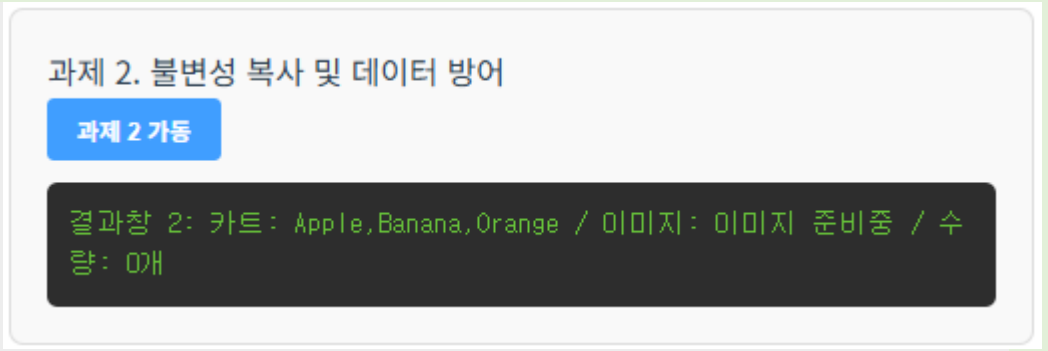
  // [미션]
  // 1. members 배열에 '박부산'이 포함(includes)되어 있는지 확인하여 담기 변수(memberContainsPark)에 담기
  // 2. rawData 객체에서 grade와 details 내부의 score를 비구조화 할당으로 한 줄로 추출하기.
  // 3. 추출한 변수들을 템플릿 리터럴을 사용하여 result1.value에 아래 양식으로 문자열 주입하기.
  //    양식: "부산 포함 여부: [결과] / 등급: [결과] / 점수: [결과]점"
}
</script>

<template>
  <div class="quiz-container">
    <div class="card">
      <h3>과제 1. 데이터 추출 및 포맷팅</h3>
      <button @click="runTask1">과제 1 가동</button>
      <div class="console">결과창 1: {{ result1 }}</div>
    </div>
 </div>
</template>
```



Code Challenge

- 다음 미션을 수행하시오




```
<script setup>
import { ref } from 'vue'

// 실시간 화면 출력을 위한 Vue 상태값 (수정 금지)
const result2 = ref('')

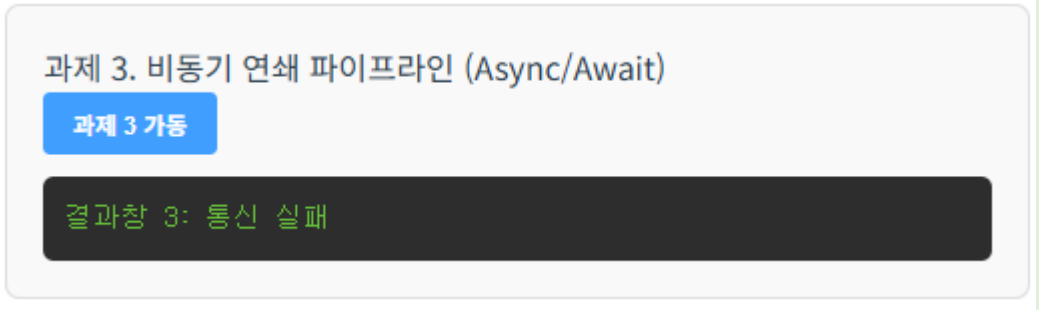
// ================================================================
// 📝 [과제 2] 장바구니 상품 추가 및 기본값 방어 시스템
// ================================================================
const runTask2 = () => {
  const currentCart = ['Apple', 'Banana']
  const newProduct = { name: 'Orange', stock: 0, preview: null }

  // [미션]
  // 1. 스프레드 연산자를 사용하여 currentCart 배열 뒤에 newProduct.name을 결합한 새 배열 'updatedCart' 만들기.
  // 2. 옵셔널 체이닝(?.)과 널 병합 연산자(??)를 연쇄로 사용하여, newProduct.preview 값이 null이거나 undefined일 때 '이미지 준비중'이라는 문자열을 변수 'imgStatus'에 담기.
  // 3. 널 병합 연산자(??)를 활용하여, newProduct.stock이 0일 때 안전하게 변수 'finalStock'에 담기도록 조치하기.
  // 4. 가공 결과를 result2.value에 템플릿 리터럴로 주입하기.
  //    양식: "카트: [updatedCart] / 이미지: [imgStatus] / 수량: [finalStock]개"
}
</script>

<template>
  <div class="quiz-container">
    <div class="card">
      <h3>과제 2. 불변성 복사 및 데이터 방어</h3>
      <button @click="runTask2">과제 2 가동</button>
      <div class="console">결과창 2: {{ result2 }}</div>
    </div>
  </div>
</template>
```

Code Challenge

- 다음 미션을 수행하시오




```vue
<script setup>
import { ref } from 'vue'

// 실시간 화면 출력을 위한 Vue 상태값 (수정 금지)
const result3 = ref('')

// ================================================================
// 📝 [과제 3] 서버 연쇄 데이터 요청 및 에러 통합 제어 (Async/Await)
// ================================================================
// 가상의 백엔드 API (수정 금지 - Promise 반환형 화살표 함수)
const fetchUserId = () => new Promise(res => setTimeout(() => res({ uid: 777 }), 400))
const fetchUserProfile = (uid) => new Promise(res => setTimeout(() => res({ uid, nick: 'Graves' }), 400))

const runTask3 = async () => {
  result3.value = '⏳ 데이터 동기화 중...'
  // [미션]
  // 1. async/await 문법을 사용하여 fetchUserId()를 먼저 호출하고, 결과 객체에서 uid를 뽑아내기.
  // 2. 뽑아낸 uid를 인자로 전달하며 fetchUserProfile(uid)을 연쇄 호출하고, 최종 결과 객체에서 nick을 뽑아내기.
  // 3. 비동기 통신 중 발생할 수 있는 에러를 안전하게 방어하기 위해 전체 로직을 try-catch 문으로 감싸기.
  // 4. 정상 완료 시 result3.value에 "동기화 성공: [닉네임]님 환영합니다." 주입, 에러 발생 시 "통신 실패" 주입.
}
</script>

<template>
  <div class="quiz-container">
    <div class="card">
      <h3>과제 3. 비동기 연쇄 파이프라인 (Async/Await)</h3>
      <button @click="runTask3">과제 3 가동</button>
      <div class="console">결과창 3: {{ result3 }}</div>
    </div>
  </div>
</template>
```

[실습] 과제 - 과제 확장

- 메뉴를 추가하고 활용 API를 추가해서 과제를 확장한다.



Full-Stack Engineering - Frontend-framework: Vue.js

10. Vite 빌드 및 실무 배포

SK 주식회사 AX

ESLint - Overview

- ESLint
  - ECMAScript 코드에서 발견되는 고유 패턴을 식별하고 보고하는 오픈소스 정적 코드 분석 도구(Static Analysis Tool)
  - 정적 분석(Static Analysis): 런타임 환경에서 코드를 실제로 실행하지 않고, 소스코드를 파싱하여 추상 구문 트리(Abstract Syntax Tree)라는 구조물로 변환한 뒤, 등록된 규칙(**Rules**)과 대조하여 문법 오류 및 잠재적 런타임 에러를 사전에 검출하는 기술
- 정적 분석이 필요한 이유
  - Interpreter 언어의 구조적 한계: Java, C# 등 Compile 기반 언어는 빌드 단계에서 구문 오류(Syntax Error)를 강제 검출하므로 결함이 있는 코드가 배포 단계에 진입하는 것이 원천 차단된다. 반면 JavaScript는 브라우저나 Node.js 환경에서 실시간으로 해석되는 인터프리터 언어이므로, 구문 오류가 포함된 상태로 배포가 가능하며 해당 라인이 실행되는 시점에 애플리케이션이 Crash를 일으킨다.
  - 동적 타이핑과 느슨한 문법 규칙의 부작용: JavaScript는 변수의 데이터 타입을 동적으로 결정하고, 세미콜론 자동 삽입(ASI) 등을 지원하는 등 문법적 허용 범위가 넓다. 이러한 특성은 개발 초기 속도를 높이지만, 대규모 코드베이스에서는 예측 불가능한 Side Effect와 메모리 누수, 가독성 저하를 유발한다.
- 프로젝트 표준화 및 CI/CD 연동 효과
  - 코드 일관성(Consistency) 유지: 개발자 개인의 성향에 따라 파편화되는 소스코드 구조를 하나의 통합된 규칙 세트(Rule Set)로 획기적으로 일관화한다.
  - 배포 파이프라인 자동 검수: 지속적 통합(CI/CD) 프로세스에 eslint 명령어를 포함시켜, 정적 검사를 통과하지 못한 결함 코드가 상용 서버에 배포(Production Deployment)되는 프로세스를 자동 차단한다.

ESLint - 주요 점검 항목

- Syntax Errors

```
// 개발자의 단순 오타 상황
const myLocation = 'Suwon';
console.log(myLocatoin); // ❌ 변수명 오타! ESLint가 없으면 배포되어 사용자가 이 라인을 실행할 때 화면이 죽습니다.
```

- Dead Code Elimination (Unused Variables)

```
// 선언만 해두고 안 쓰는 변수, import만 해두고 쓰지 않는 파일들
const secretToken = 'xyz123'; // ❌ 보안 위협 및 메모리 낭비
// ESLint는 사용하지 않는 코드(Dead Code)를 적발하여 소스코드를 경량화 상태로 유지합니다.
```

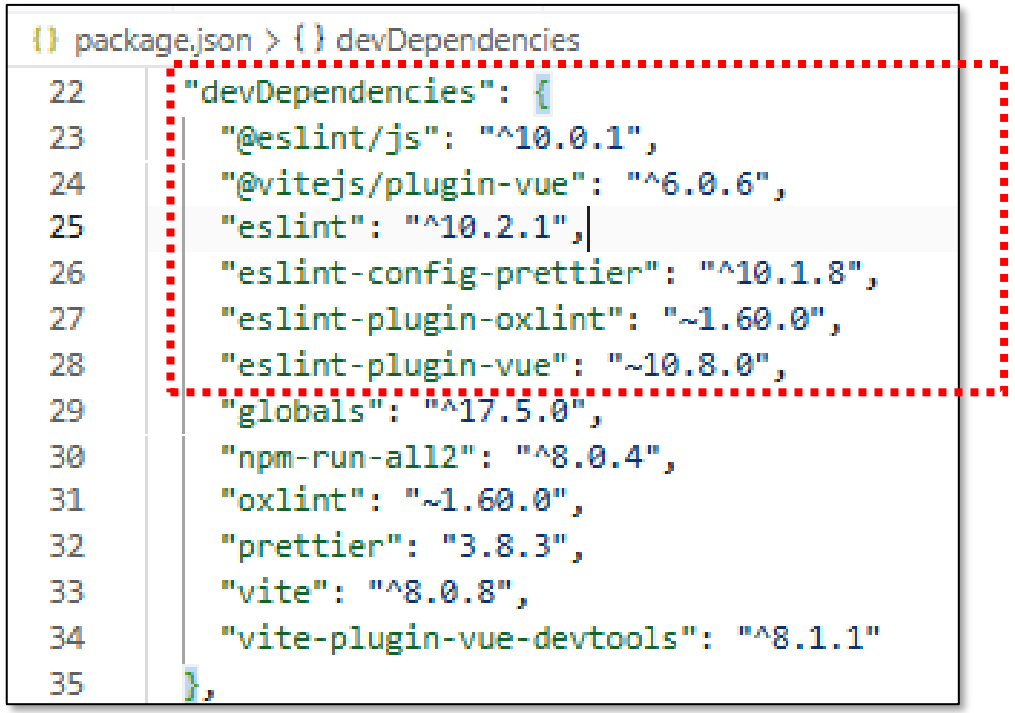
- Anti-Pattern Prevention

```
// 런타임 암묵적 타입 변환 버그 유발 구문
if (userAge == 20) { ... }
// [ESLint 검출]: Expected '===' and instead saw '=='.
// 엔진 내부의 예기치 못한 형변환을 차단하고 엄격한 비교(Strict Equality) 강제.
```

ESLint - Installation

- Vite 프로젝트 초기 생성 시 Option으로 설치 됨.
  - ESLint는 개발할 때만 감시용으로 쓰고, 최종 배포(Production) 코드에는 포함될 필요가 없다.
- 최초 생성 시 Option으로 미 설치 시

```
# Vite 프로젝트 환경에 맞는 eslint 및 Vue 전용 플러그인 설치
npm install -D eslint eslint-plugin-vue
```



ESLint - Configuration

- eslint.config.js
  - ① 설정이 적용되는 소스코드 파일을 지정
  - ② 검사 제외 대상 폴더
  - ③ JavaScript 실행 환경의 전역 변수를 등록
    …global.browse는 window, document, localStorage 같은 브라우저 객체를 정식 변수로 인정한다.
  - ④ 표준 추천 규칙: essential은 필수 문법 에러만 잡는다.
  - ⑤ 기존 ESLint보다 최대 50~100배 빠른 초고속 린터 엔진 (Oxlint)를 JavaScrpt 설정과 동기화 한다.
  - ⑥ 줄바꿈, 따옴표, 들여쓰기 등 '시각적 스타일' 규칙들은 ESLint에서 Off하고 Prettier에 전권을 위임

eslint.config.js > ...

```
1  import { defineConfig, globalIgnores } from 'eslint/config'
2  import globals from 'globals'
3  import js from '@eslint/js'
4  import pluginVue from 'eslint-plugin-vue'
5  import pluginOxlint from 'eslint-plugin-oxlint'
6  import skipFormatting from 'eslint-config-prettier/flat'
7
8  export default defineConfig([
9    {
10     name: 'app/files-to-lint',
11     files: ['**/*.{vue,js,mjs,jsx}'],
12   },
13
14   globalIgnores(['**/dist/**', '**/dist-ssr/**', '**/coverage/**']),
15
16   {
17     languageOptions: {
18       globals: {
19         ...globals.browser,
20       },
21     },
22   },
23
24   js.configs.recommended,
25   ...pluginVue.configs['flat/essential'],
26
27   ...pluginOxlint.buildFromOxlintConfigFile('.oxlintrc.json'),
28
29   skipFormatting,
30 ])
```

ESLint - Custom Rules

- Custom Rules (eslint.config.js)
  - JavaScript 배열은 하단에 위치할 수록 앞서 로드된 기본 규칙을 덮어쓴다.
  - Custom 규칙은 skipFormatting 바로 직전 설정한다

```
  js.configs.recommended,
  ...pluginVue.configs['flat/essential'],

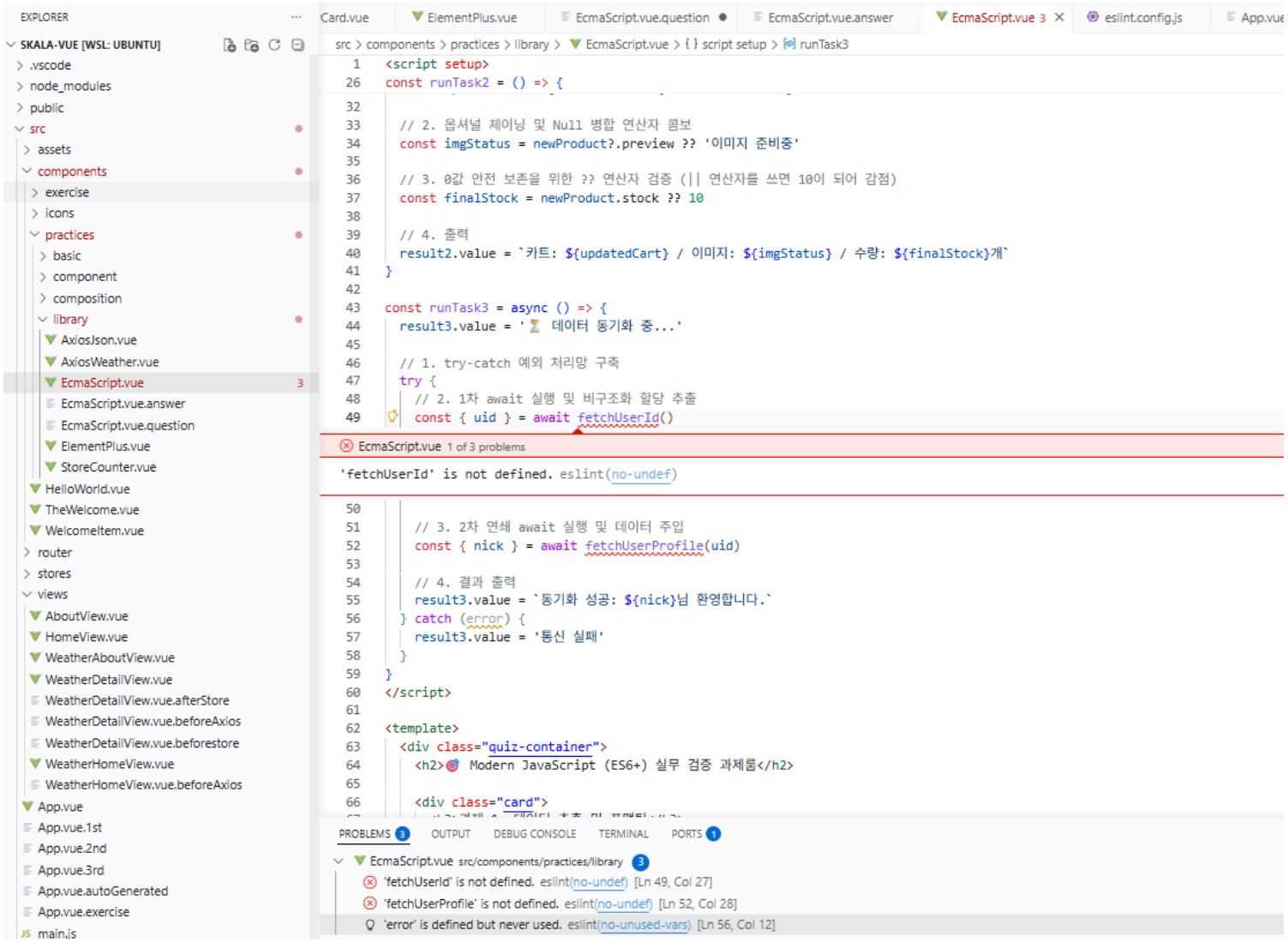
  ...pluginOxlint.buildFromOxlintConfigFile('.oxlintrc.json'),

  // 🟢 커스텀 규칙 설정 객체
  {
    name: 'app/custom-rules', // 규칙 묶음의 식별자 이름 (옵션)
    rules: {
      'no-unused-vars': 'warn', // 선언 후 사용하지 않은 변수는 경고 처리
      'no-console': 'off', // 개발 편의를 위해 console.log 허용
      'vue/multi-word-component-names': 'off', // 단일 단어로 된 컴포넌트명 허용
    },
  },

  skipFormatting,
```

ESLint - 실시간 피드백

- 소스 코드내 실시간으로 문법 오류를 점검
  - 빨간색/노란색 물결 밑줄: 코드에 마우스를 올리면 [no-unused-vars] 처럼 위반한 ESLint 규칙명과 함께 원인 메시지가 Tooltip 으로 표시.
  - 파일 이름 색상 변경: 에러가 있는 파일은 탐색기(Sidebar) 창에서 이름이 빨간색(Error) 또는 노란색(Warning)으로 변경되며 옆에 위반 개수가 표시.
  - Problems Tab: 에디터 하단에 프로젝트 전체 파일의 ESLint 위반 리스트가 파일명, 라인 번호와 함께 일괄 정렬.



ESLint - 일괄 점검

- 터미널에 npm run lint를 통해 프로젝트 소스 일괄 점검을 진행할 수 있다.

```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm run lint

> skala-vue@0.0.0 lint
> run-s lint:*


> skala-vue@0.0.0 lint:oxlint
> oxlint . --fix

Found 0 warnings and 0 errors.
Finished in 24ms on 74 files with 89 rules using 8 threads.

> skala-vue@0.0.0 lint:eslint
> eslint . --fix --cache


/home/bottletiger/projects/skala-vue/src/components/practices/library/EcmaScript.vue
  49:27  error    'fetchUserId' is not defined       no-undef
  52:28  error    'fetchUserProfile' is not defined  no-undef
  56:12  warning  'error' is defined but never used  no-unused-vars

✖ 3 problems (2 errors, 1 warning)

ERROR: "lint:eslint" exited with 1.
```

Prettier - Overview

- Prettier
  - ESLint가 문법적 에러(Bug)를 잡는다면, Prettier는 띄어쓰기, 줄바꿈, 따옴표 종류 등 코드의 시각적 스타일(Formatting)만 전문적으로 교정한다.
  - 여러 명이 협업할 때 누구는 들여쓰기를 2칸 하고, 누구는 4칸을 하거나, 문장 끝에 세미콜론을 넣고 빼는 등의 차이가 발생한다. 이는 Git에서 불필요한 코드 충돌(Conflict)을 발생시키기도 한다.
  - Prettier는 파일 저장 순간 팀이 정한 규칙대로 코드를 강제 정렬한다.

Prettier - Installation

- Vite 프로젝트 초기 생성 시 Option으로 설치 됨.
  - Prettier도 개발할 때만 사용하고, 최종 배포(Production) 코드에는 포함될 필요가 없다.
- 최초 생성 시 Option으로 미 설치 시

```
npm install -D prettier
```



Prettier - Configuration

- .prettierrc.json
  - 프로젝트 루트 디렉토리에 .prettierrc.json 파일을 생성하고, 가독성 옵션을 주입한다.

```
{
  "$schema": "https://json.schemastore.org/prettierrc",
  "semi": false,
  "singleQuote": true,
  "tabWidth": 2,
  "printWidth": 200
}
```

  - "$schema" - JSON 파일이 준수해야 하는 규격 명세서(JSON Schema)의 위치(URL)을 지정하는 메타데이터
  - "semi": false - 문장 끝에 세미콜론(;)을 붙이지 않도록 규격화합니다.
  - "singleQuote": true - 문자열을 감쌀 때 쌍따옴표(") 대신 홑따옴표(')를 강제합니다.
  - "tabWidth": 2 - 들여쓰기(Tab) 너비를 Vue 3 표준인 2칸 공간으로 고정합니다.
  - "printWidth": 200 - 한 줄이 200자를 넘어가면 가독성을 위해 자동으로 줄바꿈을 처리합니다.

Prettier - 일괄수정

- 터미널에 npm run format를 통해 프로젝트 소스 일괄 수정을 진행할 수 있다.

```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm run format

> skala-vue@0.0.0 format
> prettier --write --experimental-cli src/

src/components/exercise/UnitToggler.vue
src/components/practices/basic/SampleTwo.vue
src/components/practices/component/PropsEmitsParent.vue
src/components/practices/component/SlotDefaultParent.vue
src/components/practices/component/SlotNamedParent.vue
src/components/practices/component/SlotScopedParent.vue
src/router/index.js
src/stores/configStore.js
src/views/WeatherAboutView.vue
```

Vite Configuration

- vite.config.js
  - Vite 빌드 도구의 컴파일러 동작, 개발 서버 환경, 빌드 및 번들링 파이프라인의 명세를 정의하는 Configuration File
  - defineConfig(): 객체 형식으로 설정을 작성할 때 사용하는 내장 래퍼 함수
- 주요 속성
  - plugins: 브라우저가 직접 해석할 수 없는 .vue 확장자의 파일을 표준 JavaScript 모듈 객체로 트랜스파일(Transpile)하는 컴파일러 플러그인
  - resolve.alias: 프로젝트 루트 디렉토리의 src 폴더 물리 경로를 @ 기호로 매핑하여 모듈의 절대 경로 참조 체계를 설정

vite.config.js > ...

```
1   import { fileURLToPath, URL } from 'node:url'
2
3   import { defineConfig } from 'vite'
4   import vue from '@vitejs/plugin-vue'
5   import vueDevTools from 'vite-plugin-vue-devtools'
6
7   // https://vite.dev/config/
8   export default defineConfig({
9     plugins: [
10      vue(),
11      vueDevTools(),
12    ],
13    resolve: {
14      alias: {
15        '@': fileURLToPath(new URL('./src', import.meta.url))
16      },
17    },
18  })
```

Vite Configuration - Custom 추가

- 실무 환경에서 빈번하게 추가되는 속성 (vite.config.js)
  - server: 로컬 개발 서버의 속성
  - build: 컴파일 완료된 산출물 사양 제어

```
// 🟢 [커스텀 추가 1] 로컬 개발 서버(Dev Server) 속성 제어
server: {
  port: 3000, // 개발 서버의 네트워크 포트를 3000번으로 고정 명세
  open: true, // 프로세스 기동(npm run dev) 시 기본 웹 브라우저를 자동 실행
},
// 🟢 [커스텀 추가 2] 컴파일 완료된 산출물(Production Build) 사양 제어
build: {
  outDir: 'dist', // 최종 정적 리소스(HTML, JS, CSS)가 저장될 출력 디렉토리명 지정
},
```

- 속성 추가 후 실행 결과

```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm run dev

> skala-vue@0.0.0 dev
> vite


  VITE v8.0.16  ready in 812 ms

  ➜  Local:   http://localhost:3000/
  ➜  Network: use --host to expose
  ➜  Vue DevTools: Open http://localhost:3000/__devtools__/ as a separate window
  ➜  Vue DevTools: Press Alt(⌥)+Shift(⇧)+D in App to toggle the Vue DevTools
  ➜  press h + enter to show help
```

Environment Variables

- 환경 변수의 분리
  - Application Source Code 내부 하드코딩된 동적 설정값들을 운영체제나 빌드 스크립트 레벨의 환경 변수(Environment Variables)로 이관하여 격리하는 개발 방법
- 도입 목적
  - 보안성 확보: API 보안 토큰, 데이터베이스 접속 정보 등 민감 데이터를 소스코드(Git)에 노출하지 않고 안전하게 관리한다.
  - 환경별 유연성: 소스코드를 수정하지 않고, 빌드 명령어 조합 변경만으로 검증용(Staging) 서버와 실제 상용(Production) 서버의 API 엔드포인트를 스위칭 할 수 있다.

Environment Variables - 작성 규칙

- Vite는 루트 디렉토리에 존재하는 .env 파일들을 자동 로드한다.
- Vite 엔진은 오직 VITE_로 시작하는 환경 변수만 프론트엔드 클라이언트 코드 내부로 노출시킨다.
- 작성 Example

```
# .env.staging
VITE_API_URL=https://api-staging.skala.co.kr
VITE_APP_MODE=Staging Mode
```

```
# .env.production
VITE_API_URL=https://api.skala.co.kr
VITE_APP_MODE=Production Mode
```

- 활용 Example

```
<script setup>
// Vite 환경 변수 참조 (런타임 시 빌드 모드에 따라 값이 동적으로 주입됨)
const currentApiUrl = import.meta.env.VITE_API_URL
const currentMode = import.meta.env.VITE_APP_MODE

console.log('현재 주입된 API 서버 주소:', currentApiUrl)
</script>

<template>
  <div>
    <p>현재 빌드 모드: {{ currentMode }}</p>
    <p>연동 API 엔드포인트: {{ currentApiUrl }}</p>
  </div>
</template>
```

Environment Variables - 빌드

- 빌드 명령어 뒤에 --mode 옵션을 명시하여 필요한 환경 변수를 사용하여 빌드한다.

```
"scripts": {
  "dev": "vite",
  "build:staging": "vite build --mode staging",
  "build:production": "vite build --mode production"
}
```

Bundling and Build

- Bundling
  - 웹 애플리케이션을 구성하는 수십, 수백 개의 파일(Vue, JS, CSS, Image 등) 간의 의존성 흐름을 정적으로 분석하여, 브라우저가 로드하기 최적화된 **최소한의 파일 파일 개수군으로 묶고 압축하는 과정**
  - Vite는 개발단계에서는 ES Modules(ESM) 기반의 초고속 런타임 방식을 취하지만, 최종 프로덕션 빌드 단계에서는 Rollup 번들러 엔진을 기동하여 최적화된 정적 자산을 생성한다.
- 빌드 수행

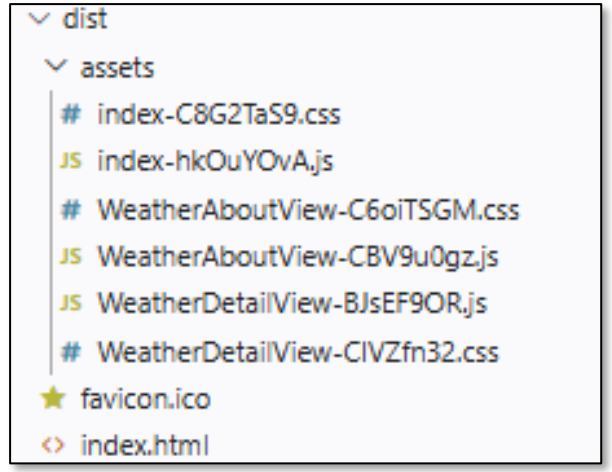
```
bottletiger@SKCC22N01104:~/projects/skala-vue$ npm run build

> skala-vue@0.0.0 build
> vite build

vite v8.0.16 building client environment for production...
✓ 1664 modules transformed.
computing gzip size...
dist/index.html                                  0.42 kB │ gzip:   0.28 kB
dist/assets/WeatherDetailView-CIVZfn32.css       0.33 kB │ gzip:   0.21 kB
dist/assets/WeatherAboutView-C6oiTSGM.css        0.48 kB │ gzip:   0.26 kB
dist/assets/index-C8G2TaS9.css                 362.63 kB │ gzip:  49.23 kB
dist/assets/WeatherAboutView-CBV9u0gz.js         0.98 kB │ gzip:   0.68 kB
dist/assets/WeatherDetailView-BJsEF9OR.js        2.04 kB │ gzip:   1.34 kB
dist/assets/index-hkOuYOvA.js                1,053.24 kB │ gzip: 345.91 kB
```

Bundling and Build

- dist
  - 빌드가 완료되면 프로젝트 루트 디렉토리에 dist/ (Distribution)라는 배포 전용 폴더가 생성된다.
  - dist 폴더 내부에는 더 이상 .vue 파일이나 개발용 모듈이 존재하지 않고 오직 웹 브라우저가 해석할 수 있는 순수 html, js, css 파일만 남는다.
  - 해시(Hash) 이름 식별자: WeatherDetailView-CIVZfn32.css처럼 파일명 뒤에 동적으로 붙는 고유 문자열은 파일의 Hash 값이다. 파일 내용이 수정되면 이 해시값이 바뀐다. 이는 브라우저가 과거의 구형 코드를 캐싱(Caching)하여 화면 갱신이 안 되는 버그를 방지하기 위한 웹 배포 표준 사양이다.
  - 이 완제품 dist 폴더 자체를 AWS S3, Nginx, Netlify, Vercel 등 정적 웹 호스팅 서버에 그대로 Upload하면, 웹 서비스 배포 파이프라인이 최종 완결된다.



Code Challenge - ESLint

- ESLint Custom 규칙 설정
  - 엄격한 비교 연산자(===) 사용을 강제하는 규칙: 'eqeqeq': ['error', 'always']
  - 개발 편의를 위해 console.log를 허용하는 규칙: 'no-console': 'off'
- ESLint Custom 규칙 적용 확인
  - 컴포넌트 소스코드 중 한 곳을 골라 if (userAge == 20)과 같이 의도적인 느슨한 비교 연산 구문을 삽입한 뒤 저장한다.
  - 에디터 화면(물결선 툴팁)과 터미널에 npm run lint를 실행했을 때 검출되는 에러 로그 내용을 확인한다

Code Challenge - Prettier

- 다음 미션을 수행한다.
  - 임의의 Vue 컴포넌트 <script setup> 내부에 아래 코드를 타이핑하여 저장한다. (정렬 및 인덴트가 엉망인 상태 유지)

```
JavaScript
const     myRegion   = `Suwon` ;
const regionGreeting = `웰컴 투 ${myRegion}`;
```

  - 터미널에서 npm run format 명령어를 실행하여 포맷팅한다.
  - 포맷팅 완료 후, myRegion 라인의 백틱(`) 기호와 공백이 어떻게 자동 변환되었는지 확인한다.

Code Challenge - env

- 다음 미션을 수행한다.
  - 프로젝트 루트 디렉토리에 .env.staging 파일과 .env.production 파일을 각각 생성하고 VITE_API_URL 변수를 설정한다.

    env.staging → VITE_API_URL=https://api-stage.skcc.com

    env.production → VITE_API_URL=https://api-prod.skcc.com
  - 임의의 컴포넌트 내부에서 import.meta.env.VITE_API_URL을 console.log로 출력하는 구문을 작성한다.
  - package.json에 빌드 스크립트를 아래와 같이 추가 등록한 후, npm run build:staging을 실행했을 때 터미널 컴파일 라인에 어떤 환경 파일이 로드되는지 관측한다

```
"build:staging": "vite build --mode staging"
```

Code Challenge - build

- 다음 미션을 수행한다.
  - 터미널에 npm run build 명령어를 실행하여 배포 완제품을 만든다.
  - 빌드 완료 후 루트 디렉토리에 새로 생성된 dist/ 폴더를 확인한다.
  - Dist/assets/ 폴더 내부에 생성된 자바스크립트 파일명(예: main-xxxx.js)을 확인한다.

[실습] 과제 - 완성

- ESLint로 점검하여 제출 과제의 Error를 없도록 한다.
- API 키는 환경 변수로 조정하고 Git에 업로드 되지 않도록 한다.
- Build 후 정적파일들(dist폴더)을 GitHug Pages에 올려놓아 Node.js가 없이 Hosting을 제공한다.

수고 하셨습니다!